GTK4 Tutorial for Beginners

Toshio Sekiya

October 23, 2026

GTK 4 Tutorial for beginners

Now that GTK 4 is widely used, "GTK" usually means GTK 4. Therefore, this tutorial simply says "GTK" except where the version needs to be explicitly mentioned.

This tutorial illustrates how to write C programs with the GTK library. It is aimed at beginners, so the contents are limited to the basics. The table of contents is at the end of this README.md.

- Section 3 to 26 describe the basics, with the example of a simple editor `tfe` (Text File Editor).
- Section 27 to 30 describe GtkDrawingArea.
- Section 31 describes Drag and Drop.
- Section 32 to 36 describe the list model and the list view (GtkListView, GtkGridView, and GtkColumnView). It also describes GtkExpression.

The latest version of the tutorial is located at the GTK4-tutorial GitHub repository. You can read it directly without downloading.

There's a GitHub Page which is the HTML version of the tutorial.

GTK Documentation

Please refer to the GTK API Reference and the GNOME Developer Documentation Website for further information. These websites were opened in August 2021.

If you want to know about GObject and the type system, please refer to the GObject tutorial. GObject is the base of GTK, so it is important for developers to understand it as well as GTK.

Contribution

This tutorial is still under development and unstable. Even though the example codes have been tested on GTK (version 4.22.4), bugs may still exist. If you find any bugs, errors, or mistakes in the tutorial and C examples, please let me know. You can post them to GitHub issues. You can also post updated files to pull request. When submitting a pull request, please ensure that you only modify the files in the `src` directory. Don't modify the files under `gfm` or `html` directories directly. After modifying some source files, run `bundle exec rake` to create GFM (GitHub Flavoured Markdown) files and run `G4T_ENV=production bundle exec rake html` to create HTML files. Please refer to the Author Guide for further information.

If you have a question, feel free to post it to `issue` of this repository. Your questions and feedback are welcome and invaluable for improving this tutorial.

How to Get Gtk 4 Tutorial in HTML or PDF Format

There is the PDF file in the pdf directory. If you want to build the tutorial in HTML or PDF format, build them with `rake` command, which is a ruby version of make. Type `bundle exec rake html` or `G4T_ENV=developmnt bundle exec rake html` for HTML for local serve. Type `bundle exec rake pdf` for PDF. For further information, see the Autho Guide.

An appendix Author Guide describes how to write source files.

License

The license of this repository is written in Section1.

In short:

- GFDL1.3 for documents
- GPL3 for programs

Contents

GTK 4 Tutorial for beginners . . . iii

1 Introduction **1**
1.1 Prerequisite and License . . . 1
1.2 Preparation . . . 2

2 GTK Basics **4**
2.1 GtkApplication and GtkApplicationWindow . . . 4
2.2 Widgets (1) . . . 9
2.3 Widgets (2) . . . 16
2.4 Strings and Memory Management . . . 19
2.5 Widgets (3) . . . 23
2.6 Defining a Final Class . . . 29
2.7 GtkBuilder and UI Files . . . 36
2.8 Build System . . . 45
2.9 Instance Initialization and Destruction . . . 46
2.10 Error Handling in GLib . . . 55
2.11 Completing the custom object implementation . . . 58
2.12 File Dialogs and the GIO Asynchronous Model . . . 64
2.13 Completing the Main Program . . . 70
2.14 Menus and Actions . . . 76
2.15 Stateful Actions . . . 82
2.16 Ui File for Menu and Action Entries . . . 92
2.17 GtkPopoverMenuBar and GtkMenuButton . . . 99
2.18 Managing the Application . . . 102
2.19 Creating TfeWindow . . . 106
2.20 Handling Unsaved Text . . . 112
2.21 Accelerators . . . 119
2.22 CSS and Pango . . . 121
2.23 GtkFontDialogButton and CSS Updates . . . 127
2.24 Gsettings . . . 131

3 GtkDrawingArea **141**
3.1 GtkDrawingArea and Cairo . . . 141
3.2 Periodic Events . . . 145
3.3 Custom Drawing . . . 152
3.4 Tiny Turtle Graphics Interpreter . . . 159

4 Drag and Drop **198**
4.1 Drag and Drop . . . 198

5 List Widgets **204**
5.1 GtkListView . . . 204
5.2 GtkGridView and Activate Signal . . . 213
5.3 GtkExpression . . . 223
5.4 GtkColumnView . . . 237
5.5 GtkSignalListItemFactory . . . 246

A Appendix **250**
A.1 Author Guide 250
A.2 TfeTextView API reference 258
A.3 Turtle Manual 261

Chapter 1

Introduction

1.1 Prerequisite and License

1.1.1 Prerequisite

1.1.1.1 GTK on a Linux OS

This tutorial describes GTK version 4 libraries. Although GTK is primarily designed for Linux using the C compiler, it is now widely used on other platforms, including Windows and macOS, through various language bindings such as Vala, Python, Rust, and Ruby. However, this tutorial describes only **C programs on Linux**.

If you want to try the examples in the tutorial, you need:

- PC with a Linux distribution like Ubuntu or Debian.
- GCC.
- GTK 4 (version 4.10.1 or later).

The stable version of GTK is 4.22.4 at present (July 13, 2026). The examples in this tutorial have been tested with GTK version 4.22.4. Version 4.10 added some new classes and functions and deprecated some old ones. Some example programs in this tutorial don't work on older versions.

1.1.1.2 Note for WSL2 (WSLg) Users

When running GTK4 applications on WSL2 (Windows Subsystem for Linux), you may encounter several warnings in your terminal. These are typically due to interactions between GTK4's rendering system and the WSLg Wayland compositor, rather than bugs in your code. These warnings are often harmless, but in some cases they may affect rendering performance or popup behavior. **In particular, menu programs do not work as expected in WSL.**

For professional-grade performance testing or pixel-perfect UI behavior, a native Linux installation is recommended.

1.1.1.3 Ruby and Rake for Making the Document

This repository includes Ruby programs. They are used to make GFM (GitHub Flavored Markdown), HTML, and PDF files.

You need:

- Linux OS (including WSL)
- Ruby 3.0 or later (tested on 4.0.5)
- Rake

There are two ways to install Ruby.

- Install your distribution's package.
- Use a Ruby version manager such as rbenv or mise.

Note: If you choose to use a Ruby version other than 4.0.5, you must update the version settings in `Gemfile`. Additionally, depending on your version manager, you may need to modify `.ruby-version` or `mise.toml` as well.

1.1.2 License

Copyright (C) 2020, 2023 ToshioCP (Toshio Sekiya)

The GTK4-tutorial repository contains tutorial documents and programs such as converters, generators and controllers. All of them comprise the 'GTK4-tutorial' package. This package is simply called 'GTK4-tutorial' in the following description.

GTK4-tutorial is free; you can redistribute it and/or modify it under the terms of the following licenses.

- The license of documents in GTK4-tutorial is the GNU Free Documentation License as published by the Free Software Foundation; either version 1.3 of the License or, at your option, any later version. The documents are Markdown, HTML and image files. If you generate a PDF file by running `rake pdf`, it is also considered part of the documents.
- The license of programs in GTK4-tutorial is the GNU General Public License as published by the Free Software Foundation; either version 3 of the License or, at your option, any later version. The programs are written in C, Ruby and other languages.

GTK4-tutorial is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU License web pages for more details.

- GNU Free Documentation License
- GNU General Public License

The licenses above are effective as of April 15, 2023. Before that, GPL covered all the contents of the GTK4-tutorial. But GFDL1.3 is more appropriate for documents so the license was changed.

1.2 Preparation

1.2.1 Installing GTK on Linux Distributions

This section describes how to install GTK on Linux distributions.

There are two ways to install GTK:

- Install from distribution packages.
- Build from source.

1.2.1.1 Installation from Distribution Packages

This is the easiest way to install. The following command installs the GTK (version 4) development files on Ubuntu.

```
$ sudo apt install libgtk-4-dev
```

It is important to install the development package (libgtk-4-dev). Otherwise, you will not be able to compile any GTK-based programs.

Fedora, Debian, Arch, Gentoo, and OpenSUSE also have GTK packages. See the website of your distribution for further information.

Package information for Arch, Debian/Ubuntu, and Fedora is described on the GTK website: Installing GTK from packages.

1.2.1.2 Installation from Source

If you want to install the development version of GTK, you need to build it from source. See the Compiling the GTK Libraries section of the GTK API reference.

1.2.2 How to Download This Repository

There are two ways: zip and git. Downloading a zip file is the easiest way. However, if you use git and clone this repository, you can easily update your local repository with the `git pull` command.

1.2.2.1 Download a zip File

- Open your browser and navigate to this repository.
- Click on the green `<> Code` button. A dropdown menu will appear; click `Download ZIP`.
- The repository will be downloaded as a zip file to your computer.
- Unzip the file.

1.2.2.2 Clone the Repository

- Click on the green `<> Code` button. A dropdown menu will appear. The first section is `Clone` with three tabs. Click the `HTTPS` tab and click the copy icon to the right of `https://github.com/ToshioCP/Gtk4-tutorial.git`.
- Open a terminal and type `git clone`, then press Ctrl+Shift+V. The command line will look like `git clone https://github.com/ToshioCP/Gtk4-tutorial.git`. Press Enter.
- The directory `Gtk4-tutorial` will be created. This directory is a local copy of the repository.

1.2.3 Examples in the Tutorial

Examples are under the `src` directory. For example, the first example of the tutorial is `pr1.c` and its path is /src/misc/pr1.c. Therefore, you do not need to type the examples manually.

Chapter 2

GTK Basics

2.1 GtkApplication and GtkApplicationWindow

2.1.1 GtkApplication

2.1.1.1 GtkApplication and g_application_run

People write programming codes to make an application. What are applications? Applications are software that runs using libraries, which are the OS, frameworks and so on. In GTK 4 programming, the GtkApplication is a program (or executable) that runs using Gtk libraries.

The basic way to write a GtkApplication is as follows.

- Create a GtkApplication instance.
- Run the application.

That's all. Very simple. The following is the C code representing the scenario above.

```
1  #include <gtk/gtk.h>
2
3  int
4  main (int argc, char **argv) {
5    GtkApplication *app;
6    int stat;
7
8    app = gtk_application_new ("com.github.ToshioCP.pr1", G_APPLICATION_DEFAULT_FLAGS);
9    stat = g_application_run (G_APPLICATION (app), argc, argv);
10   g_object_unref (app);
11   return stat;
12 }
```

The first line says that this program includes the header files of the Gtk libraries. The function `main` is a startup function in C language. The variable `app` is defined as a pointer to a GtkApplication instance. The function `gtk_application_new` creates a GtkApplication instance and returns a pointer to the instance. The GtkApplication instance is a C structure data in which the information of the application is stored. The arguments will be explained later. The function `g_application_run` runs an application that the instance defined. (We often say that the function runs `app`. Actually, `app` is not an application but a pointer to the instance of the application. However, it is simple and short, and probably no confusion occurs.)

Here I used the word `instance`. Instance, class and object are terminologies in Object Oriented Programming. I use these words in the same way. But, I will often use "object" instead of "instance" in this tutorial. That means "object" and "instance" are the same. Object is a bit ambiguous word. In a broad

sense, object has wider meaning than instance. So, readers should be careful of the contexts to find the meaning of "object". In many cases, object and instance are interchangeable.

The function `gtk_application_new` has two parameters.

- Application ID (com.github.ToshioCP.pr1). It is used to distinguish applications by the system. The format is reverse-DNS. See GNOME Developer Documentation – Application ID for further information.
- Application flag (G_APPLICATION_DEFAULT_FLAGS). If the application runs without any arguments, the flag is G_APPLICATION_DEFAULT_FLAGS. Otherwise, you need other flags. See GIO API reference for further information.

Notice: If your GLib-2.0 version is older than 2.74, use `G_APPLICATION_FLAGS_NONE` instead of `G_APPLICATION_DEFAULT_FLAGS`. It is an old flag replaced by `G_APPLICATION_DEFAULT_FLAGS` and deprecated since version 2.74.

To compile this, run the following command. The string `pr1.c` is the filename of the C source code above.

If you've downloaded this repository, you don't need to create the file. There's the same file at /src/misc/pr1.c in your local repository. All the example codes are under the /src directory as well.

```
$ gcc `pkg-config --cflags gtk4` pr1.c `pkg-config --libs gtk4`
```

The C compiler gcc generates an executable file, `a.out`. Let's run it.

```
$ ./a.out

(a.out:5084): GLib-GIO-WARNING **: 09:52:04.236: Your application does not implement
g_application_activate() and has no handlers connected to the 'activate' signal.
It should do one of these.
$
```

Oh, it just produces an error message. This error message shows that the GtkApplication object ran, without a doubt. Now, let's think about what this message means.

2.1.1.2 Signals

The message tells us that:

1. The application doesn't implement `g_application_activate()`,
2. It has no handlers connected to the "activate" signal, and
3. You will need to solve at least one of these.

These two problems are related to signals. So, I will explain that first.

A signal is emitted when something happens. For example, a window is created, a window is destroyed and so on. The signal "activate" is emitted when the application is activated. (Activated is a bit different from started, but you can think of them both as almost the same so far.) If the signal is connected to a function, which is called a signal handler or simply handler, then the function is invoked when the signal is emitted.

The flow is like this:

1. Something happens.
2. If it's related to a certain signal, then the signal is emitted.
3. If the signal has been connected to a handler in advance, then the handler is invoked.

Signals are defined in objects. For example, the "activate" signal belongs to the GApplication object, which is a parent object of GtkApplication object.

The GApplication object is a child object of the GObject object. GObject is the top object in the hierarchy of all the objects.

```
GObject -- GApplication -- GtkApplication
<---parent                      --->child
```

A child object inherits signals, functions, properties and so on from its parent object. So, GtkApplication also has the "activate" signal.

Now we can solve the problem in `pr1.c`. We need to connect the "activate" signal to a handler. We use a function `g_signal_connect` which connects a signal to a handler.

```
1  #include <gtk/gtk.h>
2
3  static void
4  app_activate (GApplication *app, gpointer *user_data) {
5    g_print ("GtkApplication is activated.\n");
6  }
7
8  int
9  main (int argc, char **argv) {
10   GtkApplication *app;
11   int stat;
12
13   app = gtk_application_new ("com.github.ToshioCP.pr2", G_APPLICATION_DEFAULT_FLAGS);
14   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
15   stat = g_application_run (G_APPLICATION (app), argc, argv);
16   g_object_unref (app);
17   return stat;
18 }
```

First, we define the handler `app_activate` which simply displays a message. The function `g_print` is defined in GLib and it's like a printf in the C standard library. In the function `main`, we add `g_signal_connect` before `g_application_run`. The function `g_signal_connect` has four arguments.

1. An instance to which the signal belongs.
2. The name of the signal.
3. A handler function (also called callback), which needs to be cast by `G_CALLBACK`.
4. Data to pass to the handler. If no data is necessary, NULL is given.

It is described in the GObject API Reference. Correctly, `g_signal_connect` is a macro (not a C function).

```
#define g_signal_connect (
  instance,
  detailed_signal,
  c_handler,
  data
)
```

You can find the description of each signal in the API reference manual. For example, "activate" signal is in the GApplication section in the GIO API Reference.

```
void
activate (
  GApplication *self,
  gpointer user_data
)
```

This is a declaration of the "activate" signal handler. You can use any name instead of "activate" in the declaration above. The parameters are:

- `self` is an instance to which the signal belongs.

- `user_data` is a data defined in the fourth argument of the `g_signal_connect` function. If it is `NULL`, then you can ignore and leave out the second parameter.

API reference manual is very important. You should see and understand it.

Let's compile the source file above (`pr2.c`) and run it.

```
$ gcc `pkg-config --cflags gtk4` pr2.c `pkg-config --libs gtk4`
$ ./a.out
GtkApplication is activated.
$
```

OK, well done. However, you may have noticed that it's painful to type such a long line to compile. It is a good idea to use shell script to solve this problem. Make a text file which contains the following line.

```
gcc `pkg-config --cflags gtk4` $1.c `pkg-config --libs gtk4`
```

Then, save it in your current directory and name it "comp".

```
$ bash comp pr2
$ ./a.out
GtkApplication is activated.
$
```

2.1.2 GtkWindow and GtkApplicationWindow

2.1.2.1 GtkWindow

A message "GtkApplication is activated." was printed out in the previous subsection. It was good in terms of a test of GtkApplication. However, it is insufficient because Gtk is a framework for graphical user interface (GUI). Now we go ahead with adding a window into this program. What we need to do is:

1. Create a GtkWindow.
2. Connect it to the GtkApplication.
3. Show the window.

Now rewrite the function `app_activate`.

2.1.2.1.1 Create a GtkWindow

```
1 static void
2 app_activate (GApplication *app, gpointer user_data) {
3   GtkWidget *win;
4
5   win = gtk_window_new ();
6   gtk_window_set_application (GTK_WINDOW (win), GTK_APPLICATION (app));
7   gtk_window_present (GTK_WINDOW (win));
8 }
```

Widget is an abstract concept that includes all the GUI interfaces such as windows, labels, buttons, multi-line text, boxes and so on. And GtkWidget is a base object from which all the GUI objects derive.

```
parent <-----> child
GtkWidget -- GtkWindow
```

GtkWindow includes GtkWidget at the top of its object.

The function `gtk_window_new` is defined as follows.

```
GtkWidget *
gtk_window_new (void);
```

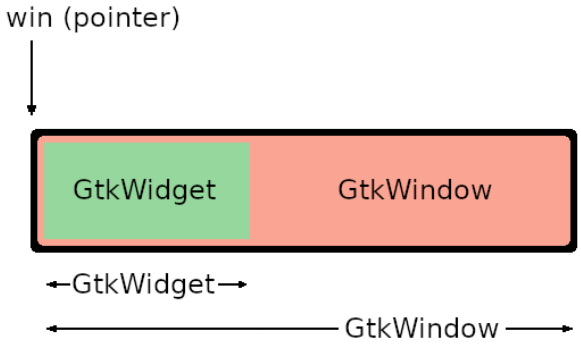


Figure 2.1: GtkWindow and GtkWidget

By this definition, it returns a pointer to GtkWidget, not GtkWindow. It actually creates a new GtkWindow instance (not GtkWidget) but returns a pointer to GtkWidget. However,the pointer points to the GtkWidget and at the same time it also points to the GtkWindow that contains the GtkWidget in it.

If you want to use `win` as a pointer to a GtkWindow type instance, you need to cast it.

```
(GtkWindow *) win
```

It works, but isn't usually used. Instead, `GTK_WINDOW` macro is used.

```
GTK_WINDOW (win)
```

The macro is recommended because it not only casts the pointer but also checks the type.

2.1.2.1.2 Connect it to the GtkApplication. The function `gtk_window_set_application` is used to connect GtkWindow to GtkApplication.

```
gtk_window_set_application (GTK_WINDOW (win), GTK_APPLICATION (app));
```

You need to cast `win` to GtkWindow and `app` to GtkApplication with `GTK_WINDOW` and `GTK_APPLICATION` macro.

GtkApplication continues to run until the related window is destroyed. If you didn't connect GtkWindow and GtkApplication, GtkApplication destroys itself immediately. Because no window is connected to GtkApplication, GtkApplication doesn't need to wait anything. As it destroys itself, the GtkWindow is also destroyed.

2.1.2.1.3 Show the Window. The function `gtk_window_present` presents the window to a user (shows it to the user).

GTK 4 changes the default widget visibility to on, so every widget doesn't need to change it to on. But, there's an exception. Top level window (this term will be explained later) isn't visible when it is created. So you need to use the function above to show the window.

You can use `gtk_widget_set_visible (win, true)` instead of `gtk_window_present`. But the behavior of these two is different. Suppose there are two windows win1 and win2 on the screen and win1 is behind win2. Both windows are visible. The function `gtk_widget_set_visible (win1, true)` does nothing because win1 is already visible. So, win1 is still behind win2. The other function `gtk_window_present (win1)` moves win1 to the top of the stack of the windows. Therefore, if you want to present the window, you should use `gtk_window_present`.

Two functions `gtk_widget_show` and `gtk_widget_hide` are deprecated since GTK 4.10. You should use `gtk_widget_set_visible` instead.

Save the program as `pr3.c`, then compile and run it.

```
$ bash comp pr3
$ ./a.out
```

A small window appears.

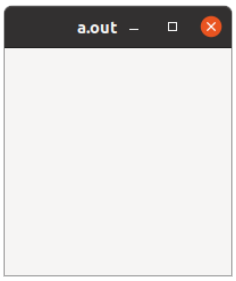


Figure 2.2: Screenshot of the window

Click on the close button then the window disappears and the program finishes.

2.1.2.2 GtkApplicationWindow

GtkApplicationWindow is a child object of GtkWindow. It has some extra feature for better integration with GtkApplication. It is recommended to use it as the top-level window of the application instead of GtkWindow.

Now rewrite the program and use GtkApplicationWindow.

```
1 static void
2 app_activate (GApplication *app, gpointer user_data) {
3   GtkWidget *win;
4
5   win = gtk_application_window_new (GTK_APPLICATION (app));
6   gtk_window_set_title (GTK_WINDOW (win), "pr4");
7   gtk_window_set_default_size (GTK_WINDOW (win), 400, 300);
8   gtk_window_present (GTK_WINDOW (win));
9 }
```

When you create GtkApplicationWindow, you need to give GtkApplication instance as an argument. Then it automatically connect these two instances. So you don't need to call `gtk_window_set_application` any more.

The program sets the title and the default size of the window. Compile it and run `a.out`, then you will see a bigger window with the title "pr4".

2.2 Widgets (1)

2.2.1 GtkLabel, GtkButton and GtkBox

2.2.1.1 GtkLabel

We made a window and showed it on the screen in the previous section. Now we go on to the next topic: widgets. The simplest widget is GtkLabel. It is a widget with text in it.

```
1 #include <gtk/gtk.h>
2
3 static void
4 app_activate (GApplication *app) {
```

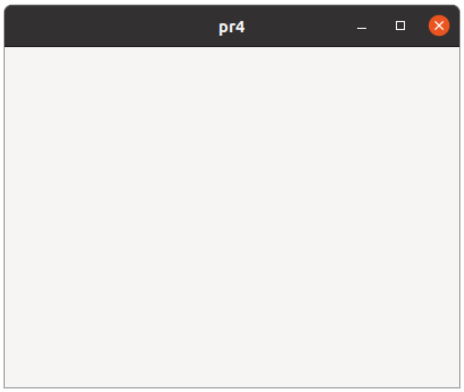


Figure 2.3: Screenshot of the window

```
5   GtkWidget *win;
6   GtkWidget *lab;
7
8   win = gtk_application_window_new (GTK_APPLICATION (app));
9   gtk_window_set_title (GTK_WINDOW (win), "lb1");
10  gtk_window_set_default_size (GTK_WINDOW (win), 400, 300);
11
12  lab = gtk_label_new ("Hello.");
13  gtk_window_set_child (GTK_WINDOW (win), lab);
14
15  gtk_window_present (GTK_WINDOW (win));
16 }
17
18 int
19 main (int argc, char **argv) {
20  GtkApplication *app;
21  int stat;
22
23  app = gtk_application_new ("com.github.ToshioCP.lb1", G_APPLICATION_DEFAULT_FLAGS);
24  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
25  stat = g_application_run (G_APPLICATION (app), argc, argv);
26  g_object_unref (app);
27  return stat;
28 }
```

Save this program to a file `lb1.c`. (You can use /src/misc/lb1.c if you've downloaded this repository.) Then compile and run it.

```
$ cd src/misc
$ bash comp lb1
$ ./a.out
```

A window with a message "Hello." appears.

There are only a few changes between `pr4.c` and `lb1.c`. A program `diff` is useful to know the difference.

```
$ cd misc; diff pr4.c lb1.c
4c4
< app_activate (GApplication *app, gpointer user_data) {
---
> app_activate (GApplication *app) {
```

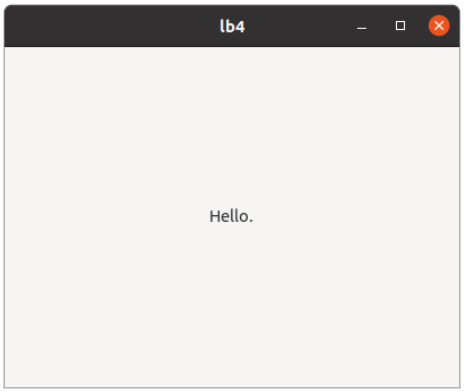


Figure 2.4: Screenshot of the label

```
5a6
>   GtkWidget *lab;
8c9
<   gtk_window_set_title (GTK_WINDOW (win), "pr4");
---
>   gtk_window_set_title (GTK_WINDOW (win), "lb1");
9a11,14
>
>   lab = gtk_label_new ("Hello.");
>   gtk_window_set_child (GTK_WINDOW (win), lab);
>
18c23
<   app = gtk_application_new ("com.github.ToshioCP.pr4", G_APPLICATION_DEFAULT_FLAGS);
---
>   app = gtk_application_new ("com.github.ToshioCP.lb1", G_APPLICATION_DEFAULT_FLAGS);
$
```

This tells us:

- A signal handler `app_activate` doesn't have `user_data` parameter. If the fourth argument of `g_signal_connect` is NULL, you can leave out `user_data`.
- The definition of a new variable `lab` is added.
- The title of the window is changed.
- A label is created and connected to the window as a child.

The function `gtk_window_set_child (GTK_WINDOW (win), lab)` makes the label `lab` a child widget of the window `win`. Be careful. A child widget is different from a child object. Objects have parent-child relationships and widgets also have parent-child relationships. But these two relationships are totally different. Don't be confused. In the program `lb1.c`, `lab` is a child widget of `win`. Child widgets are always located in their parent widget on the screen. See how the window has appeared on the screen. The application window includes the label.

The window `win` doesn't have any parents. We call such a window top-level window. An application can have more than one top-level window.

2.2.1.2 GtkButton

The next widget is GtkButton. It displays a button on the screen with a label or icon on it. In this subsection, we will make a button with a label. When the button is clicked, it emits a "clicked" signal. The following program shows how to catch the signal and do something.

```c
#include <gtk/gtk.h>

static void
click_cb (GtkButton *btn) {
  g_print ("Clicked.\n");
}

static void
app_activate (GApplication *app) {
  GtkWidget *win;
  GtkWidget *btn;

  win = gtk_application_window_new (GTK_APPLICATION (app));
  gtk_window_set_title (GTK_WINDOW (win), "lb2");
  gtk_window_set_default_size (GTK_WINDOW (win), 400, 300);

  btn = gtk_button_new_with_label ("Click me");
  gtk_window_set_child (GTK_WINDOW (win), btn);
  g_signal_connect (btn, "clicked", G_CALLBACK (click_cb), NULL);

  gtk_window_present (GTK_WINDOW (win));
}

int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new ("com.github.ToshioCP.lb2", G_APPLICATION_DEFAULT_FLAGS);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
}
```

Look at the line 17 to 19. First, it creates a GtkButton instance `btn` with a label "Click me". Then, adds the button to the window `win` as a child. Finally, connects the "clicked" signal of the button to the handler `click_cb`. So, if `btn` is clicked, the function `click_cb` is invoked. The suffix "cb" means "call back".

Name the program `lb2.c` and save it. Now compile and run it.

A window with the button appears. Click the button (it is a large button, you can click everywhere in the window), then a string "Clicked." appears on the terminal. It shows the handler was invoked by clicking the button.

It's good that we've made sure that the clicked signal was caught and the handler was invoked by using `g_print`. However, using `g_print` doesn't fit well with GTK, which is a GUI library. So, we will change the handler. The following code is extracted from `lb3.c`.

```c
static void
click_cb (GtkButton *btn, GtkWindow *win) {
  gtk_window_destroy (win);
}

static void
```

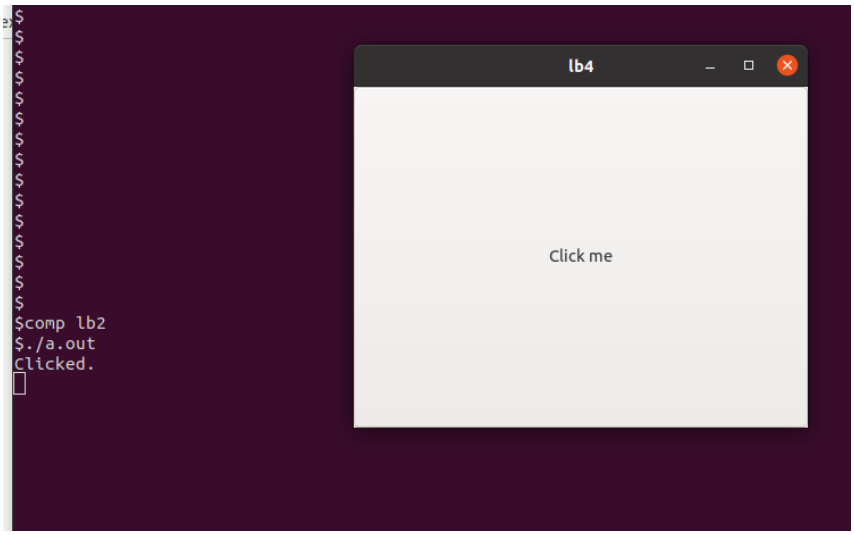


Figure 2.5: Screenshot of the label

```
7  app_activate (GApplication *app) {
8    GtkWidget *win;
9    GtkWidget *btn;
10
11   win = gtk_application_window_new (GTK_APPLICATION (app));
12   gtk_window_set_title (GTK_WINDOW (win), "lb3");
13   gtk_window_set_default_size (GTK_WINDOW (win), 400, 300);
14
15   btn = gtk_button_new_with_label ("Close");
16   gtk_window_set_child (GTK_WINDOW (win), btn);
17   g_signal_connect (btn, "clicked", G_CALLBACK (click_cb), win);
18
19   gtk_window_present (GTK_WINDOW (win));
20 }
```

And the difference between `lb2.c` and `lb3.c` is as follows.

```
$ cd misc; diff lb2.c lb3.c
4,5c4,5
< click_cb (GtkButton *btn) {
<   g_print ("Clicked.\n");
---
> click_cb (GtkButton *btn, GtkWindow *win) {
>   gtk_window_destroy (win);
14c14
<   gtk_window_set_title (GTK_WINDOW (win), "lb2");
---
>   gtk_window_set_title (GTK_WINDOW (win), "lb3");
17c17
<   btn = gtk_button_new_with_label ("Click me");
---
>   btn = gtk_button_new_with_label ("Close");
19c19
<   g_signal_connect (btn, "clicked", G_CALLBACK (click_cb), NULL);
---
>   g_signal_connect (btn, "clicked", G_CALLBACK (click_cb), win);
```

```
29c29
<   app = gtk_application_new ("com.github.ToshioCP.lb2", G_APPLICATION_DEFAULT_FLAGS);
---
>   app = gtk_application_new ("com.github.ToshioCP.lb3", G_APPLICATION_DEFAULT_FLAGS);
35d34
<
$
```

The changes are:

- The function `g_print` in `lb2.c` was deleted and two lines are inserted.
  - `click_cb` has the second parameter, which comes from the fourth argument of the `g_signal_connect` at line 17. One thing to be careful is the types are different between the second parameter of `click_cb` and the fourth argument of `g_signal_connect`. The former is `GtkWindow *` and the latter is `GtkWidget *`. The compiler doesn't complain because `g_signal_connect` uses gpointer (general type of pointer). In this program the instance pointed to by `win` is a GtkApplicationWindow object. It is a descendant of GtkWindow and GtkWidget class, so both `GtkWindow *` and `GtkWidget *` are correct types of the instance.
  - `gtk_window_destroy (win)` destroys the top-level window. Then the application quits.
- The label of `btn` is changed from "Click me" to "Close".
- The fourth argument of `g_signal_connect` is changed from `NULL` to `win`.

The most important change is the fourth argument of the `g_signal_connect`. This argument is described as "data to pass to the handler" in the definition of g_signal_connect.

2.2.1.3 GtkBox

GtkWindow and GtkApplicationWindow can have only one child. If you want to add two or more widgets in a window, you need a container widget. GtkBox is one of the containers. It arranges two or more child widgets into a single row or column. The following procedure shows the way to add two buttons in a window.

- Create a GtkApplicationWindow instance.
- Create a GtkBox instance and add it to the GtkApplicationWindow as a child.
- Create a GtkButton instance and append it to the GtkBox.
- Create another GtkButton instance and append it to the GtkBox.

After this, the widgets are connected as shown in the following diagram.

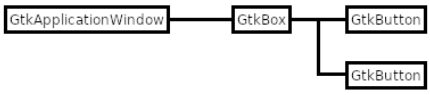


Figure 2.6: Parent-child relationship

The program `lb4.c` is as follows.

```
1  #include <gtk/gtk.h>
2
3  static void
4  click1_cb (GtkButton *btn) {
5    const char *s;
6
7    s = gtk_button_get_label (btn);
8    if (g_strcmp0 (s, "Hello.") == 0)
9      gtk_button_set_label (btn, "Good-bye.");
```

```
10    else
11      gtk_button_set_label (btn, "Hello.");
12  }
13
14  static void
15  click2_cb (GtkButton *btn, GtkWindow *win) {
16    gtk_window_destroy (win);
17  }
18
19  static void
20  app_activate (GApplication *app) {
21    GtkWidget *win;
22    GtkWidget *box;
23    GtkWidget *btn1;
24    GtkWidget *btn2;
25
26    win = gtk_application_window_new (GTK_APPLICATION (app));
27    gtk_window_set_title (GTK_WINDOW (win), "lb4");
28    gtk_window_set_default_size (GTK_WINDOW (win), 400, 300);
29
30    box = gtk_box_new (GTK_ORIENTATION_VERTICAL, 5);
31    gtk_box_set_homogeneous (GTK_BOX (box), TRUE);
32    gtk_window_set_child (GTK_WINDOW (win), box);
33
34    btn1 = gtk_button_new_with_label ("Hello.");
35    g_signal_connect (btn1, "clicked", G_CALLBACK (click1_cb), NULL);
36
37    btn2 = gtk_button_new_with_label ("Close");
38    g_signal_connect (btn2, "clicked", G_CALLBACK (click2_cb), win);
39
40    gtk_box_append (GTK_BOX (box), btn1);
41    gtk_box_append (GTK_BOX (box), btn2);
42
43    gtk_window_present (GTK_WINDOW (win));
44  }
45
46  int
47  main (int argc, char **argv) {
48    GtkApplication *app;
49    int stat;
50
51    app = gtk_application_new ("com.github.ToshioCP.lb4", G_APPLICATION_DEFAULT_FLAGS);
52    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
53    stat = g_application_run (G_APPLICATION (app), argc, argv);
54    g_object_unref (app);
55    return stat;
56  }
```

Look at the function `app_activate`.

After the creation of a GtkApplicationWindow instance, a GtkBox instance is created.

```
box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 5);
gtk_box_set_homogeneous (GTK_BOX (box), TRUE);
```

The first argument arranges the children of the box vertically. The orientation constants are defined like this:

- GTK_ORIENTATION_VERTICAL: the children widgets are arranged vertically
- GTK_ORIENTATION_HORIZONTAL: the children widgets are arranged horizontally

The second argument is the size of the space between the children. The unit of the length is pixel.

The next function `gtk_box_set_homogeneous` fills the box with the children, giving them the same space.

After that, two buttons `btn1` and `btn2` are created and the signal handlers are set. Then, these two buttons are appended to the box.

```
1  static void
2  click1_cb (GtkButton *btn) {
3    const char *s;
4
5    s = gtk_button_get_label (btn);
6    if (g_strcmp0 (s, "Hello.") == 0)
7      gtk_button_set_label (btn, "Good-bye.");
8    else
9      gtk_button_set_label (btn, "Hello.");
10 }
```

The function `gtk_button_get_label` returns the text from the label. The string is owned by the button and you can't modify or free it. The `const` qualifier is necessary for the string `s`. If you change the string, your compiler will give you a warning.

You always need to be careful with the const qualifier when you see the GTK 4 API reference.

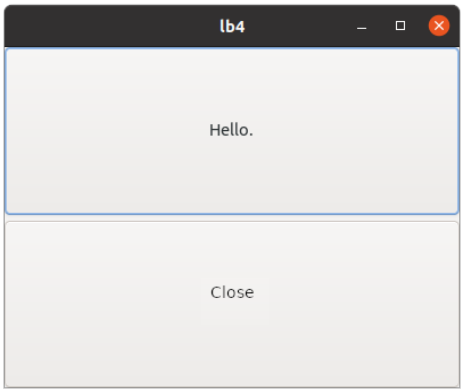


Figure 2.7: Screenshot of the box

The handler corresponding to `btn1` toggles its label. The handler corresponding to `btn2` destroys the top-level window and the application quits.

2.3 Widgets (2)

2.3.1 GtkTextView, GtkTextBuffer and GtkScrolledWindow

2.3.1.1 GtkTextView and GtkTextBuffer

GtkTextView is a widget for multi-line text editing. GtkTextBuffer is a text buffer which is connected to GtkTextView. See the sample program `tfv1.c` below.

```
1  #include <gtk/gtk.h>
2
```

```
3  static void
4  app_activate (GApplication *app) {
5    GtkWidget *win;
6    GtkWidget *tv;
7    GtkTextBuffer *tb;
8    gchar *text;
9
10   text =
11       "Once upon a time, there lived an old man known as Taketori-no-Okina. "
12       "His name, meaning \"The Bamboo Cutter,\" reflected his life's work of crafting bamboo baskets
13       "One day, while deep in the hills, he came across a bamboo stalk that shimmered with a mysteri
14       "\"How strange and wondrous!\" he exclaimed. "
15       "When he carefully cut it open, he found a tiny, beautiful baby girl nestled inside, glowing w
16       "Believing she was a gift from the heavens, he brought her home.\n"
17       "Though his wife was astonished by his tale, they were both filled with joy, for they had neve
18       ;
19   win = gtk_application_window_new (GTK_APPLICATION (app));
20   gtk_window_set_title (GTK_WINDOW (win), "Taketori");
21   gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);
22
23   tv = gtk_text_view_new ();
24   tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
25   gtk_text_buffer_set_text (tb, text, -1);
26   gtk_text_view_set_wrap_mode (GTK_TEXT_VIEW (tv), GTK_WRAP_WORD_CHAR);
27
28   gtk_window_set_child (GTK_WINDOW (win), tv);
29
30   gtk_window_present (GTK_WINDOW (win));
31 }
32
33 int
34 main (int argc, char **argv) {
35   GtkApplication *app;
36   int stat;
37
38   app = gtk_application_new ("com.github.ToshioCP.tfv1", G_APPLICATION_DEFAULT_FLAGS);
39   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
40   stat = g_application_run (G_APPLICATION (app), argc, argv);
41   g_object_unref (app);
42   return stat;
43 }
```

Look at line 25. A GtkTextView instance is created and its pointer is assigned to `tv`. When the GtkTextView instance is created, a GtkTextBuffer instance is also created and connected to the GtkTextView automatically. “GtkTextBuffer instance” will be referred to simply as “GtkTextBuffer” or “buffer”. In the next line, the pointer to the buffer is assigned to `tb`. Then, the text from line 10 to 20 is assigned to the buffer. If the third argument of `gtk_text_buffer_set_text` is a positive integer, it is the length of the text. If it is -1, the string terminates with NULL.

GtkTextView has a wrap mode. When it is set to `GTK_WRAP_WORD_CHAR`, text wraps in between words, or if that is not enough, also between graphemes.

Wrap mode is written in Gtk_WrapMode in the GTK 4 API document.

In line 30, `tv` is added to `win` as a child.

Now compile and run it. If you've downloaded this repository, its pathname is `src/tfv/tfv1.c`.

```
$ cd src/tfv
$ bash comp tfv1
$ ./a.out
```

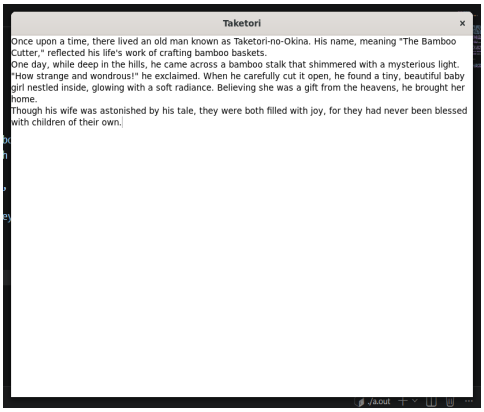


Figure 2.8: GtkTextView

There's an I-beam pointer in the window. You can add or delete any characters on the GtkTextView, and your changes are kept in the GtkTextBuffer. If you add more characters beyond the limit of the window, the height increases and the window extends. If the height gets bigger than the height of the screen, you won't be able to control the size of the window or change it back to the original size. This is essentially a bug. This can be solved by adding a GtkScrolledWindow between the GtkApplicationWindow and GtkTextView.

2.3.1.2 GtkScrolledWindow

What we need to do is:

- Create a GtkScrolledWindow and insert it as a child of the GtkApplicationWindow
- Insert the GtkTextView widget to the GtkScrolledWindow as a child.

Modify `tfv1.c` and save it as `tfv2.c`. There are only a few differences between these two files.

```
$ cd tfv; diff tfv1.c tfv2.c
5a6
>   GtkWidget *scr;
22a24,26
>   scr = gtk_scrolled_window_new ();
>   gtk_window_set_child (GTK_WINDOW (win), scr);
>
28c32
<   gtk_window_set_child (GTK_WINDOW (win), tv);
---
>   gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), tv);
38c42
<   app = gtk_application_new ("com.github.ToshioCP.tfv1", G_APPLICATION_DEFAULT_FLAGS);
---
>   app = gtk_application_new ("com.github.ToshioCP.tfv2", G_APPLICATION_DEFAULT_FLAGS);
$
```

The whole code of `tfv2.c` is as follows.

```
1  #include <gtk/gtk.h>
2
```

```
static void
app_activate (GApplication *app) {
  GtkWidget *win;
  GtkWidget *scr;
  GtkWidget *tv;
  GtkTextBuffer *tb;
  gchar *text;

  text =
      "Once upon a time, there lived an old man known as Taketori-no-Okina. "
      "His name, meaning \"The Bamboo Cutter,\" reflected his life's work of crafting bamboo baskets
      "One day, while deep in the hills, he came across a bamboo stalk that shimmered with a mysteri
      "\"How strange and wondrous!\" he exclaimed. "
      "When he carefully cut it open, he found a tiny, beautiful baby girl nestled inside, glowing w
      "Believing she was a gift from the heavens, he brought her home.\n"
      "Though his wife was astonished by his tale, they were both filled with joy, for they had neve
      ;
  win = gtk_application_window_new (GTK_APPLICATION (app));
  gtk_window_set_title (GTK_WINDOW (win), "Taketori");
  gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);

  scr = gtk_scrolled_window_new ();
  gtk_window_set_child (GTK_WINDOW (win), scr);

  tv = gtk_text_view_new ();
  tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
  gtk_text_buffer_set_text (tb, text, -1);
  gtk_text_view_set_wrap_mode (GTK_TEXT_VIEW (tv), GTK_WRAP_WORD_CHAR);

  gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), tv);

  gtk_window_present (GTK_WINDOW (win));
}

int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new ("com.github.ToshioCP.tfv2", G_APPLICATION_DEFAULT_FLAGS);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
}
```

Compile and run it.

Now, the window doesn't extend even if you type a lot of characters, it just scrolls.

2.4 Strings and Memory Management

GtkTextView and GtkTextBuffer have functions that have string parameters or return a string. The knowledge of strings and memory management is useful to understand how to use these functions.

2.4.1 Strings and Memory

A String is an array of characters that is terminated with '\0'. C has no built-in string type. Strings are represented using character arrays (or pointers to characters). It behaves like a string in other languages. So, in practice, a pointer to the first element of such a character array is often loosely called a "string".

The following is a sample program.

```
char a[10], *b;

a[0] = 'H';
a[1] = 'e';
a[2] = 'l';
a[3] = 'l';
a[4] = 'o';
a[5] = '\0';

b = a;
/* *b is 'H' */
/* *(++b) is 'e' */
```

An array `a` is defined as a `char` type array and its size is ten. The first five elements are 'H', 'e', 'l', 'l', 'o'. They are character codes. For example, 'H' is the same as 0x48 or 72. The sixth element is '\0', which is the same as zero, and indicates that the sequence of the data ends there. The array represents the string "Hello".

The size of the array is 10, so four bytes aren't used. But it's OK. They are just ignored. (If the variable `a` is defined out of functions or its storage class is static, the four bytes are assigned with zero. Otherwise, that is to say, the storage class is auto, they are undefined.)

The variable `b` is a pointer to a character. It is assigned to `a`, so `b` points to the first element of `a` (character 'H'). The array `a` itself cannot be assigned to. So `a=a+1` causes syntax error.

On the other hand, `b` is a pointer type variable, which is mutable. So, `++b`, which increases `b` by one, is allowed.

If a pointer is NULL, it points to nothing. So, the pointer is not a string. It is different from empty strings. An empty string is a pointer to a memory location where the first character is '\0'

There are four cases:

- The string is read only
- The string is in static memory area
- The string is in stack
- The string is in memory allocated from the heap area

2.4.2 Read Only Strings

A string literal is enclosed in double quotes like this:

```
char *s;
s = "Hello"
```

"Hello" is a string literal, and is read only. So, the following program is illegal.

```
*(s+1) = 'a';
```

The result is undefined. Probably something bad will happen, such as a segmentation fault.

NOTE: The memory of the literal string is allocated when the program is compiled. Literal strings can be seen using the `strings` command.

```
$ strings src/tvf/a.out
/lib64/ld-linux-x86-64.so.2
cN<5
... ... ...
... ... ...
Once upon a time, there was an old man who was called Taketori-no-Okina. It is a japanese word that r
One day, he went into a hill and found a shining bamboo. "What a mysterious bamboo it is!," he said.
His wife was surprized at his story. They were very happy because they had no children.
... ... ...
... ... ...
```

This shows that literal strings are embedded in program binary codes.

2.4.3 Strings Defined as Arrays

If a string is defined as an array, it's stored in static memory area or stack. It depends on the storage class of the array. If the array's storage class is `static`, then it's placed in static memory area. The allocated memory lives for the life of the program. This area is writable.

If the array's storage class is `auto`, it's placed in stack. If the array is defined inside a function, its default storage class is `auto`. The stack area will disappear when the function returns to the caller. Arrays defined on the stack are writable.

```
static char a[] = {'H', 'e', 'l', 'l', 'o', '\0'};

void
print_strings (void) {
  char b[] = "Hello";

  a[1] = 'a'; /* Writable because 'a' is a character array (in static storage). */
  b[1] = 'a'; /* Writable because 'b' is a character array (on the stack). */

  printf ("%s\n", a); /* Hallo */
  printf ("%s\n", b); /* Hallo */
}
```

The array `a` is defined out of functions. It is placed in the static memory area even if the `static` is left out. The compiler calculates the number of the elements (six) and allocates six bytes in the static memory area. Then, it copies "Hello" literal string data to the memory.

The array `b` is defined inside the function, so its storage class is `auto`. The compiler calculates the number of the elements in the string literal. It is six, including the null terminator. The compiler allocates six bytes in the stack and copies "Hello" litaral string to the stack memory.

Both `a` and `b` are writable.

The memory is allocated and freed by the program automatically so you don't need to allocate or free. The array `a` is alive during the program's life time. The array `b` is alive when the function is called until the function returns to the caller.

2.4.4 Strings in the Heap Area

You can get, use and release memory from the heap area. The standard C library provides `malloc` to get memory and `free` to release memory. GLib provides the functions `g_new` and `g_free`. They are similar to `malloc` and `free`.

`g_new (struct_type, n_struct)`

`g_new` is a macro to allocate memory for an array.

- `struct_type` is the type of the element of the array.
- `n_struct` is the size of the array.
- The return value is a pointer to the array. Its type is a pointer to `struct_type`.

For example,

```
char *s;
s = g_new (char, 10);
/* s points to an array of char. The size of the array is 10. */

struct tuple {int x, y;} *t;
t = g_new (struct tuple, 5);
/* t points to an array of struct tuple. */
/* The size of the array is 5. */
```

`g_free` frees memory.

```
void
g_free (gpointer mem);
```

If `mem` is NULL, `g_free` does nothing. `gpointer` is a type of general pointer. It is the same as `void *`. This pointer can be cast to any pointer type. Conversely, any pointer type can be cast to `gpointer`.

```
g_free (s);
/* Frees the memory allocated to s. */

g_free (t);
/* Frees the memory allocated to t. */
```

If the argument does not point to allocated memory, the behavior is undefined and may result in a segmentation fault.

Some GLib functions allocate memory. For example, `g_strdup` allocates memory and copies a string given as an argument.

```
char *s;
s = g_strdup ("Hello");
g_free (s);
```

The string literal “Hello” has 6 bytes because the string has ‘\0’ at the end. `g_strdup` gets 6 bytes from the heap area and copies the string to the memory. `s` is assigned the start address of the memory. `g_free` returns the memory to the heap area.

`g_strdup` is described in GLib API Reference. The following is extracted from the reference.

> The returned string should be freed with `g_free()` when no longer needed.

If you forget to free the allocated memory it will remain until the program ends. Repeated allocation and no freeing cause memory leak. It is a bug and may bring a serious problem.

2.4.5 Const Qualifier

A const-qualified variable must be initialized at definition time, and its value cannot be modified through that variable. However, this does not prevent the memory it refers to from being freed.

```
const int x = 10; /* initialization is OK. */

x = 20; /* This is illegal because x is qualified with const */
```

If a function returns `const char *` type, the string can’t be changed or freed. If a function has a `const char *` type parameter, it ensures that the parameter is not changed in the function.

```
// You never change or free the returned string.
const char *
gtk_label_get_text (
  GtkLabel *self
)

// The string 'str' remains valid while the function is executing.
void
gtk_label_set_text (
  GtkLabel *self,
  const char *str
)
```

2.5 Widgets (3)

2.5.1 Open Signal

2.5.1.1 G_APPLICATION_HANDLES_OPEN flag

We made a very simple editor in the previous section with GtkTextView, GtkTextBuffer and GtkScrolledWindow. We will add file-read ability to the program and improve it to a file viewer.

The simplest way to specify a filename is to use a command line argument.

```
$ ./a.out filename
```

The program will open the file and insert its contents into a GtkTextBuffer.

To do this, we need to know how GtkApplication (or GApplication) recognizes arguments. This is described in the GIO API Reference – Application.

When GtkApplication is created, a flag (GApplicationFlags) is given as an argument.

```
GtkApplication *
gtk_application_new (const gchar *application_id, GApplicationFlags flags);
```

This tutorial explains only two flags, `G_APPLICATION_DEFAULT_FLAGS` and `G_APPLICATION_HANDLES_OPEN`.

`G_APPLICATION_FLAGS_NONE` was used instead of `G_APPLICATION_DEFAULT_FLAGS` before GIO version2.73.3 (GLib 2.73.3 5/Aug/2022). Now it is deprecated and `G_APPLICATION_DEFAULT_FLAGS` is recommended.

For further information, see GIO API Reference – ApplicationFlags and GIO API Reference – g_application_run.

We've already used `G_APPLICATION_DEFAULT_FLAGS`, as it is the simplest option, and no command line arguments are allowed. If you give arguments in the command line, an error will occur.

The flag `G_APPLICATION_HANDLES_OPEN` is the second simplest option. It allows command line arguments, specifically filenames.

```
app = gtk_application_new ("com.github.ToshioCP.tfv3", G_APPLICATION_HANDLES_OPEN);
```

2.5.1.2 Open Signal

When `G_APPLICATION_HANDLES_OPEN` flag is given to the application, two signals are available.

- activate signal: This signal is emitted when there's no argument.
- open signal: This signal is emitted when there is at least one argument.

The handler of the "open" signal is defined as follows.

```
void
open (
  GApplication* self,
  gpointer files,
  gint n_files,
  gchar* hint,
  gpointer user_data
)
```

The parameters are:

- self: the application instance (usually GtkApplication)
- files: an array of GFiles. [array length=n_files] [element-type GFile]
- n_files: the number of the elements of `files`
- hint: an optional context string provided by the caller (often unused)
- user_data: user data that is set when the signal handler was connected.

2.5.2 File Viewer

2.5.2.1 What is a File Viewer?

A file viewer is a program that displays text files. Our file viewer is as follows.

- When arguments are given, it recognizes the first argument as a filename and opens it.
- The second argument and later are ignored.
- If there is no argument, it shows an error message and quits.
- If it successfully opens the file, it reads the content of the file, inserts it to GtkTextBuffer and shows the window.
- If it fails to open the file, it shows an error message and quit.

The program is shown below.

```
1  #include <gtk/gtk.h>
2
3  static void
4  app_activate (GApplication *app) {
5    g_printerr ("You need a filename argument.\n");
6  }
7
8  static void
9  app_open (GApplication *app, GFile ** files, int n_files, char *hint) {
10   GtkWidget *win;
11   GtkWidget *scr;
12   GtkWidget *tv;
13   GtkTextBuffer *tb;
14   char *contents;
15   gsize length;
16   char *filename_bin, *filename_utf8;
17   GError *err = NULL;
18
19   win = gtk_application_window_new (GTK_APPLICATION (app));
20   gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);
21
22   scr = gtk_scrolled_window_new ();
23   gtk_window_set_child (GTK_WINDOW (win), scr);
24
25   tv = gtk_text_view_new ();
```

```
26    tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
27    gtk_text_view_set_wrap_mode (GTK_TEXT_VIEW (tv), GTK_WRAP_WORD_CHAR);
28    gtk_text_view_set_editable (GTK_TEXT_VIEW (tv), FALSE);
29    gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), tv);
30
31    if (g_file_load_contents (files[0], NULL, &contents, &length, NULL, &err)) {
32      gtk_text_buffer_set_text (tb, contents, length);
33      g_free (contents);
34      if ((filename_bin = g_file_get_basename (files[0])) != NULL) {
35        filename_utf8 = g_filename_display_name (filename_bin);
36        gtk_window_set_title (GTK_WINDOW (win), filename_utf8);
37        g_free (filename_bin);
38        g_free (filename_utf8);
39      }
40      gtk_window_present (GTK_WINDOW (win));
41    } else {
42      g_printerr ("%s.\n", err->message);
43      g_error_free (err);
44      gtk_window_destroy (GTK_WINDOW (win));
45    }
46  }
47
48  int
49  main (int argc, char **argv) {
50    GtkApplication *app;
51    int stat;
52
53    app = gtk_application_new ("com.github.ToshioCP.tfv3", G_APPLICATION_HANDLES_OPEN);
54    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
55    g_signal_connect (app, "open", G_CALLBACK (app_open), NULL);
56    stat = g_application_run (G_APPLICATION (app), argc, argv);
57    g_object_unref (app);
58    return stat;
59  }
```

The source code is located at /src/tfv/tfv3.c in the repository. Compile and run it.

```
$ bash comp tfv3
$ ./a.out tfv3.c
```

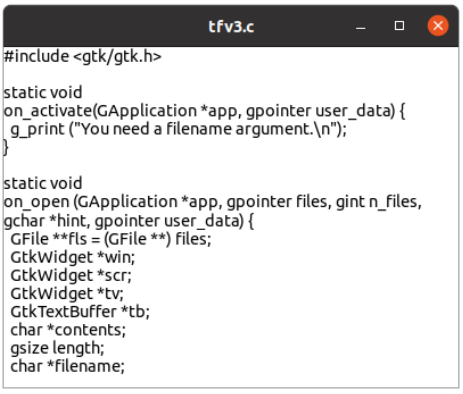


Figure 2.9: File viewer

The function `main` has only two changes from the previous version.

- `G_APPLICATION_DEFAULT_FLAGS` is replaced by `G_APPLICATION_HANDLES_OPEN`
- `g_signal_connect (app, "open", G_CALLBACK (app_open), NULL)` is added.

When the flag `G_APPLICATION_HANDLES_OPEN` is given to `gtk_application_new` function, the application behaves like this:

- If the application is run without command line arguments, it emits "activate" signal when it is activated.
- If the application is run with command line arguments, it emits "open" signal when it is activated.

The handler `app_activate` becomes very simple. It just outputs a message and returns to the caller. Then the application quits immediately because no window is created.

The main work is done in the handler `app_open`.

- Creates GtkApplicationWindow, GtkScrolledWindow, GtkTextView and GtkTextBuffer and connects them together
- Sets wrap mode to `GTK_WRAP_WORD_CHAR` in GtktextView
- Sets GtkTextView to non-editable because the program isn't an editor but only a viewer
- Reads the file and inserts the text into GtkTextBuffer (this will be explained later). The window title is set with the filename. In general, filenames are sequences of bytes. To display them on a screen, they must be converted into encoded strings. The function `g_filename_display_name` converts a filename into a valid UTF-8 string.
- If the file is not opened, outputs an error message and destroys the window. This causes the application to exit.

The following is the file reading part of the program.

```
if (g_file_load_contents (files[0], NULL, &contents, &length, NULL, &err)) {
  gtk_text_buffer_set_text (tb, contents, length);
  g_free (contents);
  if ((filename_bin = g_file_get_basename (files[0])) != NULL) {
    filename_utf8 = g_filename_display_name (filename_bin);
    gtk_window_set_title (GTK_WINDOW (win), filename_utf8);
    g_free (filename_bin);
    g_free (filename_utf8);
  }
  gtk_window_present (GTK_WINDOW (win));
} else {
  g_printerr ("%s.\n", err->message);
  g_error_free (err);
  gtk_window_destroy (GTK_WINDOW (win));
}
```

The function `g_file_load_contents` loads the file contents into a temporary buffer, allocated automatically, and sets `contents` to point to the buffer. The length of the buffer is assigned to `length`. It returns `TRUE` if the file's contents are successfully loaded. The caller takes the ownership of the buffer and is responsible for freeing it. If an error occurs, it returns `FALSE` and sets the variable `err` to point to a newly created GError structure. The caller takes the ownership of the GError structure and is responsible for freeing it. If you want to know the details about g_file_load_contents, see g file load contents.

If the file is read successfully, the contents are inserted into GtkTextBuffer. The temporary buffer `contents` is then freed. The function `g_file_get_basename` returns the basename of the pathname. Generally, a pathname of a filename is a sequence of byte data. To display the filename, it must be converted to a valid encoded string. The function `g_filename_display_name` converts a filename into a valid UTF-8 string. After the conversion, the window title is set with the filename, the memories for filenames are

freed, and the window is shown.

If it fails, `g_file_load_contents` sets `err` to point to a newly created GError structure. The structure is:

```
struct GError {
  GQuark domain;
  int code;
  char* message;
}
```

The `message` member is the most commonly used. It points to an error message. A function `g_error_free` is used to free the memory of the structure. See GError.

The program above outputs an error message, frees `err` and destroys the window and finally make the program quit.

2.5.3 GtkNotebook

GtkNotebook is a container widget that contains multiple widgets with tabs. It shows only one child at a time. Another child will be shown when its tab is clicked.

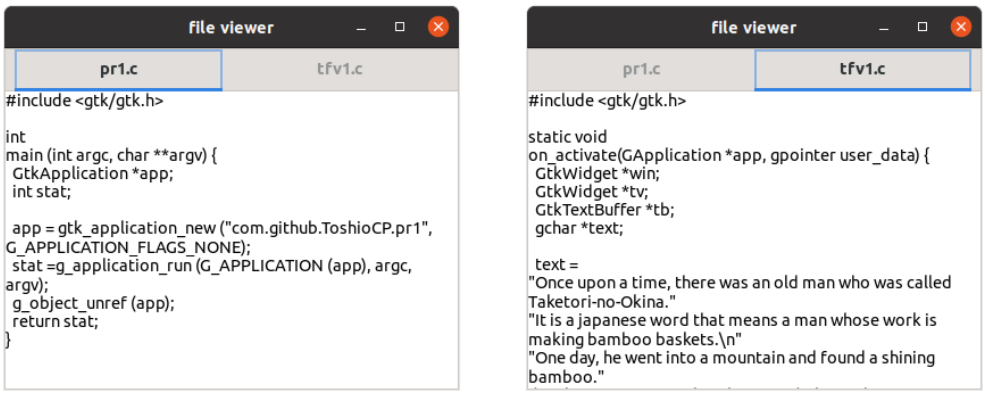


Figure 2.10: GtkNotebook

The left image is the window at the startup. The file `pr1.c` is shown and its filename is in the left tab. After clicking on the right tab, the contents of the file `tfv1.c` is shown (the right image).

The next source code is `tfv4.c`, including a GtkNoteBook widget. The widget is inserted as a child of a GtkApplicationWindow and contains multiple GtkScrolledWindow widgets.

```
1  #include <gtk/gtk.h>
2
3  static void
4  app_activate (GApplication *app) {
5    g_printerr ("You need filename arguments.\n");
6  }
7
8  static void
9  app_open (GApplication *app, GFile ** files, gint n_files, gchar *hint) {
10   GtkWidget *win;
11   GtkWidget *nb;
12   GtkWidget *lab;
13   GtkNotebookPage *nbp;
14   GtkWidget *scr;
15   GtkWidget *tv;
```

```
16    GtkTextBuffer *tb;
17    char *contents;
18    gsize length;
19    char *filename_bin, *filename_utf8;
20    int i;
21    GError *err = NULL;
22
23    win = gtk_application_window_new (GTK_APPLICATION (app));
24    gtk_window_set_title (GTK_WINDOW (win), "file viewer");
25    gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);
26    nb = gtk_notebook_new ();
27    gtk_window_set_child (GTK_WINDOW (win), nb);
28
29    for (i = 0; i < n_files; i++) {
30      if (g_file_load_contents (files[i], NULL, &contents, &length, NULL, &err)) {
31        scr = gtk_scrolled_window_new ();
32        tv = gtk_text_view_new ();
33        tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
34        gtk_text_view_set_wrap_mode (GTK_TEXT_VIEW (tv), GTK_WRAP_WORD_CHAR);
35        gtk_text_view_set_editable (GTK_TEXT_VIEW (tv), FALSE);
36        gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), tv);
37
38        gtk_text_buffer_set_text (tb, contents, length);
39        g_free (contents);
40        filename_bin = g_file_get_basename (files[i]);
41        filename_utf8 = g_filename_display_name (filename_bin);
42        lab = gtk_label_new (filename_utf8);
43        g_free (filename_bin);
44        g_free (filename_utf8);
45        gtk_notebook_append_page (GTK_NOTEBOOK (nb), scr, lab);
46        nbp = gtk_notebook_get_page (GTK_NOTEBOOK (nb), scr);
47        g_object_set (nbp, "tab-expand", TRUE, NULL);
48      } else {
49        g_printerr ("%s.\n", err->message);
50        g_clear_error (&err);
51      }
52    }
53    if (gtk_notebook_get_n_pages (GTK_NOTEBOOK (nb)) > 0)
54      gtk_window_present (GTK_WINDOW (win));
55    else
56      gtk_window_destroy (GTK_WINDOW (win));
57  }
58
59  int
60  main (int argc, char **argv) {
61    GtkApplication *app;
62    int stat;
63
64    app = gtk_application_new ("com.github.ToshioCP.tfv4", G_APPLICATION_HANDLES_OPEN);
65    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
66    g_signal_connect (app, "open", G_CALLBACK (app_open), NULL);
67    stat = g_application_run (G_APPLICATION (app), argc, argv);
68    g_object_unref (app);
```

```
69    return stat;
70  }
```

The changes are primarily located in the `app_open` function. The numbers on the left below correspond to the line numbers in the source code.

- 11-13: Variables `nb`, `lab` and `nbp` are defined. They point to a GtkNotebook, GtkLabel and GtkNotebookPage respectively.
- 24: The window's title is set to "file viewer".
- 25: The default size of the window is 800x600.
- 26-27: GtkNotebook is created and set as a child of the GtkApplicationWindow.
- 29-52: For-loop. The variable `files[i]` points to i-th GFile, which is created by the GtkApplication from the i-th command line argument.
- 31-36: A GtkScrollledWindow and a GtkTextView are created. A GtkTextBuffer is obtained from the GtkTextView. The GtkTextView is connected to the GtkScrolledWindow as a child.
- 38-39: inserts the contents of the file into the GtkTextBuffer and frees the memory referenced by `contents`.
- 40-44: Retrieves the filename from the GFile using `g_file_get_basename`. The filename is a byte-sequence without a specific encoding. It needs to be converted into a UTF-8 string with `g_filename_display_name` before setting the label text. The strings `filename_bin` and `filename_utf8` are freed.
- 45-46: Appends a GtkScrolledWindow to the GtkNotebook as a child and the GtkLabel is set as the child's tab. At the same time, a GtkNoteBookPage is created automatically. The function `gtk_notebook_get_page` returns the GtkNotebookPage of the child (GtkScrolledWindow).
- 47: GtkNotebookPage has "tab-expand" property. If it is set to TRUE then the tab expands horizontally as long as possible. If it is FALSE, then the width of the tab is determined by the size of the label. `g_object_set` is a general function to set properties of objects. See GObject API Reference – g_object_set.
- 48-50: If it fails to read the file, the error message is shown. The function `g_clear_error (&err)` works like `g_error_free (err); err = NULL;`.
- 53-56: If at least one page exists, the window is shown. Otherwise, the window is destroyed and the application quits.

2.6 Defining a Final Class

2.6.1 A Very Simple Editor

We made a very simple file viewer in the previous section. Now we go on to rewrite it and turn it into a very simple editor. Its source file is `tfe1.c` (text file editor 1) under tfe directory.

GtkTextView is a multi-line editor. So, we don't need to write the editor from scratch. We just add two things to the file viewer:

- Pointers to GFile instances.
- A text-save function.

There are a couple of ways to store the pointers.

- Use global variables
- Make a child class of GtkTextView and its each instance holds a pointer to the GFile instance.

Using global variables is easy to implement. Define a sufficient size pointer array to GFile. For example,

```
GFile *f[20];
```

The variable `f[i]` corresponds to the file associated with the i-th GtkNotebookPage.

However, there are two problems. First, the array has a fixed size. If a user provides too many arguments

(more than 20 in the example above), it becomes impossible to store all the GFile instances. Second, global variables make maintenance difficult. Our program is small so far, but as a program grows larger, maintaining global variables becomes increasingly challenging. When you need to check a global variable, you must examine every part of the code that references it.

Making a child class is a good idea in terms of maintenance. We prefer this approach over using global variables.

Note that we are discussing "child classes," not "child widgets." These two concepts are entirely different: a "class" is a term within the GObject system, whereas a "widget" refers to the UI hierarchy. If you are not familiar with GObject, please see:

- GObject API reference
- GObject tutorial for beginners

A child class inherits everything from the parent and, in addition, extends its performance. We will define TfeTextView as a child class of GtkTextView. It has everything that GtkTextView has and adds a pointer to a GFile.

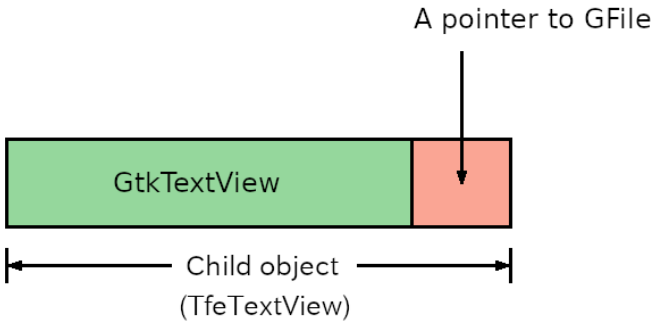


Figure 2.11: Child object of GtkTextView

2.6.2 How to Define a Child Class of GtkTextView

You need to know GObject system convention. First, look at the program below.

```
#define TFE_TYPE_TEXT_VIEW tfe_text_view_get_type ()
G_DECLARE_FINAL_TYPE (TfeTextView, tfe_text_view, TFE, TEXT_VIEW, GtkTextView)

struct _TfeTextView
{
  GtkTextView parent;
  GFile *file;
};

G_DEFINE_FINAL_TYPE (TfeTextView, tfe_text_view, GTK_TYPE_TEXT_VIEW);

static void
tfe_text_view_init (TfeTextView *tv) {
  /* Do something for instance initialization. */
}

static void
tfe_text_view_class_init (TfeTextViewClass *class) {
  /* Do something for class initialization. */
```

```
}

/* --- Some functions --- */
/* Functions here are public, used mainly for other objects. */

GtkWidget *
tfe_text_view_new (void) {
  return GTK_WIDGET (g_object_new (TFE_TYPE_TEXT_VIEW, NULL));
}
```

- The name "TfeTextView" is composed of two parts: "Tfe" and "TextView." "Tfe" serves as the prefix (or namespace), while "TextView" represents the object name.
- There are three different identifier patterns. TfeTextView (camel case), tfe_text_view (this is used for functions) and TFE_TEXT_VIEW (This is used to cast an object to TfeTextView).
- First, define the `TFE_TYPE_TEXT_VIEW` macro as `tfe_text_view_get_type()`. Macro names follow the pattern (prefix)_TYPE_(object) and are written in uppercase. The replacement text is always (prefix)_(object)_get_type () and are written in lower case. This definition must be put before `G_DECLARE_FINAL_TYPE` macro.
- The arguments of `G_DECLARE_FINAL_TYPE` macro are the child class name in camel case, lower case with underscore, prefix (upper case), object (upper case with underscore) and parent class name (camel case). The following two C structures are declared in the expansion of the macro.
  - `typedef struct _TfeTextView TfeTextView`
  - `typedef struct {GtkTextViewClass parent_class; } TfeTextViewClass;`
- These declarations tell us that TfeTextView and TfeTextViewClass are C structures. "TfeTextView" has two meanings, class name and C structure name. The C structure TfeTextView is called an instance. Similarly, TfeTextViewClass is called a class.
- Declare the structure `_TfeTextView`. The underscore is necessary. The first member is the parent object (C structure). Notice this is not a pointer but the object itself. The second member and later are members of the child object. TfeTextView structure has a pointer to a GFile instance as a member.
- `G_DEFINE_FINAL_TYPE` macro. The arguments are the child object name in camel case, lower case with underscore and parent object type (prefix)_TYPE_(module). This macro is mainly used to register the new class to the type system. The type system is a base system of GObject. Every class has its own type. The types of GObject, GtkWidget and TfeTextView are `G_TYPE_OBJECT`, `GTK_TYPE_WIDGET` and `TFE_TYPE_TEXT_VIEW` respectively. For example, `TFE_TYPE_TEXT_VIEW` is a macro and it is expanded to a function `tfe_text_view_get_type()`. It returns a unique GType identifier representing the type within the GObject.
- The instance init function `tfe_text_view_init` is called when the instance is created. It is the same as a constructor in other object oriented languages.
- The class init function `tfe_text_view_class_init` is called when the class is created.
- Some public functions can be located between `tfe_text_view_class_init` and `tfe_text_view_new` functions. Public functions are open to other instances and they can be called anywhere in the program. They are the same as public method in other object oriented languages. The variable `tv` contains a pointer to the TfeTextView object (C structure). It has a member `file` and it is pointed to by `tv->file`.
- TfeTextView instance creation function is `tfe_text_view_new`. Its name is (prefix)_(object)_new. It uses `g_object_new` function to create the instance. The arguments are (prefix)_TYPE_(object), a list to initialize properties and NULL. NULL is the end mark of the property list. No property is initialized here. And the return value is cast to GtkWidget.

This program shows the overview of how to define a child class.

2.6.3 Close-request Signal

Imagine that you are using this editor. First, you run the editor with arguments. The arguments are filenames. The editor reads the files and shows the window with the text of files in it. Then you edit the text. After you finish editing, you click on the close button of the window and quit the editor. The editor updates files just before the window closes.

GtkWindow emits the "close-request" signal when the close button is clicked. We will connect the signal and the handler `before_close`. (A handler is a C function which is connected to a signal.) The function `before_close` is called when the signal "close-request" is emitted.

```
g_signal_connect (win, "close-request", G_CALLBACK (before_close), NULL);
```

The signal "close-request" is defined on the GtkWindow class and it is inherited by the descendant classes. The first argument `win` is a GtkApplicationWindow, which is the child class of the GtkWindow class. The macro `g_signal_connect` connects the "close-request" signal to the `before_close` handler. The handler needs `G_CALLBACK` cast. The `before_close` function is as follows.

```
1  static gboolean
2  before_close (GtkWindow *win, GtkWidget *nb) {
3    GtkWidget *scr;
4    GtkWidget *tv;
5    GFile *file;
6    GtkTextBuffer *tb;
7    GtkTextIter start_iter;
8    GtkTextIter end_iter;
9    char *contents;
10   unsigned int n;
11   unsigned int i;
12   GError *err = NULL;
13
14   n = gtk_notebook_get_n_pages (GTK_NOTEBOOK (nb));
15   for (i = 0; i < n; ++i) {
16     scr = gtk_notebook_get_nth_page (GTK_NOTEBOOK (nb), i);
17     tv = gtk_scrolled_window_get_child (GTK_SCROLLED_WINDOW (scr));
18     file = tfe_text_view_get_file (TFE_TEXT_VIEW (tv));
19     tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
20     gtk_text_buffer_get_bounds (tb, &start_iter, &end_iter);
21     contents = gtk_text_buffer_get_text (tb, &start_iter, &end_iter, FALSE);
22     if (file && ! g_file_replace_contents (file, contents, strlen (contents), NULL, TRUE, G_FILE_CREA
23       g_printerr ("%s.\n", err->message);
24       g_clear_error (&err);
25     }
26     g_free (contents);
27     tfe_text_view_set_file (TFE_TEXT_VIEW (tv), NULL);
28   }
29   return FALSE;
30 }
```

The numbers on the left are line numbers.

- 15: The number of note book pages is assigned to `n`.
- 16-29: For loop with regard to the index to each page.
- 17-19: `scr`, `tv` and `file` hold the pointers to the GtkScrolledWindow, TfeTextView and GFile. The function `tfe_text_view_get_file` returns the pointer to the GFile instance that `tv` holds. The GFile instance is owned by `tv`. The function will be shown later.
- 20-22: `tb` holds a pointer to the GtkTextBuffer of the TfeTextView. The function

`gtk_text_buffer_get_bounds` assigns the start and end iterators of the buffer to `start_iter` and `end_iter` respectively. An iterator represents a specific position within the buffer. Then, `gtk_text_buffer_get_text` returns the text between `start_iter` and `end_iter`, which encompasses the entire content of the buffer.

- 23-26: The text is saved to the file. If it fails, error messages are displayed. The GError instance must be freed and the pointer `err` needs to be NULL for the next run in the loop.
- 27: `contents` are freed.
- 28: GFile is no longer needed. The function `tfe_tet_view_set_file` decrements the reference count of the GFile within `tv` and replaces it with a new GFile or NULL. Reference counts will be explained in a later section (section 11). When the reference count reaches zero, the GFile instance is automatically destroyed.

2.6.4 Source code of tfe1.c

The following is the whole source code of `tfe1.c`.

```
1  #include <gtk/gtk.h>
2
3  /* Define TfeTextView Widget which is the child class of GtkTextView */
4
5  #define TFE_TYPE_TEXT_VIEW tfe_text_view_get_type ()
6  G_DECLARE_FINAL_TYPE (TfeTextView, tfe_text_view, TFE, TEXT_VIEW, GtkTextView)
7
8  struct _TfeTextView
9  {
10   GtkTextView parent;
11   GFile *file;
12 };
13
14 G_DEFINE_FINAL_TYPE (TfeTextView, tfe_text_view, GTK_TYPE_TEXT_VIEW);
15
16 static void
17 tfe_text_view_init (TfeTextView *tv) {
18   tv->file = NULL;
19 }
20
21 static void
22 tfe_text_view_class_init (TfeTextViewClass *class) {
23 }
24
25 void
26 tfe_text_view_set_file (TfeTextView *tv, GFile *f) {
27   if (tv->file) {
28     g_object_unref (tv->file);
29   }
30   if (f)
31     g_object_ref (f); /* get the ownership of the GFile */
32   tv->file = f;
33 }
34
35 GFile *
36 tfe_text_view_get_file (TfeTextView *tv) {
37   return tv->file;
38 }
```

```
39
40  GtkWidget *
41  tfe_text_view_new (void) {
42    return GTK_WIDGET (g_object_new (TFE_TYPE_TEXT_VIEW, NULL));
43  }
44
45  /* ---------- end of the definition of TfeTextView ---------- */
46
47  static gboolean
48  before_close (GtkWindow *win, GtkWidget *nb) {
49    GtkWidget *scr;
50    GtkWidget *tv;
51    GFile *file;
52    GtkTextBuffer *tb;
53    GtkTextIter start_iter;
54    GtkTextIter end_iter;
55    char *contents;
56    unsigned int n;
57    unsigned int i;
58    GError *err = NULL;
59
60    n = gtk_notebook_get_n_pages (GTK_NOTEBOOK (nb));
61    for (i = 0; i < n; ++i) {
62      scr = gtk_notebook_get_nth_page (GTK_NOTEBOOK (nb), i);
63      tv = gtk_scrolled_window_get_child (GTK_SCROLLED_WINDOW (scr));
64      file = tfe_text_view_get_file (TFE_TEXT_VIEW (tv));
65      tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
66      gtk_text_buffer_get_bounds (tb, &start_iter, &end_iter);
67      contents = gtk_text_buffer_get_text (tb, &start_iter, &end_iter, FALSE);
68      if (file && ! g_file_replace_contents (file, contents, strlen (contents), NULL, TRUE, G_FILE_CRE
69        g_printerr ("%s.\n", err->message);
70        g_clear_error (&err);
71      }
72      g_free (contents);
73      tfe_text_view_set_file (TFE_TEXT_VIEW (tv), NULL);
74    }
75    return FALSE;
76  }
77
78  static void
79  app_activate (GApplication *app) {
80    g_print ("You need to give filenames as arguments.\n");
81  }
82
83  static void
84  app_open (GApplication *app, GFile ** files, gint n_files, gchar *hint) {
85    GtkWidget *win;
86    GtkWidget *nb;
87    GtkWidget *lab;
88    GtkNotebookPage *nbp;
89    GtkWidget *scr;
90    GtkWidget *tv;
91    GtkTextBuffer *tb;
```

```
92    char *contents;
93    gsize length;
94    char *filename_bin, *filename_utf;
95    int i;
96    GError *err = NULL;
97
98    win = gtk_application_window_new (GTK_APPLICATION (app));
99    gtk_window_set_title (GTK_WINDOW (win), "file editor");
100   gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);
101
102   nb = gtk_notebook_new ();
103   gtk_window_set_child (GTK_WINDOW (win), nb);
104
105   for (i = 0; i < n_files; i++) {
106     if (g_file_load_contents (files[i], NULL, &contents, &length, NULL, &err)) {
107       scr = gtk_scrolled_window_new ();
108       tv = tfe_text_view_new ();
109       tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
110       gtk_text_view_set_wrap_mode (GTK_TEXT_VIEW (tv), GTK_WRAP_WORD_CHAR);
111       gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), tv);
112
113       tfe_text_view_set_file (TFE_TEXT_VIEW (tv), files[i]);
114       gtk_text_buffer_set_text (tb, contents, length);
115       g_free (contents);
116       filename_bin = g_file_get_basename (files[i]);
117       filename_utf = g_filename_display_name (filename_bin);
118       lab = gtk_label_new (filename_utf);
119       g_free (filename_bin);
120       g_free (filename_utf);
121       gtk_notebook_append_page (GTK_NOTEBOOK (nb), scr, lab);
122       nbp = gtk_notebook_get_page (GTK_NOTEBOOK (nb), scr);
123       g_object_set (nbp, "tab-expand", TRUE, NULL);
124     } else {
125       g_printerr ("%s.\n", err->message);
126       g_clear_error (&err);
127     }
128   }
129   if (gtk_notebook_get_n_pages (GTK_NOTEBOOK (nb)) > 0) {
130     g_signal_connect (win, "close-request", G_CALLBACK (before_close), nb);
131     gtk_window_present (GTK_WINDOW (win));
132   } else
133     gtk_window_destroy (GTK_WINDOW (win));
134 }
135
136 int
137 main (int argc, char **argv) {
138   GtkApplication *app;
139   int stat;
140
141   app = gtk_application_new ("com.github.ToshioCP.tfe1", G_APPLICATION_HANDLES_OPEN);
142   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
143   g_signal_connect (app, "open", G_CALLBACK (app_open), NULL);
144   stat = g_application_run (G_APPLICATION (app), argc, argv);
```

```
145   g_object_unref (app);
146   return stat;
147 }
```

- 25-33: The function `tfe_text_view_set_file` decreases the reference count of `tv->file` by one (`g_object_unref`). This means `tv` releases the GFile instance ownership. Then, the function replaces `tv->file` with the new GFile `f` or NULL. If it is a GFile, the function increases the reference count of `f` by one and `tv` gets the ownership of `f`. For reference count, see Section 11 for further information.
- 131: The "close-request" signal is connected to `before_close` handler. The fourth argument is called "user data" and it will be the second argument of the signal handler. So, `nb` is given to `before_close` as the second argument.

Now it's time to compile and run.

```
$ cd src/tfe
$ bash comp tfe1
$ ./a.out taketori.txt`.
```

Modify the contents and close the window. Make sure that the file is modified.

Now we got a very simple editor. It's not smart. We need more features like open, save, saveas, change font and so on. We will add them in the following sections.

2.7 GtkBuilder and UI Files

We developed a very simple editor in the previous section. While it functions by reading and writing files at the start and end of the program, there is room for improvement. It would be more user-friendly to have "New", "Open", "Save", and "Close" buttons. This section describes how to incorporate these buttons into the window. We will focus on building the widgets and omit the file I/O functions for now.

2.7.1 New, Open, Save and Close Buttons

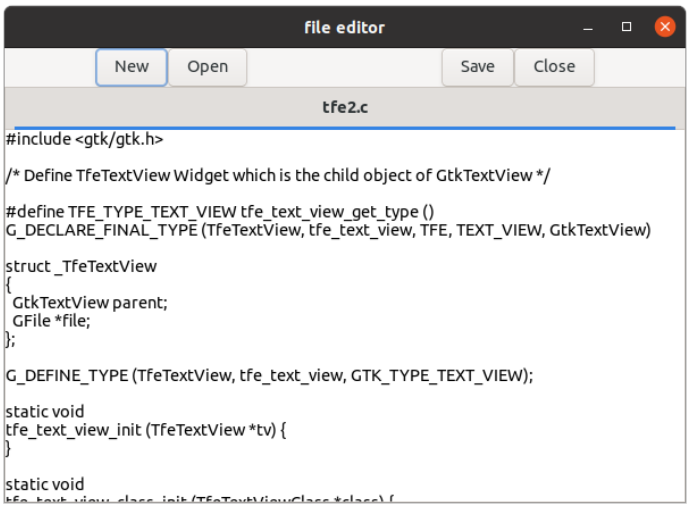


Figure 2.12: Screenshot of the file editor

The screenshot above shows the layout. The source code `tfe2.c` is as follows.

```
1 #include <gtk/gtk.h>
2
3 static void
```

```
4  app_activate (GApplication *app) {
5    GtkWidget *win, *nb, *scr, *tv, *label;
6    GtkWidget *boxv, *boxh;
7    GtkWidget *dmy1, *dmy2, *dmy3;
8    GtkWidget *btnn; /* button for new */
9    GtkWidget *btno; /* button for open */
10   GtkWidget *btns; /* button for save */
11   GtkWidget *btnc; /* button for close */
12   int i;
13   GtkNotebookPage *nbp;
14   char *files[] = {"file-1", "file-2"};
15
16   win = gtk_application_window_new (GTK_APPLICATION (app));
17   gtk_window_set_title (GTK_WINDOW (win), "file editor");
18   gtk_window_set_default_size (GTK_WINDOW (win), 600, 400);
19
20   boxv = gtk_box_new (GTK_ORIENTATION_VERTICAL, 0);
21   gtk_window_set_child (GTK_WINDOW (win), boxv);
22
23   boxh = gtk_box_new (GTK_ORIENTATION_HORIZONTAL, 0);
24   gtk_box_append (GTK_BOX (boxv), boxh);
25
26   dmy1 = gtk_label_new(NULL); /* dummy label for left space */
27   gtk_label_set_width_chars (GTK_LABEL (dmy1), 10);
28   dmy2 = gtk_label_new(NULL); /* dummy label for center space */
29   gtk_widget_set_hexpand (dmy2, TRUE);
30   dmy3 = gtk_label_new(NULL); /* dummy label for right space */
31   gtk_label_set_width_chars (GTK_LABEL (dmy3), 10);
32   btnn = gtk_button_new_with_label ("New");
33   btno = gtk_button_new_with_label ("Open");
34   btns = gtk_button_new_with_label ("Save");
35   btnc = gtk_button_new_with_label ("Close");
36
37   gtk_box_append (GTK_BOX (boxh), dmy1);
38   gtk_box_append (GTK_BOX (boxh), btnn);
39   gtk_box_append (GTK_BOX (boxh), btno);
40   gtk_box_append (GTK_BOX (boxh), dmy2);
41   gtk_box_append (GTK_BOX (boxh), btns);
42   gtk_box_append (GTK_BOX (boxh), btnc);
43   gtk_box_append (GTK_BOX (boxh), dmy3);
44
45   nb = gtk_notebook_new ();
46   gtk_widget_set_hexpand (nb, TRUE);
47   gtk_widget_set_vexpand (nb, TRUE);
48   gtk_box_append (GTK_BOX (boxv), nb);
49
50   for (i = 0; i < 2; i++) {
51     scr = gtk_scrolled_window_new ();
52     tv = gtk_text_view_new ();
53     label = gtk_label_new (files[i]);
54     gtk_text_view_set_wrap_mode (GTK_TEXT_VIEW (tv), GTK_WRAP_WORD_CHAR);
55     gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), tv);
56     gtk_notebook_append_page (GTK_NOTEBOOK (nb), scr, label);
```

```
57      nbp = gtk_notebook_get_page (GTK_NOTEBOOK (nb), scr);
58      g_object_set (nbp, "tab-expand", TRUE, NULL);
59    }
60    gtk_window_present (GTK_WINDOW (win));
61  }
62
63  int
64  main (int argc, char **argv) {
65    GtkApplication *app;
66    int stat;
67
68    app = gtk_application_new ("com.github.ToshioCP.tfe2", G_APPLICATION_DEFAULT_FLAGS);
69    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
70    stat = g_application_run (G_APPLICATION (app), argc, argv);
71    g_object_unref (app);
72    return stat;
73  }
```

This program just builds the widgets. Other features will be implemented in the later sactions.

The function `app_activate` builds the widgets in the main application window.

- 16-18: Creates a GtkApplicationWindow instance and sets the title and default size.
- 20-21: Creates a vertical GtkBox instance `boxv`as a child of GtkApplicationWindow.
- 23-24: Creates a horizontal GtkBox instance `boxh` and appends it to `boxv` as the first child.
- 26-31: Creates three dummy labels. The `dmy1` and `dmy3` labels are ten characters wide, while the `dmy2` label has its `hexpand` property set to `TRUE` to fill the available space.
- 32-35: Creates four buttons `New`, `Open`, `Save`, and `Close`.
- 37-43: Appends these label and button instances to `boxh`.
- 45-48: Creates a GtkNotebook instance with `hexpand` and `vexpand` properties set to `TRUE`, allowing it to expand in both directions. It is then appended to `boxv` as the second child.
- 50-59: Creates, GtkScrolledWindow, GtkTextView, and GtkLabel instances in the for-loop. The scrolled window `scr` takes `tv` as a child. The function `gtk_notebook_append_page` appends `scr` as a page and `label` as a page tab to the note book. The function `gtk_notebook_get_page` retrieves the GtkNotebookPage instance that corresponds to `scr`, a child widget of `nb`. GtkNotebookPage class does not have a function to set the property "tab-expand", so the general property setting function `g_object_set` is used here. The property is set to TRUE to expand the tab horizontally.

Building widgets takes 44 lines (= 59-16+1). We also needed many variables (`boxv`, `boxh`, `dmy1`, …), and most of them are used only for building the widgets. Is there a better way to reduce this overhead?

Gtk provides GtkBuilder. It reads user interface (UI) data and builds a window. It simplifies this tedious process.

2.7.2 UI Files

Look at the UI file `tfe3.ui` that defines the widget structure.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <interface>
3    <object class="GtkApplicationWindow" id="win">
4      <property name="title">file editor</property>
5      <property name="default-width">600</property>
6      <property name="default-height">400</property>
7      <child>
8        <object class="GtkBox">
9          <property name="orientation">GTK_ORIENTATION_VERTICAL</property>
```

```
10          <child>
11            <object class="GtkBox">
12              <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
13              <child>
14                <object class="GtkLabel">
15                  <property name="width-chars">10</property>
16                </object>
17              </child>
18              <child>
19                <object class="GtkButton">
20                  <property name="label">New</property>
21                </object>
22              </child>
23              <child>
24                <object class="GtkButton">
25                  <property name="label">Open</property>
26                </object>
27              </child>
28              <child>
29                <object class="GtkLabel">
30                  <property name="hexpand">TRUE</property>
31                </object>
32              </child>
33              <child>
34                <object class="GtkButton">
35                  <property name="label">Save</property>
36                </object>
37              </child>
38              <child>
39                <object class="GtkButton">
40                  <property name="label">Close</property>
41                </object>
42              </child>
43              <child>
44                <object class="GtkLabel">
45                  <property name="width-chars">10</property>
46                </object>
47              </child>
48            </object>
49          </child>
50          <child>
51            <object class="GtkNotebook" id="nb">
52              <property name="hexpand">TRUE</property>
53              <property name="vexpand">TRUE</property>
54              <child>
55                <object class="GtkScrolledWindow">
56                  <property name="hscrollbar-policy">automatic</property>
57                  <property name="vscrollbar-policy">automatic</property>
58                  <child>
59                    <object class="GtkTextView">
60                    </object>
61                  </child>
62                </object>
```

```
63              </child>
64              <child type="tab">
65                <object class="GtkLabel">
66                  <property name="label">file-1</property>
67                </object>
68              </child>
69              <child>
70                <object class="GtkScrolledWindow">
71                  <property name="hscrollbar-policy">automatic</property>
72                  <property name="vscrollbar-policy">automatic</property>
73                  <child>
74                    <object class="GtkTextView">
75                    </object>
76                  </child>
77                </object>
78              </child>
79              <child type="tab">
80                <object class="GtkLabel">
81                  <property name="label">file-2</property>
82                </object>
83              </child>
84            </object>
85          </child>
86        </object>
87      </child>
88    </object>
89  </interface>
```

This is an XML file. The extension "ui" means User Interface.

Tags are enclosed in angle brackets, beginning with `<` and ending with `>`. They generally come in pairs: a start tag and an end tag. For instance, `<interface>` opens the element, while `</interface>` closes it.

The end tag is identified by a forward slash immediately following the opening bracket, and its name must match the corresponding start tag. Additionally, certain tags (such as object tags) can include class and id attributes within their start tag to provide extra metadata.

- 1: XML declaration. This is a special tag and doesn't follow the "start and end tag" rule. It specifies that the XML version is 1.0 and the encoding is UTF-8.
- 2, 89: The UI file begins and ends with interface tags.
- 3: The object tag defines an instance with the class of `GtkApplicationWindow` and the id of `win`. The class attribute specifies the type of the object to create, while the id attribute provides a unique name used to retrieve the object within your C code.
- 4-6: Property tags define the values of the properties on the object that is defined with an object tag just before the property definitions. These three lines define the properties of the top-level window `win`.
  - title: "file editor"
  - default-width: 600
  - default-height: 400
- 7: Child tag means that the following object is a child widget. For example, line 7 defines the GtkBox as a child widget of the GtkApplicationWindow `win`.
- 8-49: Defines a horizontal box comprising labels and buttons.
- 50-85: Defines the GtkNotebook as a child of the vertical box. It contains two pairs of children, where each pair consists of a content area (a scrolled window) and a tab (a label). While a standard `<child>` tag defines the content, a `<child>` tag with `type="tab"` defines the tab itself. Note that

GtkNotebookPage cannot be defined directly in UI files because it is not a widget; it is an auxiliary object derived directly from GObject.

Compare this UI file with the `app_activate` function of `tfe2.c`. While both methods produce the same window and widget hierarchy, the UI file approach offers superior readability and easier maintenance.

2.7.2.1 Gtk4-Builder-Tool

You can verify and optimize your UI files using the `gtk4-builder-tool` utility.

- **Validation:** `gtk4-builder-tool validate <ui_file_name>` checks the file for errors. If the UI file contains syntax errors or invalid object definitions, the tool will point them out.
- **Simplification:** `gtk4-builder-tool simplify <ui_file_name>` cleans up the UI file and prints the result to the terminal.
  - The `--replace` option: Using this option replaces the original file with the simplified version.
  - Removing Defaults: If a property is set to its default value, the tool removes it to save space. For example, since the default orientation for GtkBox widgets is horizontal, that line would be deleted (ex. line 12 in `tfe3.ui`).
  - Value Conversion: Some values are shortened; for instance, "TRUE" and "FALSE" are converted to "1" and "0".

Example:

```
$ gtk-builder-tool validate tfe/tfe3.ui
$
```

Since `tfe3.ui` contains no errors, `gtk-builder-tool` produced no output.

A Note on Maintenance

While the simplify tool makes the file smaller, it can sometimes make it harder to read. For example, it converts "TRUE" and "FALSe" into "1" and "0" respectively. Keeping "TRUE" or "FALSE" is better for maintenance because it is more readable for humans than "1" or "0".

It is always a good practice to validate your UI file before deploying your application.

2.7.3 GtkBuilder

The file tfe3.c builds the same widget as `tfe2.c` using `tfe3.ui`. It is far shorter than `tfe2.c` (39 lines against 73 lines).

```
$ wc tfe/tfe2.c tfe/tfe3.c
  73  274 2557 tfe/tfe2.c
  38  113 1093 tfe/tfe3.c
 111  387 3650 total
$
```

The following is the file `tfe3.c`

```
1  #include <gtk/gtk.h>
2
3  static void
4  app_activate (GApplication *app) {
5    GtkWidget *win;
6    GtkWidget *nb;
7    GtkWidget *scr;
8    GtkNotebookPage *nbp;
9    int n_pages;
10   GtkBuilder *build;
11
```

```
12    build = gtk_builder_new_from_file ("tfe3.ui");
13    win = GTK_WIDGET (gtk_builder_get_object (build, "win"));
14    nb = GTK_WIDGET (gtk_builder_get_object (build, "nb"));
15    gtk_window_set_application (GTK_WINDOW (win), GTK_APPLICATION (app));
16    g_object_unref(build);
17
18    n_pages = gtk_notebook_get_n_pages (GTK_NOTEBOOK (nb));
19    for (int i = 0; i < n_pages; i++) {
20      scr = gtk_notebook_get_nth_page (GTK_NOTEBOOK (nb), i);
21      nbp = gtk_notebook_get_page (GTK_NOTEBOOK (nb), scr);
22      g_object_set (nbp, "tab-expand", TRUE, NULL);
23    }
24
25    gtk_window_present (GTK_WINDOW (win));
26  }
27
28  int
29  main (int argc, char **argv) {
30    GtkApplication *app;
31    int stat;
32
33    app = gtk_application_new ("com.github.ToshioCP.tfe3", G_APPLICATION_DEFAULT_FLAGS);
34    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
35    stat = g_application_run (G_APPLICATION (app), argc, argv);
36    g_object_unref (app);
37    return stat;
38  }
```

GtkBuilder builds widgets based on a UI file.

- 12: The function `gtk_builder_new_from_file` reads the file `tfe3.ui`. Then, it builds the widgets and creates a GtkBuilder object. All the widgets are connected based on the parent-children relationship described in the UI file.
- 13, 14: We can retrieve objects from the builder object using the `gtk_builder_get_object` function. The top level window, which has an id of "win" in the UI file, is taken and assigned to the variable `win`. Similarly, The GtkNotbook instance with the id "nb" is assignd to the variable `nb`.
- 15: The function `gtk_window_set_application` adds `win` to the application's list of managed windows. You can use `gtk_application_add_window` instead of this function. The result is identical.
- 16: After the window and the application are connected, we no longer need the GtkBuilder instance. It is released with the `g_object_unref` function. 18-23: Sets the "tab-expand" properties of the GtkNotebookPage instances to TRUE. Since GtkNotebookPage is not a widget, its properties cannot be set in the UI file; you must set them in your C source code.

2.7.3.1 Using UI String

GtkBuilder can build widgets with a string. Use `gtk_builder_new_from_string` instead of `gtk_builder_new_from_file`.

```
char *uistring;

uistring =
"<interface>"
  "<object class=\"GtkApplicationWindow\" id=\"win\">"
    "<property name=\"title\">file editor</property>"
```

```
    "<property name=\"default-width\">600</property>"
    "<property name=\"default-height\">400</property>"
    "<child>"
      "<object class=\"GtkBox\">"
        "<property name=\"orientation\">GTK_ORIENTATION_VERTICAL</property>"
... ... ...
... ... ...
"</interface>";

build = gtk_builder_new_from_string (uistring, -1);
```

This method has an advantage and disadvantage. The advantage is that the UI string is written in the source code. So, no UI file is needed at runtime. The disadvantage is that defining the UI as a C string can be tedious, as the xml needs quoting and special characters need escaping. If you want to use this method, you should write a script that converts UI files into C string literals.

- Replace backslashes with two backslashes.
- Add a backslash before each double quote.
- Add double quotes at the left and right of the string in each line.

Or, if you have `jq` installed, you can use `jq -R < tfe3.ui` to do the quoting and escaping for you.

2.7.3.2 GResources

A GResource is similar to a UI string, except that a GResource is a binary data. The `glib-compile-resources` program compiles UI files into GResources. It can compile not only text files but also binary files such as images, sounds and so on. After compilation, it bundles them up into one GResource object.

An xml file is necessary for the resource compiler `glib-compile-resources`. It describes resource files.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <gresources>
3   <gresource prefix="/com/github/ToshioCP/tfe3">
4     <file>tfe3.ui</file>
5   </gresource>
6 </gresources>
```

- 2: `<gresources>` tag can include multiple `<gresource>` tags. However, this xml has only one `<gresource>` tag.
- 3: A `<gresource>` tag can have a prefix attribute. This tag defines that the prefix for `tfe3.ui` is `/com/github/ToshioCP/tfe3`. A prefix works like a namespace. As a best practice, you should always use your Application ID (with dots replaced by slashes) as the prefix. In this program, the Application ID is "com.github.ToshioCP.tfe3" and the prefix is "/com/github/ToshioCP/tfe3". While it is technically possible to define multiple different prefixes within a single application to distinguish resources with the same name, this is not recommended. If you need to organize multiple resources, you should keep a single Application ID prefix and distinguish your files by giving them different filenames or placing them in different subdirectories.
- 4: The name of the GRresource is `tfe3.ui`. The resource will be referred to as "/com/github/ToshioCP/tfe3/tfe3.ui" by GtkBuilder. The pattern is "prefix" + "name". If you want to add more files, insert them between line 4 and 5.

Save this xml text to `tfe3.gresource.xml`. The GResource compiler `glib-compile-resources` shows its usage with the argument `--help`.

```
$ glib-compile-resources --help
 :
  glib-compile-resources [   …]
```

```
         .gresource.xml
        .gresource

    :
  -h, --help

       :
  --version
  --target=
  --sourcedir=DIRECTORY               (FILE)        (   :      )
  --generate
  --generate-header
  --generate-source
  --generate-dependencies
  --dependency-file=
  --generate-phony-targets                PHONY
  --manual-register
  --internal                             (G_GNUC_INTERNAL      )
  --external-data                       C        (           )
  --c-name=IDENTIFIER                      C
  -C, --compiler=              C     ( : CC   )

$
```

Usually, the resources are converted to a C source file. Two options `--target` and `--generate-source` are added to the command line. Move your current directory to /src/tfe and run the command below, then a C source file `resources.c` will be generated.

```
$ glib-compile-resources tfe3.gresource.xml --target=resources.c --generate-source
```

Modify `tfe3.c` and save it as `tfe3_r.c`.

```
#include "resources.c"
... ... ...
... ... ...
build = gtk_builder_new_from_resource ("/com/github/ToshioCP/tfe3/tfe3.ui");
... ... ...
... ... ...
```

The function `gtk_builder_new_from_resource` builds widgets from a resource.

Then, compile and run it.

```
$ bash comp tfe3_r
$ ./a.out
```

A window appears and it is the same as the screen shown by `tfe3.c`.

Generally, using GResources is the best approach for C programs. They allow you to use embedded assets in various widgets. For example, a GtkImage can be created from a resource path, provided that the original image file has been compiled into the bundle using `glib-compile-resources`.

```
GtkWidget *image;
image = gtk_image_new_from_resource (const char* resource_path)
```

2.8 Build System

2.8.1 Managing Big Source Files

We've compiled a small editor so far. The program is not complicated yet, but if it grows bigger, it will be difficult to maintain. So, we should do the following now.

- We've had only one C source file and put everything in it. We need to divide it into several files.
- There are two compilers, `gcc` and `glib-compile-resources`. We should control them with one build tool.

2.8.2 Divide a C Source File into Two Parts.

When you divide C source file into several parts, each file should contain one thing. For example, our C source has two parts, one is for TfeTextView and the other is for application, including GtkApplication and GtkApplicationWindow. So, we divide it into:

- `tfetextview.h` and `tfetextview.c`
- `tfeapplication.c`

2.8.2.1 Header Files

In `tfeapplication.c`, the function `tfe_text_view_new` is called to create a TfeTextView instance. However, since this function is defined in `tfetextview.c`, using it in `tfeapplication.c` without a prior declaration will cause a compilation error. To prevent this, the function must be declared. In C, such declarations (along with variables and macros) are typically written in header files. For example, by placing the declaration of `tfe_text_view_new` in `tfetextview.h` and including this header file in both `tfeapplication.c` and `tfetextview.c`, we can successfully avoid compilation errors.

2.8.2.2 compilation process

In addition, we have some other files.

- `tfe.ui`: a UI file.
- `tfe.gresource.xml`: a resource description file

Now we face another problem. The number of build steps has increased.

- Compiling the UI file `tfe.ui` into `resources.c`, using `tfe.gresource.xml` by resource compiler.
- Compiling `tfeapplication.c` into `tfe.o` (object file).
- Compiling `tfetextview.c` into `tfetextview.o`.
- Compiling `resources.c` into `resources.o`.
- Linking all the object files into application `tfe`.

How can we manage the build process above? It is time to use Build tools to manage the steps.

2.8.3 Meson and Ninja

Meson and Ninja are one of the most popular build systems. Many developers, includeing GTK developers, use them.

First, You need to create `meson.build` file.

```
1 project('tfe', 'c')
2
3 gtkdep = dependency('gtk4')
4
5 gnome=import('gnome')
6 resources = gnome.compile_resources('resources','tfe.gresource.xml')
```

```
7
8  sourcefiles=files('tfeapplication.c', 'tfetextview.c')
9
10 executable('tfe', sourcefiles, resources, dependencies: gtkdep, install: false)
```

- 1: The function `project` defines things about the project. The first argument is the name of the project and the second is the programming language.
- 3: The function `dependency` defines a dependency that is taken by `pkg-config`. We put `gtk4` as an argument.
- 5: The function `import` imports a module. In line 5, the gnome module is imported and assigned to the variable `gnome`. The gnome module provides helper tools to build GTK programs.
- 6: The method `.compile_resources` is from the gnome module. It compiles files to resources, as instructed by the xml file. In line 6, the resource filename is `resources`, which means `resources.c` and `resources.h`, and the xml file is `tfe.gresource.xml`. This method generates C source file by default.
- 8: Defines source files.
- 10: The executable function generates a target file by compiling the source files. The first argument is the filename of the target, and the following arguments are the source files. The last two arguments are provided as "key-value" pairs. For example, the fourth argument consists of the key `dependencies` and the value `gtkdep`, separated by a colon (:). This type of argument is called a **keyword argument** (or **kwargs**). The value `gtkdep` is defined on line 3. The final argument specifies that this executable will not be installed to a system directory (such as `/usr/local/bin`). Instead, it is only built and kept within the build directory.

Now, build the project by running Meson and Ninja. The whole programs are in src/tfe4 directory. Change your current directory to `src/tfe4` and type the following on your command line.

```
$ meson setup _build
$ ninja -C _build
```

Here, the meson command takes two arguments:

- `setup`: The primary command for Meson. Although it is the default action, explicitly specifying `setup` has been recommended since Meson version 0.64.0.
- `_build`: The name of the build directory.

Once the build is complete, the executable file tfe will be generated inside the `_build` directory. You can run it using the following command:

```
$ _build/tfe tfeapplication.c tfetextview.c
```

A window with two pages appears. One is `tfeapplication.c` and the other is `tfetextview.c`.

This application has four buttons but their signal handlers are not implemented yet. So, you can just display the files. Even you can edit the files on the screen, the files will not be updated.

For further information, see The Meson Build system.

2.9 Instance Initialization and Destruction

2.9.1 Encapsulation and Object

We've divided C source file into two parts.

- `tfetextview.h` and `tfetextview.c`
- `tfeapplication.c`

The TfeTextView object manages a GFile instance internally. Outside of this object, the only way to access the GFile is through the `tfe_text_view_set_file` and `tfe_text_view_get_file` functions. This demonstrates encapsulation.

By keeping variables hidden and controlling access through specific functions, this object-oriented design divides a program into manageable parts, making it easier to maintain.

In this section, we complete the TfeTextView program to show how the GObject system implements object-oriented encapsulation.

2.9.2 Class and Instance

The actual object created in memory is called an "instance." When an instance is created, the system allocates the necessary memory based on the definition of its object structure. For example, the object structure for TfeTextView is defined as follows:

```
struct _TfeTextView
{
  GtkTextView parent;
  GFile *file;
};
```

When the instance `tv` is created using `tv = tfe_text_view_new ();`, memory for this structure is allocated internally, allowing it to manage its state. Multiple instances can exist simultaneously. Since each instance has its own memory area, it can hold its own unique state.

On the other hand, some functions used for managing the instances are stored as "function pointers" in a separate structure called a "class". Because these functions perform common operations for all instances, every instance shares a single class structure. In short, while multiple instances can be created, there is only one class in memory.

2.9.3 Inheritance

Objects have a mechanism called inheritance, which can be represented as a parent-child or ancestor-descendant relationship.

Note: Please be careful not to confuse this with the parent-child relationship of widgets, as they are completely different concepts.

Most objects are derived from GObject (although there are exceptions, such as GtkExpression, which is not derived from GObject). TfeTextView is also derived from GObject. Its inheritance hierarchy looks like this:

```
GObject - GInitiallyUnowned - GtkWidget - GtkTextView - TfeTextView
```

A child object inherits all the features of its parent object. As we will explain in detail in the later subsection, object inheritance is based on the inheritance of both the class structure and the object structure. For instance, the GObject class holds function pointers required for object destruction, and the TfeTextView class inherits these pointers. Furthermore, instead of simply inheriting them, a child class can rewrite these functions to suit its own needs. This is called "overriding."

We will explain the mechanism of inheritance in more detail in the next subsection, using TfeTextView as an example.

2.9.4 GObject Structure Inheritance

An instance is a block of memory allocated for the object structure. The following is the structure of `TfeTextView`.

```
/* This typedef statement is automatically generated by the macro G_DECLARE_FINAL_TYPE */
typedef struct _TfeTextView TfeTextView;

struct _TfeTextView {
```

```
  GtkTextView parent;
  GFile *file;
};
```

The members of the structure are:

- The member `parent` is a GtkTextView C structure. It is declared in `gtktextview.h`. GtkTextView is the parent object of TfeTextView. Note that this member is not a pointer, but the structure itself.
- The member `file` is a pointer to a GFile. It can be NULL if the TfeTextView instance has no file associated with it. For example, it is initially NULL when the instance is created.

You can find the structural declarations of these ancestor objects in the GTK and GLib source files. The following snippets are extracted from the source files (simplified for clarity):

```
typedef struct _GObject GObject;
typedef struct _GObject GInitiallyUnowned;
struct  _GObject
{
  GTypeInstance  g_type_instance;
  volatile guint ref_count;
  GData         *qdata;
};

typedef struct _GtkWidget GtkWidget;
struct _GtkWidget
{
  GInitiallyUnowned parent_instance;
  GtkWidgetPrivate *priv;
};

typedef struct _GtkTextView GtkTextView;
struct _GtkTextView
{
  GtkWidget parent_instance;
  GtkTextViewPrivate *priv;
};
```

In each structure, the parent instance is declared as the very first member. Because of this layout, the child object naturally includes all of its ancestors. The structure of a TfeTextView instance looks like the following diagram:

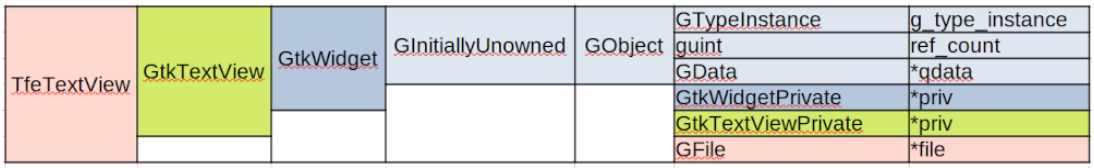


Figure 2.13: The structure of the instance TfeTextView

Derivable objects (ancestor objects) have their own private data areas, which are not directly included in the structures shown above. For example, GtkWidget uses the GtkWidgetPrivate C structure for its private data.

In C, a structure declaration merely defines a new type; it does not allocate any memory. Therefore, memory is only allocated from the heap when the `tfe_text_view_new` function is called. At the same time, the ancestors' private data areas are also allocated for the TfeTextView instance. These private areas are hidden from TfeTextView, meaning it cannot access them directly. This entire allocated memory

block is called the instance. When a TfeTextView instance is created, it is allocated three distinct data areas:

- The TfeTextView instance structure itself.
- The GtkWidgetPrivate structure.
- The GtkTextViewPrivate structure.

TfeTextView functions can only access its own instance structure. The GtkWidgetPrivate and GtkTextViewPrivate structures are managed exclusively by the functions of their respective ancestors. Consider the following example:

```
GtkWidget *tv = tfe_text_view_new ();
GtkTextBuffer *buffer = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
```

The parent function `gtk_text_view_get_buffer` accesses the GtkTextViewPrivate data associated with the `tv` instance. This private area contains a pointer to the GtkTextBuffer, which the function retrieves and returns. (The actual internal behavior is slightly more complicated, but this illustrates the concept).

This is how a TfeTextView instance effectively inherits the functions and data of its ancestors.

2.9.5 GObjectClass Structure Inheritance

Objects have class structure such as GObjectclass, GtkWidgetClass, GtkTextViewClass, and TfeTextViewClass. They are C structures, mainly including pointers to C functions. The functions are called **class methods** and used by the object itself or its descendant objects.

For example, GObject class is declared in `gobject.h` in GLib source files.

```
typedef struct _GObjectClass             GObjectClass;
typedef struct _GObjectClass             GInitiallyUnownedClass;

struct  _GObjectClass
{
  GTypeClass   g_type_class;

  /*< private >*/
  GSList      *construct_properties;

  /*< public >*/
  /* seldom overridden */
  GObject*   (*constructor)     (GType                  type,
                                 guint                  n_construct_properties,
                                 GObjectConstructParam *construct_properties);
  /* overridable methods */
  void       (*set_property)        (GObject        *object,
                                     guint           property_id,
                                     const GValue   *value,
                                     GParamSpec     *pspec);
  void       (*get_property)        (GObject        *object,
                                     guint           property_id,
                                     GValue         *value,
                                     GParamSpec     *pspec);
  void       (*dispose)         (GObject        *object);
  void       (*finalize)        (GObject        *object);
  /* seldom overridden */
  void       (*dispatch_properties_changed) (GObject      *object,
                          guint     n_pspecs,
```

```
                        GParamSpec  **pspecs);
  /* signals */
  void       (*notify)          (GObject    *object,
                                 GParamSpec *pspec);

  /* called when done constructing */
  void       (*constructed)     (GObject    *object);

  /*< private >*/
  gsize     flags;

  gsize         n_construct_properties;

  gpointer pspecs;
  gsize n_pspecs;

  /* padding */
  gpointer  pdummy[3];
};
```

Most of the `_GObjectClass` structure members are pointers to functions. Two functions `dispose` and `finalaize` are called when the object is destructed. The details will be explained later.

Other ancestors' classes of the TfeTextView class are shown below. They are extracted from the GTK source codes, but not exactly the same.

```
struct _GtkWidgetClass
{
  GInitiallyUnownedClass parent_class;

  /*< public >*/

  /* basics */
  void (* show)                (GtkWidget        *widget);
  void (* hide)                (GtkWidget        *widget);
  void (* map)                 (GtkWidget        *widget);
  void (* unmap)               (GtkWidget        *widget);
  void (* realize)             (GtkWidget        *widget);
  void (* unrealize)           (GtkWidget        *widget);
  void (* root)                (GtkWidget        *widget);
  void (* unroot)              (GtkWidget        *widget);
  void (* size_allocate)       (GtkWidget           *widget,
                                int                  width,
                                int                  height,
                                int                  baseline);
  void (* state_flags_changed) (GtkWidget        *widget,
                                GtkStateFlags     previous_state_flags);
  void (* direction_changed)   (GtkWidget        *widget,
                                GtkTextDirection  previous_direction);

  /* size requests */
  GtkSizeRequestMode (* get_request_mode)              (GtkWidget      *widget);
  void               (* measure) (GtkWidget      *widget,
                                  GtkOrientation  orientation,
                                  int             for_size,
```

```
                                int             *minimum,
                                int             *natural,
                                int             *minimum_baseline,
                                int             *natural_baseline);

  /* Mnemonics */
  gboolean (* mnemonic_activate)        (GtkWidget           *widget,
                                         gboolean             group_cycling);

  /* explicit focus */
  gboolean (* grab_focus)               (GtkWidget           *widget);
  gboolean (* focus)                    (GtkWidget           *widget,
                                         GtkDirectionType     direction);
  void     (* set_focus_child)          (GtkWidget           *widget,
                                         GtkWidget           *child);

  /* keyboard navigation */
  void     (* move_focus)               (GtkWidget           *widget,
                                         GtkDirectionType     direction);
  gboolean (* keynav_failed)            (GtkWidget           *widget,
                                         GtkDirectionType     direction);

  gboolean     (* query_tooltip)      (GtkWidget  *widget,
                                       int         x,
                                       int         y,
                                       gboolean    keyboard_tooltip,
                                       GtkTooltip *tooltip);

  void         (* compute_expand)     (GtkWidget  *widget,
                                       gboolean   *hexpand_p,
                                       gboolean   *vexpand_p);

  void         (* css_changed)                (GtkWidget            *widget,
                                               GtkCssStyleChange    *change);

  void         (* system_setting_changed)     (GtkWidget            *widget,
                                               GtkSystemSetting      settings);

  void         (* snapshot)                   (GtkWidget            *widget,
                                               GtkSnapshot          *snapshot);

  gboolean     (* contains)                   (GtkWidget *widget,
                                               double     x,
                                               double     y);

  /*< private >*/

  GtkWidgetClassPrivate *priv;

  gpointer padding[8];
};

struct _GtkTextViewClass
```

```
83  {
84    GtkWidgetClass parent_class;
85
86    /*< public >*/
87
88    void (* move_cursor)           (GtkTextView      *text_view,
89                                    GtkMovementStep   step,
90                                    int               count,
91                                    gboolean          extend_selection);
92    void (* set_anchor)            (GtkTextView      *text_view);
93    void (* insert_at_cursor)      (GtkTextView      *text_view,
94                                    const char       *str);
95    void (* delete_from_cursor)    (GtkTextView      *text_view,
96                                    GtkDeleteType     type,
97                                    int               count);
98    void (* backspace)             (GtkTextView      *text_view);
99    void (* cut_clipboard)         (GtkTextView      *text_view);
100   void (* copy_clipboard)        (GtkTextView      *text_view);
101   void (* paste_clipboard)       (GtkTextView      *text_view);
102   void (* toggle_overwrite)      (GtkTextView      *text_view);
103   GtkTextBuffer * (* create_buffer) (GtkTextView   *text_view);
104   void (* snapshot_layer)        (GtkTextView      *text_view,
105                       GtkTextViewLayer  layer,
106                       GtkSnapshot      *snapshot);
107   gboolean (* extend_selection)  (GtkTextView            *text_view,
108                                   GtkTextExtendSelection  granularity,
109                                   const GtkTextIter      *location,
110                                   GtkTextIter            *start,
111                                   GtkTextIter            *end);
112   void (* insert_emoji)          (GtkTextView      *text_view);
113
114   /*< private >*/
115
116   gpointer padding[8];
117 };
118
119 /* The following definition is generated by the macro G_DECLARE_FINAL_TYPE */
120 typedef struct {
121   GtkTextView parent_class;
122 } TfeTextViewClass;
```

- 120-122: These three lines are generated by the macro `G_DECLARE_FINAL_TYPE`. So, they are not written in either `tfetextview.h` or `tfetextview.c`.
- 3, 84, 121: Each class has its parent class at the first member of its structure. It is the same as instance structures.
- Class members in ancestors are open to the descendant class. So, they can be changed in `tfe_text_view_class_init` function. For example, the `dispose` pointer in GObjectClass will be overridden later in `tfe_text_view_class_init`. (Override is an object oriented programming terminology. Override is rewriting ancestors' class methods in the descendant class.)
- Some class methods are often overridden. `set_property`, `get_property`, `dispose`, `finalize` and `constructed` are such methods.

TfeTextViewClass includes its ancestors' class in it. It is illustrated in the following diagram.

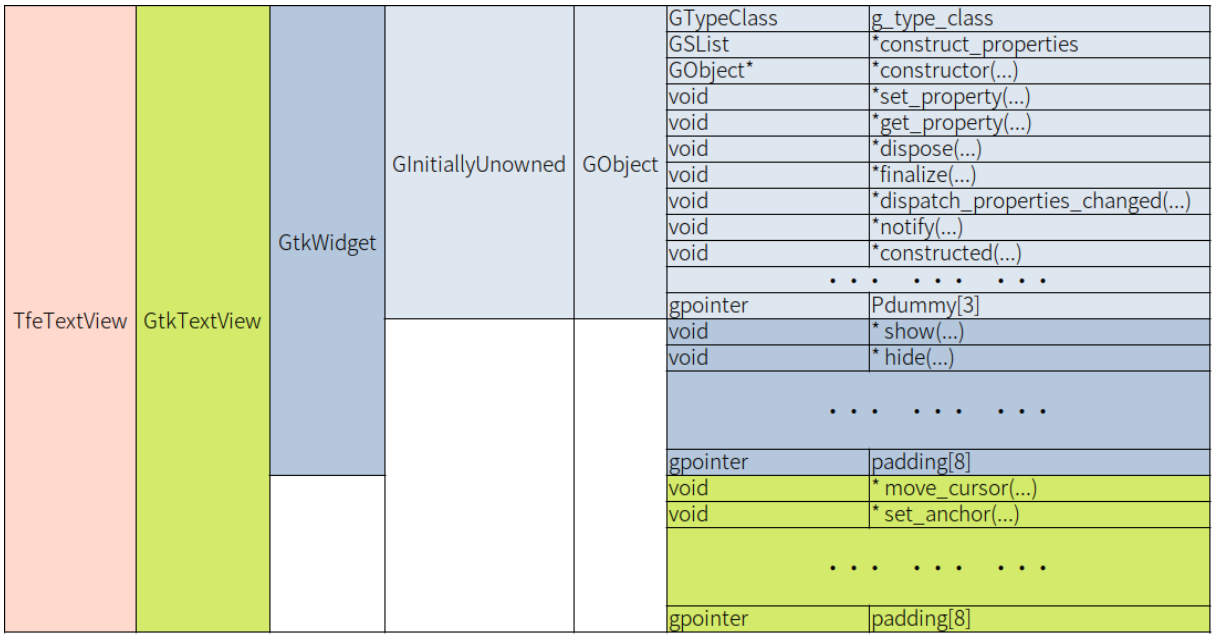


Figure 2.14: The structure of TfeTextView Class

2.9.6 Instance Initialization Process

By convention, instances are created using a function named `object_name_new` (for example, `tfe_text_view_new`). However, because this function calls `g_object_new` internally, it is actually `g_object_new` that serves as the core function for creating instances in the system.

When this function is called, an initialization process takes place as follows (the actual behavior is more complex, but this is a simplified overview):

- If the class has not been created yet, memory for the class structure is allocated and initialized.
- Memory required for the instance is allocated and initialized.

Both class and instance initializations are performed in a top-down order, from parent to child.

Objects generally fall into two categories: “derivable types”, which can have child objects, and “final types”, which cannot. TfeTextView is a final type.

Since final type objects do not expose their class structures, you don’t add new class members to them. In fact, you don’t need to write the class structure definition yourself at all; a macro (`G_DECLARE_FINAL_TYPE`) handles it for you.

When using the `G_DEFINE_FINAL_TYPE` macro, the instance initialization function must be named `object_name_init`. For TfeTextView, this function is `tfe_text_view_init`.

```
1 static void
2 tfe_text_view_init (TfeTextView *tv) {
3   /* The allocated memory is set to zero by the system.*/
4   /* Therefore, "tv->file" is automatically initialized to NULL; setting it explicitly is unnecessar
5 }
```

The pointer to a GFile must be initialized to be NULL.

Note: When the system allocates memory for an instance, it automatically clears all variables to 0 or NULL. Therefore, explicitly writing `tv->file = NULL;` is actually unnecessary. You only need to write code here if you are allocating dynamic memory or setting specific default values.

The system calls these initialization functions automatically. As long as you define the functions with the correct names according to the rules, the system will handle the rest.

2.9.7 Reference Count

Every Object derived from GObject has a reference count. If an object A refers to an object B, then A keeps a pointer to B in A and at the same time increases the reference count of B by one with the function `g_object_ref (B)`. If A doesn't need B any longer, A discards the pointer to B (usually it is done by assigning NULL to the pointer) and decreases the reference count of B by one with the function `g_object_unref (B)`.

If two objects A and B refer to C, then the reference count of C is two. If A no longer needs C, A discards the pointer to C and decreases the reference count in C by one. Now the reference count of C is one. In the same way, if B no longer needs C, B discards the pointer to C and decreases the reference count in C by one. At this moment, no object refers to C and the reference count of C is zero. This means C is no longer useful. Then C destructs itself and finally the memories allocated to C will be freed.

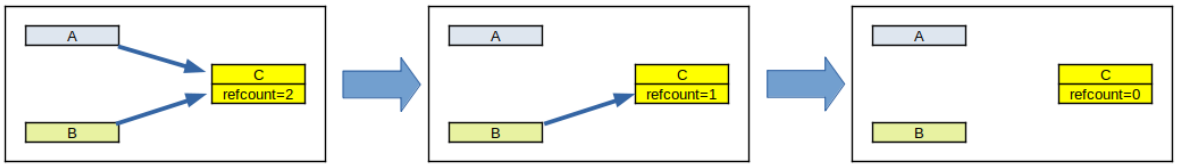


Figure 2.15: Reference count of B

The idea above is based on an assumption that an object referred by nothing has reference count of zero. When the reference count drops to zero, the object starts its destruction process.

2.9.8 Instance Destruction Process

The destruction process is split into two phases: disposing and finalizing. In the disposing process, the object calls the function pointed to by `dispose` in its class to release all references to other instances. After that, it calls the function pointed to by `finalize` in its class to free memory used by the instance and complete the destruction process.

At the beginning of the creation of the instance, these class member points to the dispose and finalize functions **for GObject**. You need to override these functions **for TfeTextView** in the class initialization.

- Dispose functions release objects with `g_clear_object()`. The functions decrease the reference count by one and clear the pointer to the objects.
- Finalize functions free memory with `g_free()` that has allocated in the initialization process.

```
1  static void
2  tfe_text_view_dispose (GObject *gobject) {
3    TfeTextView *tv = TFE_TEXT_VIEW (gobject);
4
5    g_clear_object (&tv->file);
6    G_OBJECT_CLASS (tfe_text_view_parent_class)->dispose (gobject);
7  }
8
9  static void
10 tfe_text_view_class_init (TfeTextViewClass *class) {
11   GObjectClass *object_class = G_OBJECT_CLASS (class);
12
13   object_class->dispose = tfe_text_view_dispose;
14 }
```

In the destruction process, TfeTextView needs to unref the GFile pointed to by `tv->file` and clear the pointer.

- 5: The function `g_clear_object` do nothing if `tv->file == NULL` (not `&tv->file == NULL`). If `tv->file` points to a GFile, it decreases the reference count of the GFile and assigns NULL to `tv->file`. In dispose handlers, we usually use `g_clear_object` rather than `g_object_unref`.
- 6: calls parent's dispose handler.
- The dispose method needs to be overridden by `tfe_text_view_dispose` when the TfeTextView class is initialized. The function `tfe_text_view_class_init` first cast the variable `class` to point the GObjectclass. Because `dispose` is defined as a member of GObjectclass.

Each ancestors' class has been created before the TfeTextView class is created. Therefore, there are four classes and each class has a pointer to each dispose handler. Look at the following diagram. There are four classes – GObjectClass (GInitiallyUnownedClass), GtkWidgetClass, GtkTextViewClass, and TfeTextViewClass. Each class has its own dispose handler – `dh1`, `dh2`, `dh3` and `tfe_text_view_dispose`.

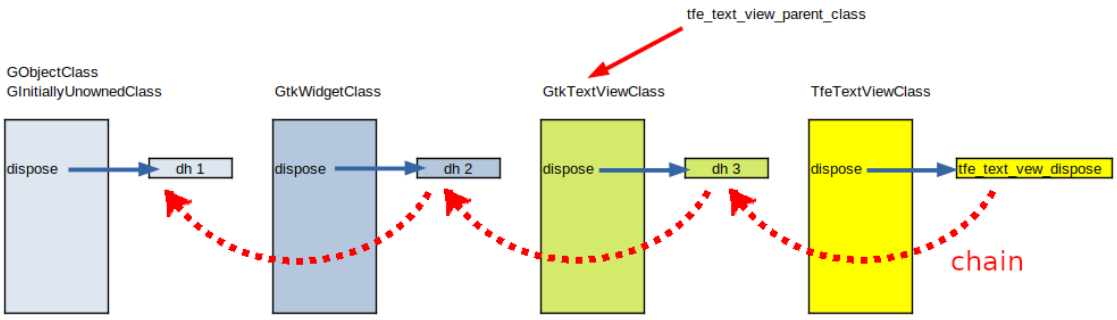


Figure 2.16: dispose handlers

Now, look at the `tfe_text_view_dispose` program above. It first releases the reference to GFile object pointed to by `tv->file`. Then it calls its parent's dispose handler.

```
G_OBJECT_CLASS (tfe_text_view_parent_class)->dispose (gobject);
```

A variable `tfe_text_view_parent_class`, which is made by `G_DEFINE_FINAL_TYPE` macro, is a pointer that points to the parent object class. The variable `gobject` is a pointer to TfeTextView instance which is cast as a GObject instance. Therefore, `G_OBJECT_CLASS (tfe_text_view_parent_class)->dispose` points to the handler `dh3` in the diagram above. The statement `G_OBJECT_CLASS (tfe_text_view_parent_class)->dispose (gobject)` is the same as `dh3 (gobject)`, which means it releases all the reference to the other instances in the GtkTextViewPrivate in the TfeTextView instance. After that, `dh3` calls `dh2`, and `dh2` calls `dh1`. Finally all the references are released.

This process is called "chaining up".

2.10 Error Handling in GLib

2.10.1 Programmer Errors and Runtime Errors

In GLib, errors are generally categorized into two types: Programmer Errors and Runtime Errors.

Programmer Errors are bugs within your source code. You should check for these errors during development; if one is detected, it is a sign that you must find the bug and fix your code immediately.

Runtime Errors, on the other hand, occur due to external factors during program execution, such as user operations or environmental issues (e.g., trying to read a file that does not exist). These are not bugs in your code. When a runtime error occurs, you should notify the user or prompt them to take action; you should never simply terminate the program.

When designing objects, you must consider both types of error handling.

2.10.2 Programmer Error Handling

A well-known programmer error is passing an argument of the wrong type. While the C compiler can detect some of these issues, it cannot verify GType because they are determined at runtime, not at compile time.

To perform runtime type checking, we use the following macros:

- `g_return_if_fail (cond)`: If the condition `cond` is false, it prints an error message and returns from the function.
- `g_return_val_if_fail (cond, val)`: If the condition `cond` is false, it prints an error message, returns from the function with the value val.

While internal helper functions are less prone to type mismatches, public functions are highly susceptible. You should treat these checks as mandatory for all public-facing functions. If these errors are triggered, it almost always indicates a bug in the code calling your library.

```
GFile *
tfe_text_view_get_file (TfeTextView *tv) {
  g_return_val_if_fail (TFE_IS_TEXT_VIEW (tv), NULL);
```

In addition to type checking, GLib provides functions to report other anomalies that suggest programmer error:

- `g_warning (...)`: Used for non-fatal but important issues, such as system configuration errors or anomalies detected in trusted files. The `...` follows the printf format. Example: `g_warning ("%s is not a GType name.", string);`
- `g_error (...)`: Used for fatal errors caused by programming mistakes. This function reports the error and terminates the program immediately.

Note: These functions are intended for developers, not for application end-users.

2.10.3 Runtime Error Handling

Runtime errors are recoverable issues that occur during execution, such as failing to find a file you are trying to open. In many other languages, these are called “exceptions”. GLib provides a structured way to handle these using the GError structure. You can also define custom error domains for your own objects.

2.10.3.1 GError

GError is a C structure that carries three key pieces of information:

- Domain: The category or library to which the error belongs.
- Code: An integer identifying the specific type of error.
- Message: A human-readable string explaining the error.

In most cases, error handling involves displaying this error message in a dialog box to inform the user.

```
void
tfe_error_alert (GtkWindow *win, GError *err) {
  GtkAlertDialog *alert_dialog;

  alert_dialog = gtk_alert_dialog_new ("%s", err->message);
  gtk_alert_dialog_show (alert_dialog, win);
  g_object_unref (alert_dialog);
}

void
example_function (GtkWidget *win, GtkWidget *tv) {
  GError *err = NULL;
```

```
  if (! tfe_text_view_write (TFE_TEXT_VIEW (tv), &err)) {
    tfe_error_alert (GTK_WINDOW (win), err);
    g_clear_error (&err);
  }
}
```

The function `tfe_text_view_write` saves the content of the buffer to the GFile held by the TfeTextView. It may fail if:

- No GFile is set (i.e., the target file is undefined).
- A write error occurs (e.g., lack of write permissions).

When an error occurs, `tfe_text_view_write` creates a GError, populates it with details, assigns it to the pointer provided by the caller, and returns FALSE.

Steps for handling runtime errors:

- Define a GError pointer and initialize it to NULL.
- Pass the address of that pointer (`&err`) to the function.
- Check the return value. If it is FALSE, handle the error (e.g., showing an alert dialog).
- Free the GError using `g_error_free` or, preferably, `g_clear_error (&err)`, which also sets the pointer back to NULL.

If you don't need detailed error reports, you can pass NULL to the error argument:

```
tfe_text_view_write (TFE_TEXT_VIEW (tv), NULL);
```

All functions that support error reporting must be designed to accept NULL for this argument.

2.10.4 Customizing Error Domains

To define errors specific to your own objects, you need to create a custom error domain.

In your header file (`tfetextview.h`):

```
/* User Error Definition */
#define TFE_TEXT_VIEW_ERROR (tfe_text_view_error_quark ())
GQuark tfe_text_view_error_quark (void);

typedef enum {
  TFE_TEXT_VIEW_ERROR_NO_FILE,    /* No file is set */
  TFE_TEXT_VIEW_ERROR_FAILED,     /* Generic or unrecoverable error */
} TfeTextViewError;
```

In your source file (tfetextview.c):

```
G_DEFINE_QUARK (tfe-text-view-error-quark, tfe_text_view_error)
```

Key Rules for Custom Errors:

- Error Domain Macro: Defined as `<NAMESPACE>_<MODULE>_ERROR`. For TfeTextView, it is `TFE_TEXT_VIEW_ERROR`. It maps to a quark function named `namespace_module_error_quark()`.
- Error Codes: Use an enum following the `<Namespace><Module>Error` convention. Individual error codes should be named `<NAMESPACE>_<MODULE>_ERROR_<CODE>`. Always include a `_FAILED` member for generic or unrecognized errors.
- `G_DEFINE_QUARK`: This macro automatically defines the `..._quark()` function. The first argument is the domain name string (kebab-case: `tfe-text-view-error-quark`). The second argument is the C function prefix (snake_case: `tfe_text_view_error`).

To set a custom error within a function, use `g_set_error_literal`:

```
if (tv->file == NULL) {
  g_set_error_literal (err, TFE_TEXT_VIEW_ERROR, TFE_TEXT_VIEW_ERROR_NO_FILE,
                       "No file is set in TfeTextView.");
  return FALSE;
}
```

The function `g_error_set_literal` creates a GError structure and sets the domain, code, and message with the second to fourth arguments. Furthermore, it updates err to point to the GError. If you want to create a message using printf type arguments, use `g_eror_set` instead.

Refer to the files `tfetextview.h` and `tfetextview.c` in src/tfe5 for further information.

2.11 Completing the custom object implementation

The TfeTextView class will be finally completed in this section.

The source files are in the directory src/tfe5. You can get them by downloading the repository.

2.11.1 Header File

The header file `tfetextview.h` includes:

- The macro definition for TfeTextView type (`TFE_TYPE_TEXT_VIEW`).
- The macro `G_DECLARE_FINAL_TYPE`.
- GError domain and code definition and declaration.
- Public function declarations. They are instance methods and constructors.

```
1  #pragma once
2
3  #include <gtk/gtk.h>
4
5  #define TFE_TYPE_TEXT_VIEW tfe_text_view_get_type ()
6  G_DECLARE_FINAL_TYPE (TfeTextView, tfe_text_view, TFE, TEXT_VIEW, GtkTextView)
7
8  /* User Error Definition */
9  #define TFE_TEXT_VIEW_ERROR (tfe_text_view_error_quark ())
10 GQuark tfe_text_view_error_quark (void);
11 typedef enum {
12   TFE_TEXT_VIEW_ERROR_NO_FILE,    /* No file is set */
13   TFE_TEXT_VIEW_ERROR_FAILED,    /* Other invalid state */
14 } TfeTextViewError;
15
16 GFile *
17 tfe_text_view_get_file (TfeTextView *tv);
18
19 void
20 tfe_text_view_set_file (TfeTextView *tv, GFile *file);
21
22 gboolean
23 tfe_text_view_read (TfeTextView *tv, GError **err);
24
25 gboolean
26 tfe_text_view_write (TfeTextView *tv, GError **err);
27
28 GtkWidget *
29 tfe_text_view_new (void);
```

```
30
31 GtkWidget *
32 tfe_text_view_new_with_file (GFile *file, GError **err);
```

- 1: The preprocessor directive `#pragma once` makes the header file be included only once. It is non-standard but widely used.
- 3: Includes gtk4 header files.
- 5: Defines TfeTextView type (`TFE_TYPE_TEXT_VIEW`).
- 6: The macro `G_DECLARE_FINAL_TYPE` defines the class structure and some useful functions.
  - `TfeTextView` and `TfeTextViewClass` are declared as typedef of C structures.
  - You need to define a structure `_TfeTextView` later.
  - The class structure `_TfeTextViewClass` is defined here. You don't need to define it by yourself.
  - Convenience functions `TFE_TEXT_VIEW ()` for casting and `TFE_IS_TEXT_VIEW` for type check are defined.
- 8-14: Custom GError domain and code definition as you learned in the previous section.
- 16-32: Declarations of public functions on TfeTextView.

The content of header files are open to C source files. So, the functions here are public functions. Don't write any private functions (functions for only the corresponding C file) here.

2.11.2 Constructors

A TfeTextView instance is created with `tfe_text_view_new` or `tfe_text_view_new_with_file`. These functions are called constructors.

```
1  GtkWidget *
2  tfe_text_view_new (void) {
3    return GTK_WIDGET (g_object_new (TFE_TYPE_TEXT_VIEW, "wrap-mode", GTK_WRAP_WORD_CHAR, NULL));
4  }
5
6  GtkWidget *
7  tfe_text_view_new_with_file (GFile *file, GError **err) {
8    g_return_val_if_fail (G_IS_FILE (file) || file == NULL, NULL);
9    g_return_val_if_fail (err == NULL || *err == NULL, NULL);
10
11   GtkWidget *tv;
12
13   tv = tfe_text_view_new();
14   if (file == NULL)
15     return tv;
16   tfe_text_view_set_file (TFE_TEXT_VIEW (tv), file);
17   if (tfe_text_view_read (TFE_TEXT_VIEW (tv), err))
18     return tv;
19   /* An error occurs during the read operation */
20   g_object_ref_sink (tv);
21   g_object_unref (tv);
22   return NULL;
23 }
```

- 1-4: `tfe_text_view_new` function. It just returns the value from the function `g_object_new` and casts it to the pointer to GtkWidget. It is a GTK convention that constructors of GtkWidget subclasses return a `GtkWidget *`. The function `g_object_new` creates any instances of its descendant class. The arguments are the type of the class, property list and NULL, which is the end mark of the property list. Here, the property "wrap-mode" is set to `GTK_WRAP_WORD_CHAR` for breaking lines in between words, or if that is not enough, also between characters (graphemes). The default value of the property "wrap-mode" in GtkTextView (not TfeTextView) is `GTK_WRAP_NONE` (no wrapping).

- 6-23: `tfe_text_view_new_with_file` function.
- 8: `g_return_val_if_fail` tests whether the argument `file` is a pointer to GFile. If it's true, the program goes on to the next line. If it's false, it returns NULL (the second argument) immediately. And at the same time it logs out the error message (usually the log is outputted to stderr).
- 9: Another `g_return_val_if_fail` tests `err`. This can be NULL when the caller does not need detailed error information. Otherwise it must point to NULL (Note: Being NULL and pointing to NULL is totally different).
- 13: Creates a TfeTextView instance.
- 14-15: If `file` is NULL, just returns `tv`.
- 16: sets the file of the TfeTextView instance to `file`.
- 17-18: If the function `tfe_text_view_read` successfully reads the content of the file, returns the TfeTextView instance.
- 20-22: If an error happens during the read process, Releases `tv` and returns NULL. The function `g_object_ref_sink` will be explained in the next subsection.

2.11.3 Floating References

All widgets are derived from GInitiallyUnowned. While GObject and GInitiallyUnowned are very similar, they differ in their initial reference count behavior.

When an instance of GInitiallyUnowned is created, it has a "floating reference", whereas a GObject instance has a "normal reference". The reference count of both the objects is one, but the type of the references is different.

The descendants of GInitiallyUnowned inherit this behavior, so every widget has a floating reference immediately after the creation. On the other hand, non-widget classes, such as GtkTextBuffer, are direct subclasses of GObject and do not have a floating reference.

The function `g_object_ref_sink (object)` behaves:

- If the object has a floating reference, it converts the floating reference to the normal reference. The reference count will be one.
- If the object does not have a floating reference, it increases the normal reference count by one. In this case, `g_object_ref_sink (object)` behaves the same as `g_object_ref (object)`.

The function is used when an widget is added to another widget as a child.

```
GtkTextView *tv = gtk_text_view_new (); /* Floating reference */
GtkScrolledWindow *scr = gtk_scrolled_window_new ();
gtk_scrolled_window_set_child (scr, tv); /* Scrolled window sinks the tv's floating and tv's normal
```

This program works because `gtk_scrolled_window_set_child` uses `g_object_ref_sink` instead of `g_object_ref`. If `tv` does not have a floating reference, the program needs to decrement the `tv` reference count:

```
GtkTextView *tv = gtk_text_view_new ();  /* Floating reference: tv is not owned by anyone */
g_object_ref_sink (tv); /* Normal reference count == 1: this program owns tv. */
GtkScrolledWindow *scr = gtk_scrolled_window_new ();
gtk_scrolled_window_set_child (scr, tv); /* tv's reference count becomes 2. */
/* At this moment, both of the caller of the function and GtkScrolledWindow instance refer to tv. */
/* But the caller does not use tv any more. So, it releases tv using g_object_unref. */
g_object_unref (tv); /* tv's reference count becomes 1 because the caller decrements tv's reference
```

From the comparison of these two programs, we can say that floating reference makes the program simpler because we don't need to release `tv`.

Conversely, it gets more complicated to release an InitiallyUnowned instance just after the creation. Because we cannot use `g_object_unref` to floating object.

```
GtkTextView *tv = gtk_text_view_new (); /* Floating reference */
/* We need to convert the floating reference to the normal reference before releasing it. */
g_object_ref_sink (tv); /* Floating reference is converted to normal reference. The reference count
g_object_unref (tv); /* Now reference count becomes zero and tv will be destroyed. */
```

See GObject API reference - g_object_ref_sink for further information.

2.11.4 Instance Methods

2.11.4.1 Getting and setting the GFile

A TfeTextView instance holds the GFile instance in it. External code can access the GFile with `tfe_text_view_get_file` and `tfe_text_view_set_file`.

```
1  GFile *
2  tfe_text_view_get_file (TfeTextView *tv) {
3    g_return_val_if_fail (TFE_IS_TEXT_VIEW (tv), NULL);
4
5    /* The instance holds the ownership of the returned GFile. */
6    return tv->file;
7  }
8
9  void
10 tfe_text_view_set_file (TfeTextView *tv, GFile *file) {
11   g_return_if_fail (TFE_IS_TEXT_VIEW (tv));
12   g_return_if_fail (G_IS_FILE (file) || file == NULL);
13
14   GtkTextBuffer *tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
15
16   /* The instance gets the ownership of the new GFile by increment the reference count. */
17   g_set_object (&tv->file, file);
18   gtk_text_buffer_set_modified (tb, TRUE);
19 }
```

There are two new functions in the code.

- `g_set_object (&tv->file, file)`: Updates `tv->file` to refer to `file`. It increments the reference count of the new `file` (if non-NULL), decrements the reference count of the current `tv->file` (if non-NULL), and updates the pointer. This reference counting ensures that the instance releases the old `GFile` and assumes ownership of the new one.
- `gtk_text_buffer_set_modified (tb, TRUE)`: Sets the "modified" flag to `TRUE`. `GtkTextBuffer` maintains an internal flag to track whether its content has been modified. A value of `TRUE` indicates that the buffer has been modified since it was last saved. When `tfe_text_view_set_file` changes the `GFile`, the content is considered unsaved, so the modified flag must be set to `TRUE`.

2.11.4.2 Ownership of Data

When working with getter and setter functions, understanding data "ownership" is crucial.

For "get" functions, there are two patterns for handling the ownership of the returned object:

- **The caller gets ownership:** In this case, the documentation usually states something like, "The caller of the method takes ownership of the returned data, and is responsible for freeing it." Example: `g_list_model_get_item`.
- **The caller does NOT get ownership:** In this case, the documentation typically says, "The returned data is owned by the instance." Example: `gtk_text_view_get_buffer`.

If you are given ownership, you must free or unreference the data when you no longer need it. Conversely, if you are not given ownership, you must **never** free it.

2.11.4.2.1 The `const` Qualifier as an Ownership Hint for Strings For string data, the `const` qualifier serves as a very helpful built-in hint for ownership in C:

- **Get functions:** If a function returns a `const char *`, it means the caller does not get ownership. You must not free it.
- **Set functions:** If a function argument is `const char *`, it means the instance will not take ownership of your string (it will usually make its own internal copy if it needs the ownership).

This convention is standard throughout the GTK and GLib documentation, and you should apply this same `const` rule when writing your own programs to make ownership clear.

2.11.4.2.2 How to Acquire Ownership Manually But what if you aren't given ownership of the return value, yet you need to keep the data for a while? You must explicitly acquire ownership yourself:

- For objects with a **normal reference**, use `g_object_ref` to increase the reference count by one.
- For objects that might have a **floating reference**, use `g_object_ref_sink`. This converts a floating reference into a normal one, or simply increments the reference count if it's already a normal reference.
- For data without reference counts, such as **strings** (especially those returned as `const char *`), you need to make a copy. Use `g_strdup` to duplicate the string, and later free it using `g_free`.
- **Note:** In the method above, the caller and the instance share the object. Therefore, any modification made by one affects the other. In contrast, the string is duplicated rather than shared. Because the caller and the instance own their respective copies, changing one string will not affect the other.

In the case of `tfe_text_view_get_file`, the function simply returns `tv->file` without increasing its reference count. It does not give you ownership.

2.11.4.2.3 Ownership in "Set" Functions On the other hand, in "set" functions, the instance receiving the data usually claims its own ownership by calling `g_object_ref` or `g_object_ref_sink` internally. This means the caller's ownership remains unchanged before and after the function call. For example, `tfe_text_view_set_file (tv, file)` does not change the caller's ownership of the `file`.

However, be aware that some GTK and GLib functions completely transfer ownership. For example, `GValue` has a function called `g_value_take_object (GValue* value, gpointer v_object)`. In this case, the ownership of `v_object` moves entirely to `value`. The caller no longer owns it and must **not** free it after setting it.

In short, always pay close attention to the ownership of arguments and return values to avoid memory bugs.

2.11.4.3 Reading the File Content

The function `tfe_text_view_read` reads the file content into the buffer.

```
1  gboolean
2  tfe_text_view_read (TfeTextView *tv, GError **err) {
3    g_return_val_if_fail (TFE_IS_TEXT_VIEW (tv), FALSE);
4    g_return_val_if_fail (err == NULL || *err == NULL, FALSE);
5
6    GtkTextBuffer *tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
7    gboolean stat;
8    char *contents;
9    gsize length;
10
```

```
11    if (! G_IS_FILE (tv->file)) {
12      g_set_error_literal (err, TFE_TEXT_VIEW_ERROR, TFE_TEXT_VIEW_ERROR_NO_FILE, "No file is set in T
13      return FALSE;
14    }
15
16    stat = g_file_load_contents (tv->file, NULL, &contents, &length, NULL, err);
17    if (stat) {
18      if (length > G_MAXINT) { /* File is too large to put it in the buffer*/
19        g_set_error_literal (err, TFE_TEXT_VIEW_ERROR, TFE_TEXT_VIEW_ERROR_FAILED, "The file is too la
20        stat = FALSE;
21      }
22      else if (!g_utf8_validate (contents, length, NULL)) { /* Only valid UTF-8 strings are accepted *
23        g_set_error_literal (err, TFE_TEXT_VIEW_ERROR, TFE_TEXT_VIEW_ERROR_FAILED, "The file is not va
24        stat = FALSE;
25      }
26      else {
27        gtk_text_buffer_set_text (tb, contents, (int) length);
28        gtk_text_buffer_set_modified (tb, FALSE);
29      }
30      g_free (contents);
31    }
32    return stat;
33  }
```

- 11-14: If `tv->file` is `NULL`, the function sets `err` and returns `FALSE`. `g_set_error_literal` creates a `GError` structure with the domain, code, and message. Then it sets `*err` to point to the `GError`.
- 16: `g_file_load_contents` reads the file content into memory. If an error occurs, it returns `FALSE` and sets `err` to a newly created `GError`. See the GIO documentation for details.
- 17-31: After successfully loading the content, the function performs two checks before updating the GtkTextBuffer. First, it checks the file size. Since the maximum capacity of the buffer is `G_MAXINT` (the upper limit of the int type), if length exceeds this value, `err` is set and `stat` becomes FALSE. Second, it validates the UTF-8 encoding. The `g_utf8_validate` function ensures the content is a valid UTF-8 string. If it is not, `err` is set and `stat` becomes FALSE. If both checks pass, the function updates the buffer with the loaded text and sets the modified flag to FALSE, as the buffer now exactly matches the file. Finally, the memory for `contents` is freed.

2.11.4.4 Writing the buffer content

The function `tfe_text_view_write` saves the content of the text buffer to the file.

```
1   gboolean
2   tfe_text_view_write (TfeTextView *tv, GError **err) {
3     g_return_val_if_fail (TFE_IS_TEXT_VIEW (tv), FALSE);
4     g_return_val_if_fail (err == NULL || *err == NULL, FALSE);
5
6     GtkTextBuffer *tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
7     GtkTextIter start_iter;
8     GtkTextIter end_iter;
9     char *contents;
10    gboolean stat;
11
12    if (! G_IS_FILE (tv->file)) {
13      g_set_error_literal (err, TFE_TEXT_VIEW_ERROR, TFE_TEXT_VIEW_ERROR_NO_FILE, "No file is set in T
14      return FALSE;
15    }
```

```
16
17    gtk_text_buffer_get_bounds (tb, &start_iter, &end_iter);
18    contents = gtk_text_buffer_get_text (tb, &start_iter, &end_iter, FALSE);
19    stat = g_file_replace_contents (tv->file, contents, strlen (contents), NULL, TRUE, G_FILE_CREATE_NO
20    g_free (contents);
21    if (stat)
22      gtk_text_buffer_set_modified (tb, FALSE);
23    return stat;
24  }
```

- 12-15: If `tv->file` is not set, it means there is no file to write to. The function sets `err` with a specific error message and returns `FALSE`.
- 17: To extract text from a `GtkTextBuffer`, you need to specify a range using **iterators**. A `GtkTextIter` is a structure that represents a specific position between characters within a buffer. `gtk_text_buffer_get_bounds` initializes `start_iter` to the first position in the buffer, and `end_iter` to the end.
- 18: `gtk_text_buffer_get_text` retrieves the text between the start and end iterators. Note that this function allocates memory for the returned string, meaning the caller takes ownership and is responsible for freeing it later. See the GIO documentation for details.
- 19: `g_file_replace_contents` writes the `contents` string to the file, replacing any existing data. It returns `TRUE` on success or `FALSE` on failure, and sets `err` appropriately if an error occurs.
- 20: Since the text has been written to the file and is no longer needed in memory, the function frees the `contents` using `g_free`.
- 21-22: If the write operation was successful (`stat` is `TRUE`), it sets the modified flag of the buffer to `FALSE`. This is because the content in the buffer is now safely saved and exactly matches the file.
- 23: Finally, it returns the status of the write operation.

2.11.5 Summary

We have successfully completed the `TfeTextView` class by implementing its header, constructors, and instance methods. We also learned essential GTK memory management concepts, such as floating references and data ownership. With our custom widget fully operational, we are now ready to integrate it into the main application.

2.12 File Dialogs and the GIO Asynchronous Model

2.12.1 What is Asynchronous Operation?

GUI applications, like those built with GTK, run in a continuous loop called the **main loop**. This loop constantly waits for user events — such as mouse clicks or key presses — and redraws the screen as needed.

Because the main loop must keep running quickly to keep the application responsive, you should never perform heavy or time-consuming tasks (like reading a large file or downloading data from the network) **directly inside it**. If you use a **synchronous** function for a slow I/O operation, the main loop will block and wait for the operation to finish. During this time, your application will freeze, and the user will not be able to click or type anything.

To solve this problem, we use **asynchronous** operations. When you call an asynchronous function, it immediately requests the slow operation to start in the background and instantly returns control to the main loop. The application remains responsive. Once the background operation is complete, the application is notified so it can handle the result.

2.12.2 The GIO Asynchronous Model

GIO is the underlying I/O library used by GTK. It provides a consistent, powerful model for handling asynchronous operations using a **Callback** mechanism.

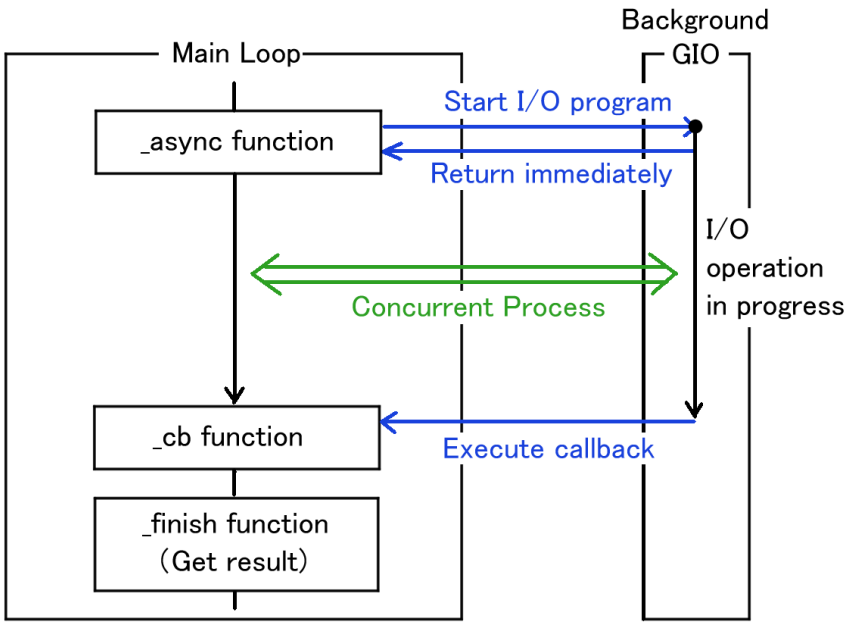


Figure 2.17: Async-Finish Pattern

The GIO asynchronous model always follows a strict pattern using a pair of functions: `_async` and `_finish`.

1. Start the operation (`_async`): You call a function ending in `_async` (e.g., `do_something_async`). You pass your parameters, along with a pointer to a callback function that you have written.
2. Wait in the background: The `_async` function returns immediately. The main loop continues running smoothly while GIO does the hard work in the background.
3. Execute the callback: When the operation completes (either successfully or with an error), GIO automatically calls your callback function.
4. Get the result (`_finish`): Inside your callback function, you must call the corresponding `_finish` function (e.g., `do_something_finish`). This function collects the actual result (like a `GFile`) or any `GError` that occurred during the process.

2.12.3 Getting a File Name with GtkFileDialog

In GTK4, `GtkFileDialog` is the modern way to ask the user to choose a file. It uses the native file chooser dialog of the user's operating system or desktop environment and is designed entirely around the GIO asynchronous model.

There are two main operations when dealing with files:

- Opening a file: Uses `gtk_file_dialog_open_async` and `gtk_file_dialog_open_finish`.
- Saving a file: Uses `gtk_file_dialog_save_async` and `gtk_file_dialog_save_finish`.

Here is a practical code example of how to show an "Open" file dialog and handle the user's selection using the `_async` and `_finish` pattern.

2.12.3.1 The Callback Function

First, we define the callback function that will run when the user selects a file or cancels the dialog.

```
static void
open_dialog_cb (GObject *source_object, GAsyncResult *res, gpointer data) {
  /* The source_object is the GtkFileDialog that started the operation */
  GtkFileDialog *dialog = GTK_FILE_DIALOG (source_object);
  GError *err = NULL;
  GFile *file;

  /* Call the _finish function to get the result of the async operation */
  /* If an error occurs during the interaction process, the function returns NULL. */
  file = gtk_file_dialog_open_finish (dialog, res, &err);

  if (file != NULL) {
    /* The user selected a file successfully */
    g_print ("File selected: %s\n", g_file_get_path (file));

    /* We own the GFile, so we must unreference it when done */
    g_object_unref (file);
  } else {
    /* The user cancelled the dialog or an error occurred */
    g_print ("Dialog cancelled or error: %s\n", err->message);
    g_error_free (err);
  }
}
```

2.12.3.2 Setting up and showing the dialog

Next, we write the function that creates the dialog and starts the asynchronous operation.

```
static void
show_open_dialog (GtkWindow *parent_window) {
  /* Create a new GtkFileDialog instance */
  GtkFileDialog *dialog = gtk_file_dialog_new ();

  /* Start the asynchronous open operation.
   * We pass 'open_dialog_cb' as the callback function. */
  gtk_file_dialog_open_async (dialog,
                              parent_window,
                              NULL,           /* GCancellable (NULL means we won't cancel it program
                              open_dialog_cb, /* The callback function to run when done */
                              NULL);          /* User data to pass to the callback (NULL here) */
}
```

If you wanted to implement a “Save As” feature instead, the structure would be exactly the same. You would simply change `gtk_file_dialog_open_async` to `gtk_file_dialog_save_async`, and inside the callback, you would use `gtk_file_dialog_save_finish`.

With this pattern, your application remains fully responsive while the user browses their folders, and you safely receive the selected GFile once they are ready.

Keep in mind that GIO uses this exact same asynchronous pattern for almost all input and output operations, such as reading and writing files. Therefore, mastering this `_async` and `_finish` pattern is an absolutely essential skill for whenever you write any I/O-related code in the future.

2.12.4 Clicked Signal Handler on Open Button

Now, we will look at the TFE (Text File Editor) main program `tfeapplication.c`. First, we will focus on the "clicked" signal handler on "Open" button. The handler shows the file dialog then reads the file to the newly created TfeTextView instance.

```
1  static void
2  open_cb (GtkNotebook *nb) {
3    GtkFileDialog *dialog;
4    GtkWidget *win = gtk_widget_get_ancestor (GTK_WIDGET (nb), GTK_TYPE_WINDOW);
5
6    dialog = gtk_file_dialog_new ();
7    gtk_file_dialog_open (dialog, GTK_WINDOW (win), NULL, open_dialog_cb, nb);
8    g_object_unref (dialog);
9  }
```

The `open_cb` function is called when the "Open" button is clicked. It creates a file dialog and calls `gtk_file_dialog_open`, passing `open_dialog_cb` as the callback function. This starts a background operation that displays the dialog and waits for user input, while the function itself returns immediately to the caller. Once the user interaction is complete, the background operation invokes the callback.

```
1  static void
2  open_dialog_cb (GObject *source_object, GAsyncResult *res, gpointer data) {
3    GtkFileDialog *dialog = GTK_FILE_DIALOG (source_object);
4    GtkNotebook *nb = GTK_NOTEBOOK (data);
5    GtkWidget *win;
6    GFile *file;
7    GError *err = NULL;
8
9    if ((file = gtk_file_dialog_open_finish (dialog, res, &err)) == NULL) {
10     if (!g_error_matches (err, GTK_DIALOG_ERROR, GTK_DIALOG_ERROR_DISMISSED)) {
11       win = gtk_widget_get_ancestor (GTK_WIDGET (nb), GTK_TYPE_WINDOW);
12       tfe_error_alert (GTK_WINDOW (win), err);
13     }
14     g_clear_error (&err);
15     return;
16   }
17   notebook_page_new_with_file (nb, file);
18   g_object_unref (file);
19 }
```

- 9-15: Calls `gtk_file_dialog_open_finish` to retrieve the selected GFile. A NULL return value indicates that the operation failed or the dialog was dismissed by the user. If the error was not caused by cancellation, it displays an alert using `tfe_error_alert`. The function then clears the error and returns.
- 17: Calls `notebook_page_new_with_file` to create a new TfeTextView instance and build a new notebook page with it.
- 18: Frees `file`

The function `tfe_error_alert` is as follows. This is a private helper function.

```
1  static void
2  tfe_error_alert (GtkWindow *win, GError *err) {
3    GtkAlertDialog *alert_dialog;
4    GtkWindow *parent;
5
6    if (win != NULL && gtk_widget_get_mapped (GTK_WIDGET (win)))
```

```
7       parent = win;
8     else
9       parent = NULL;
10    alert_dialog = gtk_alert_dialog_new ("%s", err->message);
11    gtk_alert_dialog_show (alert_dialog, parent);
12    g_object_unref (alert_dialog);
13  }
```

- 6-9: The argument `win` can be NULL. Otherwise it must be the main window instance and also mapped (drawn on the screen). If it is not mapped, an error possibly occurs even if the window instance actually exists.
- 10: Creates an alert dialog.
- 11: Shows the dialog. The second argument specifies the "transient parent window." While passing NULL is generally discouraged, it is mandatory here. Since `tfe_error_alert` can be called from the "open" handler before the main window is mapped, passing NULL is the only safe way to display the dialog. See GTK documentation for further information about transient parent.
- 12: releases the alert dialog.

2.12.4.1 Page build Utility

A private function `notebook_page_new_with_file` creates a new TfeTextView instance and builds a new page in the GtkNotebook.

```
1   static void
2   notebook_page_new_with_file (GtkNotebook *nb, GFile *file) {
3     g_return_if_fail (GTK_IS_NOTEBOOK (nb));
4     g_return_if_fail (G_IS_FILE (file) || file == NULL);
5
6     GtkWidget *win;
7     GtkNotebookPage *nbp;
8     GtkWidget *scr;
9     GtkWidget *tv;
10    GtkWidget *lab;
11    int i;
12    GError *err = NULL;
13
14    if ((tv = tfe_text_view_new_with_file (file, &err)) == NULL) {
15      win = gtk_widget_get_ancestor (GTK_WIDGET (nb), GTK_TYPE_WINDOW);
16      tfe_error_alert (GTK_WINDOW (win), err);
17      g_clear_error (&err);
18      return;
19    }
20    lab = tfe_label_from_file (file); /* lab is floating. lab can be NULL */
21    scr = gtk_scrolled_window_new ();
22    gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), GTK_WIDGET (tv));
23    i = gtk_notebook_append_page (nb, scr, lab);
24    nbp = gtk_notebook_get_page (nb, scr);
25    g_object_set (nbp, "tab-expand", TRUE, NULL);
26    gtk_notebook_set_current_page (nb, i);
27  }
```

- 14-19: Calls `tfe_text_view_new_with_file` to create a new TfeTextView with the GFile `file`. The argument `file` can be NULL and the function returns an empty TfeTextView instance. If an error occurs, it calls `tfe_error_alert` to show the error message and returns.
- 20: Calls `tfe_label_from_file`. This function creates a GtkLabel instance with the filename

obtained from the GFile. If the argument (GFile) is NULL, the function returns NULL. -21-22: Creates a scrolled window instance and sets the child to `tv`.

- 23: Appends the scroll window and label to the notebook as the child and tab respectively.
- 24-25: Sets the "tab-expand" property of the notebook page to TRUE using `g_object_set`
- 26: Sets the current page to the new page.

2.12.5 Clicked Signal Handler on Save Button

Next, we will go on to the "clicked" signal handler on the "Save" button.

```
1  static void
2  save_cb (GtkNotebook *nb) {
3    int i;
4    GtkWidget *win;
5    GtkWidget *scr;
6    GtkWidget *tv;
7    GError *err = NULL;
8    GtkFileDialog *dialog;
9    GtkAlertDialog *alert_dialog;
10
11   i = gtk_notebook_get_current_page (nb);
12   win = gtk_widget_get_ancestor (GTK_WIDGET (nb), GTK_TYPE_WINDOW);
13
14   if (i == -1) {
15     alert_dialog = gtk_alert_dialog_new ("No page to save.");
16     gtk_alert_dialog_show (alert_dialog, GTK_WINDOW (win));
17     g_object_unref (alert_dialog);
18     return;
19   }
20
21   scr = gtk_notebook_get_nth_page (nb, i);
22   tv = gtk_scrolled_window_get_child (GTK_SCROLLED_WINDOW (scr));
23
24   if (tfe_text_view_get_file (TFE_TEXT_VIEW (tv))) { /* File is already set */
25     if (!tfe_text_view_write (TFE_TEXT_VIEW (tv), &err)) {
26       tfe_error_alert (GTK_WINDOW (win), err);
27       g_clear_error (&err);
28     }
29   } else {
30     dialog = gtk_file_dialog_new ();
31     gtk_file_dialog_save (dialog, GTK_WINDOW (win), NULL, save_dialog_cb, tv);
32     g_object_unref (dialog);
33   }
34 }
```

- 11-19: If the GtkNotebook has no page, it shows the alert dialog with "No page to save." and returns.
- 24-28: If the TfeTextView holds a GFile, it just writes the content of the buffer to the file.
- 30-32: If the TfeTextView does not hold a GFile, it shows a Filedialog. The callback function is `save_dialog_cb`.

The `save_dialog_cb` function is shown below.

```
1  static void
2  save_dialog_cb (GObject *source_object, GAsyncResult *res, gpointer data) {
3    GtkFileDialog *dialog = GTK_FILE_DIALOG (source_object);
```

```
4   GtkWidget *tv = GTK_WIDGET (data);
5   GtkWidget *win;
6   GtkWidget *scr;
7   GtkWidget *nb;
8   GtkWidget *lab;
9   GFile *file;
10  GError *err = NULL;
11
12  win = gtk_widget_get_ancestor (tv, GTK_TYPE_WINDOW);
13  if ((file = gtk_file_dialog_save_finish (dialog, res, &err)) == NULL) {
14    if (!g_error_matches (err, GTK_DIALOG_ERROR, GTK_DIALOG_ERROR_DISMISSED)) {
15      tfe_error_alert (GTK_WINDOW (win), err);
16    }
17    g_clear_error (&err);
18    return;
19  }
20  tfe_text_view_set_file (TFE_TEXT_VIEW (tv), file);
21  lab = tfe_label_from_file (file); /* lab is floating. lab can be NULL */
22  nb = gtk_widget_get_ancestor (tv, GTK_TYPE_NOTEBOOK);
23  scr = gtk_widget_get_parent (tv);
24  gtk_notebook_set_tab_label (GTK_NOTEBOOK (nb), scr, lab);
25  g_object_unref (file);
26
27  if (!tfe_text_view_write (TFE_TEXT_VIEW (tv), &err)) {
28    tfe_error_alert (GTK_WINDOW (win), err);
29    g_clear_error (&err);
30  }
31 }
```

- 13-19: Calls `gtk_file_dialog_save_finish` to get a GFile. If the return value is NULL and the user did not cancel the operation (`GTK_DIALOG_ERROR_DISMISSED`), it calls `tfe_error_alert` to show the error dialog.
- 20: Sets the file of `tv` to the GFile that is returned by `gtk_file_dialog_save_finish`.
- 21-25: Sets the tab of the notebook page to the new filename. The Gfile instance is released.
- 27-30: Calls `tfe_text_view_write` to write the content of the buffer to the file. If an error occurs, it calls `tfe_error_alert` to show the error dialog. Then, it frees GError.

2.13 Completing the Main Program

The file `tfeapplication.c` is the main program of Tfe. Its main roles are:

- Application initialization, including command-line support (opening files from the terminal).
- Building the user interface using a UI resource file.
- Managing multiple text views with tabs (GtkNotebook).
- Providing signal handlers for buttons and file dialog operations.

2.13.1 The Function main

The program `tfe` is executed from the command line.

```
$ tfe file1 file2 ...
```

The function `main` is called first.

```
1 int
2 main (int argc, char **argv) {
```

```
3   GtkApplication *app;
4   int stat;
5
6   app = gtk_application_new (APPLICATION_ID, G_APPLICATION_HANDLES_OPEN);
7   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
8   g_signal_connect (app, "open",     G_CALLBACK (app_open),     NULL);
9   stat = g_application_run (G_APPLICATION (app), argc, argv);
10  g_object_unref (app);
11  return stat;
12 }
```

- 6: Creates a GtkApplication object. `APPLICATION_ID` is defined as "com.github.ToshioCP.tfe5" before the `main` function. The second argument `G_APPLICATION_HANDLES_OPEN` is a flag and that means that the application accepts commandline arguments but only pathnames.
- 7-8: Connects the "activate" and "open" signals to their handlers.
- 9: Runs the application.
- 10-11: Releases the application and returns the status.

2.13.2 Startup Signal Handler

A startup signal is emitted just after the GtkApplication instance is registered. The purpose of this handler is to initialize the application. This does not include window initialization, which is handled by the "activate" or "open" signal handlers.

This application does not need a "startup" handler for now, but it will be implemented in a later version.

2.13.3 Activate and open signal handler

The "activate" and "open" signal handlers are named `app_activate` and `app_open`, respectively. When the application is launched without any command-line arguments, the "activate" signal is emitted. Conversely, if arguments are provided, the "open" signal is emitted.

```
1  static GtkWidget *
2  tfe_get_notebook (GtkWidget *win) {
3    GtkWidget *boxv = gtk_window_get_child (GTK_WINDOW (win));
4    return gtk_widget_get_last_child (boxv);
5  }
6
7  void
8  app_activate (GApplication *application) {
9    GtkWidget *win = get_main_window (application);
10   GtkWidget *nb  = tfe_get_notebook (win);
11
12   notebook_page_new_with_file (GTK_NOTEBOOK (nb), NULL);
13   gtk_window_present (GTK_WINDOW (win));
14 }
15
16 void
17 app_open (GApplication *application, GFile **files, gint n_files, const gchar *hint) {
18   GtkWidget *win = get_main_window (application);
19   GtkWidget *nb  = tfe_get_notebook (win);
20   int i;
21
22   for (i = 0; i < n_files; i++)
23     notebook_page_new_with_file (GTK_NOTEBOOK (nb), files[i]);
```

```
24    if (gtk_notebook_get_n_pages (GTK_NOTEBOOK (nb)) == 0) /* No files were opened */
25      notebook_page_new_with_file (GTK_NOTEBOOK (nb), NULL);
26    gtk_window_present (GTK_WINDOW (win));
27  }
```

- 1-5: The private function `tfe_get_notebook` returns the notebook instance. This function is a private utility used by `app_activate` and `app_open`. This function relies on the assumption that the notebook is the last child of the box. Therefore, it will no longer work if another widget is added to the box in the future version. There are several ways to solve this issue, which will be explained in a later section.
- 7-14: `app_activate`.
- 9: The main window instance is obtained via the `get_main_window` function, which will be discussed later.
- 10: The notebook instance `nb` is obtained by calling the function `tfe_get_notebook`.
- 12: A new empty page is created, and finally, the main window is displayed. The `notebook_page_new_with_file` function will be explained later.
- 16-27: The `app_open` function.
- 22-23: In this for-loop, notebook pages are created with files using the `notebook_page_new_with_file` function. `files[i]` is the i-th commandline argument.
- 24-25: If no page has been created, maybe because of read error, then it creates an empty page.
- 26: Shows the window.

2.13.4 The Notebook page building function

The function `notebook_page_new_with_file` builds a new notebook page from a GFile. If the GFile is NULL, it creates a new empty page. The function is called by the "activate" and "open" handlers when the application starts.

```
1  static void
2  notebook_page_new_with_file (GtkNotebook *nb, GFile *file) {
3    g_return_if_fail (GTK_IS_NOTEBOOK (nb));
4    g_return_if_fail (G_IS_FILE (file) || file == NULL);
5
6    GtkWidget *win;
7    GtkNotebookPage *nbp;
8    GtkWidget *scr;
9    GtkWidget *tv;
10   GtkWidget *lab;
11   int i;
12   GError *err = NULL;
13
14   if ((tv = tfe_text_view_new_with_file (file, &err)) == NULL) {
15     win = gtk_widget_get_ancestor (GTK_WIDGET (nb), GTK_TYPE_WINDOW);
16     tfe_error_alert (GTK_WINDOW (win), err);
17     g_clear_error (&err);
18     return;
19   }
20   lab = tfe_label_from_file (file); /* lab is floating. lab can be NULL */
21   scr = gtk_scrolled_window_new ();
22   gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), GTK_WIDGET (tv));
23   i = gtk_notebook_append_page (nb, scr, lab);
24   nbp = gtk_notebook_get_page (nb, scr);
25   g_object_set (nbp, "tab-expand", TRUE, NULL);
26   gtk_notebook_set_current_page (nb, i);
```

```
27 }
```

- 14-19: Calls `tfe_text_view_new_with_file` to create a new TfeTextView instance. If it returns NULL, shows the error message through the alert dialog using `tfe_error_alert`. -20: Gets the GtkLabel instance with the filename. If no file is assigned to TfeTextView, `tfe_label_from_file` returns NULL.
- 21-22: assigns `tv` to the newly created scrolled window `scr`. Then, appends the scrolled window and the label to the notebook as a child and tab respectively.
- 23-25: Sets the page property "tab-expand" to TRUE.
- 26: Sets the current page to the new page.

2.13.5 Primary and Secondary Instances

Only one GApplication instance can be run at a time in a session. The session is a somewhat complex concept and also platform-dependent, but roughly speaking, it corresponds to a graphical desktop login. When you use your PC, you probably log in first, and then your desktop appears until you log out. This is the session.

However, Linux is a multi-process OS and you can run two or more instances of the same application. Isn't it a contradiction?

When the first instance is launched, it registers itself with its application ID (for example, "com.github.ToshioCP.tfe5"). Just after registration, the "startup" signal is emitted, followed by the "activate" or "open" signal, and finally, the instance's main loop starts.

If another instance with the same application ID is launched, it also tries to register itself. Because this is the second instance, the registration of the ID has already been done, so it fails. Because of the failure, the "startup" signal isn't emitted. After that, the "activate" or "open" signal is emitted in the primary instance, not in the second instance. The primary instance receives the signal and its handler is invoked. On the other hand, the second instance doesn't receive the signal and immediately quits.

2.13.5.1 Building a Single Window

The `activate` and `open` signal handlers are responsible for creating and displaying the window. However, because these handlers can be called multiple times, creating a window unconditionally would result in multiple main windows. Our TFE application is designed to have only one main window, using notebook pages to manage multiple files. Therefore, the handlers must follow this logic:

- If the application already has a window, use it.
- If it does not, create a new one.

Since the window is built from a UI file, creating it also involves instantiating child widgets and connecting button signals to their handlers. The `get_main_window` helper function takes care of all these steps.

```
1  static GtkWidget *
2  get_main_window (GApplication *application) {
3    GtkApplication *app = GTK_APPLICATION (application);
4    GList *windows = gtk_application_get_windows (app);
5    GtkWidget *win;
6    GtkWidget *nb;
7    GtkButton *btno;
8    GtkButton *btnn;
9    GtkButton *btns;
10   GtkButton *btnc;
11   GtkBuilder *build;
12
13   /* Return the existing window if one already exists */
14   if (windows)
```

```
15      return GTK_WIDGET (windows->data);
16
17    build = gtk_builder_new_from_resource ("/com/github/ToshioCP/tfe5/tfe.ui");
18    win  = GTK_WIDGET (gtk_builder_get_object (build, "win"));
19    gtk_window_set_application (GTK_WINDOW (win), app);
20    nb   = GTK_WIDGET (gtk_builder_get_object (build, "nb"));
21    btno = GTK_BUTTON (gtk_builder_get_object (build, "btno"));
22    btnn = GTK_BUTTON (gtk_builder_get_object (build, "btnn"));
23    btns = GTK_BUTTON (gtk_builder_get_object (build, "btns"));
24    btnc = GTK_BUTTON (gtk_builder_get_object (build, "btnc"));
25
26    g_signal_connect_swapped (btno, "clicked", G_CALLBACK (open_cb), nb);
27    g_signal_connect_swapped (btnn, "clicked", G_CALLBACK (new_cb), nb);
28    g_signal_connect_swapped (btns, "clicked", G_CALLBACK (save_cb), nb);
29    g_signal_connect_swapped (btnc, "clicked", G_CALLBACK (close_cb), nb);
30
31    g_object_unref (build);
32    return win;
33  }
```

- 4: An application can generally have multiple main windows. These are stored in a list, which can be retrieved using the `gtk_application_get_windows` function. The return type is a `GList *` pointer. A `GList` structure contains `data` (pointing to the window in this case), `next`, and `prev` pointers, forming a doubly linked list. If the list is empty, the function returns `NULL`. For more details, refer to the GLib Documentation - Doubly Linked Lists and GList.
- 14-15: If `windows` is not `NULL`, the application already has a window. The function simply returns this existing window (`windows->data`).
- 17-32: Otherwise, it uses `GtkBuilder` to generate all the widgets, registers the new window with the application, connects the "clicked" signals to their handlers, and finally returns the window.

2.13.5.2 Running and Observing Primary and Secondary Instances

For compiling, see "Build and Execute the Program" subsection below.

Try running two instances in a row:

```
$ ./_build/tfe & ./_build/tfe tfeapplication.c
$
```

First, the primary instance opens a window. Then, after the second instance is run, a new notebook page with the contents of `tfeapplication.c` appears in the primary instance's window. This is because the "open" signal is emitted in the primary instance. The second instance immediately quits, so the shell prompt soon appears.

2.13.6 New and CLose Button Signal Handlers

Open and save button handlers were explained in the previous section. Now we will explore the remaining handlers: the new and close handlers

```
1  static void
2  new_cb (GtkNotebook *nb) {
3    notebook_page_new_with_file (nb, NULL);
4  }
5
6  static void
7  close_cb (GtkNotebook *nb) {
8    if (gtk_notebook_get_n_pages (nb) >= 2)
```

```
9       gtk_notebook_remove_page (nb, gtk_notebook_get_current_page (nb));
10    else  /* If the page is the last page or no page exists */
11      gtk_window_destroy (GTK_WINDOW (gtk_widget_get_ancestor (GTK_WIDGET (nb), GTK_TYPE_WINDOW)));
12  }
```

- 1-4: Handles the “clicked” signal for the “New” button. The `new_cb` function simply calls `notebook_page_new_with_file` to create a new, empty page.
- 6-12: Handles the “clicked” signal for the “Close” button. If two or more pages exist, it removes the current page. If it is the last remaining page, the program destroys the window, which consequently quits the application.

2.13.7 meson.build

```
1  project('tfe', 'c')
2
3  gtkdep = dependency('gtk4')
4
5  gnome=import('gnome')
6  resources = gnome.compile_resources('resources','tfe.gresource.xml')
7
8  # Main application
9  sourcefiles=files('tfeapplication.c', 'tfetextview.c')
10 executable('tfe', sourcefiles, resources, dependencies: gtkdep)
11
12 # Test executable for tfetextview
13 test_sourcefiles = files('test/test_tfetextview.c', 'tfetextview.c')
14 test_exe = executable('test_tfetextview', test_sourcefiles, dependencies: gtkdep)
15
16 # Test registration
17 test('tfetextview_test', test_exe)
```

- 1-10: In this file, just the source file names are modified from the prior version.
- 12-17: These lines are for a test program for `tfetextview.c`. Test is not covered by this tutorial. If you want to know Glib test framework, See the GLib documentation.

2.13.8 Source Files

You can download the files from the repository.

The source files are under the /src/tfe5 directory.

2.13.9 Build and Execute the Program

You can build and execute the program using Meson and Ninja. To install the applications, type:

```
$ sudo apt install meson ninja-build
```

Change your current directory to `Gtk4-tutorial/src/tfe5` and type the following to build the program.

```
$ meson setup _build
$ ninja -C _build
```

To execute the program, type:

```
$ _build/tfe
```

You can add pathnames as arguments. A window will appear, containing four buttons and an editing area.

If you want to run the test program for `tfetextview.c`, type:

```
$ meson test -C _build
ninja: Entering directory `/home/username/Gtk4-tutorial/src/tfe5/_build'
ninja: no work to do.
1/1 tfetextview_test        OK              0.53s

Ok:                 1
Expected Fail:      0
Fail:               0
Unexpected Pass:    0
Skipped:            0
Timeout:            0

Full log written to /home/username/Gtk4-tutorial/src/tfe5/_build/meson-logs/testlog.txt
```

2.14 Menus and Actions

2.14.1 Menus

Users often use menus to give a command to the application. It is like this:

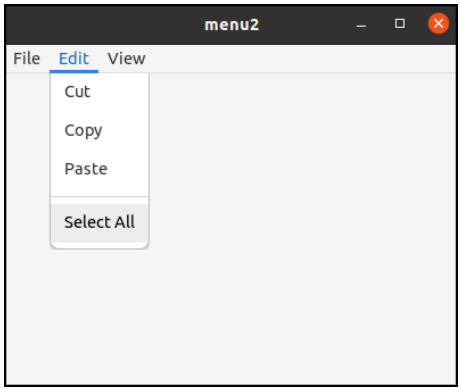


Figure 2.18: Menu

There are two types of objects.

- “File”, “Edit”, “View”, “Cut”, “Copy”, “Paste” and “Select All”. They are called “menu item” or simply “item”. When the user clicks one of these items, then something will happen.
- Menubar, submenu referenced by “Edit” item and two sections. They are called “menu”. Menu is an ordered list of items. They are similar to arrays.

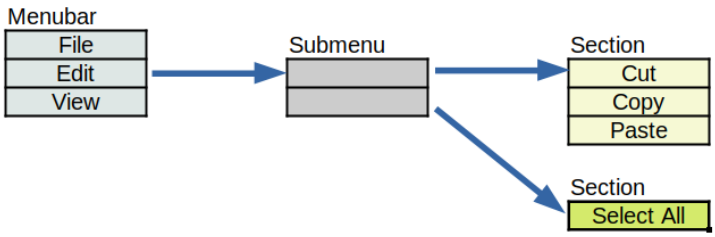


Figure 2.19: Menu structure

- Menubar has three items, which are “File”, “Edit” and “View”.
- The menu item labeled “Edit” has a link to the submenu which has two items. These two items don’t have labels. Each item refers to a section.

- The first section has three items – “Cut”, “Copy” and “Paste”.
- The second section has one item – “Select All”.

Menus can build a complicated structure thanks to the links of menu items.

2.14.2 GMenuModel, GMenu and GMenuItem

GMenuModel is an abstract object which represents a menu. GMenu is a simple implementation of GMenuModel and a child object of GMenuModel.

```
GObject -- GMenuModel -- GMenu
```

Because GMenuModel is an abstract object, it isn’t instantiatable. Therefore, it doesn’t have any functions to create its instance. If you want to create a menu, use `g_menu_new` to create a GMenu instance. GMenu inherits all the GMenuModel functions.

GMenuItem is an object directly derived from GObject. GMenuItem and Gmenu (or GMenuModel) don’t have a parent-child relationship.

```
GObject -- GMenuModel -- GMenu
GObject -- GMenuItem
```

GMenuItem has attributes. One of the attributes is label. For example, One of the menu item in the diagram above has “Edit” label. “Cut”, “Copy”, “Paste” and “Select All” are also the labels of the menu items. Other attributes will be explained later.

Some menu items have a link to another GMenu. There are two types of links, submenu and section.

GMenuItem can be inserted, appended or prepended to GMenu. When it is inserted, all of the attributes and link values are copied and stored in the menu. The GMenuItem itself is not really inserted. Therefore, after the insertion, GMenuItem is useless and it should be freed. The same goes for appending or prepending.

The following code shows how to append GMenuItem to GMenu.

```
GMenu *menu = g_menu_new ();
GMenuItem *menu_item_quit = g_menu_item_new ("Quit", "app.quit");
g_menu_append_item (menu, menu_item_quit);
g_object_unref (menu_item_quit);
```

2.14.3 Menu and Action

One of the menu item attributes is an action and this attribute points to an action object.

There are two action objects, GSimpleAction and GPropertyAction. GSimpleAction is often used. And it is used with a menu item. This section only covers GSimpleAction.

An action that corresponds to a menu item will be activated when the menu item is clicked. Then the action emits an activate signal.

1. menu item is clicked.
2. The corresponding action is activated.
3. The action emits a signal.
4. The connected handler is invoked.

The following code is an example.

```
static void
quit_activated(GSimpleAction *action, GVariant *parameter, gpointer app) { ... ... ...}

GSimpleAction *act_quit = g_simple_action_new ("quit", NULL);
g_action_map_add_action (G_ACTION_MAP (app), G_ACTION (act_quit));
```

```
g_signal_connect (act_quit, "activate", G_CALLBACK (quit_activated), app);
GMenuItem *menu_item_quit = g_menu_item_new ("Quit", "app.quit");
```

- `act_quit` represents an action named "quit". The function `g_simple_action_new` creates a stateless action, meaning `act_quit` does not maintain any internal state. Stateless actions are like simple push buttons: when clicked, they perform an operation (like "Quit", "Copy", or "Paste") and nothing more. Most menu items are connected to stateless actions that take no parameters, which is exactly what the NULL argument specifies here. (Stateful actions, on the other hand, will be explained in the next section.)
- The action `act_quit` is added to the GtkApplication instance using `g_action_map_add_action`. This means the action's scope is application-wide, which is precisely what the "app" prefix in "app.quit" signifies.
- Finally, the "activate" signal of the action is connected to the callback handler `quit_activated`.
- The variable `menu_item_quit` points to a menu item (though we often simply say it is the menu item). It has the label "Quit" and is connected to the action "app.quit". Here, "app" is a prefix indicating that the action belongs to the GtkApplication instance, and "quit" is the name of the action. Therefore, "app.quit" refers to the `act_quit` action.

If the menu is clicked, the corresponding action "quit" will be activated and emits an "activate" signal. Then, the handler `quit_activated` is called.

2.14.4 Menu Bar

While modern applications often use menu buttons instead of traditional menu bars, the classic style is still widely used.

An application typically defines only one menu model. If an application has multiple windows with menu bars, they display exactly the same menus because every window creates its own menu bar instance based on the same shared menu model.

An application's menu bar typically remains unchanged once set. Therefore, the ideal place to configure it is in the startup handler, as this handler is called only once in the primary application instance.

It is helpful for readers to understand how application instances behave.

- When an application runs for the first time, this instance is called the primary instance.
- It registers itself with the system and, upon success, emits the "startup" signal.
- After that, an "activate" or "open" signal is emitted.
- If the application is launched again while the primary instance is running, this new instance is called a remote instance.
- A remote instance does not emit the "startup" signal.
- Instead of emitting "activate" or "open" signals locally, it forwards these events to the primary instance and then quits.

Therefore, the "activate" or "open" handlers can be called multiple times, whereas the "startup" handler is called exactly once. This is why the menu bar should be set in the startup handler.

```
static void
app_startup (GApplication *app) {
... ... ...
  gtk_application_set_menubar (GTK_APPLICATION (app), G_MENU_MODEL (menubar));
... ... ...
}
```

2.14.5 Simple Example

The following is a simple example of menus and actions. The source file `menu1.c` is located in the /src/menu directory.

```c
#include <gtk/gtk.h>

static void
quit_activated(GSimpleAction *action, GVariant *parameter, GApplication *application) {
  g_application_quit (application);
}

static void
app_activate (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GtkWidget *win = gtk_application_window_new (app);
  gtk_window_set_title (GTK_WINDOW (win), "menu1");
  gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);
  gtk_application_window_set_show_menubar (GTK_APPLICATION_WINDOW (win), TRUE);
  gtk_window_present (GTK_WINDOW (win));
}

static void
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);

  GSimpleAction *act_quit = g_simple_action_new ("quit", NULL);
  g_signal_connect (act_quit, "activate", G_CALLBACK (quit_activated), application);
  g_action_map_add_action (G_ACTION_MAP (app), G_ACTION (act_quit));
  g_object_unref (act_quit);

  GMenu *menubar = g_menu_new ();
  GMenuItem *menu_item_menu = g_menu_item_new ("Menu", NULL);
  GMenu *menu = g_menu_new ();
  GMenuItem *menu_item_quit = g_menu_item_new ("Quit", "app.quit");
  g_menu_append_item (menu, menu_item_quit);
  g_object_unref (menu_item_quit);
  g_menu_item_set_submenu (menu_item_menu, G_MENU_MODEL (menu));
  g_object_unref (menu);
  g_menu_append_item (menubar, menu_item_menu);
  g_object_unref (menu_item_menu);

  gtk_application_set_menubar (GTK_APPLICATION (app), G_MENU_MODEL (menubar));
  g_object_unref (menubar);
}

#define APPLICATION_ID "com.github.ToshioCP.menu1"

int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
  g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);

  stat = g_application_run (G_APPLICATION (app), argc, argv);
```

```
54    g_object_unref (app);
55    return stat;
56  }
```

- 3–6: `quit_activated` is the handler for the “activate” signal on the `act_quit` action. The “activate” signal handler takes three parameters:
  - Action: The action instance that emitted the signal.
  - Parameter: The parameter passed to the action. In this example, it is NULL because the second argument of `g_simple_action_new` (line 22) is NULL, so we can safely ignore it.
  - User Data: The user data, which is provided as the fourth argument to `g_signal_connect` (line 23) when connecting the signal to the handler.
- 5: The `g_application_quit` function immediately quits the application.
- 8–16: `app_activate` is the handler for the application’s “activate” signal.
- 11–13: Creates a GtkApplicationWindow named `win`, and sets its title and default size.
- 14: Instructs the GtkApplicationWindow to display the menu bar.
- 15: Shows the window on the screen.
- 18–40: `app_startup` is the handler for the application’s “startup” signal.
- 22: Creates a stateless GSimpleAction named `act_quit`. The first argument of `g_simple_action_new` is the action’s name, and the second is its parameter type. By passing NULL, we specify that the “quit” action takes no parameters.
- 23: Connects the “activate” signal of the action to the `quit_activated` handler.
- 24: Adds the action to the GtkApplication instance `app`. Because GtkApplication implements the GActionMap and GActionGroup interfaces, it can hold a group of actions using `g_action_map_add_action` (see the Gio API Reference). Since this action belongs to the application, its scope is “app”. Therefore, it is referred to as “app.quit” when a prefix is required.
- 25: Frees the initial reference to `act_quit`. Because `g_action_map_add_action` acquires its own reference to the action when it is added to the application, our local reference is no longer needed.
- 27–30: Creates the GMenu and GMenuItem instances. `menubar` and `menu` are GMenu objects, while `menu_item_menu` and `menu_item_quit` are GMenuItem objects. `menu_item_menu` has the label “Menu” but no action, whereas `menu_item_quit` has the label “Quit” and is linked to the “app.quit” action.
- 31–32: Appends `menu_item_quit` to `menu`. As discussed previously, the menu model copies all attributes and links to form a new item internally. Thus, the original `menu_item_quit instance` is no longer needed and is safely freed using `g_object_unref`.
- 33–34: Sets `menu` as a submenu of `menu_item_menu`. Once linked, the original `menu` is no longer needed and is freed.
- 35–36: Appends `menu_item_menu` to the `menubar` and then frees `menu_item_menu`. Through this process, the entire menu structure is built and anchored to the `menubar` variable. (The resulting menu structure is shown in the diagram below.)
- 38–39: The completed menubar model is set into the GtkApplication. Finally, the initial reference to `menubar` is freed, completing the startup initialization safely without memory leaks.

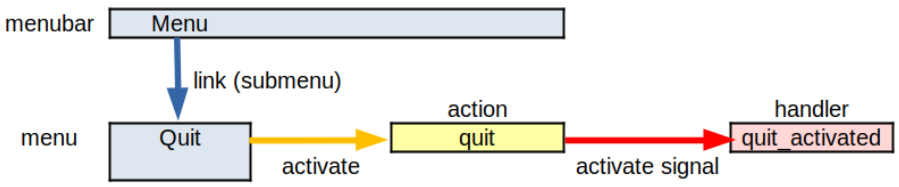


Figure 2.20: menu and action

2.14.6 Compiling and Running

Change your current directory to `src/menu`. Use `comp` to compile `menu1.c`.

```
$ bash comp menu1
$ ./a.out
```

A window will appear. Click "Menu" on the menu bar to open the drop-down menu. Click "Quit" to exit the application.

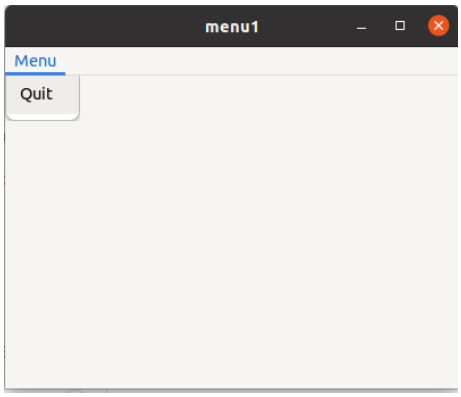


Figure 2.21: Screenshot of menu1

2.14.7 Testing Primary and Remote Application Instances

Let's try running the application twice simultaneously to see how multiple instances behave. You can do this elegantly by executing both commands on a single line using `&`.

```
$ ./a.out & ./a.out
```

Two identical windows will appear at once. Here is what happens behind the scenes:

- The First Command (./a.out &): This runs the application in the background and creates the primary instance. It executes the startup and activate handlers, displaying the first window.
- The Second Command (./a.out): This runs immediately after in the foreground, creating a remote instance. Because the primary instance is already running, the remote instance does not emit the startup signal. Instead, it forwards its activate signal to the primary instance and immediately quits.
- The Result: The primary instance receives the forwarded signal and executes its activate handler a second time, creating the second window.

Because both windows belong to the single primary instance and are generated from the same shared menu model, they look exactly the same. Finally, if you click the "Quit" item on either window's menu, the primary instance quits, and both windows will close simultaneously.

The reason a second window appeared in our test is specifically because of how our `activate` handler is written. Currently, it calls `gtk_application_window_new` every time it is invoked.

This behavior is entirely up to you as a developer. Depending on your application's design, you can change what happens during subsequent executions:

- **Single Window Design:** If you want your application to only ever have one window, you should create the window in the `startup` handler. Your `activate` handler would then only be responsible for presenting it (e.g., using `gtk_window_present`). With this design, a second execution will simply bring the existing window to the front instead of creating a new one.
- **Tabbed Interface Design:** Alternatively, you can design the `activate` handler to add a new notebook page (tab). For example, a text file editor (like `tfe`) might open a new blank tab in the existing window rather than spawning a second window. In practice, this is often a much better user experience.

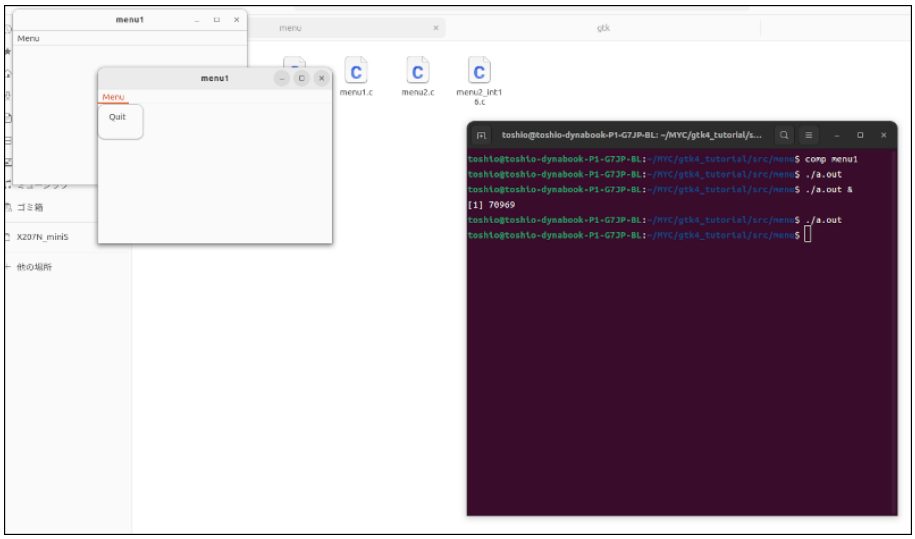


Figure 2.22: menu1 – two windows

In real-world desktop environments, users frequently launch applications multiple times, such as by double-clicking an icon repeatedly. Therefore, it is crucial to carefully consider how you divide your logic between the `startup` and `activate` (or `open`) handlers to ensure your application behaves exactly as intended.

2.15 Stateful Actions

Actions that maintain a state are called stateful actions. While these states are typically boolean or string values, other data types can also be used when necessary.

Stateful actions fall into two main categories:

- **Stateful actions without parameters:** Often used for toggle menus. The state is usually a boolean (TRUE or FALSE), and activating the action simply flips the state to its opposite value.
- **Stateful actions with parameters:** Used when an action can take on one of several distinct states. For example, a background color action might accept “red”, “green”, or “blue”. To change the state, the caller must pass the target value as a parameter.

2.15.1 Action Activation and State Changes

GIO provides two primary functions for working with actions:

- `g_action_activate()`: activates the action. This function emits the “activate” signal.
- `g_action_change_state()`: requests a state change on the action. Note that this function itself doesn’t change the state. This function just emits the “change-state” signal.

2.15.1.1 Menus and Shortcuts Always Go Through Activate

Menu items and keyboard shortcuts work by calling `g_action_activate()` internally. This means the starting point is always the `"activate"` signal — even for stateful actions. Menu items do **not** call `g_action_change_state()` directly.

2.15.1.2 Aciivate and Change-state Default Behavior

2.15.1.2.1 Forwarding Activate to Change-state So how do stateful actions work naturally with menus? `GSimpleAction` has a helpful default behavior built in.

If **no handler is connected** to the "activate" signal, `GSimpleAction` will automatically forward the activation to "change-state" (since GLib 2.40), under the following conditions:

Action type	Default behavior
Boolean state, no parameter type	Toggles the state via "change-state"
State type matches the parameter type (*)	Forwards directly to "change-state"

Note:

- This forwarding is specific to `GSimpleAction`. It is not a general feature of the `GAction` interface.
- (*) If the caller's parameter type doesn't match the state type of the action, no signal will be emitted.

2.15.1.2.2 change-state's Default Behavior When "change-state" signal is emitted and no handler is connected, GSimpleAction will automatically call `g_simple_action_set_state()` to apply the requested value. This means that for simple use cases where no validation is needed, you don't have to write any handler at all — the state just updates itself.

2.15.1.3 A Recommended Pattern: Put Your Logic in change-state

Taking advantage of this default behavior leads to a clean and robust design pattern:

Leave `"activate"` unhandled, and connect your logic to `"change-state"` instead.

```
static void
on_change_state (GSimpleAction *action,
                 GVariant      *value,
                 gpointer       user_data)
{
    /* Optionally validate the requested value here */
    g_simple_action_set_state (action, value);  /* Apply the new state */
    /* Perform tasks related to the state change */
}

g_signal_connect (action, "change-state", G_CALLBACK (on_change_state), NULL);
```

With this pattern, activations from menus, keyboard shortcuts, and direct calls to `g_action_change_state()` all flow through the same handler consistently.

- **If you connect your own `"activate"` handler:** The automatic forwarding to `"change-state"` is disabled. You become responsible for handling the state update yourself.
- **If you connect your own `"change-state"` handler:** The default behavior of automatically calling `g_simple_action_set_state()` is also disabled. If you want the state to actually change, you must call `g_simple_action_set_state()` explicitly from within your handler. If you don't, the state will remain unchanged — which is intentional, as it allows you to validate or reject requests before applying them.

2.15.2 Stateful Action without Parameters

A common example of a stateful action without a parameter is a toggle menu. Its state, represented by a boolean value (TRUE or FALSE), changes each time the menu is clicked.

The following example demonstrates how to implement a toggle menu that updates a GtkLabel. The label is initialized to "OFF", and its text alternates between "ON" and "OFF" with each click.

This is a part of the `menu2.c` program, defining the toggle action and the menu item.

```
GSimpleAction *act_toggle_switch
  = g_simple_action_new_stateful ("toggle-switch", NULL, g_variant_new_boolean (FALSE));
g_signal_connect (act_toggle_switch, "change-state", G_CALLBACK (toggle_switch_changed), label);
g_action_map_add_action (G_ACTION_MAP (win), G_ACTION (act_toggle_switch));
... ... ...
GMenuItem *menu_item_toggle_switch = g_menu_item_new ("Toggle Switch", "win.toggle-switch");
```

`act_toggle_switch` is a GSimpleAction instance created with `g_simple_action_new_stateful`. This function takes three arguments:

- The first argument, "toggle-switch", is the name of the action.
- The second argument is the parameter type. NULL means this action does not take any parameters.
- The third argument is the initial state of the action. It is provided as a GVariant value (which will be explained in the next subsection). Here, `g_variant_new_boolean (FALSE)` sets the initial state to FALSE.

The action's "change-state" signal is connected to the handler `toggle_switch_changed`. Notice that we pass `label` (the GtkLabel) as the user_data so we can update its text later. As explained in the previous section, we intentionally leave the "activate" signal unconnected to take advantage of forwarding it to the "change-state" signal .

The action is added to the GtkWindow (`win`) using `g_action_map_add_action`. Therefore, the scope of this action is `win`.

`menu_item_toggle_switch` is a GMenuItem instance created with two arguments.

- The first argument, "Toggle Switch", is the label displayed on the menu.
- The second argument, "win.toggle-switch", is the action name. The prefix "win." indicates that the action belongs to the window.

```
1  static void
2  toggle_switch_changed(GSimpleAction *action, GVariant *value, gpointer user_data) {
3    GtkWidget *label = GTK_WIDGET (user_data);
4
5    if (g_variant_get_boolean (value))
6      gtk_label_set_text (GTK_LABEL (label), "ON");
7    else
8      gtk_label_set_text (GTK_LABEL (label), "OFF");
9    g_simple_action_set_state (action, value);
10 }
```

This is the signal handler connected to the "change-state" signal of the "toggle-switch" action.

The handler `toggle_switch_changed` has three parameters:

- The first parameter is the action that emits the "change-state" signal.
- The second parameter, value, is the requested new state of the action.
- The third parameter is the `user_data` that was passed to `g_signal_connect`. In this case, it is the pointer to the GtkLabel.

The handler checks the boolean value of the requested state. If it is TRUE, it sets the label's text to "ON". Otherwise, it sets the text to "OFF".

Finally, it sets the actual state of the action using `g_simple_action_set_state`.

Note: Before this function is called, the action still holds its original state. You must explicitly call `g_simple_action_set_state` to update the action to the new state.

As discussed in the previous section, by exclusively handling the "change-state" signal and omitting the handler for the "activate" signal, this single handler gracefully manages both menu clicks and any direct state-change requests.

2.15.3 GVariant and GVariantType

2.15.3.1 GVariant

Before moving on to the "Stateful Actions with Parameters" subsection, we will discuss GVariant, as these parameters are passed as GVariant instances. This subsection and the next subsection describes GVariant and GVariantType.

GVariant is a fundamental data type in GLib. Unlike many other classes in GTK, it is not a child of GObject. It acts as a strongly-typed container that can hold various types of data, such as booleans, strings, and numeric values (e.g., int16, int32, int64, double).

You might wonder why we need a specialized container like GVariant instead of using standard C types. There are three primary reasons:

- **Serialization and Portability:** GVariant is designed to be easily "serialized" (converted into a stream of bytes). This makes it perfect for storing data in files, sending data over a network (like D-Bus for inter-process communication), or passing parameters between different programming languages (language bindings).
- **Self-Describing Types:** In C, data types are defined at compile time, not at runtime. Consequently, a receiver cannot natively determine the data type of a generic pointer at runtime. In contrast, GVariant packages the data along with its type information, ensuring that the receiver always knows exactly what kind of data they are handling.
- **Flexibility in UI Actions:** In GTK, actions (like "Quit", "Copy", or "Change Color") are often shared across different UI elements (menus, shortcuts, buttons). Since these UI elements might pass different types of data (e.g., a color name as a string, or a zoom level as a double), GVariant serves as a universal, type-safe wrapper that allows any action to receive any data format safely.

The following code creates a GVariant holding a boolean value of TRUE:

```
GVariant *value = g_variant_new_boolean (TRUE);
```

Similarly, you can create a GVariant containing a string:

```
GVariant *value2 = g_variant_new_string ("Hello");
```

To extract the original data from a GVariant, you use the `g_variant_get_*` family of functions. For instance, you can retrieve the boolean value from our first example like this:

```
gboolean bool_val = g_variant_get_boolean (value); /* bool_val is now TRUE */
```

In the same way, you can extract the string from value2:

```
const char *str = g_variant_get_string (value2, NULL);
```

The second parameter of `g_variant_get_string` is a pointer to a `gsize` variable (`gsize` is defined as an `unsigned integer`). If you pass a valid pointer, the function stores the length of the string in that variable. If you pass NULL, the length is simply ignored.

Note: The returned string (`str`) is owned by the GVariant instance. It must not be modified or freed by the caller.

2.15.3.2 GVariantType

GVariantType represents the type of a GVariant. It is initialized using a type string. Type string examples are:

- "b": Boolean type.
- "s": String type.

See GLib documentation for details about type string.

The following program demonstrates how to create, use, and free a GVariantType instance. This example outputs the string “s”.

```c
#include <glib.h>

int
main (int argc, char **argv) {
  GVariantType *vtype = g_variant_type_new ("s");
  const char *type_string = g_variant_type_peek_string (vtype);
  g_print ("%s\n",type_string);
  g_variant_type_free (vtype);
}
```

- `g_variant_type_new` creates a GVariantType structure based on the provided type string (e.g., “s” for a string). The caller assumes ownership of the returned structure and must release it using `g_variant_type_free` when it is no longer needed.
- `g_variant_type_peek_string` retrieves the type string (in this case, “s”) from the GVariantType instance. The returned string is owned by the GVariantType instance itself, so the caller **must not** modify or free it.
- `g_print` outputs the type string to the terminal. Note that `vtype` must not be freed before this function is called, as the string pointer depends on the lifetime of the `vtype` instance.
- `g_variant_type_free` frees the GVariantType structure.

Alternatively, for standard types, GLib provides predefined macros. This is generally preferred as it avoids dynamic allocation.

```c
#include <glib.h>

int
main (int argc, char **argv) {
  const GVariantType *vtype = G_VARIANT_TYPE_STRING;
  g_print ("%s\n", g_variant_type_peek_string (vtype));
}
```

`G_VARIANT_TYPE_STRING` is a macro that provides a pointer to a pre-allocated GVariantType structure representing a string. Because this is a constant, it is managed by the system; you do not need to (and should not) call `g_variant_type_free` on it. Using these macros is more efficient and safer as it eliminates the risk of memory leaks or double-free errors.

2.15.4 Stateful Actions with Parameters

Next example of stateful actions is an action used with color-selection menus. Imagine three separate menus, each offering the options “red,” “green,” and “blue”. These menus are used to set the background color of a GtkLabel widget.

In this scenario, a single action can be connected to all three menus. The action maintains the selected color as its state, using a string value such as ‘red’, ‘green’, or ‘blue’ When a user makes a selection, the chosen color is passed as a parameter to the signal handler, which then updates the label accordingly.

```c
... ... ...
GSimpleAction *act_color
  = g_simple_action_new_stateful ("color", G_VARIANT_TYPE_STRING, g_variant_new_string ("red"));
GMenuItem *menu_item_red = g_menu_item_new ("Red", "app.color::red");
GMenuItem *menu_item_green = g_menu_item_new ("Green", "app.color::green");
GMenuItem *menu_item_blue = g_menu_item_new ("Blue", "app.color::blue");
GtkCssProvider *provider = gtk_css_provider_new ();
```

```
g_signal_connect (act_color, "change-state", G_CALLBACK (color_state_changed), provider);
... ... ...
```

- The variable `act_color` points to a GSimpleAction instance. It is created with `g_simple_action_new_stateful`.
  - The first argument, "color", is the name of the action.
  - The second argument is the parameter type. `G_VARIANT_TYPE_STRING` represents a string type for GVariant.
  - The third argument is the initial state of the action. `g_variant_new_string ("red")` returns a GVariant holding the string "red". GVariant uses a floating reference system; `g_simple_action_new_stateful` consumes this floating reference, so you do not need to manually release the GVariant instance.
- The variable `menu_item_red` points to a GMenuItem instance. The function `g_menu_item_new` takes two arguments: the label ("Red") and a detailed action name ("app.color::red"). The suffix after the `::` is the target, which is passed to the action as a parameter when the menu item is activated. The same logic applies to `menu_item_green` and `menu_item_blue`.
- The signal handler `color_state_changed` is connected to the "change-state" signal. This signal is emitted when the action's state is requested to be changed. Typically, clicking one of the menu items activates the action, and this activation is subsequently forwarded to the "change-state" signal. We pass `provider` to the handler so that the CSS can be updated accordingly. We will discuss the details of GtkCssProvider and how it handles CSS later.
- Signal connection: `g_signal_connect` links the "change-state" signal of `act_color` to the `color_state_changed` callback. When a menu item is selected, the handler is called with the new target value (the color).

```
1  static void
2  color_state_changed(GSimpleAction *action, GVariant *value, gpointer user_data) {
3    GtkCssProvider *provider = GTK_CSS_PROVIDER (user_data);
4    char *color = g_strdup_printf ("label.lb {background-color: %s; font-size: 72pt; color: white;}", g
5    /* Change the CSS data in the provider. */
6    /* Previous data is thrown away. */
7    gtk_css_provider_load_from_string (provider, color);
8    g_free (color);
9    g_simple_action_set_state (action, value);
10 }
```

- The handler takes three parameters.
  - action: The GSimpleAction that emitted the "change-state" signal.
  - value: The new state (a GVariant string) passed to the action, representing the selected color.
  - user_data: The data passed from `g_signal_connect`. In this case, it is the GtkCssProvider instance.
- We use `g_strdup_printf` to construct a CSS rule. `g_variant_get_string` extracts the raw string (e.g., "red") from the GVariant parameter. This color is then injected into the CSS rule "label.lb { background-color: %s; ... }".
- `gtk_css_provider_load_from_string` updates the provider with the new CSS rule. This automatically overrides any previous styles loaded into this provider. See the following "GTK CSS styling" subsection below.
- Finally, `g_simple_action_set_state` updates the action's internal state to the new value.

2.15.4.1 GTK CSS Styling

GTK uses a CSS-based styling system that works very similarly to web CSS (CSS is an abbreviation of Cascading Style Sheet). To style a widget, you typically follow these three steps:

- GtkCssProvider: This acts as a container for your CSS rules. You load your CSS strings or files into this provider.

- Display-wide Application: Register your GtkCssProvider with the GdkDisplay using `gtk_style_context_add_provider_for_display`. This applies the CSS rules globally to all widgets on that display. **GdkDisplay** represents your application's connection to the underlying windowing system. Because of this, it manages not only the output screens (monitors) but also the input devices (keyboards and mice) you use to interact with the app. The **"default" display** is simply the primary connection your computer assigned when the app launched. Standard applications only use this default connection. We call `gdk_display_get_default()` to access this main connection. In a web browser, CSS styles a single webpage. In GTK, adding a CSS provider to the GdkDisplay acts as a "global stylesheet". It ensures that **your CSS rules are automatically applied to every widget associated with that GdkDisplay**.
- Selectors: Like in HTML, you use selectors to target widgets (e.x. `label.lb`):
  - `label`: Targets the node name of all GtkLabel widgets.
  - `.lb`: Targets any widget that has the "lb" class added (using `gtk_widget_add_css_class`).
  - `label.lb`: Combines both, targeting only GtkLabel widgets that also have the `lb` class.

By updating the GtkCssProvider dynamically via `gtk_css_provider_load_from_string`, you can change the look of your UI in real-time. Since the provider is registered at the display level, all widgets matching the selectors will automatically reflect the updated styles.

2.15.5 Example Program for Stateful Actions

This is an example program to demonstrate for the stateful actions.

```c
#include <gtk/gtk.h>

static void
toggle_switch_changed(GSimpleAction *action, GVariant *value, gpointer user_data) {
  GtkWidget *label = GTK_WIDGET (user_data);

  if (g_variant_get_boolean (value))
    gtk_label_set_text (GTK_LABEL (label), "ON");
  else
    gtk_label_set_text (GTK_LABEL (label), "OFF");
  g_simple_action_set_state (action, value);
}

static void
color_state_changed(GSimpleAction *action, GVariant *value, gpointer user_data) {
  GtkCssProvider *provider = GTK_CSS_PROVIDER (user_data);
  char *color = g_strdup_printf ("label.lb {background-color: %s; font-size: 72pt; color: white;}", g
  /* Change the CSS data in the provider. */
  /* Previous data is thrown away. */
  gtk_css_provider_load_from_string (provider, color);
  g_free (color);
  g_simple_action_set_state (action, value);
}

static void
app_shutdown (GApplication *app, GtkCssProvider *provider) {
  gtk_style_context_remove_provider_for_display (gdk_display_get_default(), GTK_STYLE_PROVIDER (provi
  g_object_unref (provider);
}

static void
```

```c
app_activate (GApplication *app) {
  GtkWidget *win = gtk_application_window_new (GTK_APPLICATION (app));
  gtk_window_set_title (GTK_WINDOW (win), "menu2");
  gtk_window_set_default_size (GTK_WINDOW (win), 800, 600);

  GtkWidget *label = gtk_label_new ("OFF");
  gtk_widget_add_css_class (label, "lb"); /* the class is used by CSS Selector */
  gtk_window_set_child (GTK_WINDOW (win), label);

  GSimpleAction *act_toggle_switch
    = g_simple_action_new_stateful ("toggle-switch", NULL, g_variant_new_boolean (FALSE));
  g_signal_connect (act_toggle_switch, "change-state", G_CALLBACK (toggle_switch_changed), label);
  g_action_map_add_action (G_ACTION_MAP (win), G_ACTION (act_toggle_switch));
  g_object_unref (act_toggle_switch);
  gtk_application_window_set_show_menubar (GTK_APPLICATION_WINDOW (win), TRUE);

  gtk_window_present (GTK_WINDOW (win));
}

static void
app_startup (GApplication *app) {
  /* The provider below provides application wide CSS data. */
  GtkCssProvider *provider = gtk_css_provider_new ();
  GSimpleAction *act_color
    = g_simple_action_new_stateful ("color", G_VARIANT_TYPE_STRING, g_variant_new_string ("red"));
  GSimpleAction *act_quit
    = g_simple_action_new ("quit", NULL);
  g_signal_connect (act_color, "change-state", G_CALLBACK (color_state_changed), provider);
  g_signal_connect_swapped (act_quit, "activate", G_CALLBACK (g_application_quit), app);
  g_action_map_add_action (G_ACTION_MAP (app), G_ACTION (act_color));
  g_action_map_add_action (G_ACTION_MAP (app), G_ACTION (act_quit));
  g_object_unref (act_color);
  g_object_unref (act_quit);

  GMenu *menubar = g_menu_new ();
  GMenu *menu = g_menu_new ();
  GMenu *section1 = g_menu_new ();
  GMenu *section2 = g_menu_new ();
  GMenu *section3 = g_menu_new ();
  GMenuItem *menu_item_toggle_switch = g_menu_item_new ("Toggle Switch", "win.toggle-switch");
  GMenuItem *menu_item_red = g_menu_item_new ("Red", "app.color::red");
  GMenuItem *menu_item_green = g_menu_item_new ("Green", "app.color::green");
  GMenuItem *menu_item_blue = g_menu_item_new ("Blue", "app.color::blue");
  GMenuItem *menu_item_quit = g_menu_item_new ("Quit", "app.quit");

  g_menu_append_item (section1, menu_item_toggle_switch);
  g_menu_append_item (section2, menu_item_red);
  g_menu_append_item (section2, menu_item_green);
  g_menu_append_item (section2, menu_item_blue);
  g_menu_append_item (section3, menu_item_quit);
  g_object_unref (menu_item_red);
  g_object_unref (menu_item_green);
  g_object_unref (menu_item_blue);
```

```
85    g_object_unref (menu_item_toggle_switch);
86    g_object_unref (menu_item_quit);
87
88    g_menu_append_section (menu, NULL, G_MENU_MODEL (section1));
89    g_menu_append_section (menu, "Color", G_MENU_MODEL (section2));
90    g_menu_append_section (menu, NULL, G_MENU_MODEL (section3));
91    g_menu_append_submenu (menubar, "Menu", G_MENU_MODEL (menu));
92    g_object_unref (section1);
93    g_object_unref (section2);
94    g_object_unref (section3);
95    g_object_unref (menu);
96
97    gtk_application_set_menubar (GTK_APPLICATION (app), G_MENU_MODEL (menubar));
98    g_object_unref (menubar);
99
100   /* Initialize the css data */
101   gtk_css_provider_load_from_string (provider, "label.lb {background-color: red; font-size: 72pt; co
102   /* Add CSS to the default GdkDisplay. */
103   gtk_style_context_add_provider_for_display (gdk_display_get_default (),
104         GTK_STYLE_PROVIDER (provider), GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);
105   g_signal_connect (app, "shutdown", G_CALLBACK (app_shutdown), provider);
106 }
107
108 #define APPLICATION_ID "com.github.ToshioCP.menu2"
109
110 int
111 main (int argc, char **argv) {
112   GtkApplication *app;
113   int stat;
114
115   app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
116   g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
117   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
118
119   stat = g_application_run (G_APPLICATION (app), argc, argv);
120   g_object_unref (app);
121   return stat;
122 }
```

- 3-22: Action signal handlers. The `toggle_switch_changed` handler toggles the label text between "ON" and "OFF" depending on the boolean state. The `color_state_changed` handler dynamically generates a CSS string to change the background color of the label and loads it into the GtkCssProvider. In both handlers, `g_simple_action_set_state()` is called at the end to explicitly update the action's state.
- 25-29: The handler `app_shutdown` is called right before the application quits. It safely cleans up by removing the CSS provider from the default display and unreferencing it.
- 31-49: The activate signal handler.
- 33-35: A new application window is created and assigned to `win`. Its title and default size are set to "menu2" and 800x600, respectively.
- 37-39: A new label is created with the initial text "OFF" and assigned to `label`. It is given a CSS class "lb" and added to `win` as its child widget.
- 41-44: A stateful toggle action is created and assigned to `act_toggle_switch` with an initial boolean value of FALSE. It is connected to the signal handler `toggle_switch_changed` and added to the window. Because it is added to the window's action map, its scope is "win" (i.e., "win.toggle-switch").

The action object is then unreferenced.

- 46: The function `gtk_application_window_set_show_menubar` is called to ensure the menubar is displayed in the window.
- 48: Shows the window using `gtk_window_present()`.
- 51-106: The startup signal handler.
- 53-64: A new GtkCssProvider is created to handle application-wide CSS. Two actions, `act_color` (stateful) and `act_quit` (stateless), are created. Since the startup handler is called only once, there is only one instance of each action. They are connected to their respective handlers, added to the application, and then unreferenced. Because they are added to the application's action map, their scope is "app".
- 66-95: The menus are built. GMenuItem objects are created and appended to distinct sections (section1, section2, section3) to logically group them (e.g., Toggle Switch, Colors, Quit). Unnecessary references to the menu items and sections are freed.
- 97-98: The fully constructed menubar is applied to the application.
- 100-104: The CSS provider is initialized with default CSS data (setting the label's background to red and font size to 72pt) and added to the default display so it affects all widgets.
- 105: Finally, the "shutdown" signal on the application is connected to the `app_shutdown` handler.

2.15.6 Compiling and Running

Change your current directory to `src/menu`.

```
$ bash comp menu2
$./a.out
```

You will see a window with the text "OFF" in the center and a red background. You can change the text to "ON" and back to "OFF" by clicking the "Toggle Switch" menu item. You can change the background color by clicking the "Red", "Green", or "Blue" menu items.

If you run the application a second time while the first instance is still running, another window will appear on the screen. Note that both windows belong to the primary application instance. The second instance simply emits the "activate" signal to the primary instance and then immediately quits.

```
$ ./a.out & ./a.out
$
```

The "ON" and "OFF" states in the two windows are independent of each other because each window has its own GtkLabel instance.

However, both windows share the same GdkDisplay (the overall screen environment). Since the background color is determined by the CSS applied to the GdkDisplay, both windows will always share the same background color.

Note for WSL users:

You may notice that the menu remains on the screen after being clicked, and only disappears on your next click. This is a known issue specific to WSL. On a native Linux environment, the menu closes immediately after a selection is made.

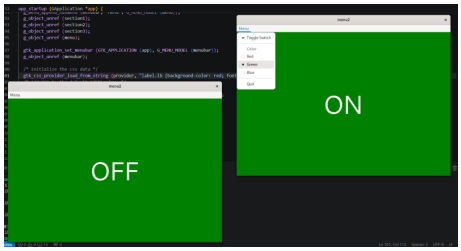


Figure 2.23: menu2

2.16 Ui File for Menu and Action Entries

2.16.1 Simplifying Menu Creation with UI Files

Constructing menus purely in C can be a tedious process that results in lengthy, hard-to-read code. Fortunately, there is a better way. Just as we used UI files to simplify the creation of widgets, we can use XML-based UI files to define our menus. This approach improves code readability and saves development time.

2.16.2 Rewriting menu2 with a UI File

Let's take the `menu2` program from the previous section and rewrite its menu construction using a UI file. To do this, we need to understand a few basic XML tags used for GTK menus.

2.16.2.1 Basic Tags for Menu

- `<menu>`: The root element of a menu. It corresponds to a GMenu object. The `id` attribute defines an identifier for this object. GtkBuilder uses this identifier to retrieve the menu in your C code.
- `<item>`: Corresponds to an item in a GMenu. In the C code in the previous section, this item was created using a GMenuItem.
- `<attribute>`: Defines the properties of an item or a menu. We use the name attribute to specify what property we are setting. The most common ones are:
  - `label`: The text displayed to the user.
  - `action`: The action triggered when the item is clicked (e.g., `app.quit`).
  - `target`: The parameter passed to a stateful action (e.g., "red", "green", "blue").
- `<submenu>`: A convenient shortcut tag to create a hierarchical menu (a menu within a menu).
- `<section>`: A shortcut tag to logically group menu items. GTK usually inserts a visual separator between sections to make the menu easier to read.

2.16.2.2 Rewriting menu2 with a UI File

Here is the UI file `menu2.ui` that mirrors the menu structure we built purely in C in the previous section.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <menu id="menubar">
    <submenu>
      <attribute name="label">Menu</attribute>
      <section>
        <item>
          <attribute name="label">Toggle Switch</attribute>
          <attribute name="action">win.toggle-switch</attribute>
        </item>
      </section>
      <section>
        <attribute name="label">Color</attribute>
        <item>
          <attribute name="label">Red</attribute>
          <attribute name="action">app.color</attribute>
          <attribute name="target">red</attribute>
        </item>
        <item>
          <attribute name="label">Green</attribute>
          <attribute name="action">app.color</attribute>
          <attribute name="target">green</attribute>
```

```
23          </item>
24          <item>
25            <attribute name="label">Blue</attribute>
26            <attribute name="action">app.color</attribute>
27            <attribute name="target">blue</attribute>
28          </item>
29        </section>
30        <section>
31          <item>
32            <attribute name="label">Quit</attribute>
33            <attribute name="action">app.quit</attribute>
34          </item>
35        </section>
36      </submenu>
37    </menu>
38  </interface>
```

As you can see, the hierarchical structure is much easier to grasp at a glance compared to reading C code.

- 1: This line declares the XML version and encoding.
- 2: The `<interface>` tag is the root element of all GTK UI files.
- 3: The top-level `<menu>` tag must have an id attribute. Here, the ID is “menubar”, which allows GtkBuilder to retrieve it in the C code.
- 4-5: The `<submenu>` tag creates a hierarchical menu. The `<attribute>` tag inside it sets the label of this submenu to “Menu”. This submenu contains the sections defined in lines 6 to 35.
- 6-11: The first `<section>` contains a single menu `<item>`. Its label is “Toggle Switch” and its action name is “win.toggle-switch”.
- 12-29: The next section has a label attribute set to “Color” (which acts as a section heading) and contains three menu items for changing the color.
- 14-18: The first item’s label is “Red”, its action is “app.color”, and its target is “red”. This corresponds to “app.color::red” in the previous `menu2.c` code.
- The remaining code follows the same pattern.

For further information, see GTK 4 API reference – PopoverMenu.

Now, let’s look at how to load this UI file in the C program. The following snippet shows the relevant part of `menu2_ui.c` inside the `app_startup` function:

```
GtkBuilder *builder;
GMenuModel *menubar;

builder = gtk_builder_new_from_file ("menu2.ui");
menubar = G_MENU_MODEL (gtk_builder_get_object (builder, "menubar"));
gtk_application_set_menubar (GTK_APPLICATION (app), menubar);
g_object_unref (builder);
```

First, `gtk_builder_new_from_file` reads the `menu2.ui` file and constructs the menu objects in memory. Then, `gtk_builder_get_object` retrieves the root menu using the ID “menubar”. After setting the menu to the application with `gtk_application_set_menubar`, we no longer need the builder, so we safely release it with `g_object_unref`.

Note: `gtk_builder_get_object` simply returns a pointer to the object without incrementing its reference count. The builder retains full ownership, so you must not unref `menubar`. In addition, `gtk_application_set_menubar` adds its own reference to the `menubar`, making it safe to unref the builder immediately afterward.

2.16.2.3 A Quick Note on Menu Shortcuts

As mentioned earlier, `<submenu>` and `<section>` are actually convenient shortcut tags provided by GtkBuilder. Under the hood, a menu is strictly a tree of `<item>` nodes connected by `<link>` tags.

When you use a `<section>` or `<submenu>`, GTK internally translates it into an `<item>` that contains a `<link>` with the name "section" or "submenu". To illustrate this, look at the two examples below. Both snippets produce the exact same menu UI:

Example 1: Using the `<section>` shortcut (Recommended)

```
<section>
  <item>
    <attribute name="label">Copy</attribute>
  </item>
  <item>
    <attribute name="label">Paste</attribute>
  </item>
</section>
```

Example 2: The actual internal structure using `<link>`

```
<item>
  <link name="section">
    <item>
      <attribute name="label">Copy</attribute>
    </item>
    <item>
      <attribute name="label">Paste</attribute>
    </item>
  </link>
</item>
```

Both approaches are perfectly valid, but developers almost always use the shortcuts (`<section>` and `<submenu>`) because they are much easier to read and write.

2.16.3 Building a Practical Menu

Now let's look at a more practical example. Most desktop applications feature a standard menu bar with categories like "File", "Edit", and "View".

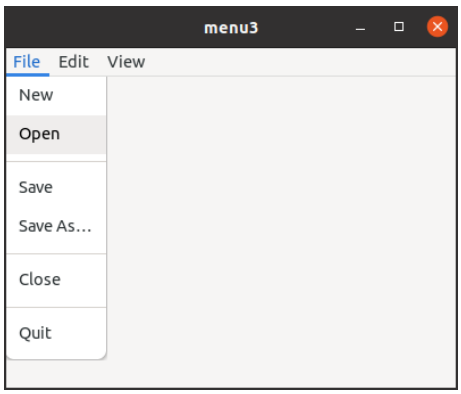


Figure 2.24: menu3

The `menu3.ui` file implements this familiar structure.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <menu id="menubar">
    <submenu>
      <attribute name="label">File</attribute>
      <section>
        <item>
          <attribute name="label">New</attribute>
          <attribute name="action">app.new</attribute>
        </item>
        <item>
          <attribute name="label">Open</attribute>
          <attribute name="action">app.open</attribute>
        </item>
      </section>
      <section>
        <item>
          <attribute name="label">Save</attribute>
          <attribute name="action">win.save</attribute>
        </item>
        <item>
          <attribute name="label">Save As…</attribute>
          <attribute name="action">win.saveas</attribute>
        </item>
      </section>
      <section>
        <item>
          <attribute name="label">Close</attribute>
          <attribute name="action">win.close</attribute>
        </item>
      </section>
      <section>
        <item>
          <attribute name="label">Quit</attribute>
          <attribute name="action">app.quit</attribute>
        </item>
      </section>
    </submenu>
    <submenu>
      <attribute name="label">Edit</attribute>
      <section>
        <item>
          <attribute name="label">Cut</attribute>
          <attribute name="action">win.cut</attribute>
        </item>
        <item>
          <attribute name="label">Copy</attribute>
          <attribute name="action">win.copy</attribute>
        </item>
        <item>
          <attribute name="label">Paste</attribute>
          <attribute name="action">win.paste</attribute>
        </item>
```

```
54        </section>
55        <section>
56          <item>
57            <attribute name="label">Select All</attribute>
58            <attribute name="action">win.selectall</attribute>
59          </item>
60        </section>
61      </submenu>
62      <submenu>
63        <attribute name="label">View</attribute>
64        <section>
65          <item>
66            <attribute name="label">Fullscreen</attribute>
67            <attribute name="action">win.fullscreen</attribute>
68          </item>
69        </section>
70      </submenu>
71    </menu>
72  </interface>
```

Although this UI file creates a much larger and deeper tree structure compared to our previous example, notice that it uses the exact same basic XML tags: `<menu>`, `<submenu>`, `<section>`, `<item>`, and `<attribute>`. By simply nesting these familiar elements, you can build complex and practical menu systems without learning any new syntax.

For more detailed information and advanced usage of menu UI files, please refer to the GtkPopoverMenu section in the GTK documentation.

Note: To load this UI file in your C code, you use `gtk_builder_new_from_file ("menu3.ui")` exactly as we did in the previous example. Since the C code for extracting the menu from the builder is identical to `menu2`, we will skip repeating it here.

2.16.4 Automating Action Creation with GActionEntry

In the previous examples, we created actions, connected their signals, and added them to an action map one by one. While this works, it requires a lot of repetitive boilerplate code. To automate and simplify this process, GTK provides the GActionEntry structure and the `g_action_map_add_action_entries` function. GActionEntry allows you to define all the properties of an action — its name, signal handlers, parameter type, and initial state — in a single array element.

```
typedef struct _GActionEntry GActionEntry;

struct _GActionEntry
{
  /* action name */
  const char *name;
  /* activate handler */
  void (* activate) (GSimpleAction *action, GVariant *parameter, gpointer user_data);
  /* the type of the parameter given as a single GVariant type string */
  const char *parameter_type;
  /* initial state given in GVariant text format */
  const char *state;
  /* change-state handler */
  void (* change_state) (GSimpleAction *action, GVariant *value, gpointer user_data);
  /*< private >*/
```

```
  gsize padding[3];
};
```

Let's look at how the actions in `menu3.c` are defined using this structure.

2.16.4.1 Non-stateful Actions

Most standard commands like "new", "open", or "quit" are non-stateful and do not require parameters.

```
{ "quit", quit_activated, NULL, NULL, NULL }
```

1. "quit": The name of the action.
2. `quit_activated`: The "activate" signal handler.
3. `NULL`: The parameter type. Since this action takes no parameters, it is NULL.
4. `NULL`: The initial state. Since this is a non-stateful action, it is NULL.
5. `NULL`: The "change-state" handler. Only stateful actions need this.

2.16.4.2 Stateful Actions without parameters

Actions that toggle a state (like switching fullscreen mode on and off) are defined slightly differently.

```
{ "fullscreen", NULL, NULL, "false", fullscreen_state_changed }
```

1. "fullscreen": The name of the action.
2. `NULL`: The "activate" handler. For stateful actions, we usually handle state changes instead of basic activations, so this is NULL.
3. `NULL`: The parameter type. This action doesn't have parameters.
4. "false": The initial state. It is provided in GVariant text format. As a boolean, the initial state is set to "false".
5. `fullscreen_state_changed`: The "change-state" handler. This handler is called in response to a request to toggle the action's state.

2.16.4.3 Stateful Actions with Parameters

Although `menu3.c` does not use them, some actions require parameters (for example, an action that changes a color based on the user's selection). If you were to recreate the "color" action from our earlier examples, the entry would look like this:

```
{ "color", NULL, "s", "'red'", color_state_changed }
```

1. "color": The name of the action.
2. `NULL`: The "activate" handler. For stateful actions, we usually handle state changes instead of basic activations, so this is NULL.
3. "s": The parameter type. This uses a GVariant format string, where "s" stands for a standard string. See GVariant format strings for details.
4. " 'red' ": The initial state. This must be written in GVariant text format. Because GVariant expects string values to be explicitly quoted, you must include quotes inside your C string. You can write this using single quotes like `"'red'"` or by escaping double quotes like `"\"red\""`. See GVariant text format for details.
5. `color_state_changed`: The "change-state" handler. This handler is called in response to a request to toggle the action's state.

2.16.4.4 Registering the Entries

Once you define an array of GActionEntry, you can register all of them at once using `g_action_map_add_action_entries()`

For the window-scoped actions, we do this inside `app_activate`:

```
const GActionEntry win_entries[] = {
  { "save", save_activated, NULL, NULL, NULL },
```

```
  { "saveas", saveas_activated, NULL, NULL, NULL },
  { "close", close_activated, NULL, NULL, NULL },
  { "cut", cut_activated, NULL, NULL, NULL },
  { "copy", copy_activated, NULL, NULL, NULL },
  { "paste", paste_activated, NULL, NULL, NULL },
  { "selectall", selectall_activated, NULL, NULL, NULL },
  { "fullscreen", NULL, NULL, "false", fullscreen_state_changed }
};
g_action_map_add_action_entries (G_ACTION_MAP (win), win_entries, G_N_ELEMENTS (win_entries), win);
```

This single function call performs all the heavy lifting:

- It iterates through the win_entries array.
- It creates the GSimpleAction instances.
- It sets up their initial states.
- It connects the "activate" and "change-state" signal handlers.
- It adds all the newly created actions to the `win` action map.

We do the exact same thing for the application-scoped actions inside `app_startup`:

```
const GActionEntry app_entries[] = {
  { "new", new_activated, NULL, NULL, NULL },
  { "open", open_activated, NULL, NULL, NULL },
  { "quit", quit_activated, NULL, NULL, NULL }
};
g_action_map_add_action_entries (G_ACTION_MAP (app), app_entries, G_N_ELEMENTS (app_entries), app);
```

Important Note on Signal Handler Signatures

Strictly speaking, your handler functions must exactly match the standard GObject callback signature, accepting a gpointer for the user data:

```
static void
handler (GSimpleAction *action, GVariant *parameter, gpointer user_data) {
  // ...
}
```

Even if you know that `user_data` will be a GApplication, you cannot change the function signature to `(..., GApplication *app)`. The `g_action_map_add_action_entries` function expects standard GObject callback signatures. Instead, you should accept gpointer `user_data` and cast it inside the function body, like this:

```
static void
quit_activated (GSimpleAction *action, GVariant *parameter, gpointer user_data) {
  GApplication *app = G_APPLICATION (user_data);
  g_application_quit (app);
}
```

2.16.5 Full Source Code

The complete source files, including `menu3.c` and `menu3.ui`, are located in the src/menu directory. It is helpful to review the full code to see how the UI definitions and the GActionEntry implementations come together.

2.17 GtkPopoverMenuBar and GtkMenuButton

2.17.1 GtkPopoverMenuBar

As explained in the previous sections, `GMenuModel` is strictly a model, not a widget. This means it cannot be displayed on the screen by itself. In earlier examples, the widget assembly was handled automatically in the background by `gtk_application_window_set_show_menubar`.

However, you can explicitly build and place this menu within your program using `GtkPopoverMenuBar`. The `menu4.c` source file demonstrates how to display a menu bar equivalent to `menu3.c` using this manual approach.

Because this method is more complex than using `gtk_application_window_set_show_menubar`, it is generally not recommended unless you specifically need to customize the placement of the menu bar (e.g., placing it somewhere other than the top of the window). For this reason, we will skip a detailed explanation of `GtkPopoverMenuBar` here, but interested readers are encouraged to examine the menu4.c code.

2.17.2 GtkMenuButton

This section introduces the modern approach to displaying menus: using a menu button that reveals a popup menu when clicked. This is the recommended design pattern in modern GTK applications.

We will demonstrate this by adding a menu button to our Text File Editor (TFE) program.

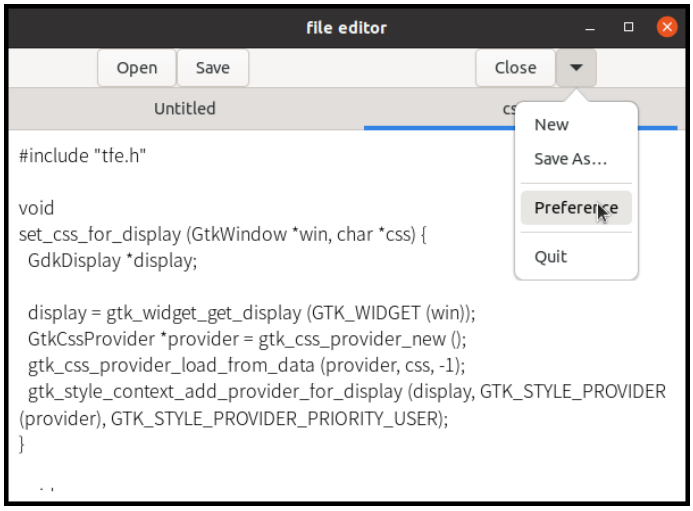


Figure 2.25: tfe6

As shown in the image, the toolbar contains four buttons:

- Open
- Save
- Close
- Menu (with a downward-pointing triangle icon)

The rightmost button is the menu button. Clicking it reveals a dropdown menu with the following items:

- New
- Save as
- Preference
- Quit

In modern UI design, frequently used functions are placed as standard buttons, while less common features are tucked away in menus. This is because accessing a menu requires a two-step operation: (1) clicking

the menu button, and (2) clicking the menu item. Traditional menu bars also require two steps (e.g., clicking “Edit”, then “Copy”).

In contrast, standard buttons require only a single click. Eliminating just one step might seem trivial, but it makes quite a big difference in user experience. Modern applications favor this approach to keep the interface clean and efficient.

2.17.3 Implementing the Menu in TFE

2.17.3.1 UI Files

It is good practice to separate the UI file for the menu from the UI file for the main widgets. Because our popup menu is quite simple and introduces no new concepts, we will skip the explanation of its XML structure. Please refer to src/tfe6/gtk/menus.ui for the code.

2.17.3.2 Automatic Resources

In previous examples, you might have used GtkBuilder to manually load UI files and extract the objects. However, GtkApplication provides a very convenient feature called “Automatic Resources” for your menu UI resources.

If you place your menu definition file at exactly `gtk/menus.ui` relative to your application’s resource base path, GtkApplication will automatically load it for you in the background and you don’t need to write any GtkBuilder code. The application’s resource base path is normally created from the application ID. For example, if the application ID is “com.github.ToshioCP.tfe6”, the application’s resource base path is “/com/github/ToshioCP/tfe6”. Therefore, the menu definition UI resource path is “/com/github/ToshioCP/tfe6/gtk/menus.ui”.

```
<?xml version="1.0" encoding="UTF-8"?>
<gresources>
  <gresource prefix="/com/github/ToshioCP/tfe6">
    <file>tfe.ui</file>
    <file alias="gtk/menus.ui">menus.ui</file>
  </gresource>
</gresources>
```

The file path in the project source is `menus.ui` but the resource path is `gtk/menus.ui`. We use an alias to resolve this in the `tfe.gresource.xml` file.

```
<file alias="gtk/menus.ui">menus.ui</file>
```

Inside this `menus.ui`, you can define multiple menus and assign them unique IDs. When you need a specific menu in your C code, you simply call `gtk_application_get_menu_by_id()` with the desired ID.

In the following code, the menu model is retrieved using `gtk_application_get_menu_by_id()`. The argument “menu” is the ID of the menu tag in the UI resource.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <menu id="menu">
  ... ... ...
```

You don’t need to write any GtkBuilder boilerplate code.

```
GMenuModel *menu = G_MENU_MODEL (gtk_application_get_menu_by_id (app, "menu"));
```

2.17.3.3 The Menu Button

Connecting a menu to a menu button is easy and straightforward. Once you have built your menu model, you simply attach it using the `gtk_menu_button_set_menu_model` function:

```
gtk_menu_button_set_menu_model (GtkMenuButton *menu_button, G_MENU_MODEL *menu_model);
```

In the widget UI file, we configure the property of the menu button:

```
<object class="GtkMenuButton" id="btnm">
  <property name="direction">none</property>
</object>
```

The direction property dictates which way the menu pops up relative to the button. Its default value is `GTK_ARROW_DOWN` ("down" in UI file), which automatically displays a downward-pointing triangle icon, as seen in the image at the beginning of this section. This triangle adapts depending on the chosen direction (up, down, left, or right).

By setting the direction property to "none" (`GTK_ARROW_NONE`), the menu still pops up downward, but the button's icon changes to "open-menu-symbolic", the standard "hamburger" menu icon. You can also explicitly change the icon by setting the "icon-name" property. Some common alternatives include:

- **view-more-symbolic:** The "three dots" (kebab) menu icon.
- **emblem-system-symbolic:** The gear icon, typically used for settings.

For example, to create a "three-dot" menu button, your XML would look like this:

```
<object class="GtkMenuButton" id="btnm">
  <property name="direction">none</property>
  <property name="icon-name">view-more-symbolic</property>
</object>
```

To explore the available icons, you can use the `gtk4-icon-browser` tool. Run the following command in your terminal:

```
$ gtk4-icon-browser
```

Once the window opens, click the "Symbolic" tab at the top to find icons suited for UI elements.

2.17.3.4 GtkActionable

Unlike in previous examples, the buttons in this widget UI file are directly bound to actions. These buttons implement the GtkActionable interface, which provides the functionality to link a widget to a specific action.

We have three standard buttons: "Open", "Save", and "Close". There are two ways to handle a button click event.

- Connect the "clicked" signal directly to your handler. This is a simpler way.
- Register an action to the button, making it behave similarly to a menu item. When the button is clicked, its corresponding action is activated. This approach is slightly more complex because it requires creating the action and its activate handler in advance. However, it offers a big advantage: **you can connect multiple triggers to a single action**.

For instance, you can link a keyboard accelerator to the action. Accelerators are keystrokes (like Ctrl+O) mapped to specific actions. By using this method, clicking the "Open" button or pressing Ctrl+O will activate the same file-opening action. This is the primary reason why actions are designed to be independent of specific buttons or menus.

For example, the UI definition for the "Open" button looks like this:

```
<object class="GtkButton">
  <property name="label">Open</property>
  <property name="action-name">win.open</property>
</object>
```

By setting the "action-name" property to "win.open", the "open" action is activated whenever the button is clicked. You simply need to write the file-opening logic inside the action's activate signal handler. The workflow is identical to using actions within a menu.

2.17.4 A Note for WSL Users

When running GTK applications on WSL (Windows Subsystem for Linux), you may notice that the menu button fails to display the menu on the second and subsequent clicks. Because this bug does not occur in native Linux environments, it is likely an issue specific to WSL's rendering system. Currently, developing and testing GTK applications that rely heavily on popup menus can be problematic under WSL.

2.17.5 Source files

The source files are under the src/tfe6 directory. You can build and run the program by changing your current directory to `src/tfe6` and typing:

```
$ meson setup _build
$ ninja -C _build
$ _build/tfe
```

2.18 Managing the Application

A good rule of thumb in programming is to split your code into separate files based on what they do. Following this idea, we should reorganize TFE's `application.c`. We're going to divide it into three main parts:

- `main`: This function handles the command-line interface.
- `application`: This is the core application object. It creates and manages things like windows.
- `window`: This object handles the window itself and the widgets inside it.

In this section, we'll focus on the first two parts and save the window management for later.

2.18.1 Creating an Application Class

When you write a GTK application, you have two main options:

- Use GtkApplication exactly as it is.
- Create your own subclass of GtkApplication.

Creating a subclass is generally the better choice because it lets you use "instance variables". An instance variable is simply a piece of data attached to a specific object. For example, remember how we made TfeTextView (a subclass of GtkTextView)? We gave it an instance variable called `file` to store a GFile object. If we had a TfeTextView object named `tv`, we could easily access that file using `tv->file`.

Applications usually need to keep track of objects other than just windows. For example, TFE will eventually need to manage settings using a GSettings object. The standard way to keep track of these objects is by saving pointers to them as instance variables. So, let's define our own TfeApplication class as a subclass of GtkApplication.

If you remember how we made TfeTextView, you'll find this process very familiar.

2.18.2 Writing the Header and C Files.

Let's quickly review how to write the header and C files for our new class. We'll start with the header file.

```
1 #pragma once
2
3 #include <gtk/gtk.h>
```

```
4
5  #define TFE_TYPE_APPLICATION tfe_application_get_type ()
6  G_DECLARE_FINAL_TYPE (TfeApplication, tfe_application, TFE, APPLICATION, GtkApplication)
7
8  TfeApplication *
9  tfe_application_new (const char *application_id, GApplicationFlags flag);
```

- 5: We define the GType for TfeApplication using a macro. There are strict naming rules here:
  - The macro name must be ALL CAPS like this: `PREFIX_TYPE_OBJECT_NAME`.
  - The function name must be all lowercase like this: `prefix_object_name_get_type ()`.
- 6: We use the `G_DECLARE_FINAL_TYPE` macro to set up our class.
  - 1st argument: CamelCase (`PrefixObjectname`) like: `TfeApplication`
  - 2nd argument: lowercase (`prefix_objectname`) like: `tfe_application`
  - 3rd argument: ALL CAPS (`PREFIX`) like: `TFE`
  - 4th argument: ALL CAPS (`OBJECTNAME`) like: `APPLICATION`
  - 5th argument: CamelCase (`ParentObjectname`) like: `GtkApplication`
- 8-9: The only function we need to make public is the one that creates our application, `tfe_application_new`.

Now, let's look at the C file. The beginning looks just like what we did for TfeTextView.

```
#include <gtk/gtk.h>
#include "tfeapplication.h"

struct _TfeApplication {
  GtkApplication parent;
  GSettings *settings;
};

G_DEFINE_FINAL_TYPE (TfeApplication, tfe_application, GTK_TYPE_APPLICATION)
```

- 1: Include the main GTK header.
- 2: You must include your object's own header file.
- 4-7: Here is where we declare the data for our object. The struct name must be CamelCase, starting with an underscore: `_TfeApplication`. The very first member must be the parent object's structure. You can name it whatever you want, but calling it "parent" is the standard practice. After "parent", you can add your instance variables. We're adding a pointer for GSettings here because we plan to use it later.
- 9: The `G_DEFINE_FINAL_TYPE` macro.
  - 1st argument: CamelCase. `TfeApplication`
  - 2nd argument: lowercase. `tfe_application`
  - 3rd argument: ALL CAPS (The parent's type macro). `GTK_TYPE_APPLICATION`

These steps are the standard recipe for creating a subclass in GObject. If you want to know exactly what these macros do behind the scenes, check out the GObject tutorial.

2.18.3 Initializing the Instance and ClassInstance

2.18.3.1 Instance Initialization

Right now, we don't actually need to do any special setup when a new instance is created. However, GTK expects an initialization function to exist, so we still have to write one, even if it's completely empty inside.

2.18.3.2 Instance Disposal

```
static void
tfe_application_dispose (GObject *object) {
  /* In the future version, settings will be disposed here */

  G_OBJECT_CLASS (tfe_application_parent_class)->dispose (object);
}
```

We already covered how disposal works in detail back in Section 11, so we'll keep it brief here. For now, our application doesn't need to clean anything up. But later, when we start using GSettings (which we'll set up during the application's startup), we will need to free that memory here. So, we've just left a comment as a reminder. The most important thing to remember about your dispose function is to always call the parent class's dispose function at the end to chain up the cleanup process.

2.18.3.3 Class Initialization

2.18.3.3.1 Overriding the Dispose Method

```
static void
tfe_application_class_init (TfeApplicationClass *class) {
  G_OBJECT_CLASS (class)->dispose = tfe_application_dispose;
  G_APPLICATION_CLASS (class)->startup = app_startup;
  G_APPLICATION_CLASS (class)->activate = app_activate;
  G_APPLICATION_CLASS (class)->open = app_open;
}
```

When the class is initialized, we can override class methods.

First, we need to override the dispose method so GTK knows to call our custom cleanup code. We did the exact same thing for TfeTextView in Section 11, so feel free to look back at that if you need a refresher.

2.18.3.3.2 Overriding the Startup, Activate, and Open Methods Take a look at the family tree for our TfeApplication class:

`GObject - GApplication - GtkApplication - TfeApplication`

It's actually the GApplication class that defines the "startup", "activate", and "open" signals. In GTK, there are generally two ways to handle signals:

- Default signal handlers: These are built right into the class that created the signal.
- User-defined signal handlers: These are attached from the outside using functions like `g_signal_connect`.

Let's look at the "startup" signal. The GApplication class owns the default handler for it. Inside the class structure, there is a member named `startup` that points to the GApplication startup function. GTK uses this startup pointer to run the code; it doesn't call the function directly by its name.

When you make a subclass, the beginning of its class structure is an exact copy of the parent's class structure. For example, the start of the GtkApplication class structure looks just like the GApplication class structure:

```
struct _GtkApplicationClass
{
  GApplicationClass parent_class; /* This member has the same structure as GApplicationClass */
  ... ... ...
}
```

Because of this, the `startup` member is included in this GApplicationClass area. As a subclass, we can write our own custom startup function and change this `startup` member so it points to our new function instead. This is called overriding.

```
static void
gtk_application_class_init (GtkApplicationClass *class)
{
  ... ... ...
  application_class->startup = gtk_application_startup;
  ... ... ...
}
```

When we override the startup process, it's crucial that we still run the parent class's startup code. We do this by calling the parent's startup function right at the beginning of our custom function.

```
static void
gtk_application_startup (GApplication *g_application)
{
  ... ... ...
  G_APPLICATION_CLASS (gtk_application_parent_class)->startup (g_application);
  ... ... ...
}
```

So, to set up our own startup process in TfeApplication, we need to do two things:

- Inside `tfe_application_class_init`, override the startup member so it points to our custom function.
- Inside our custom startup function, call the parent class's (GtkApplication) startup function first.

We follow the exact same steps to override the "activate" and "open" handlers.

```
static void
tfe_application_class_init (TfeApplicationClass *class) {
  G_OBJECT_CLASS (class)->dispose = tfe_application_dispose;
  G_APPLICATION_CLASS (class)->startup = app_startup;
  G_APPLICATION_CLASS (class)->activate = app_activate;
  G_APPLICATION_CLASS (class)->open = app_open;
}

static void
app_startup (GApplication *application) {
  ... ... ...
  G_APPLICATION_CLASS (tfe_application_parent_class)->startup (application);
  ... ... ...
}
```

2.18.3.3.3 Looking Ahead: TfeWindow In the next section, we'll build TfeWindow, a class completely dedicated to managing our application's window. Since the "activate" and "open" handlers are the ones that actually create the window, building TfeWindow will change how those handlers are written. Once we have our window class ready, we'll come back to `tfeapplication.c` and show how they connect.

2.18.4 The Main Function

Finally, let's put our main function in its own file, `main.c`. The only real difference from before is that we are now creating and running our brand new TfeApplication instead of a plain GtkApplication.

```
1 #include <gtk/gtk.h>
2 #include "tfeapplication.h"
```

```
3
4  #define APPLICATION_ID "com.github.ToshioCP.tfe7"
5
6  int
7  main (int argc, char **argv) {
8    TfeApplication *app;
9    int stat;
10
11   app = tfe_application_new (APPLICATION_ID, G_APPLICATION_HANDLES_OPEN);
12   stat = g_application_run (G_APPLICATION (app), argc, argv);
13   g_object_unref (app);
14   return stat;
15 }
```

2.19 Creating TfeWindow

Restructuring the window not just as a separate file from the application, but as a subclass of GtkApplicationWindow, has the following advantages:

- Better encapsulation. It makes it easier to distinguish between external interfaces and internal processing functions.
- Ability to have instance variables. TfeWindow often refers to its internal GtkNotebook. Keeping its pointer in an instance variable is quite useful.
- By introducing composite widgets along with subclassing, you can handle all widgets within the window together.

2.19.1 Defining the Subclass

The method for defining a subclass of GtkApplicationWindow is the same as for TfeTextView or TfeApplication.

2.19.1.1 Writing the Header File

```
1  #pragma once
2
3  #include <gtk/gtk.h>
4
5  #define TFE_TYPE_WINDOW tfe_window_get_type ()
6  G_DECLARE_FINAL_TYPE (TfeWindow, tfe_window, TFE, WINDOW, GtkApplicationWindow)
7
8  void
9  tfe_window_append_page (TfeWindow *win, GFile *file);
10
11 GtkWidget *
12 tfe_window_new (GtkApplication *app);
```

- 5-6: The macro definition of `TFE_TYPE_WINDOW` and how to write the `G_DECLARE_FINAL_TYPE` macro are the same as before.
- 8-12: This class only needs to be accessed from the application (TfeApplication) through:
  - `tfe_window_new()`: Used when creating a window.
  - `tfe_window_append_page()`: Used within the "open" handler to add the specified file to a page.

2.19.1.2 Writing the C File

First, in the C file, you need to write the object structure and the `G_DEFINE_FINAL_TYPE` macro.

```
#include <gtk/gtk.h>
#include "tfewindow.h"
#include "tfetextview.h"

struct _TfeWindow {
  GtkApplicationWindow parent;
  GtkMenuButton *btnm; /* menu button */
  GtkNotebook *nb; /* notebook */
};

G_DEFINE_FINAL_TYPE (TfeWindow, tfe_window, GTK_TYPE_APPLICATION_WINDOW)
```

- 5-9: The first member of the object structure is the parent object's structure. If a pointer to a TfeWindow object `win` is passed as an argument to functions in the class, you can access the menu button with `win->btnm` and the notebook with `win->nb`.
- 11: Uses the `G_DEFINE_FINAL_TYPE` macro.

2.19.2 Composite Widgets

Until now, we have built windows by combining widgets like buttons and notebooks, treating them as a single object. There was no way to represent them as a cohesive unit. A composite widget provides a way to represent this group; it is a single widget made by combining multiple widgets.

With a composite widget, you create a template for a "special window containing a button and a notebook". This template is similar to a class. When you instantiate a composite widget, all its internal widgets are created and assembled based on this template.

Note that existing widgets like GtkWindow cannot be converted into composite widgets directly. Composite widgets must be implemented programmatically as subclasses.

The template is created from UI data by GtkBuilder, which is similar to how we've used the builder so far, but their roles are clearly different:

- **Creating objects with a builder:** Creates instances of widgets from an XML file and combines them. The output is a set of instances.
- **Creating a template with a builder:** Generates a template for the widget combination from an XML file. The template is not a group of instances.

When a composite widget is instantiated, the individual widgets it contains are created within its instance initialization function (for example, `tfe_window_init`). This is handled by the `gtk_widget_init_template()` function.

The main advantage of this approach is encapsulation — keeping multiple widgets contained within a single composite widget. For example, if the application needs to open multiple editor windows, you would normally have to create and manage individual widgets for each window. With a composite widget, creating the main widget automatically creates its child widgets. By letting the composite widget manage its internal components, external code doesn't need to do anything extra.

This clarifies the division of responsibilities in the code, leading to cleaner programs with fewer bugs.

In the following subsections, let's see how to actually create a composite window using TfeWindow as an example.

2.19.2.1 Structure of the UI File

The UI file `tfewindow.ui` for the composite widget is almost the same as before. The only difference is using the `<template>` tag instead of the usual `<object>` tag for the top window.

```
<template class="TfeWindow" parent="GtkApplicationWindow">
  ... ... ...
</template>
```

- The `class` attribute specifies TfeWindow, the class name of the composite widget.
- The `parent` attribute specifies GtkApplicationWindow, the parent class of TfeWindow.

2.19.2.2 Binding the Template in the C File

Next, we bind the template and child widgets defined in the UI file to the TfeWindow class in the C file. This is done in the class initialization function (`tfe_window_class_init`).

```
  ... ... ...
struct _TfeWindow {
  GtkApplicationWindow parent;
  GtkMenuButton *btnm; /* menu button */
  GtkNotebook *nb; /* notebook */
};

  ... ... ...

static void
tfe_window_class_init (TfeWindowClass *class) {
  GObjectClass *object_class = G_OBJECT_CLASS (class);

  object_class->dispose = tfe_window_dispose;
  gtk_widget_class_set_template_from_resource (GTK_WIDGET_CLASS (class), "/com/github/ToshioCP/tfe7/
  gtk_widget_class_bind_template_child (GTK_WIDGET_CLASS (class), TfeWindow, btnm); /* bind "btnm" i
  gtk_widget_class_bind_template_child (GTK_WIDGET_CLASS (class), TfeWindow, nb);  /* bind "nb" in _
}
```

- `gtk_widget_class_set_template_from_resource()` loads the template from the UI data into the class.
- `gtk_widget_class_bind_template_child()` maps the pointer to the widget with `id="btnm"` in the XML to the `win->btnm` variable. Because of this binding, when a composite widget instance is created, `win->btnm` is automatically initialized to point to that specific widget. The notebook (`nb`) is handled in the same way.

This function is called only once when the TfeWindow class is created, establishing the connection between the class and the template.

2.19.2.3 Creating an Instance

When an instance is created, the initialization function `tfe_window_init()` is called.

```
static void
tfe_window_init (TfeWindow *win) {
  GtkBuilder *builder;

  gtk_widget_init_template (GTK_WIDGET (win));

  ... ... ...
}
```

`gtk_widget_init_template()` instantiates the widgets inside the window based on the template, setting up the proper parent-child relationships.

2.19.2.4 Destroying an Instance

The internal widgets of the window must be destroyed just before the composite widget itself is destroyed. This is handled in the `tfe_window_dispose()` function.

```
static void
tfe_window_dispose (GObject *gobject) {
  gtk_widget_dispose_template (GTK_WIDGET (gobject), TFE_TYPE_WINDOW);
  G_OBJECT_CLASS (tfe_window_parent_class)->dispose (gobject);
}
```

The `gtk_widget_dispose_template()` function destroys the widgets created from the template. It also sets the instance variables bound by `gtk_widget_class_bind_template_child()` to NULL.

To summarize the process of creating a composite widget:

- Use the `<template>` tag as the root tag in your UI file.
- Load the template into the class using `gtk_widget_class_set_template_from_resource()` during class initialization.
- To link objects defined by IDs in the UI file to instance variables in the object structure:
  - Use the same name for the UI ID and the instance variable.
  - Bind them using `gtk_widget_class_bind_template_child()` during class initialization.
- Call `gtk_widget_init_template()` during object instantiation to create the child widgets.
- Call `gtk_widget_dispose_template()` during object destruction to clean up the child widgets.

2.19.3 The C File of TfeWindow

This subsection covers the changes made to TfeWindow that are not directly related to composite widgets.

2.19.3.1 Setting the Menu to the Menu Button

The menu is set inside `tfe_window_init()`.

```
builder = gtk_builder_new_from_resource ("/com/github/ToshioCP/tfe7/menus.ui");
gtk_menu_button_set_menu_model (GTK_MENU_BUTTON (win->btnm), G_MENU_MODEL (gtk_builder_get_object (bu
g_object_unref (builder);
```

Here, we did not use GtkApplication's Automatic resources (which involves `gtk_application_get_menu_by_id()`). This was done to avoid adding a dependency on GtkApplication. Setting up the menu entirely within TfeWindow aligns better with the concept of encapsulation.

2.19.3.2 Defining Actions

Actions are also defined inside `tfe_window_init()`.

```
const GActionEntry win_entries[] = {
  { "open", open_activated, NULL, NULL, NULL },
  { "save", save_activated, NULL, NULL, NULL },
  { "close", close_activated, NULL, NULL, NULL },
  { "new", new_activated, NULL, NULL, NULL },
  { "saveas", saveas_activated, NULL, NULL, NULL },
};
g_action_map_add_action_entries (G_ACTION_MAP (win), win_entries, G_N_ELEMENTS (win_entries), win);
```

This is the same code that was previously in the `get_main_window()` function in version `tfe6`. Since TfeWindow is now a standalone class, the code was simply moved to TfeWindow's initialization.

2.19.3.3 Constructor

```
GtkWidget *
tfe_window_new (GtkApplication *app) {
  GtkWidget *win;

  win = GTK_WIDGET (g_object_new (TFE_TYPE_WINDOW, "application", app, NULL));
  tfe_window_append_page (TFE_WINDOW (win), NULL);
  return win;
}
```

In the constructor, the instance is created using `g_object_new()`, just like with TfeTextView. The "application" property is set to the application instance here. We also added code to open an empty page by default. This mimics the behavior of many text editors that start with a blank document.

2.19.3.4 Instance Methods

The only instance method is `tfe_window_append_page()`. This is a slightly modified version of the old `notebook_page_new_with_file()` function, keeping the same core logic.

2.19.3.5 Saveas Action Handler

```
static void
saveas_activated (GSimpleAction *action, GVariant *parameter, gpointer user_data) {
  TfeWindow *win = TFE_WINDOW (user_data);
  int i;
  GtkWidget *scr;
  GtkWidget *tv;
  GtkFileDialog *dialog;
  GtkAlertDialog *alert_dialog;

  i = gtk_notebook_get_current_page (win->nb);

  if (i == -1) {
    alert_dialog = gtk_alert_dialog_new ("No page to save.");
    gtk_alert_dialog_show (alert_dialog, GTK_WINDOW (win));
    g_object_unref (alert_dialog);
    return;
  }

  scr = gtk_notebook_get_nth_page (win->nb, i);
  tv = gtk_scrolled_window_get_child (GTK_SCROLLED_WINDOW (scr));

  dialog = gtk_file_dialog_new ();
  gtk_file_dialog_save (dialog, GTK_WINDOW (win), NULL, save_dialog_cb, tv);
  g_object_unref (dialog);
}
```

This handler is very similar to the Save action handler (`save_activated()`). The main difference is that it always opens a file dialog before saving. Since the code overlaps significantly with the Save handler, we'll skip a detailed explanation here.

2.19.4 The C File of TfeApplication

With TfeWindow covered, let's go over the remaining changes in TfeApplication.

2.19.4.1 Function to Get the Window

Both the “activate” and “open” handlers need to get a reference to the main window first. This logic was extracted into a separate `get_main_window()` function. Since window initialization is now handled by TfeWindow, this function has become much simpler.

```
static GtkWidget *
get_main_window (GApplication *application) {
  GList *windows = gtk_application_get_windows (GTK_APPLICATION (application));
  GtkWidget *win;

  if (windows)
    win = GTK_WIDGET (windows->data);
  else
    win = tfe_window_new(GTK_APPLICATION (application));
  return win;
}
```

If the application already has a window (i.e., the list of top windows is not empty), it retrieves it; otherwise, it creates a new one. Notice that we don’t need to explicitly link the window to the application here using `gtk_window_set_application()`. This is because the application is already passed to the “application” property when `tfe_window_new()` is called, which establishes the connection.

```
GtkWidget *
tfe_window_new (GtkApplication *app) {
  GtkWidget *win;

  win = GTK_WIDGET (g_object_new (TFE_TYPE_WINDOW, "application", app, NULL));
  tfe_window_append_page (TFE_WINDOW (win), NULL);
  return win;
}
```

2.19.4.2 Activate and Open Functions

By moving some logic to `get_main_window()`, these handlers are now shorter as well.

```
static void
app_activate (GApplication *application) {
  GtkWidget *win;

  win = get_main_window (application);
  gtk_window_present (GTK_WINDOW (win));
}

static void
app_open (GApplication *application, GFile ** files, gint n_files, const gchar *hint) {
  GtkWidget *win;
  int i;

  win = get_main_window (application);
  for (i = 0; i < n_files; i++) {
    tfe_window_append_page (TFE_WINDOW (win), files[i]);
  }
  gtk_window_present (GTK_WINDOW (win));
}
```

You may notice that chaining up is omitted at the beginning of both `app_activate ()` and `app_open ()`.

To understand why, let's look at what GApplication's default implementation of "activate" handler actually does. The source code of `g_application_real_activate ()` (GApplication's "activate" handler) in GLib shows that it only checks whether any handler is connected.

- Handlers are implemented using `g_signal_connect`.
- Subclasses implement handlers by overriding methods.

And if not, it emits a warning.

In other words, GApplication itself provides no meaningful implementation for "activate" or "open" handler. It simply requires that the application developer implement them, either externally with `g_signal_connect ()` or internally by overriding the virtual function in a subclass. This design is analogous to a pure virtual function in C++: the base class declares the interface but intentionally leaves the implementation to the derived class.

Since the default implementation is effectively empty, chaining up would execute no meaningful code. For this reason, we chose to omit chaining up in these specific functions.

However, chaining up is not always optional. The necessity and timing depend on the parent class's implementation. For instance, chaining up is required for "startup" handler and must be done at the beginning of the override. It is also required for "dispose" handler, but must be done at the end. When overriding virtual functions, always check the documentation or source code to determine whether chaining up is necessary.

2.19.5 Compilation and Execution

All the files are located in the src/tfe7 directory. Navigate to this directory in your terminal and run the following commands to compile and start the application:

```
$ meson setup _build
$ ninja -C _build
$ _build/tfe
```

As noted earlier, menus does not work correctly in WSL (Windows Subsystem for Linux). They will work as expected on a native Linux environment.

2.20 Handling Unsaved Text

Up to this point in TFE, pages could be closed or the application quit even if there was unsaved text. This is a poor design that risks discarding the user's hard work and needs to be improved. To fix this, we should check for unsaved text beforehand and, if any exists, ask the user whether they want to cancel the closing process.

2.20.1 Checking for Text Modifications

To check for unsaved text, we use the `gtk_text_buffer_get_modified ()` function from `GtkTextBuffer`. This returns `TRUE` if the text buffer has been modified, and `FALSE` otherwise.

However, since the buffer is internal to `TfeTextView`, querying `GtkTextBuffer` directly breaks the principle of encapsulation. Instead, we will create a new instance method in `TfeTextView` to serve as a public interface.

Public functions must be declared in the header file.

```
gboolean
tfe_text_view_get_modified (TfeTextView *tv);
```

This function queries the buffer, returning `TRUE` if modified and `FALSE` if not. The corresponding implementation in `tfetextview.c` is shown below.

```
gboolean
tfe_text_view_get_modified (TfeTextView *tv) {
  g_return_val_if_fail (TFE_IS_TEXT_VIEW (tv), FALSE);

  GtkTextBuffer *tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));

  return gtk_text_buffer_get_modified (tb);
}
```

2.20.2 Closing a Page

The current page is closed when its close button is clicked. Before closing, we check if the page has been modified. If it has, we prompt the user with an alert dialog; if not, we simply close it.

Because we are using an asynchronous dialog, we need to split this process into two functions:

- One for setting up and displaying the dialog.
- Another for handling the steps after the user makes a selection (the callback handler).

First, let's look at the close action's handler, located in `tfewindow.c`.

```
static void
close_activated (GSimpleAction *action, GVariant *parameter, gpointer user_data) {
  TfeWindow *win = TFE_WINDOW (user_data);
  GtkWidget *scr = gtk_notebook_get_nth_page (win->nb, gtk_notebook_get_current_page (win->nb));
  GtkWidget *tv = gtk_scrolled_window_get_child (GTK_SCROLLED_WINDOW (scr));
  GtkAlertDialog *dialog;
  const char *buttons[] = {"Cancel", "Close Without Saving", NULL};

    if (!tfe_text_view_get_modified (TFE_TEXT_VIEW (tv))) {
      close_page (win);
      return;
    }

  dialog = gtk_alert_dialog_new ("Unsaved changes");
  gtk_alert_dialog_set_detail (dialog, "Are you sure to close this page without saving?");

  gtk_alert_dialog_set_buttons (dialog, buttons);
  gtk_alert_dialog_set_cancel_button (dialog, 0); /* cancel button */
  gtk_alert_dialog_set_default_button (dialog, 0); /* default focus */

  gtk_alert_dialog_choose (dialog, GTK_WINDOW (win), NULL, page_close_alert_cb, win);
  g_object_unref (dialog);
}
```

- 9-12: If the text is unmodified, call `close_page ()` to close the page and return. (This function will be explained later).
- 14-19: Create the alert dialog.
  - 14: Create the dialog and set its title to "Unsaved changes".
  - 15: Set the alert's detailed description.
  - 17: Set the two buttons using the string array prepared on line 7. The final `NULL` serves as the termination marker.
  - 18: Assign the cancel button to index 0. This ensures that pressing the `ESC` key acts exactly like clicking the Cancel button.
  - 19: Assign the default button to index 0. This ensures that pressing the Return key also acts like clicking the Cancel button.

- 21: We use `gtk_alert_dialog_choose` to asynchronously receive the user's selection. The arguments are as follows:
  - `dialog`: The alert dialog instance.
  - `GTK_WINDOW (win)`: The transient parent window. The dialog will be displayed modally on top of this window.
  - `NULL`: No `GCancellable` is set. If provided, this would allow the dialog to be cancelled programmatically from the outside.
  - `page_close_alert_cb`: The callback function, which is invoked after the user operation (dialog selection) is completed or cancelled.
  - `win`: The user data passed into the callback function.
- 22: Free the dialog instance, as it is no longer needed in this function.

Next, let's look at the callback function.

```
1  static void
2  page_close_alert_cb (GObject *source_object, GAsyncResult *res, gpointer user_data)
3  {
4    GtkAlertDialog *dialog = GTK_ALERT_DIALOG (source_object);
5    GtkWidget *win;
6    GError *err = NULL;
7    int response = gtk_alert_dialog_choose_finish (dialog, res, &err);
8
9    if (err != NULL) {
10       g_warning ("Alert Dialog Error: %s", err->message);
11       g_error_free (err);
12       return;
13   }
14
15   win = GTK_WIDGET (user_data); /* This assignment must be done after the error check */
16   if (response == 1) { /* Close Without Saving */
17       close_page (TFE_WINDOW (win));
18   }
19   /* Cancel => do nothing*/
20 }
```

- 7: Use `gtk_alert_dialog_choose_finish ()` to determine which button was pressed. The return value indicates the index (starting from 0) of the selected button.
- 9-13: If an error occurs, print the message to standard error and return.
- 16-17: If the button at index 1 is selected (which corresponds to "Close Without Saving"), call `close_page ()` to close the tab.

The `close_page ()` function is implemented as follows:

```
1  static void
2  close_page (TfeWindow *win) {
3    if (gtk_notebook_get_n_pages (win->nb) >= 2)
4      gtk_notebook_remove_page (win->nb, gtk_notebook_get_current_page (win->nb));
5    else {  /* If the page is the last one or no page exists */
6      /*
7      * Delay the window destruction until the main loop becomes idle.
8      * This function must be called after all other pending operations are completed.
9      * For example, we need to wait for the GtkAlertDialog's asynchronous
10     * cleanup and internal signal disconnections to safely finish.
11     * If the window is destroyed immediately here, it will cause
12     * a "GLib-GObject-CRITICAL" error.
13     */
```

```
14      g_idle_add_once ((GSourceOnceFunc) gtk_window_destroy, win);
15    }
16  }
```

- 3-4: If two or more pages remain, the current page is removed.
- 5-15: Otherwise (if only one page remains), destroy the window and terminate the program.

However, we must be careful about the order in which the window is destroyed and the dialog is cleaned up. At the moment the dialog button is pressed and the callback function is invoked, GTK's internal cleanup process for the dialog has not completely finished. GTK performs this remaining cleanup (such as disconnecting signals from the parent window) *after* the callback function returns. If we destroy the window immediately inside the callback, the dialog's parent window will no longer exist during GTK's cleanup phase, triggering a `GLib-GObject-CRITICAL` warning.

Therefore, we must wait for all of GTK's internal processes to finish before executing the main window's termination process. We achieve this using `g_idle_add_once ()`. This function schedules our destroy operation to run on the main loop once it becomes idle. By the time it executes, all high-priority operations (like the dialog cleanup) are guaranteed to be finished, allowing the window to be destroyed safely.

2.20.3 Closing the Window

The "x" button in the top right corner closes the window. If there is unsaved text when this button is pressed, we must notify the user and let them choose how to proceed, just as we did for a single page.

Since there may be unsaved changes across multiple pages, we will handle it as follows:

1. Intercept the "close-request" signal, which is emitted just before the window closes.
2. Iterate through all pages to create a list of unsaved documents.
3. If there are no unsaved pages, close the window normally.
4. If there are unsaved pages, display the list in an alert dialog and ask the user whether to proceed.
5. Handle the result in a callback function.

2.20.3.1 The close-request Signal

The `close-request` signal is defined in `GtkWindow`, and its default handler is a class method. The following code is an excerpt from `gtkwindow.h`.

```
struct _GtkWindowClass
{
  GtkWidgetClass parent_class;

  ... ... ...

  /**
   * GtkWindowClass::close_request:
   *
   * Class handler for the [signal@Window::close-request] signal.
   *
   * Returns: Whether the window should be destroyed
   */
  gboolean (* close_request)    (GtkWindow *window);

  ... ... ...
}
```

To use this signal in a subclass, we override the class method rather than using `g_signal_connect` (which is intended for connecting signals **externally**). This override is performed in the class initialization function.

```
1  static void
2  tfe_window_class_init (TfeWindowClass *class) {
3    GObjectClass *object_class = G_OBJECT_CLASS (class);
4    GtkWindowClass *window_class = GTK_WINDOW_CLASS (class);
5
6    object_class->dispose = tfe_window_dispose;
7    window_class->close_request = tfe_window_close_request;
8    gtk_widget_class_set_template_from_resource (GTK_WIDGET_CLASS (class), "/com/github/ToshioCP/tfe8/
9    gtk_widget_class_bind_template_child (GTK_WIDGET_CLASS (class), TfeWindow, btnm);
10   gtk_widget_class_bind_template_child (GTK_WIDGET_CLASS (class), TfeWindow, nb);
11 }
```

- 4: Since the defining class of the "close-request" signal is `GtkWindow`, we cast it to a `GtkWindowClass` pointer and assign it to the `window_class` variable.
- 7: Override the handler.

2.20.3.2 The close-request Signal Handler

The code for the handler is as follows:

```
1  static gboolean
2  tfe_window_close_request (GtkWindow *window)
3  {
4    TfeWindow *win = TFE_WINDOW (window);
5    GtkWidget *scr;
6    GtkWidget *tv;
7    const char *tab_label;
8    int n_pages = gtk_notebook_get_n_pages (win->nb);
9    int unsaved_count = 0;
10   GString *unsaved_list = g_string_new ("");
11   GtkAlertDialog *dialog;
12   const char *buttons[] = {"Cancel", "Close Window Without Saving", NULL};
13
14   for (int i = 0; i < n_pages; i++) {
15     scr = gtk_notebook_get_nth_page (win->nb, i);
16     tv = gtk_scrolled_window_get_child (GTK_SCROLLED_WINDOW (scr));
17
18     if (tfe_text_view_get_modified (TFE_TEXT_VIEW (tv))) {
19       unsaved_count++;
20       tab_label = gtk_notebook_get_tab_label_text (win->nb, scr);
21       if (tab_label != NULL) {
22         g_string_append_printf (unsaved_list, "- %s\n", tab_label);
23       } else {
24         g_string_append_printf (unsaved_list, "- Page %d\n", i + 1);
25       }
26     }
27   }
28
29   if (unsaved_count == 0) {
30     g_string_free (unsaved_list, TRUE);
31
32     /* Chain up to the parent's close_request handler */
33     return GTK_WINDOW_CLASS (tfe_window_parent_class)->close_request (window);
34   }
35
```

```
36    dialog = gtk_alert_dialog_new (
37      unsaved_count == 1 ? "1 unsaved change" : "%d unsaved changes",
38      unsaved_count
39    );
40    gtk_alert_dialog_set_detail (dialog, unsaved_list->str);
41    gtk_alert_dialog_set_buttons (dialog, buttons);
42    gtk_alert_dialog_set_cancel_button (dialog, 0);
43    gtk_alert_dialog_set_default_button (dialog, 0);
44    gtk_alert_dialog_choose (dialog, GTK_WINDOW (win), NULL, window_close_alert_cb, win);
45
46    g_string_free (unsaved_list, TRUE);
47    g_object_unref (dialog);
48
49    /* Stop the close request */
50    return TRUE;
51  }
```

- 10: Use a `GString` to generate the description for the alert dialog. A `GString` is a GLib structure that automatically handles memory allocation as you append text. It is initialized as an empty string and is perfect for safely building dynamic strings.
- 14-27: Iterate through the pages. If any text is unsaved, append the page's tab name to `unsaved_list`. If the page tab lacks a `GtkLabel`, it defaults to displaying a placeholder like "Page 1", so we construct that string in place of a filename.
- 29-34: If there are no unsaved pages, free the `GString`, chain up to the parent class's `close-request` signal handler, and finish. Since the parent class's handler returns a `gboolean`, we simply return its value directly.
- 36-44: Construct the alert dialog and call `gtk_alert_dialog_choose ()`. This process is nearly identical to how we constructed the dialog for closing a single page.
- 46-47: Free the `GString` and the dialog instance.
- 50: By returning `TRUE`, we halt the closing process. The actual window closure will be handled later in the callback function, depending on the user's selection.

2.20.3.3 The callback function

The callback function looks like this:

```
1   static void
2   window_close_alert_cb (GObject *source_object, GAsyncResult *res, gpointer user_data)
3   {
4     GtkAlertDialog *dialog = GTK_ALERT_DIALOG (source_object);
5     GtkWidget *win;
6     GError *err = NULL;
7
8     int response = gtk_alert_dialog_choose_finish (dialog, res, &err);
9
10    if (err != NULL) {
11        g_warning ("Alert Dialog Error: %s", err->message);
12        g_error_free (err);
13        return;
14    }
15
16    win = GTK_WIDGET (user_data);
17    if (response == 1) { /* Close Without Saving */
18        g_idle_add_once ((GSourceOnceFunc) gtk_window_destroy, win);
19    }
```

```
20 }
```

The logic here is almost identical to the `page_close_alert_cb` explained in the previous section, except that it destroys the entire window at the end instead of closing a single page. As discussed earlier, we again use `g_idle_add_once ()` to safely wait for the alert dialog's cleanup process to finish.

2.20.4 Application Quit Process

Quitting the application is triggered by clicking the "quit" menu item. The corresponding action is `app.quit`, defined in `tfeapplication.c`.

Previously, its handler called the `g_application_quit ()` function, but once invoked, the application termination process cannot be aborted. To fix this, we will replace it with a routine that closes the main windows instead, allowing the `close-request` handler to intercept the process if there is unsaved text.

```
1  static void
2  quit_activated (GSimpleAction *action, GVariant *parameter, gpointer user_data) {
3    GtkApplication *app = GTK_APPLICATION (user_data);
4
5    GList *windows = gtk_application_get_windows (app);
6    /*
7    * Closing a window removes it from the application's internal list.
8    * To avoid mutating the list while iterating over it, we must use a copy.
9    */
10   GList *list_copy = g_list_copy (windows);
11
12   for (GList *l = list_copy; l != NULL; l = l->next) {
13     gtk_window_close (GTK_WINDOW (l->data));
14   }
15
16   g_list_free (list_copy);
17 }
```

- 5: Obtains the list of top-level windows of the application.
- 10: Makes a shallow copy of the list. Because closing a window removes it from the application's internal list (`windows`), the original list will change during the loop. To safely iterate over the windows without the list mutating under us, we must use a copy.
- 12-14: Iterate through the list of top-level windows held by `GtkApplication`, closing all of them with the `gtk_window_close ()` function. Since this function emits the "close-request" signal, any unsaved text will automatically be caught and handled by our custom handler. Because TFE only has one main window, iterating through a list like this isn't strictly necessary, but the code is written this way to maintain generality.

2.20.5 Compiling and Running

All the source files are in the src/tfe8 directory. You can build and execute the program as follows.

First, change your current directory to `src/tfe8`. Then, type below in your command line.

```
$ meson setup _build
$ ninja -C _build
$ _build/tfe
```

Try modifying the page and clicking the close button. An alert dialog will appear, prompting you to choose either Cancel or Close.

2.21 Accelerators

2.21.1 Accelerators

Accelerators are the mechanism used to implement keyboard shortcuts. For instance, `Ctrl+c` is a common shortcut for copying text.

Because accelerators are incredibly convenient, you should definitely implement them for frequently used features. Especially in a keyboard-centric application like our Text File Editor (TFE), allowing users to complete tasks without reaching for the mouse is essential for a comfortable user experience.

2.21.2 Accelerators and Actions

Accelerators are rarely used in isolation; they are usually paired with buttons or menu items. For example, most text editors have an "Open File" button or menu item, but you can achieve the same result by pressing `Ctrl+o`.

In GTK, the UI elements that trigger events (like buttons, menus, and shortcut keys) are separated from the "actions" that perform the actual work. Thanks to this architecture, you can connect multiple types of events to a single action, keeping your code clean and straightforward. Without actions, you would have to write the handling logic for every single event individually, which would lead to messy and complicated code.

Since accelerators function across the entire application, they should be defined in the application source file (e.g., `tfeapplication.c`). To link an accelerator to an action, we use the `gtk_application_set_accels_for_action ()` function.

```
void
gtk_application_set_accels_for_action (
  GtkApplication* application,
  const char* detailed_action_name,
  const char* const* accels
)
```

- `application`: The application instance.
- `detailed_action_name`: The name of the action (e.g., `"app.quit"`, `"win.save"`).
- `accels`: An array of strings representing the shortcut keys (the syntax is explained in the next subsection). This array must be `NULL`-terminated.

2.21.3 Shortcut Key Strings

The GTK parser for shortcut keys can properly interpret strings like `<control>o`, `<Shift><Alt>F2`, and `<Ctrl>minus`. You can find detailed information about these strings in the documentation for the `gtk_accelerator_parse ()` function.

This syntax is divided into two parts: a modifier key and a character key. For example, `<control>o` is split into `<control>` and `o`.

- The syntax for modifier keys is quite flexible. It allows both uppercase and lowercase letters, as well as abbreviations. For instance, `<control>`, `<CONTROL>`, `<Ctrl>`, and `<Ctl>` all represent the Control key.
- For the character part, the parser calls the `gdk_keyval_from_name ()` function to translate the "key name string" into a key code. This key name string **must be written in lowercase**. This is because typing an uppercase 'O' on a keyboard requires pressing the Shift key and the 'o' key together, meaning an uppercase `O` is treated as being equivalent to `<Shift>o`. Therefore, if you write `<Ctrl>O`, GTK will interpret it as `<Ctrl><Shift>o`.
- Because the character part must be a lowercase alphabet name, you cannot use symbols directly. To represent the minus (`-`) key, for example, you must spell it out as `minus`. This means `<Ctrl>-` will

not work; you must write `<Ctrl>minus`. The underlying definitions for these names can be found in the GTK source code within gdkkeysyms.h. The key string is based on the macro name defined there (such as `GDK_KEY_minus`), simply with the `GDK_KEY_` prefix removed.

2.21.4 Implementation in the TFE Program

Let's incorporate accelerators into our TFE (Text File Editor) C code. Since shortcuts should be registered when the application initializes, we will add them to the `"startup"` signal handler.

For example, to link `Ctrl+o` to the `"win.open"` action, you would write:

```
const char *open_accels[] = {"<Ctrl>o", NULL};
gtk_application_set_accels_for_action(GTK_APPLICATION(app), "win.open", open_accels);
```

When you have multiple shortcut keys, the best practice is to define them in an array and register them using a loop.

```
  struct {
    const char *action;
    const char *accels[2];
  } action_accels[] = {
    { "win.open", { "<Control>o", NULL } },
    { "win.save", { "<Control>s", NULL } },
    { "win.close", { "<Control>w", NULL } },
    { "win.new", { "<Control>n", NULL } },
    { "win.saveas", { "<Shift><Control>s", NULL } },
    { "app.quit", { "<Control>q", NULL } },
  };

  for (i = 0; i < G_N_ELEMENTS(action_accels); i++)
    gtk_application_set_accels_for_action(GTK_APPLICATION(app), action_accels[i].action, action_accel
```

- 1-4: This is the structure definition. It might look a bit complex, but it becomes clearer when you look at the initialization values starting on line 5. The first member is the action name, like `"win.open"`. The second member is an array of accelerators (shortcut strings), ending with a `NULL` terminator. Since we are only registering one shortcut per action here, an array size of 2 (the shortcut plus `NULL`) is sufficient. If you wanted to assign two shortcuts to a single action, you would need to increase the array size to 3. The empty brackets `[]` after `action_accels` tell the compiler to calculate the array size automatically based on the initialization data, so we can omit the number.
- 5-10: This is the actual list pairing each action with its corresponding accelerator. Note that we will add a menu and action for a Preferences dialog later on, but since that isn't a frequently used feature, we are omitting an accelerator for it here.

2.21.5 Build and Run

The complete source code is located in the src/tfe9 directory. Navigate into that directory and type the following commands in your terminal to compile and run the application:

```
$ meson setup _build
$ ninja -C _build
$ _build/tfe
```

Try pressing shortcut keys like `Ctrl+o` to confirm that they are working correctly.

2.22 CSS and Pango

GTK uses CSS (Cascading Style Sheets) to style widgets, just like web browsers do. GTK supports the W3C CSS Fonts Module Level 3, but not necessarily every feature in the specification. Refer to the official documentation for details:

- CSS in GTK
- GTK CSS Properties

We briefly used CSS in Section 17 to change a label's background color, focusing mostly on CSS providers. Now, let's look closer at how CSS itself works in GTK.

2.22.1 CSS Basics

2.22.1.1 CSS Syntax

CSS uses very simple rules. It combines three main elements: "Where" (Selector), "What" (Property), and "How" (Value).

```
selector {
  property: value;
  another-property: another-value;
}
```

- selector: The target widget you want to style.
- property: The attribute you want to change (like color, margin, or font).
- value: The specific setting for that property (like `red` or `10px`).

```
button {
  color: white;             /* Text color */
  background-color: blue;   /* Background color */
  font-weight: bold;        /* Text weight */
}
```

2.22.1.2 Types of Selectors

GTK CSS uses three main selectors to target widgets:

2.22.1.2.1 Node Targets the widget type itself, applying the style to all widgets of that kind. You can find these node names in the CSS section of the official GTK documentation for each widget. For example, "window" is the node for GtkWindow, "button" for GtkButton, and "textview" for GtkTextView.

```
/* Makes the background of ALL text views light gray */
textview {
  background-color: lightgray;
}
```

2.22.1.2.2 Class Targets a specific group of widgets. You define a class by adding a dot (`.`) before the class name. For example, if you have four stacked labels, you could assign an `.upper` class to the top two and a `.lower` class to the bottom two.

```
/* Applies only to labels with the "upper" class */
label.upper {
  background-color: green;
}
```

2.22.1.2.3 ID Targets one unique widget in your entire application. You define an ID by adding a hash (`#`) before the name.

```
/* Applies ONLY to the specific widget named "my-main-button" */
#my-main-button {
  font-size: 20pt;
}
```

2.22.1.3 Combining Selectors

In a real app, you might want to style a "button inside a header bar" differently from a normal button. By **separating selectors with a space**, you can target "something inside something else" (called a descendant selector).

```
/* Applies ONLY to buttons inside a header bar */
headerbar button {
  color: red;
}

/* Applies ONLY to buttons inside a box with the "upper" class */
box.upper button {
  font-weight: bold;
}
```

2.22.1.4 The Box Model

GTK CSS uses the standard "Box Model" for sizing. From the inside out, it consists of:

1. Content: The actual item, like text or an icon.
2. Padding: The inner spacing between the content and the border.
3. Border: The frame surrounding the widget (even if invisible).
4. Margin: The outer spacing that pushes neighboring widgets away.

```
button {
  padding: 8px 16px;      /* Inner spacing: 8px top/bottom, 16px left/right */
  border: 2px solid red; /* Border: 2px solid red line */
  margin: 10px;           /* Outer spacing: 10px gap all around */
}
```

2.22.2 Widgets and CSS

To apply styles, you must assign a node, class, or ID to a specific widget using C functions:

- Node: Fixed by the widget type (e.g., `GtkLabel` node is "label"). No function is needed.
- Class: Manage the widget classes with `gtk_widget_add_css_class()`, `gtk_widget_set_css_classes()`, or `gtk_widget_remove_css_class()`.
- ID: Corresponds to the widget's name, which you set using `gtk_widget_set_name()`.

2.22.3 Applying CSS

We covered applying CSS in Section 17, but here is a quick recap:

- Load your CSS into a `GtkCssProvider` using `gtk_css_provider_load_from_string()`.
- Apply that provider to the default `GdkDisplay` using `gtk_style_context_add_provider_for_display()`.
- A display can have multiple providers. If styles conflict, the provider with the higher priority wins. The `GTK_STYLE_PROVIDER_PRIORITY_APPLICATION` priority is appropriate for application style providers. (Tip: You can check the exact priority rankings in the `gtkstyleprovider.h` in the GTK source files).

2.22.4 TFE Example: Applying Base CSS

Let's look at how TFE (Text File Editor) applies CSS. TFE uses two separate providers:

- Base CSS: Loaded at startup. Right now, it just adds a 10px padding to the GtkTextView to make it look better.
- Font CSS: Allows users to change fonts dynamically via a font button (we will build this in the next section).

Let's apply the Base CSS in our startup handler.

```
#include "tfestylemanager.h"

  ... ... ...

static void
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  int i;
  const char *base_css = "textview {padding: 10px;}";

  G_APPLICATION_CLASS (tfe_application_parent_class)->startup (application);

  tfe_style_manager_initialize (base_css); /* initialize the style manager helper functions */

  ... ... ...

}

static void
app_shutdown (GApplication *application) {
  tfe_style_manager_terminate ();

 G_APPLICATION_CLASS (tfe_application_parent_class)->shutdown (application);
}
```

The files `tfestylemanager.h` and `tfestylemanager.c` contain our CSS helper functions.

- `tfe_style_manager_initialize()` initializes the library and loads the Base CSS at startup.
- `tfe_style_manager_terminate()` cleans it up during the application's dispose phase.

The two functions are called in the startup and shutdown handlers respectively.

Here is the implementation from `tfestylemanager.c`:

```
static gboolean initialized = FALSE;
static GdkDisplay *display;
static GtkCssProvider *provider_base;
static GtkCssProvider *provider_font;

void
tfe_style_manager_initialize (const char *base_css) {
  if (initialized)
    return;

  display = gdk_display_get_default (); /* This is a singleton and should not be freed */
  if (display == NULL) {
    g_warning ("Failed to get default display");
```

```
    return;
  }

  if (base_css != NULL) {
    provider_base = gtk_css_provider_new ();
    gtk_css_provider_load_from_string (provider_base, base_css); /* Recommended for GTK 4.12+ */
    gtk_style_context_add_provider_for_display (display, GTK_STYLE_PROVIDER (provider_base), GTK_STYL
  }

  provider_font = gtk_css_provider_new ();
  gtk_style_context_add_provider_for_display (display, GTK_STYLE_PROVIDER (provider_font), GTK_STYLE_

  initialized = TRUE;
}

void
tfe_style_manager_terminate (void) {
  if (provider_base)
    gtk_style_context_remove_provider_for_display (display, GTK_STYLE_PROVIDER (provider_base));
  if (provider_font)
    gtk_style_context_remove_provider_for_display (display, GTK_STYLE_PROVIDER (provider_font));

  g_clear_object (&provider_base);
  g_clear_object (&provider_font);
  display = NULL;
  initialized = FALSE;
}
```

Here is what the initialization does:

1. Grabs the default `GdkDisplay`.
2. Loads the Base CSS into `provider_base` and applies it to the display.
3. Applies an *empty* `provider_font` to the display. We can safely attach this empty provider now and inject the Font CSS into it later using `tfe_style_manager_set_font()`.

Add `tfestylemanager.c` to your `meson.build` file:

```
project('tfe', 'c')

gtkdep = dependency('gtk4')

gnome=import('gnome')
resources = gnome.compile_resources('resources','tfe.gresource.xml')

# Main application
sourcefiles=files('main.c', 'tfeapplication.c', 'tfewindow.c', 'tfetextview.c', 'tfestylemanager.c')
executable('tfe', sourcefiles, resources, dependencies: gtkdep)
```

THe source files are in the src/tfe10 directory.

Compile and run the app. You should now see a 10px padding around your text view.

2.22.4.1 Pango

Pango is the library GTK uses under the hood to handle text rendering and layout.

When we add a font dialog later, the user's selected font will be returned as a `PangoFontDescription`

structure. We need to convert this Pango format into CSS, which requires a bit of manual work since GTK doesn't do it automatically.

Here are the key functions to extract font data from Pango:

- `pango_font_description_get_family()`: Gets the font family (e.g., "sans-serif").
- `pango_font_description_get_size()`: Gets the font size. (Note: Divide this by the `PANGO_SCALE` constant to get standard points or pixels.)
- `pango_font_description_get_size_is_absolute()`: Returns `TRUE` for pixels (px) and `FALSE` for points (pt).
- `pango_font_description_get_weight()`: Returns the weight (e.g., `PANGO_WEIGHT_NORMAL`).
- `pango_font_description_get_style()`: Returns the style (e.g., `PANGO_STYLE_ITALIC`).

You can also easily convert a font description to and from a readable string using `pango_font_description_from_string()` and `pango_font_description_to_string()`.

The string format looks like this:

`[FAMILY-LIST] [STYLE-OPTIONS] [SIZE] [VARIATIONS] [FEATURES]`

You don't need to fill out every option. A simple string like `"sans-serif Bold 12"` works perfectly.

2.22.4.2 Pango to CSS Conversion

To make our font button work, we need to bridge the gap between Pango and CSS. The helper function `font_desc_to_css()` does exactly this.

```
static int
pango_weight2int (PangoWeight pango_weight) {
  switch (pango_weight) {
  case PANGO_WEIGHT_THIN:
    return 100;
  case PANGO_WEIGHT_ULTRALIGHT:
    return 200;
  case PANGO_WEIGHT_LIGHT:
    return 300;
  case PANGO_WEIGHT_SEMILIGHT:
    return 350;
  case PANGO_WEIGHT_BOOK:
    return 380;
  case PANGO_WEIGHT_NORMAL:
    return 400; /* or "normal" */
  case PANGO_WEIGHT_MEDIUM:
    return 500;
  case PANGO_WEIGHT_SEMIBOLD:
    return 600;
  case PANGO_WEIGHT_BOLD:
    return 700; /* or "bold" */
  case PANGO_WEIGHT_ULTRABOLD:
    return 800;
  case PANGO_WEIGHT_HEAVY:
    return 900;
  case PANGO_WEIGHT_ULTRAHEAVY:
    /* CSS Fonts Level 4 defines 1000, but GTK only supports Level 3 (max 900). */
    return 900;
  default:
    return 400; /* "normal" */
```

```
  }
}

static char *
font_desc_to_css (const PangoFontDescription *desc) {
  const char *family;
  double size;
  const char *size_unit;
  int weight;
  PangoStyle style;
  const char *style_str;
  char *css;

  family = pango_font_description_get_family (desc); /* The return string is owned by the instance.
  size = (double) pango_font_description_get_size (desc) / PANGO_SCALE;
  if (size <= 0)
    size = 11.0; /* Default font size */
  size_unit = pango_font_description_get_size_is_absolute (desc) ? "px" : "pt";
  weight = pango_weight2int (pango_font_description_get_weight (desc));
  style = pango_font_description_get_style (desc);

  switch (style) {
    case PANGO_STYLE_ITALIC:
      style_str = "italic";
      break;
    case PANGO_STYLE_OBLIQUE:
      style_str = "oblique";
      break;
    case PANGO_STYLE_NORMAL:
    default:
      style_str = "normal";
      break;
  }

  css = g_strdup_printf ("textview {font-family: \"%s\"; font-size: %.1f%s; font-weight: %d; font-sty
                         family ? family : "sans-serif", /* family can be NULL */
                         size,
                         size_unit,
                         weight,
                         style_str);

  return css;
}
```

This function turns a PangoFontDescription (which is a C structure) into proper CSS. For example, if the description contains “sans-serif”, “Bold”, and “12pt”, the CSS string will be:

```
textview {font-family: "sans-serif"; font-size: 12pt; font-weight: 700; font-style: normal;}
```

Finally, here is our public interface, `tfe_style_manager_set_font()`:

```
void
tfe_style_manager_set_font (const PangoFontDescription *desc) {
  g_return_if_fail (initialized);
  g_return_if_fail (desc != NULL);
```

```
5
6     char *css_string;
7
8     css_string = font_desc_to_css (desc);
9
10    gtk_css_provider_load_from_string (provider_font, css_string);
11    g_free (css_string);
12  }
```

This function creates the CSS string and feeds it directly into the `provider_font` (GtkCssProvider instance). Since the provider is already linked to the display, the screen updates instantly.

Writing these conversion functions from scratch takes time because you have to cross-reference Pango and CSS documentation. Thankfully, our helper library handles the heavy lifting.

In the next section, we'll use these helpers to build the actual font selection feature.

2.23 GtkFontDialogButton and CSS Updates

2.23.1 The Preference Dialog

When you click "Preferences" menu on the new TFE application, the following preference dialog appears.

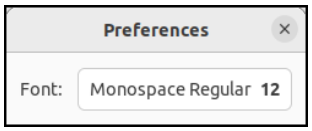


Figure 2.26: Preference dialog

A preference dialog allows users to customize settings, such as fonts. Nowadays, it is often called a "Settings" dialog.

This dialog contains only one bUtton: a `GtkFontDialogButton`. You can add more widgets to this dialog, but this simple layout is enough for our first version.

Clicking this button opens `GtkFontDialog` to select a font.

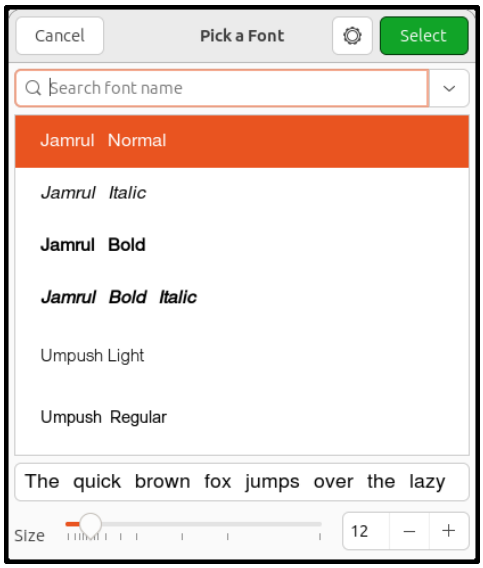


Figure 2.27: Font dialog

When the user selects a font and clicks the "Select" button, the application's font updates.

Note that `GtkFontDialogButton` and `GtkFontDialog` are new widgets introduced in GTK 4.10. They replace the older `GtkFontButton` and `GtkFontChooserDialog`, which were deprecated in version 4.10.

2.23.1.1 The Composite Widget

This preference dialog contains a `GtkBox`, a `GtkLabel`, and a `GtkFontDialogButton`, and is defined as a composite widget. Here is the template UI file for `TfePref`:

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="TfePref" parent="GtkWindow">
    <property name="title">Preferences</property>
    <property name="resizable">FALSE</property>
    <property name="modal">TRUE</property>
    <child>
      <object class="GtkBox">
        <property name="orientation">horizontal</property>
        <property name="spacing">12</property>
        <property name="halign">center</property>
        <property name="margin-start">12</property>
        <property name="margin-end">12</property>
        <property name="margin-top">12</property>
        <property name="margin-bottom">12</property>
        <child>
          <object class="GtkLabel">
            <property name="label">Font:</property>
            <property name="xalign">1</property>
          </object>
        </child>
        <child>
          <object class="GtkFontDialogButton" id="font_dialog_btn">
            <property name="dialog">
              <object class="GtkFontDialog"/>
            </property>
          </object>
        </child>
      </object>
    </child>
  </template>
</interface>
```

- The `<template>` tag specifies that this is a composite widget.
- The `class` attribute is set to “TfePref” and the `parent` attribute to GtkWindow. This makes `TfePref` a child class of GtkWindow.
- It sets three properties: `title`, `resizable`, and `modal`. When `modal` is set to `TRUE`, the user cannot interact with the parent window while this dialog is open.
- Inside, there is a horizontal `GtkBox` containing a `GtkLabel` and a `GtkFontDialogButton`.

2.23.1.2 The Header File

The `tfepref.h` file defines the types and declares the public function.

```
#pragma once

#include <gtk/gtk.h>

```

```
5  #define TFE_TYPE_PREF tfe_pref_get_type ()
6  G_DECLARE_FINAL_TYPE (TfePref, tfe_pref, TFE, PREF, GtkWindow)
7
8  GtkWidget *
9  tfe_pref_new (GtkApplication *application);
```

- 5: Defines the `TFE_TYPE_PREF` macro, which is replaced by `tfe_pref_get_type()`.
- 6: Uses the `G_DECLARE_FINAL_TYPE` macro to automatically generate standard boilerplate code.
- 8-9: Declares the public function `tfe_pref_new()`, which creates a new `TfePref` instance. It takes a GtkApplication argument, which will be assigned to "application" property.

2.23.1.3 The C Source File for the Composite Widget

The following code is the `tfepref.c` file.

```
1  #include <gtk/gtk.h>
2  #include "tfestylemanager.h"
3  #include "tfepref.h"
4
5  struct _TfePref
6  {
7    GtkWindow parent;
8    GtkFontDialogButton *font_dialog_btn;
9  };
10
11 G_DEFINE_FINAL_TYPE (TfePref, tfe_pref, GTK_TYPE_WINDOW);
12
13 static void
14 font_dialog_btn_notify_cb (GtkFontDialogButton *btn, GParamSpec *pspec, gpointer user_data) {
15   PangoFontDescription *font_desc = gtk_font_dialog_button_get_font_desc (btn);
16
17   tfe_style_manager_set_font (font_desc);
18 }
19
20 static void
21 tfe_pref_dispose (GObject *gobject) {
22   TfePref *pref = TFE_PREF (gobject);
23
24   gtk_widget_dispose_template (GTK_WIDGET (pref), TFE_TYPE_PREF);
25   G_OBJECT_CLASS (tfe_pref_parent_class)->dispose (gobject);
26 }
27
28 static void
29 tfe_pref_init (TfePref *pref) {
30   gtk_widget_init_template (GTK_WIDGET (pref));
31   g_signal_connect (pref->font_dialog_btn, "notify::font-desc", G_CALLBACK (font_dialog_btn_notify_cl
32 }
33
34 static void
35 tfe_pref_class_init (TfePrefClass *class) {
36   G_OBJECT_CLASS (class)->dispose = tfe_pref_dispose;
37   gtk_widget_class_set_template_from_resource (GTK_WIDGET_CLASS (class), "/com/github/ToshioCP/tfe11/
38   gtk_widget_class_bind_template_child (GTK_WIDGET_CLASS (class), TfePref, font_dialog_btn);
39 }
40
```

```
41 GtkWidget *
42 tfe_pref_new (GtkApplication *application) {
43   return GTK_WIDGET (g_object_new (TFE_TYPE_PREF, "application", application, NULL));
44 }
```

- 5-9: The `_TfePref` structure includes a `font_dialog_btn` member. This holds a pointer to the `GtkFontDialogButton` instance defined in the XML file (`tfepref.ui`). This member name must exactly match the `id` attribute in the XML.
- 11: The `G_DEFINE_FINAL_TYPE` macro expands into standard boilerplate code.
- 13-18: `font_dialog_btn_notify_cb()` is the notify signal handler for the font dialog button's `font-desc` property. This signal is emitted whenever the property changes. The handler retrieves the `font-desc` property (which is a `PangoFontDescription`) and passes it to our `tfe_style_manager_set_font()` style manager function. Because `tfe_style_manager_set_font()` not only converts this data into a CSS string but also applies it to the `GdkDisplay`, simply calling this function instantly updates the screen's style.
- 20-26: The dispose handler uses `gtk_widget_dispose_template()` to destroy the instance built from the template.
- 28-32: The instance initialization function uses `gtk_widget_init_template()` to build the instance from the template. It also connects the notify signal for the `font-desc` property to our callback handler.
- 34-39: `tfe_pref_class_init()` loads the template and binds the `font_dialog_btn` instance variable to the UI file.
- 41-44: The constructor `tfe_pref_new()` creates the new instance using `g_object_new()`. The argument `application` is assigned to the "application" property.

2.23.1.4 Handling it in the Application

The preference dialog opens when the user clicks "Preferences" in the application menu. Therefore, we need to add code to generate and display the dialog inside the application's action handler.

```
1  static void
2  pref_activated (GSimpleAction *action, GVariant *parameter, gpointer user_data) {
3    GtkApplication *app = GTK_APPLICATION (user_data);
4    GtkWindow *win = gtk_application_get_active_window (app);
5    GtkWidget *pref_dialog;
6
7    pref_dialog = tfe_pref_new (app);
8    gtk_window_set_transient_for (GTK_WINDOW (pref_dialog), win);
9    gtk_window_present (GTK_WINDOW (pref_dialog));
10 }
```

The logic here is very simple: it creates the preference dialog, sets the main window as its transient parent window, and shows the dialog. The user closes the dialog by clicking the window's close (X) button. Once closed, the dialog instance is destroyed.

2.23.2 Compile and Run

The source code is located in the src/tfe11 directory.

Before compiling, you need to make a few changes:

- Change the Application ID in `main.c` to "com.github.ToshioCP.tfe11".
- Change the prefix in `tfe.gresource.xml` to "/com/github/ToshioCP/tfe11". Be sure to update the UI file loading prefix in `tfewindow.c` to match.
- Add `tfepref.c` to the source files list in `meson.build`.

Move your current directory to `src/tfe11` and type the following commands:

```
$ meson setup _build
$ ninja -C _build
$ _build/tfe
```

Select "Preferences" from the menu to open the dialog. Click the font button to open the font chooser, pick a font and size, and the chooser will close, returning you to the preference dialog. Now, click the close (X) button at the top right to close the preference dialog. You should see that the text view in the main window is now using your selected font.

This setting remains active while the program is running, but once you exit the application, all settings are lost. The next time you launch the app, it will revert to the default font.

To save these settings so they persist across restarts, the best practice in GTK is to use GSettings. In the next section, we will wrap up the TFE project by covering how to implement GSettings.

2.24 Gsettings

This section introduces GSettings, a convenient API that allows applications to save and retrieve settings persistently.

2.24.1 GSettings

We want to save the user's font preferences so they persist even after the application quits. There are a few ways to achieve this:

- Create a custom configuration file: For example, we could manually read and write font information to a simple text file like `~/.config/tfe/font_desc.cfg`.
- Use the GSettings API: While the basic concept is similar to a configuration file, GSettings handles all the heavy lifting. It safely stores the configuration data in a system-specific backend database (such as dconf on Linux). This means we don't have to write our own text parsers or worry about file I/O errors.

Using GSettings in your code is simple and highly effective, but its underlying architecture can be a bit tricky to grasp at first. This subsection will explain the core concepts of GSettings before diving into the actual programming.

2.24.1.1 GSettings Schema

A GSettings schema defines a set of keys, their data types, and other metadata. The `GSettings` object uses this schema to correctly read and write key values to the backend database.

- Schema ID: Every schema must have a unique identifier. It is formatted as a reverse-DNS string delimited by periods (e.g., `com.github.ToshioCP.tfe`). While the schema ID and the application ID are technically distinct, it is a common and highly recommended convention to use the exact same string for both.
- Path: A schema typically has a path, which acts as its location directory in the database. A path must start and end with a slash (`/`), and its internal segments are delimited by slashes. For example, if a key `font-desc` is defined under the path `/com/github/ToshioCP/tfe/`, its absolute location in the database becomes `/com/github/ToshioCP/tfe/font-desc`.
- Keys and Values: GSettings stores information as key-value pairs.
  - Keys: A key name must begin with a lowercase letter, followed by lowercase letters, digits, or dashes (`-`), and end with a lowercase letter or digit. Consecutive dashes are not allowed.
  - Values: Values are stored as `GVariant` types, meaning they can be simple types (integers, doubles, booleans, strings) or complex types (like arrays). The exact type for each key must be explicitly defined in the schema.
- Default Value: Every key must have a mandatory default value.

- Summary and Description: You can optionally provide a short summary and a detailed description for each key to document its purpose.

Schemas are written in XML format. For example:

```
<?xml version="1.0" encoding="UTF-8"?>
<schemalist>
  <schema path="/com/github/ToshioCP/tfe/" id="com.github.ToshioCP.tfe">
    <key name="font-desc" type="s">
      <default>'Sans Regular 12'</default>
      <summary>Font</summary>
      <description>A font for textview.</description>
    </key>
  </schema>
</schemalist>
```

- 4: The `type` attribute is `"s"`, which represents a `GVariant` string.

For more details on GVariant type strings, see the GLib API Reference – GVariant Type Strings.

Other common type strings include:

- `"b"`: boolean (`gboolean`)
- `"i"`: 32-bit integer (`gint32`)
- `"d"`: double-precision floating point (`double`)

For further information, refer to the official documentation:

- GLib API Reference – GVariant Format Strings
- GLib API Reference – GVariant Text Format
- GLib API Reference – GVariant
- GLib API Reference – VariantType

2.24.1.2 Gsettings Command

2.24.1.3 The gsettings Command

First, let's try the `gsettings` command-line utility. It is a useful tool for inspecting and modifying GSettings configurations directly from the terminal.

```
$ gsettings help
Usage:
  gsettings --version
  gsettings [--schemadir SCHEMADIR] COMMAND [ARGS?]

Commands:
  help                        Show this information
  list-schemas                List installed schemas
  list-relocatable-schemas    List relocatable schemas
  list-keys                   List keys in a schema
  list-children               List children of a schema
  list-recursively            List keys and values, recursively
  range                       Queries the range of a key
  describe                    Queries the description of a key
  get                         Get the value of a key
  set                         Set the value of a key
  reset                       Reset the value of a key
  reset-recursively           Reset all values in a given schema
  writable                    Check if a key is writable
```

```
  monitor                   Watch for changes

Use "gsettings help COMMAND" to get detailed help.
```

To see all available schemas on your system, use the `list-schemas` command:

```
$ gsettings list-schemas
org.gnome.rhythmbox.podcast
ca.desrt.dconf-editor.Demo.Empty
org.gnome.gedit.preferences.ui
org.gnome.evolution-data-server.calendar
org.gnome.rhythmbox.plugins.generic-player

... ...
```

Each line represents a unique schema ID. Since schemas contain key-value configuration data, you can inspect their contents using the `list-recursively` command. Let's look at the keys and values for the `org.gnome.calculator` schema:

```
$ gsettings list-recursively org.gnome.calculator
org.gnome.calculator accuracy 9
org.gnome.calculator angle-units 'degrees'
org.gnome.calculator base 10
org.gnome.calculator button-mode 'basic'
org.gnome.calculator number-format 'automatic'
org.gnome.calculator precision 2000
org.gnome.calculator refresh-interval 604800
org.gnome.calculator show-thousands false
org.gnome.calculator show-zeroes false
org.gnome.calculator source-currency ''
org.gnome.calculator source-units 'degree'
org.gnome.calculator target-currency ''
org.gnome.calculator target-units 'radian'
org.gnome.calculator window-position (-1, -1)
org.gnome.calculator word-size 64
```

This schema is used by the GNOME Calculator application. Let's run the calculator, change its mode, and see how the schema updates.

```
$ gnome-calculator
```

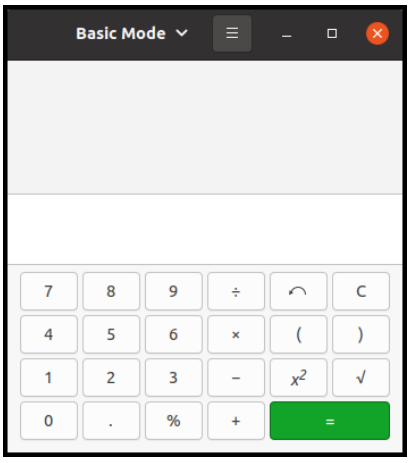


Figure 2.28: gnome-calculator basic mode

Change the calculator mode to “Advanced” and quit the application.

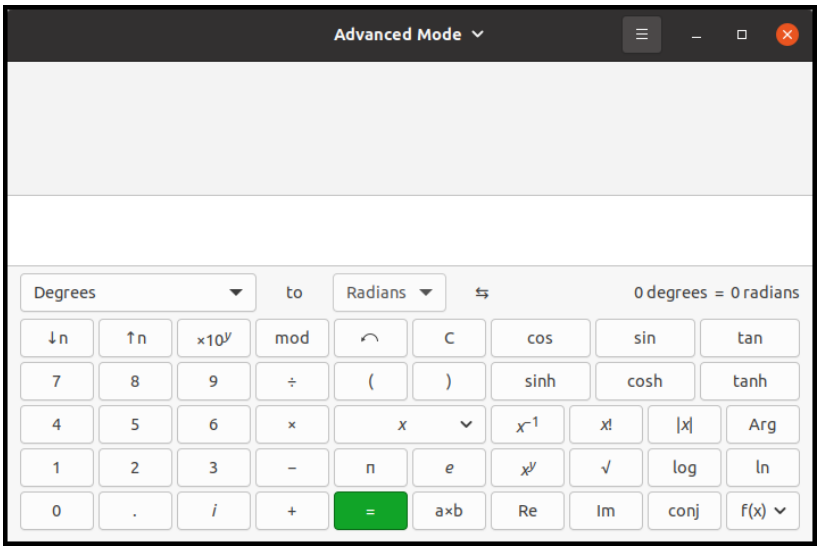


Figure 2.29: gnome-calculator advanced mode

Run `gsettings` again and check the value of the `button-mode` key:

```
$ gsettings list-recursively org.gnome.calculator

... ...

org.gnome.calculator button-mode 'advanced'

... ...
```

This demonstrates that GNOME Calculator uses GSettings. It updated the `button-mode` key to `'advanced'`, and this value persists even after the application is closed. Consequently, the next time you launch the calculator, it will read this setting and automatically open in Advanced mode.

2.24.1.4 The glib-compile-schemas Utility

GSettings schemas are specified in an XML format. The schema files must have the `.gschema.xml` extension. The following is the XML schema file for the `tfe` application:

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <schemalist>
3    <schema path="/com/github/ToshioCP/tfe/" id="com.github.ToshioCP.tfe">
4      <key name="font-desc" type="s">
5        <default>'Sans Regular 12'</default>
6        <summary>Font</summary>
7        <description>A font for textview.</description>
8      </key>
9    </schema>
10 </schemalist>
```

The filename is `com.github.ToshioCP.tfe.gschema.xml`. Schema XML filenames typically consist of the schema ID followed by the `.gschema.xml` extension. While you can use a name different from the schema ID, it is unconventional and not recommended.

- 2: The top-level element is `<schemalist>`.

- 3: The `<schema>` tag has `path` and `id` attributes. The `path` determines where the settings are stored in the conceptual global tree of settings, while the `id` uniquely identifies the schema.
- 4: The `<key>` tag has `name` and `type` attributes. The `name` is the identifier of the key, and the `type` specifies the data type of the key's value using a GVariant Format String.
- 5: The `<default>` tag sets the initial value of the `font-desc` key to `Sans Regular 12`. Note that the value must be enclosed in quotes because it is a GVariant string.
- 6-7: The `<summary>` and `<description>` elements describe the key. Although they are optional, it is recommended to include them in your XML file to document the settings.

These XML files must be compiled using the `glib-compile-schemas` utility. When executed, it compiles all files with the `.gschema.xml` extension in the specified directory and converts them into a single binary file named `gschemas.compiled`.

For example, suppose the XML file above is located in the `tfe12` directory:

```
$ glib-compile-schemas tfe12
```

This command generates the `gschemas.compiled` file inside the `tfe12` directory. When testing your application locally, you must set the `GSETTINGS_SCHEMA_DIR` environment variable so the `GSettings` object can find your newly compiled schema:

```
$ GSETTINGS_SCHEMA_DIR=(path_to_directory_with_gschemas.compiled)
```

At runtime, the `GSettings` object looks for the compiled schema using the following process:

- It searches the `glib-2.0/schemas` subdirectories of all the directories specified in the `XDG_DATA_DIRS` environment variable. Common directories are `/usr/share/glib-2.0/schemas` and `/usr/local/share/glib-2.0/schemas`.
- If `$HOME/.local/share/glib-2.0/schemas` exists, it is also searched.
- If the `GSETTINGS_SCHEMA_DIR` environment variable is defined, it searches all the directories specified within it. `GSETTINGS_SCHEMA_DIR` can specify multiple directories delimited by a colon (`:`).

System schema directories typically contain many `.gschema.xml` files. Therefore, when installing your application, follow these steps to properly install your schemas:

1. Create your `.gschema.xml` file.
2. Copy it to one of the system schema directories mentioned above (for example, `$HOME/.local/share/glib-2.0/schemas`).
3. Run `glib-compile-schemas` on that directory. This compiles all the schema files in the directory and creates or updates the `gschemas.compiled` binary file. This binary file acts as a fast index for the schema definitions (such as data types and default values). Note that this file is not the database itself. On Linux systems, the database that contains the actual preferences is usually dconf.

2.24.1.5 GSettings Object and Binding

Now, let's move on to the next topic: how to use GSettings in your C code.

Before writing the code, ensure your schema file is compiled. Let's assume the following identifiers for our example:

- GSettings ID: `com.github.ToshioCP.sample`
- GSettings key: `sample_key`
- Class name: `Sample`
- Property to bind: `sample_property`

The example below uses `g_settings_bind`. To use this function, the GSettings key and the instance property must have the exact same data type. For this example, we assume `sample_key` and `sample_property` share the same type.

```
GSettings *settings;
Sample *sample_object;
```

```
settings = g_settings_new ("com.github.ToshioCP.sample");
sample_object = sample_new ();
g_settings_bind (settings, "sample_key", sample_object, "sample_property", G_SETTINGS_BIND_DEFAULT);
```

The `g_settings_bind` function creates a bidirectional binding between the GSettings key and the object's property. If the property value changes, the GSettings database updates automatically, and vice versa. They are always kept in sync.

While `g_settings_bind` is simple and convenient, it cannot be used in every situation. GSettings keys are restricted to GVariant data types. However, some object properties use complex types that cannot be directly mapped to a GVariant.

For example, GtkFontDialogButton has a "font-desc" property of type PangoFontDescription. Since PangoFontDescription is a C structure, it is wrapped in a boxed GValue for the object property system. GVariant does not natively support these boxed types.

In these cases, you must use `g_settings_bind_with_mapping`. This function allows you to bind a GVariant-based GSettings key to an object property by providing custom mapping functions that translate the data back and forth. See the GIO documentation for further details.

```
void
g_settings_bind_with_mapping (
  GSettings* settings,
  const gchar* key,
  GObject* object,
  const gchar* property,
  GSettingsBindFlags flags, // G_SETTINGS_BIND_DEFAULT is commonly used
  GSettingsBindGetMapping get_mapping, // GSettings => property. See the example below.
  GSettingsBindSetMapping set_mapping, // property => GSettings. See the example below.
  gpointer user_data, // NULL if not needed
  GDestroyNotify destroy // NULL if not needed
)
```

The mapping functions are defined as follows:

```
gboolean
(* GSettingsBindGetMapping) (
  GValue* value,
  GVariant* variant,
  gpointer user_data
)

GVariant*
(* GSettingsBindSetMapping) (
  const GValue* value,
  const GVariantType* expected_type,
  gpointer user_data
)
```

The following code is extracted from `tfepref.c`:

```
1 static gboolean // GSettings => property
2 get_mapping (GValue *value, GVariant *variant, gpointer user_data) {
3   const char *s = g_variant_get_string (variant, NULL);
4   PangoFontDescription *font_desc = pango_font_description_from_string (s);
5   g_value_take_boxed (value, font_desc);
6   return TRUE;
```

```
7  }
8
9  static GVariant * // Property => GSettings
10 set_mapping (const GValue *value, const GVariantType *expected_type, gpointer user_data) {
11   PangoFontDescription *font_desc = g_value_get_boxed (value);
12   if (font_desc == NULL)
13     return NULL;  // Cancel the binding (GSettings will not be updated)
14   char *font_desc_string = pango_font_description_to_string (font_desc);
15   return g_variant_new_take_string (font_desc_string);
16 }
17
18 GtkWidget *
19 tfe_pref_new (GtkApplication *application) {
20   g_return_val_if_fail (GTK_IS_APPLICATION (application), NULL);
21
22   GtkWidget *pref =  GTK_WIDGET (g_object_new (TFE_TYPE_PREF, "application", application, NULL));
23   TfePref *self = TFE_PREF (pref);
24
25   self->settings = tfe_application_get_settings (TFE_APPLICATION (application));
26   g_object_ref (self->settings); /* get the ownership */
27
28   g_settings_bind_with_mapping (self->settings, "font-desc", self->font_dialog_btn, "font-desc", G_SE
29       get_mapping, set_mapping, NULL, NULL);
30
31   return pref;
32 }
```

- 1-7: These lines define the mapping function from GSettings to the object property. The first argument, `value`, is an empty `GValue` where the property data will be stored. The second argument, `variant`, is the `GVariant` structure retrieved from GSettings.
- 3: Retrieves the string from the `GVariant` structure.
- 4: Builds a `PangoFontDescription` structure from the string and assigns its address to `font_desc`.
- 5: Places `font_desc` into the `GValue` using `g_value_take_boxed`. Notice that this function transfers the ownership of `font_desc` to the `GValue`, meaning we do not need to free it manually.
- 6: Returns `TRUE` to indicate that the mapping was successful.
- 9-16: These lines define the mapping function from the object property back to GSettings. The first argument, `value`, holds the current property data. The second argument, `expected_type`, specifies the expected `GVariant` type for GSettings. It is unused in this specific function.
- 11: Retrieves the `PangoFontDescription` structure from the `GValue`.
- 12-13: If the font description is `NULL`, the function returns `NULL`. This safely cancels the binding operation, meaning the corresponding GSettings value will not be updated.
- 14: Converts the font description back into a string.
- 15: Creates a new `GVariant` from the string using `g_variant_new_take_string`. This function transfers the ownership of the newly allocated `font_desc_string` to the `GVariant`, which is then returned.
- 18-32: The `tfe_pref_new ()` function creates a new `TfePref` instance and initializes the GSettings binding.
- 25: Obtains the `GSettings` instance from the application.
- 26: Increments the reference count of the `GSettings` instance using `g_object_ref ()` to ensure the `TfePref` instance safely takes its ownership.
- 28-29: Binds the GSettings `"font-desc"` key to the `GtkFontDialogButton`'s `"font-desc"` property, utilizing our custom `get_mapping` and `set_mapping` functions.

2.24.2 TfeApplication and GSettings

When the application starts, it needs to read the font data from GSettings and apply it to the CSS. This should be handled inside the `startup` handler.

While the program is running, if the user changes the font in the preference dialog, there are two ways to reflect that change in the CSS:

1. In `tfepref.c`, catch the `notify` signal of the font button's "font-desc" property and call `tfe_style_manager_set_font ()`.
2. In `tfeapplication.c`, catch the "changed::font-desc" signal from GSettings and call `tfe_style_manager_set_font ()`. The "changed" signal on a GSettings object is emitted when a key has potentially changed, but this signal does not guarantee that the value has actually changed. You should call one of the g_settings_get() calls to check the new value. This signal supports detailed connections. You can connect to the detailed signal "changed::font-desc" in order to only receive callbacks when key "font-desc" changes. **Note that settings only emits this signal if you have read key at least once while a signal handler was already connected for key.**

We will choose approach 2. This ensures a consistent rule across our application: "The CSS always reflects GSettings."

The process from clicking the font button to updating the CSS is slightly complex. Let's review the exact steps:

- The user clicks the `GtkFontDialogButton` and the `GtkFontDialog` appears.
- The user selects a new font.
- The "font-desc" property of the `GtkFontDialogButton` instance changes.
- The value of the "font-desc" key in the GSettings database changes because it is bound to the property.
- The "changed::font-desc" signal on the GSettings instance is emitted.
- The signal handler is called, and the CSS is updated.

In the `startup` handler of `tfeapplication.c`, we will first initialize the style manager, then create the GSettings instance, connect the signal, and finally apply the initial font to the CSS.

```
  tfe_style_manager_initialize (base_css);

  tfe_app->settings = g_settings_new ("com.github.ToshioCP.tfe");
  g_signal_connect (tfe_app->settings, "changed::font-desc", G_CALLBACK (changed_font_cb), tfe_app);
  changed_font_cb (tfe_app->settings, "font-desc", tfe_app); /* set the initial font */
```

Since the callback function `changed_font_cb` relies on the style manager, it is important that the style manager is initialized beforehand.

The callback function is implemented as follows:

```
1 static void
2 changed_font_cb (GSettings *settings, const char *prop_name, gpointer user_data) {
3   const char *font_desc = g_settings_get_string (settings, "font-desc");
4   PangoFontDescription *desc = pango_font_description_from_string (font_desc);
5
6   tfe_style_manager_set_font (desc);
7   pango_font_description_free (desc);
8 }
```

This callback is first called from the startup handler. At that time, it calls `g_settings_get_string ()` to get the value of the "font-desc" key. This ensures that the "changed::font-desc" signal will be emitted for future changes.

Up until now, we assumed that the cleanup for GSettings would be written in the dispose handler. However, we are going to change this and put the cleanup code in the shutdown handler instead. Here are the simplified reasons why:

- Symmetry: As a general rule, objects created in the startup handler should be cleaned up in the shutdown handler.
- Application Lifecycle: startup and shutdown run only once during the lifetime of the primary application instance. Secondary instances do not run them. Since our GSettings initialization only happens in startup, the cleanup must logically happen in shutdown.
- Execution Order: The GSettings signal handler uses the style manager. Therefore, we must clean up GSettings (which disconnects the handler) before terminating the style manager. If we put the cleanup in dispose, this order gets reversed and causes issues.

The shutdown handler now looks like this:

```
static void
app_shutdown (GApplication *application) {
  TfeApplication *app = TFE_APPLICATION (application);

  g_clear_object (&app->settings);
  tfe_style_manager_terminate ();

  G_APPLICATION_CLASS (tfe_application_parent_class)->shutdown (application);
}
```

At this point, the following parts are no longer needed, so you can delete them from your code:

- The dispose handler.
- The dispose handler override inside the class initialization function.

2.24.3 Building the Program

Building the program involves several steps:

- Compile the schema file.
- Compile the XML file into a C resource file.
- Compile the C source files.
- Run the executable file.
- (Optional:) If you install the program, copy the executable binary to a directory like `/usr/local/bin`, copy the schema file to a schema directory like `/usr/local/share/glib-2.0/schemas`, and run `glib-compile-schemas` in that directory.

Meson wraps all these commands up for us. Create the following text and save it as `meson.build`:

```
project('tfe', 'c', license : 'GPL-3.0-or-later', meson_version:'>=1.0.1', version: '0.5')

gtkdep = dependency('gtk4')

gnome = import('gnome')
resources = gnome.compile_resources('resources','tfe.gresource.xml')
gnome.compile_schemas(depend_files: 'com.github.ToshioCP.tfe.gschema.xml')

sourcefiles=files('main.c', 'tfeapplication.c', 'tfewindow.c', 'tfetextview.c', 'tfestylemanager.c',

executable(meson.project_name(), sourcefiles, resources, dependencies: gtkdep, export_dynamic: true,

schema_dir = get_option('prefix') / get_option('datadir') / 'glib-2.0/schemas/'
```

```
14 install_data('com.github.ToshioCP.tfe.gschema.xml', install_dir: schema_dir)
15 gnome.post_install (glib_compile_schemas: true)
```

To build and run the program, change your current directory to src/tfe12 and type the following commands in your terminal:

```
$ meson setup _build
$ ninja -C _build
$ GSETTINGS_SCHEMA_DIR=_build _build/tfe
```

In the last line, the `GSETTINGS_SCHEMA_DIR` environment variable points to the directory where the compiled schema file is located.

Our `meson.build` also supports installing the application to your system. The executable will be installed in `/usr/local/bin` and the schema file in `/usr/local/share/glib-2.0/schemas`. Keep in mind that you need administrator privileges to install it:

```
$ sudo ninja -C _build install
```

Once installed, you no longer need to set `GSETTINGS_SCHEMA_DIR`. You can run the program from anywhere simply by typing its name:

```
$ tfe
```

Chapter 3

GtkDrawingArea

3.1 GtkDrawingArea and Cairo

To dynamically draw shapes or images on the screen, use the GtkDrawingArea widget.

GtkDrawingArea provides a Cairo drawing context, allowing you to render graphics using Cairo library functions. This section covers:

1. A brief introduction to Cairo
2. A simple example of using GtkDrawingArea

3.1.1 Cairo

Cairo is a 2D graphics library. You can find extensive documentation on the Cairo website. If you are new to Cairo, we highly recommend reading the tutorial.

Below is a brief introduction to the Cairo library.

First, you need to know surfaces, sources, masks, destinations, cairo context and transformations.

- A surface represents an image. It is like a canvas. We can draw shapes and images with different colors on surfaces.
- The source pattern, or simply source, is like paint, which will be transferred to destination surface by cairo functions.
- The mask describes the area to be used in the copy;
- The destination is a target surface;
- The cairo context manages the transfer from source to destination through the mask, using its functions; For example, `cairo_stroke` is a function that transfers the paint to the destination along a path.
- A transformation can be applied before the transfer completes. This kind of transformation is called an affine transformation, which is a mathematical term meaning transformations that preserve straight lines. Scaling, rotating, reflecting, shearing and translating are examples of affine transformations. They are mathematically represented by matrix multiplication and vector addition. In this section we don't use it: instead we will only use the identity transformation. This means that the coordinates in the source and mask map directly to the same coordinates in the destination.

The instructions is as follows:

1. Create a surface. This will be the destination.
2. Create a cairo context with the surface. This surface will be the destination of the context.
3. Create a source pattern within the context.
4. Create paths, which are lines, rectangles, arcs, text or more complicated shapes in the mask.
5. Use a drawing operator such as `cairo_stroke` to transfer the paint in the source to the destination.

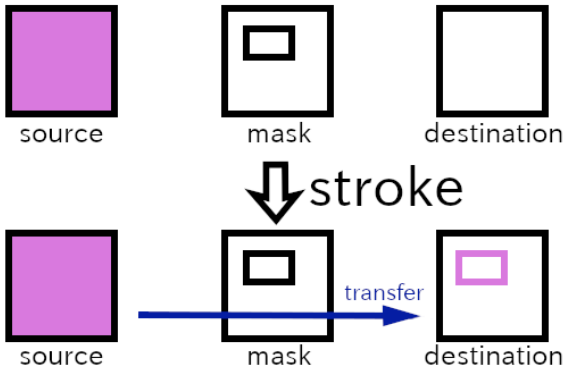


Figure 3.1: Stroke a rectangle

6. Save the destination surface to a file if necessary.

Here's a simple example program that draws a small square and saves it as a png file. The path of the file is `/src/misc/cairo.c`.

```
1  #include <cairo.h>
2
3  int
4  main (int argc, char **argv)
5  {
6    cairo_surface_t *surface;
7    cairo_t *cr;
8    int width = 100;
9    int height = 100;
10   double square_size = 40.0;
11
12   /* Create surface and cairo */
13   surface = cairo_image_surface_create (CAIRO_FORMAT_RGB24, width, height);
14   cr = cairo_create (surface);
15
16   /* Drawing starts here. */
17   /* Paint the background white */
18   cairo_set_source_rgb (cr, 1.0, 1.0, 1.0);
19   cairo_paint (cr);
20   /* Draw a black rectangle */
21   cairo_set_source_rgb (cr, 0.0, 0.0, 0.0);
22   cairo_set_line_width (cr, 2.0);
23   cairo_rectangle (cr,
24                    width/2.0 - square_size/2,
25                    height/2.0 - square_size/2,
26                    square_size,
27                    square_size);
28   cairo_stroke (cr);
29
30   /* Write the surface to a png file and clean up cairo and surface. */
31   cairo_surface_write_to_png (surface, "rectangle.png");
32   cairo_destroy (cr);
33   cairo_surface_destroy (surface);
```

```
34
35   return 0;
36 }
```

- 1: Includes the header file of Cairo.
- 6: `cairo_surface_t` is the type of a surface.
- 7: `cairo_t` is the type of a cairo context.
- 8-10: `width` and `height` are the size of `surface`. `square_size` is the size of a square to be drawn on the surface.
- 13: `cairo_image_surface_create` creates an image surface. `CAIRO_FORMAT_RGB24` is a constant that means each pixel has red, green, and blue components, and each component is an 8-bit number (24 bits in total). This is the typical color depth of modern displays. Width and height are in pixels and given as integers.
- 14: Creates a cairo context. The surface given as an argument will be the destination of the context.
- 18: `cairo_set_source_rgb` creates a source pattern, which is a solid white paint. The second to fourth arguments are red, green and blue color values respectively, and they are of type double. The values are between zero (0.0) and one (1.0). Black is (0.0,0.0,0.0) and white is (1.0,1.0,1.0).
- 19: `cairo_paint` copies the entire source to the destination. The destination is filled with white pixels with this command.
- 21: Sets the source color to black.
- 22: `cairo_set_line_width` sets the width of lines. In this case, the line width is set to two pixels, and it actually appears as that width because the transformation is the identity transformation. (It is because the transformation is identity. If the transformation isn't identity, for example scaling with the factor three, the actual width in destination will be six (2x3=6) pixels.)
- 23: Draws a rectangle (square) on the mask. The square is located at the center.
- 28: `cairo_stroke` transfers the source to destination through the rectangle in the mask.
- 31: Outputs the image to a png file `rectangle.png`.
- 32: Destroys the context. At the same time the source is destroyed.
- 33: Destroys the surface.

To compile this, change your current directory to `/src/misc` and type the following.

```
$ gcc `pkg-config --cflags cairo` cairo.c `pkg-config --libs cairo`
```

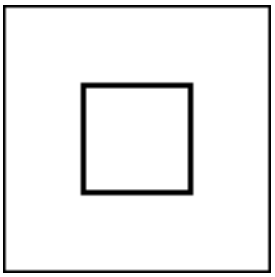


Figure 3.2: rectangle.png

See Cairo's website for further information.

3.1.2 GtkDrawingArea

The following is a very simple example.

```
1  #include <gtk/gtk.h>
2
3  static void
4  draw_function (GtkDrawingArea *area, cairo_t *cr, int width, int height, gpointer user_data) {
5    int square_size = 40.0;
```

```
6
7     cairo_set_source_rgb (cr, 1.0, 1.0, 1.0); /* white */
8     cairo_paint (cr);
9     cairo_set_line_width (cr, 2.0);
10    cairo_set_source_rgb (cr, 0.0, 0.0, 0.0); /* black */
11    cairo_rectangle (cr,
12                     width/2.0 - square_size/2,
13                     height/2.0 - square_size/2,
14                     square_size,
15                     square_size);
16    cairo_stroke (cr);
17  }
18
19  static void
20  app_activate (GApplication *app, gpointer user_data) {
21    GtkWidget *win = gtk_application_window_new (GTK_APPLICATION (app));
22    GtkWidget *area = gtk_drawing_area_new ();
23
24    gtk_window_set_title (GTK_WINDOW (win), "da1");
25    gtk_drawing_area_set_draw_func (GTK_DRAWING_AREA (area), draw_function, NULL, NULL);
26    gtk_window_set_child (GTK_WINDOW (win), area);
27
28    gtk_window_present (GTK_WINDOW (win));
29  }
30
31  #define APPLICATION_ID "com.github.ToshioCP.da1"
32
33  int
34  main (int argc, char **argv) {
35    GtkApplication *app;
36    int stat;
37
38    app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
39    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
40    stat = g_application_run (G_APPLICATION (app), argc, argv);
41    g_object_unref (app);
42    return stat;
43  }
```

The `main` function is almost the same as those in the previous sections. The two functions `app_activate` and `draw_function` are important in this example.

- 22: Creates a GtkDrawingArea instance.
- 25: Sets a drawing function of the widget. GtkDrawingArea widget uses the function `draw_function` to draw the contents of itself whenever it is necessary. For example, when a user drags a mouse pointer and resizees a top-level window, GtkDrawingArea also changes the size. Then, the whole window needs to be redrawn. For more information on `gtk_drawing_area_set_draw_func`, see Gtk API Reference – gtk_drawing_area_set_draw_func.

The drawing function has five parameters.

```
void drawing_function (GtkDrawingArea *drawing_area, cairo_t *cr, int width, int height,
                       gpointer user_data);
```

The first parameter is the GtkDrawingArea widget. You can't change any properties, for example `content-width` or `content-height`, in this function. The second parameter is a cairo context given by

the widget. The destination surface of the context is connected to the contents of the widget. What you draw to this surface will appear in the widget on the screen. The third and fourth parameters give the width and height of the destination surface. Now, look at the program again.

- 3-17: The drawing function.
- 7-8: Sets the source to be white and paints the destination white.
- 9: Sets the line width to be 2.
- 10: Sets the source to be black.
- 11-15: Adds a rectangle to the mask.
- 16: Draws the rectangle in black on the destination.

The program is /src/misc/da1.c. Compile and run it, and a window with a black rectangle (square) will appear. When you resize the window, the square always appears at the center of the window because the drawing function is invoked each time the window is resized.

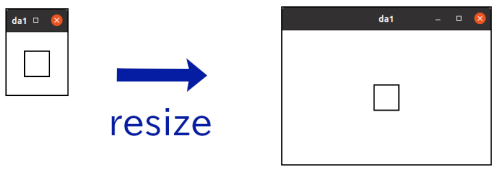


Figure 3.3: Square in the window

3.2 Periodic Events

This chapter was written by Paul Schulz paul@mawsonlakes.org.

3.2.1 How do we create an animation?

In this section we will continue to build on our previous work. We will create an analog clock application. By adding a function which periodically redraws GtkDrawingArea, the clock will be able to continuously display the time.

The application uses a compiled in 'resource' file, so if the GTK4 libraries and their dependencies are installed and available, the application will run from anywhere.

The program also makes use of some standard mathematical and time handling functions.

The clocks mechanics were taken from a Cairo drawing example, using gtkmm4, which can be found here.

The complete code is at the end.

3.2.2 Drawing the clock face, hour, minute and second hands

The `draw_clock()` function does all the work. See the in-file comments for an explanation of how the Cairo drawing works.

For a detailed reference of what each of the Cairo functions does see the cairo_t reference.

```
1  static void
2  draw_clock (GtkDrawingArea *area, cairo_t *cr, int width, int height, gpointer user_data) {
3
4      // Scale to unit square and translate (0, 0) to be (0.5, 0.5), i.e.
5      // the center of the window
6      cairo_scale(cr, width, height);
```

```
7       cairo_translate(cr, 0.5, 0.5);
8
9       // Set the line width and save the cairo drawing state.
10      cairo_set_line_width(cr, m_line_width);
11      cairo_save(cr);
12
13      // Set the background to a slightly transparent green.
14      cairo_set_source_rgba(cr, 0.337, 0.612, 0.117, 0.9);   // green
15      cairo_paint(cr);
16
17      // Resore back to precious drawing state and draw the circular path
18      // representing the clockface. Save this state (including the path) so we
19      // can reuse it.
20      cairo_restore(cr);
21      cairo_arc(cr, 0.0, 0.0, m_radius, 0.0, 2.0 * M_PI);
22      cairo_save(cr);
23
24      // Fill the clockface with white
25      cairo_set_source_rgba(cr, 1.0, 1.0, 1.0, 0.8);
26      cairo_fill_preserve(cr);
27      // Restore the path, paint the outside of the clock face.
28      cairo_restore(cr);
29      cairo_stroke_preserve(cr);
30      // Set the 'clip region' to the inside of the path (fill region).
31      cairo_clip(cr);
32
33      // Clock ticks
34      for (int i = 0; i < 12; i++)
35      {
36          // Major tick size
37          double inset = 0.05;
38
39          // Save the graphics state, restore after drawing tick to maintain pen
40          // size
41          cairo_save(cr);
42          cairo_set_line_cap(cr, CAIRO_LINE_CAP_ROUND);
43
44          // Minor ticks are shorter, and narrower.
45          if(i % 3 != 0)
46          {
47              inset *= 0.8;
48              cairo_set_line_width(cr, 0.03);
49          }
50
51          // Draw tick mark
52          cairo_move_to(
53              cr,
54              (m_radius - inset) * cos (i * M_PI / 6.0),
55              (m_radius - inset) * sin (i * M_PI / 6.0));
56          cairo_line_to(
57              cr,
58              m_radius * cos (i * M_PI / 6.0),
59              m_radius * sin (i * M_PI / 6.0));
```

```
60          cairo_stroke(cr);
61          cairo_restore(cr); /* stack-pen-size */
62      }
63
64      // Draw the analog hands
65
66      // Get the current Unix time, convert to the local time and break into time
67      // structure to read various time parts.
68      time_t rawtime;
69      time(&rawtime);
70      struct tm * timeinfo = localtime (&rawtime);
71
72      // Calculate the angles of the hands of our clock
73      double hours   = timeinfo->tm_hour * M_PI / 6.0;
74      double minutes = timeinfo->tm_min * M_PI / 30.0;
75      double seconds = timeinfo->tm_sec * M_PI / 30.0;
76
77      // Save the graphics state
78      cairo_save(cr);
79      cairo_set_line_cap(cr, CAIRO_LINE_CAP_ROUND);
80
81      cairo_save(cr);
82
83      // Draw the seconds hand
84      cairo_set_line_width(cr, m_line_width / 3.0);
85      cairo_set_source_rgba(cr, 0.7, 0.7, 0.7, 0.8);   // gray
86      cairo_move_to(cr, 0.0, 0.0);
87      cairo_line_to(cr,
88                    sin(seconds) * (m_radius * 0.9),
89                    -cos(seconds) * (m_radius * 0.9));
90      cairo_stroke(cr);
91      cairo_restore(cr);
92
93      // Draw the minutes hand
94      cairo_set_source_rgba(cr, 0.117, 0.337, 0.612, 0.9);   // blue
95      cairo_move_to(cr, 0, 0);
96      cairo_line_to(cr,
97                    sin(minutes + seconds / 60) * (m_radius * 0.8),
98                    -cos(minutes + seconds / 60) * (m_radius * 0.8));
99      cairo_stroke(cr);
100
101     // draw the hours hand
102     cairo_set_source_rgba(cr, 0.337, 0.612, 0.117, 0.9);   // green
103     cairo_move_to(cr, 0.0, 0.0);
104     cairo_line_to(cr,
105                   sin(hours + minutes / 12.0) * (m_radius * 0.5),
106                   -cos(hours + minutes / 12.0) * (m_radius * 0.5));
107     cairo_stroke(cr);
108     cairo_restore(cr);
109
110     // Draw a little dot in the middle
111     cairo_arc(cr, 0.0, 0.0, m_line_width / 3.0, 0.0, 2.0 * M_PI);
112     cairo_fill(cr);
```

```
113 }
```

In order for the clock to be drawn, the drawing function `draw_clock()` needs to be registered with GTK4. This is done in the `app_activate()` function (on line 24).

Whenever the application needs to redraw the GtkDrawingArea, it will now call `draw_clock()`.

There is still a problem though. In order to animate the clock we need to also tell the application that the clock needs to be redrawn every second. This process starts by registering (on the next line, line 15) a timeout function with `g_timeout_add()` that will wakeup and run another function `time_handler`, every second (or 1000ms).

```
1  static void
2  app_activate (GApplication *app, gpointer user_data) {
3      GtkWidget *win;
4      GtkWidget *clock;
5      GtkBuilder *build;
6
7      build = gtk_builder_new_from_resource ("/com/github/ToshioCP/tfc/tfc.ui");
8      win = GTK_WIDGET (gtk_builder_get_object (build, "win"));
9      gtk_window_set_application (GTK_WINDOW (win), GTK_APPLICATION (app));
10
11     clock = GTK_WIDGET (gtk_builder_get_object (build, "clock"));
12     g_object_unref(build);
13
14     gtk_drawing_area_set_draw_func(GTK_DRAWING_AREA (clock), draw_clock, NULL, NULL);
15     g_timeout_add(1000, (GSourceFunc) time_handler, (gpointer) clock);
16     gtk_window_present(GTK_WINDOW (win));
17
18 }
```

Our `time_handler()` function is very simple, as it just calls `gtk_widget_queue_draw()` which schedules a redraw of the widget.

```
1  gboolean
2  time_handler(GtkWidget* widget) {
3      gtk_widget_queue_draw(widget);
4
5      return TRUE;
6  }
```

.. and that is all there is to it. If you compile and run the example you will get a ticking analog clock.

If you get this working, you can try modifying some of the code in `draw_clock()` to tweak the application (such as change the color or size and length of the hands) or even add text, or create a digital clock.

3.2.3 The Complete code

You can find the source files in the `tfc` directory. it can be compiled with `comp` shell script.

`tfc.c`

```
1  #include <gtk/gtk.h>
2  #include <math.h>
3  #include <time.h>
4
5  float m_radius     = 0.42;
6  float m_line_width = 0.05;
7
```

```
8  static void
9  draw_clock (GtkDrawingArea *area, cairo_t *cr, int width, int height, gpointer user_data) {
10
11     // Scale to unit square and translate (0, 0) to be (0.5, 0.5), i.e.
12     // the center of the window
13     cairo_scale(cr, width, height);
14     cairo_translate(cr, 0.5, 0.5);
15
16     // Set the line width and save the cairo drawing state.
17     cairo_set_line_width(cr, m_line_width);
18     cairo_save(cr);
19
20     // Set the background to a slightly transparent green.
21     cairo_set_source_rgba(cr, 0.337, 0.612, 0.117, 0.9);   // green
22     cairo_paint(cr);
23
24     // Resore back to precious drawing state and draw the circular path
25     // representing the clockface. Save this state (including the path) so we
26     // can reuse it.
27     cairo_restore(cr);
28     cairo_arc(cr, 0.0, 0.0, m_radius, 0.0, 2.0 * M_PI);
29     cairo_save(cr);
30
31     // Fill the clockface with white
32     cairo_set_source_rgba(cr, 1.0, 1.0, 1.0, 0.8);
33     cairo_fill_preserve(cr);
34     // Restore the path, paint the outside of the clock face.
35     cairo_restore(cr);
36     cairo_stroke_preserve(cr);
37     // Set the 'clip region' to the inside of the path (fill region).
38     cairo_clip(cr);
39
40     // Clock ticks
41     for (int i = 0; i < 12; i++)
42     {
43         // Major tick size
44         double inset = 0.05;
45
46         // Save the graphics state, restore after drawing tick to maintain pen
47         // size
48         cairo_save(cr);
49         cairo_set_line_cap(cr, CAIRO_LINE_CAP_ROUND);
50
51         // Minor ticks are shorter, and narrower.
52         if(i % 3 != 0)
53         {
54             inset *= 0.8;
55             cairo_set_line_width(cr, 0.03);
56         }
57
58         // Draw tick mark
59         cairo_move_to(
60             cr,
```

```
61              (m_radius - inset) * cos (i * M_PI / 6.0),
62              (m_radius - inset) * sin (i * M_PI / 6.0));
63          cairo_line_to(
64              cr,
65              m_radius * cos (i * M_PI / 6.0),
66              m_radius * sin (i * M_PI / 6.0));
67          cairo_stroke(cr);
68          cairo_restore(cr); /* stack-pen-size */
69      }
70
71      // Draw the analog hands
72
73      // Get the current Unix time, convert to the local time and break into time
74      // structure to read various time parts.
75      time_t rawtime;
76      time(&rawtime);
77      struct tm * timeinfo = localtime (&rawtime);
78
79      // Calculate the angles of the hands of our clock
80      double hours   = timeinfo->tm_hour * M_PI / 6.0;
81      double minutes = timeinfo->tm_min * M_PI / 30.0;
82      double seconds = timeinfo->tm_sec * M_PI / 30.0;
83
84      // Save the graphics state
85      cairo_save(cr);
86      cairo_set_line_cap(cr, CAIRO_LINE_CAP_ROUND);
87
88      cairo_save(cr);
89
90      // Draw the seconds hand
91      cairo_set_line_width(cr, m_line_width / 3.0);
92      cairo_set_source_rgba(cr, 0.7, 0.7, 0.7, 0.8);   // gray
93      cairo_move_to(cr, 0.0, 0.0);
94      cairo_line_to(cr,
95                    sin(seconds) * (m_radius * 0.9),
96                    -cos(seconds) * (m_radius * 0.9));
97      cairo_stroke(cr);
98      cairo_restore(cr);
99
100     // Draw the minutes hand
101     cairo_set_source_rgba(cr, 0.117, 0.337, 0.612, 0.9);   // blue
102     cairo_move_to(cr, 0, 0);
103     cairo_line_to(cr,
104                   sin(minutes + seconds / 60) * (m_radius * 0.8),
105                   -cos(minutes + seconds / 60) * (m_radius * 0.8));
106     cairo_stroke(cr);
107
108     // draw the hours hand
109     cairo_set_source_rgba(cr, 0.337, 0.612, 0.117, 0.9);   // green
110     cairo_move_to(cr, 0.0, 0.0);
111     cairo_line_to(cr,
112                   sin(hours + minutes / 12.0) * (m_radius * 0.5),
113                   -cos(hours + minutes / 12.0) * (m_radius * 0.5));
```

```
114     cairo_stroke(cr);
115     cairo_restore(cr);
116 
117     // Draw a little dot in the middle
118     cairo_arc(cr, 0.0, 0.0, m_line_width / 3.0, 0.0, 2.0 * M_PI);
119     cairo_fill(cr);
120 }
121 
122 
123 gboolean
124 time_handler(GtkWidget* widget) {
125     gtk_widget_queue_draw(widget);
126 
127     return TRUE;
128 }
129 
130 
131 static void
132 app_activate (GApplication *app, gpointer user_data) {
133     GtkWidget *win;
134     GtkWidget *clock;
135     GtkBuilder *build;
136 
137     build = gtk_builder_new_from_resource ("/com/github/ToshioCP/tfc/tfc.ui");
138     win = GTK_WIDGET (gtk_builder_get_object (build, "win"));
139     gtk_window_set_application (GTK_WINDOW (win), GTK_APPLICATION (app));
140 
141     clock = GTK_WIDGET (gtk_builder_get_object (build, "clock"));
142     g_object_unref(build);
143 
144     gtk_drawing_area_set_draw_func(GTK_DRAWING_AREA (clock), draw_clock, NULL, NULL);
145     g_timeout_add(1000, (GSourceFunc) time_handler, (gpointer) clock);
146     gtk_window_present(GTK_WINDOW (win));
147 
148 }
149 
150 static void
151 app_open (GApplication *app, GFile **files, gint n_files, gchar *hint, gpointer user_data) {
152     app_activate(app,user_data);
153 }
154 
155 int
156 main (int argc, char **argv) {
157     GtkApplication *app;
158     int stat;
159 
160     app = gtk_application_new ("com.github.ToshioCP.tfc", G_APPLICATION_HANDLES_OPEN);
161     g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
162     g_signal_connect (app, "open", G_CALLBACK (app_open), NULL);
163     stat = g_application_run (G_APPLICATION (app), argc, argv);
164     g_object_unref (app);
165     return stat;
166 }
```

tfc.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="GtkApplicationWindow" id="win">
    <property name="title">Clock</property>
    <property name="default-width">200</property>
    <property name="default-height">200</property>
    <child>
      <object class="GtkDrawingArea" id="clock">
        <property name="hexpand">TRUE</property>
        <property name="vexpand">TRUE</property>
      </object>
    </child>
  </object>
</interface>
```

tfc.gresource.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<gresources>
  <gresource prefix="/com/github/ToshioCP/tfc">
    <file>tfc.ui</file>
  </gresource>
</gresources>
```

comp

```
glib-compile-resources $1.gresource.xml --target=$1.gresource.c --generate-source
gcc `pkg-config --cflags gtk4` $1.gresource.c $1.c `pkg-config --libs gtk4` -lm
```

3.3 Custom Drawing

Custom drawing refers to drawing shapes dynamically. This section shows a custom drawing example that allows you to draw rectangles by dragging the mouse. Here is how to draw a rectangle with the example program.

Press down the button.



Figure 3.4: Press down the button

Move the mouse.

Release the button.

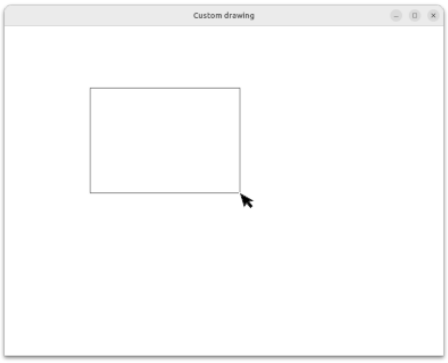


Figure 3.5: Move the mouse

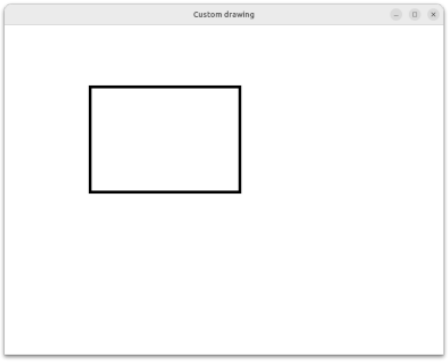


Figure 3.6: Release the button

The programs are at `src/custom_drawing` directory. Download the repository and see the directory. There are four files.

- meson.build
- rect.c
- rect.gresource.xml
- rect.ui

3.3.1 rect.gresource.xml

This file is a resource manifest used by the glib-compile-resources compiler.

```
<?xml version="1.0" encoding="UTF-8"?>
<gresources>
  <gresource prefix="/com/github/ToshioCP/rect">
    <file>rect.ui</file>
  </gresource>
</gresources>
```

The prefix is `/com/github/ToshioCP/rect` and the file is `rect.ui`. Therefore, the resource compiler compiles and stores the resource at `/com/github/ToshioCP/rect/rect.ui`. Later, GtkBuilder reads the resource from `/com/github/ToshioCP/rect/rect.ui`.

3.3.2 rect.ui

The following is the UI file that defines the widgets which are GtkApplicationWindow and GtkDrawingArea. The ids are `win` and `da` respectively.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="GtkApplicationWindow" id="win">
    <property name="default-width">800</property>
    <property name="default-height">600</property>
    <property name="resizable">FALSE</property>
    <property name="title">Custom drawing</property>
    <child>
      <object class="GtkDrawingArea" id="da">
        <property name="hexpand">TRUE</property>
        <property name="vexpand">TRUE</property>
      </object>
    </child>
  </object>
</interface>
```

3.3.3 rect.c

3.3.3.1 GtkApplication

This program uses GtkApplication. The application ID is `com.github.ToshioCP.rect`.

```
#define APPLICATION_ID "com.github.ToshioCP.rect"
```

See GNOME Developer Documentation for further information.

The function `main` is called at the beginning of the application.

```
int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
  g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
  g_signal_connect (app, "shutdown", G_CALLBACK (app_shutdown), NULL);
  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
}
```

It connects three signals and handlers.

- startup: Emitted after the application is registered to the system.
- activate: Emitted when the application is activated.
- shutdown: Emitted just before the application quits.

```
static void
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GtkBuilder *build;
  GtkWindow *win;
  GtkDrawingArea *da;
  GtkGesture *drag;

  build = gtk_builder_new_from_resource ("/com/github/ToshioCP/rect/rect.ui");
  win = GTK_WINDOW (gtk_builder_get_object (build, "win"));
```

```
  da = GTK_DRAWING_AREA (gtk_builder_get_object (build, "da"));
  gtk_window_set_application (win, app);
  g_object_unref (build);

  gtk_drawing_area_set_draw_func (da, draw_cb, NULL, NULL);
  g_signal_connect (da, "resize", G_CALLBACK (resize_cb), NULL);

  drag = gtk_gesture_drag_new ();
  gtk_widget_add_controller (GTK_WIDGET (da), GTK_EVENT_CONTROLLER (drag));
  g_signal_connect (drag, "drag-begin", G_CALLBACK (drag_begin), NULL);
  g_signal_connect (drag, "drag-update", G_CALLBACK (drag_update), da);
  g_signal_connect (drag, "drag-end", G_CALLBACK (drag_end), da);
  g_signal_connect (drag, "cancel", G_CALLBACK (drag_cancel), da);
}
```

The startup handler performs the following three tasks:

- Builds the widgets.
- Initializes the GtkDrawingArea instance.
  - Sets the drawing function
  - Connects the "resize" signal and the handler.
- Creates the GtkGestureDrag instance and connects it to the drawing area object. Gestures will be explained in this section later.

```
static void
app_activate (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GtkWindow *win;

  win = gtk_application_get_active_window (app);
  gtk_window_present (win);
}
```

The activate handler just shows the window.

3.3.3.2 GtkDrawingArea

The program has two cairo surfaces and they are pointed to by the global variables.

```
static cairo_surface_t *surface = NULL;
static cairo_surface_t *surface_save = NULL;
```

The drawing process is as follows.

- Creates an image on `surface`.
- Copies `surface` to the cairo surface of the GtkDrawingArea.
- Calls `gtk_widget_queue_draw (da)`.

They are created in the "resize" signal handler.

```
static void
resize_cb (GtkWidget *widget, int width, int height, gpointer user_data) {
  cairo_t *cr;
  int scale = gtk_widget_get_scale_factor (widget);

  if (surface)
    cairo_surface_destroy (surface);
  surface = cairo_image_surface_create (CAIRO_FORMAT_RGB24, width*scale, height*scale);
```

```
  cairo_surface_set_device_scale (surface, scale, scale);

  if (surface_save)
    cairo_surface_destroy (surface_save);
  surface_save = cairo_image_surface_create (CAIRO_FORMAT_RGB24, width*scale, height*scale);
  cairo_surface_set_device_scale (surface_save, scale, scale);

  /* Paint the surface white. It is the background color. */
  cr = cairo_create (surface);
  cairo_set_source_rgb (cr, 1.0, 1.0, 1.0);
  cairo_paint (cr);
  cairo_destroy (cr);
}
```

This callback is called when the GtkDrawingArea is shown. This is called only once since the window is not resizable. It creates image surfaces for `surface` and `surface_save`. The `surface` surface is painted white, which is the background color.

The drawing function copies `surface` to the GtkDrawingArea surface.

3.3.3.2.1 Scale factor Modern displays, especially HiDPI (or "Retina") displays, render more physical pixels per logical pixel than traditional displays. GTK reports this ratio as the scale factor of a widget, obtained with `gtk_widget_get_scale_factor()`. A scale factor of 1 means one logical pixel equals one physical pixel, while a scale factor of 2 means one logical pixel is rendered using a 2x2 block of physical pixels.

The `surface` surface must have the same scale factor and the same physical pixel dimensions as the drawing area's cairo destination surface, since the former is copied onto the latter in the draawing function. If `surface` is created using only the logical width and height passed to the "resize" signal, the surface will contain far fewer physical pixels than the display can actually show on a HiDPI screen. As a result, the drawn image will look blurry when it is scaled up to fill the display.

The physical pixel sizes can be obtained by multiplying logical pixel sizes by the scale factor. The surface scale is set with `cairo_surface_set_device_scale`.

```
static void
draw_cb (GtkDrawingArea *da, cairo_t *cr, int width, int height, gpointer user_data) {
  if (surface) {
    cairo_set_source_surface (cr, surface, 0.0, 0.0);
    cairo_paint (cr);
  }
}
```

This function is called by the system when it needs to redraw the drawing area.

Two surfaces `surface` and `surface_save` are destroyed before the application quits.

```
static void
app_shutdown (GApplication *application) {
  if (surface)
    cairo_surface_destroy (surface);
  if (surface_save)
    cairo_surface_destroy (surface_save);
}
```

3.3.3.3 GtkGestureDrag

Gesture class is used to recognize human gestures such as click, drag, pan, swipe and so on. It is a subclass of GtkEventController. GtkGesture class is abstract and there are several implementations.

- GtkGestureClick
- GtkGestureDrag
- GtkGesturePan
- GtkGestureSwipe
- And others

The program `rect.c` uses GtkGestureDrag. It is the implementation for drag gestures. The parent-child relationship is as follows.

`GObject -- GtkEventController -- GtkGesture -- GtkGestureSingle -- GtkGestureDrag`

GtkGestureSingle is a subclass of GtkGesture and optimized for single-touch and mouse interactions.

A GtkGestureDrag instance is created and initialized in the startup signal handler in `rect.c`. See line 18 to 23 in the following.

```
1  static void
2  app_startup (GApplication *application) {
3    GtkApplication *app = GTK_APPLICATION (application);
4    GtkBuilder *build;
5    GtkWindow *win;
6    GtkDrawingArea *da;
7    GtkGesture *drag;
8
9    build = gtk_builder_new_from_resource ("/com/github/ToshioCP/rect/rect.ui");
10   win = GTK_WINDOW (gtk_builder_get_object (build, "win"));
11   da = GTK_DRAWING_AREA (gtk_builder_get_object (build, "da"));
12   gtk_window_set_application (win, app);
13   g_object_unref (build);
14
15   gtk_drawing_area_set_draw_func (da, draw_cb, NULL, NULL);
16   g_signal_connect (da, "resize", G_CALLBACK (resize_cb), NULL);
17
18   drag = gtk_gesture_drag_new ();
19   gtk_widget_add_controller (GTK_WIDGET (da), GTK_EVENT_CONTROLLER (drag));
20   g_signal_connect (drag, "drag-begin", G_CALLBACK (drag_begin), NULL);
21   g_signal_connect (drag, "drag-update", G_CALLBACK (drag_update), da);
22   g_signal_connect (drag, "drag-end", G_CALLBACK (drag_end), da);
23   g_signal_connect (drag, "cancel", G_CALLBACK (drag_cancel), da);
24 }
```

- The function `gtk_gesture_drag_new` creates a new GtkGestureDrag instance. The default button number is 1 (`GDK_BUTTON_PRIMARY`), which is the left button of a mouse or touch events.
- The function `gtk_widget_add_controller` adds an event controller, which is a base class for gestures.
- Four signals and handlers are connected.
  - drag-begin: Emitted when dragging starts.
  - drag-update: Emitted when the dragging pointer moves.
  - drag-end: Emitted when the dragging ends.
  - cancel: Emitted when the dragging was canceled. This signal belongs to GtkGesture.

The process during the drag is as follows.

- start: save the surface and start points
- update: restore the surface and draw a thin rectangle between the start point and the current point of the mouse
- end: restore the surface and draw a thick rectangle between the start and end points
- cancel: restore the surface

We need two global variables for the start point.

```
static double start_x;
static double start_y;
```

The following is the handler for the “drag-begin” signal.

```
1  static void
2  copy_surface (cairo_surface_t *src, cairo_surface_t *dst) {
3    if (!src || !dst)
4      return;
5    cairo_t *cr = cairo_create (dst);
6    cairo_set_source_surface (cr, src, 0.0, 0.0);
7    cairo_paint (cr);
8    cairo_destroy (cr);
9  }
10
11 static void
12 drag_begin (GtkGestureDrag *gesture, double x, double y, gpointer user_data) {
13   // save the surface and record (x, y)
14   copy_surface (surface, surface_save);
15   start_x = x;
16   start_y = y;
17 }
```

- Copies `surface` to `surface_save`, which is an image just before the dragging.
- Stores the points to `start_x` and `start_y`.

```
1  static void
2  drag_update  (GtkGestureDrag *gesture, double offset_x, double offset_y, gpointer user_data) {
3    GtkWidget *da = GTK_WIDGET (user_data);
4    cairo_t *cr;
5
6    copy_surface (surface_save, surface);
7    cr = cairo_create (surface);
8    cairo_rectangle (cr, start_x, start_y, offset_x, offset_y);
9    cairo_set_line_width (cr, 1.0);
10   cairo_stroke (cr);
11   cairo_destroy (cr);
12   gtk_widget_queue_draw (da);
13 }
```

- Restores `surface` from `surface_save`.
- Draws a rectangle with thin lines.
- Calls `gtk_widget_queue_draw` to add the GtkDrawingArea to the queue to redraw.

```
1  static void
2  drag_end  (GtkGestureDrag *gesture, double offset_x, double offset_y, gpointer user_data) {
3    GtkWidget *da = GTK_WIDGET (user_data);
4    cairo_t *cr;
5
```

```
6     copy_surface (surface_save, surface);
7     cr = cairo_create (surface);
8     cairo_rectangle (cr, start_x, start_y, offset_x, offset_y);
9     cairo_set_line_width (cr, 6.0);
10    cairo_stroke (cr);
11    cairo_destroy (cr);
12    gtk_widget_queue_draw (da);
13  }
```

- Restores `surface` from `surface_save`.
- Draws a rectangle with thick lines.
- Calls `gtk_widget_queue_draw` to add the GtkDrawingArea to the queue to redraw.

```
1  static void
2  drag_cancel (GtkGesture *gesture, GdkEventSequence *sequence, gpointer user_data) {
3    GtkWidget *da = GTK_WIDGET (user_data);
4
5    copy_surface (surface_save, surface);
6    gtk_widget_queue_draw (da);
7  }
```

- Restores `surface` from `surface_save`.
- Calls `gtk_widget_queue_draw` to add the GtkDrawingArea to the queue to redraw.

3.3.4 Build and Run

Download the repository. Change your current directory to `src/custom_drawing`. Run meson and ninja to build the program. Type `_build/rect` to run the program. Try to draw rectangles.

```
$ cd src/custom_drawing
$ meson setup _build
$ ninja -C _build
$ _build/rect
```

3.4 Tiny Turtle Graphics Interpreter

The Turtle program is an example that combines TfeTextView and GtkDrawingArea objects. It is a very small interpreter, but it allows you to draw fractal curves. The following diagram illustrates a Koch curve, which is a well-known fractal curve.

The following image displays a snowflake curve, which consists of six Koch curves.

The Turtle program uses Flex, a lexical analyzer, and Bison, a parser generator. These are open-source alternatives to Lex and Yacc, which were proprietary tools developed at Bell Labs. Because this section focuses mainly on these tools rather than GTK, feel free to skip it.

- Flex GitHub repository
- Flex Documentation
- Bison Project Page

3.4.1 How to Use Turtle

You can find The Turtle documentation here. Here is a simple example.

```
fc (1,0,0) # Foreground color is red, rgb = (1,0,0).
pd         # Pen down.
rp (4) {   # Repeat four times.
  fd 100   # Go forward by 100 pixels.
```

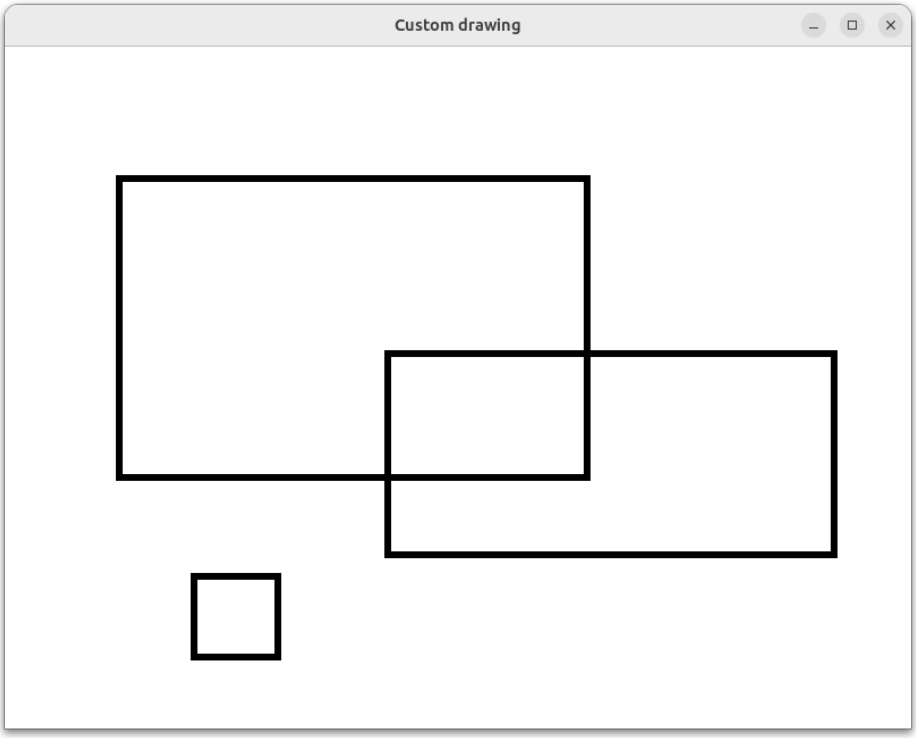


Figure 3.7: The screen of rect program

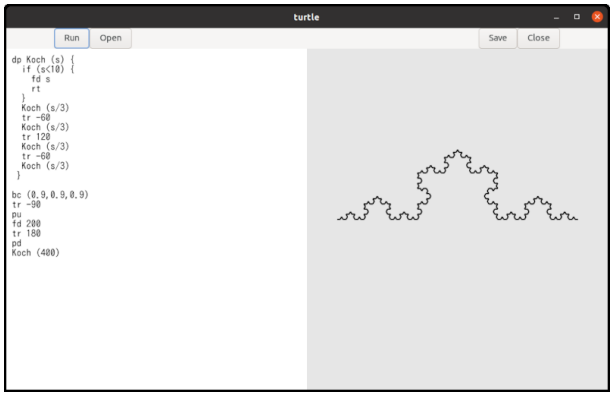


Figure 3.8: Koch curve

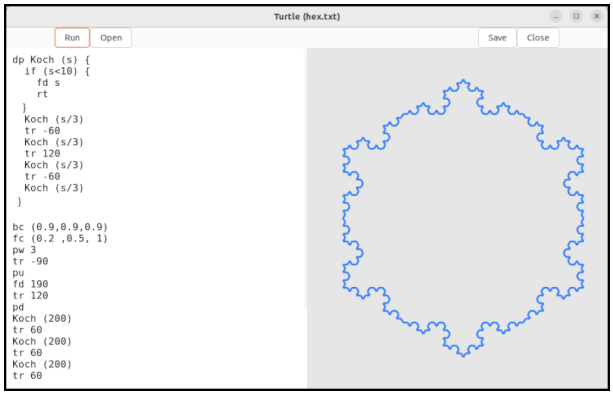


Figure 3.9: Snow

```
  tr 90    # Turn right by 90 degrees.
}
```

1. Compile, install, and run `turtle` (See the documentation above for details).
2. Type the program above into the editor on the left pane of the window.
3. Click the `Run` button. A red square with 100-pixel sides will appear on the right pane of the window.

You can draw other shapes and curves using a similar approach. The Turtle documentation includes examples of fractal curves, such as a tree, a Koch curve, and a square Koch curve. The source code for these examples is located in the /src/turtle/example directory. You can load these files into the `turtle` editor by clicking the `Open` button.

3.4.2 Combining TfeTextView and GtkDrawingArea

Turtle uses TfeTextView and GtkDrawingArea. Here is how they interact:

1. The user types or loads a Turtle program into the TfeTextView buffer.
2. The user clicks the "Run" button.
3. The parser reads the code and generates a data tree.
4. The interpreter executes this data step by step, drawing shapes onto a separate, off-screen surface rather than directly onto the GtkDrawingArea widget.
5. The widget is then queued for a redraw. A drawing function is triggered, which simply copies the off-screen surface onto the GtkDrawingArea.

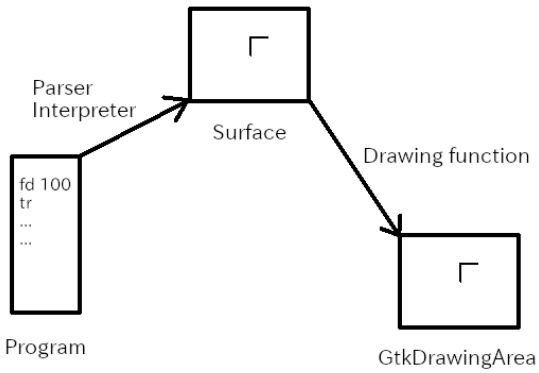


Figure 3.10: Parser, interpreter and drawing function

The core of the interpreter is written using flex and bison. Because this code is not thread-safe, the `run_cb` callback function—which handles the "clicked" signal for the "Run" button—prevents re-entrancy.

```
1  void
2  run_cb (GtkWidget *btnr) {
3    GtkTextBuffer *tb = gtk_text_view_get_buffer (GTK_TEXT_VIEW (tv));
4    GtkTextIter start_iter;
5    GtkTextIter end_iter;
6    char *contents;
7    int stat;
8    static gboolean busy = FALSE; /* initialized only once */
9
10   /* yyparse() and run() are NOT thread safe. */
11   /* The variable busy avoids reentrance. */
12   if (busy)
13     return;
14   busy = TRUE;
```

```
15    gtk_text_buffer_get_bounds (tb, &start_iter, &end_iter);
16    contents = gtk_text_buffer_get_text (tb, &start_iter, &end_iter, FALSE);
17    if (surface && contents[0] != '\0') {
18      init_flex (contents);
19      stat = yyparse ();
20      if (stat == 0) { /* No error */
21        run ();
22      }
23      finalize_flex ();
24    }
25    g_free (contents);
26    gtk_widget_queue_draw (GTK_WIDGET (da));
27    busy = FALSE;
28  }
29
30  static void
31  resize_cb (GtkDrawingArea *drawing_area, int width, int height, gpointer user_data) {
32    if (surface)
33      cairo_surface_destroy (surface);
34    surface = cairo_image_surface_create (CAIRO_FORMAT_ARGB32, width, height);
35    run_cb (NULL); // NULL is a fake (run button).
36  }
```

- 8, 12-14: The static variable `busy` tracks the status of the interpreter. If it is `TRUE`, the interpreter is already running and cannot be called again, as the code is not re-entrant. If it is `FALSE`, it is safe to call the interpreter, at which point `busy` is set to `TRUE`.
- 15-16: Retrieves the contents of `tb`.
- 17-27: The static variable `surface` points to a `cairo_surface_t` instance. Because it is created when the GtkDrawingArea is realized and recreated upon resizing, it should always be non-NULL at this stage. However, as a safety check, if it is `NULL`, the interpreter will not be called.
- 17-24: If `surface` is not `NULL` and `contents` is not an empty string, the interpreter is invoked to perform the following steps:
  - Initializes the lexical analyzer.
  - Calls the parser, which analyzes the program's syntax and generates a data tree.
  - If parsing is successful, it calls the `run` runtime routine.
  - Finalizes the lexical analyzer.
- 25: Frees `contents`.
- 26: Queues the drawing area widget for a redraw.
- 27: Sets `busy` back to `FALSE`.
- 30-36: The "resize" signal handler. If `surface` is not NULL, it is destroyed. A new surface is then created to match the dimensions of the GtkDrawingArea, and the `run_cb` callback is called to redraw the shapes.

TfeTextView has been updated to include a "change-file" signal, which is emitted whenever its GFile changes. For details on how to create and emit signals on an object, please refer to the GObject Tutorial. The updated TfeTextView files (`tfetextview.h` and `tfetextview.c`) are located in the turtle directory.

When the "Open" button is clicked and a file is loaded, the filename is displayed in the header bar.

```
1   static void
2   show_filename (TfeTextView *tv) {
3     GFile *file;
4     char *filename_bin, *filename_utf;
5     char *title;
6
```

```
  /* The instance (TfeTextView) holds the ownership of the returned GFile. */
  file = tfe_text_view_get_file (tv);
  if (G_IS_FILE (file)) {
    filename_bin = g_file_get_basename (file);
    filename_utf = g_filename_display_name (filename_bin);
    title = g_strdup_printf ("Turtle (%s)", filename_utf);
    g_free (filename_bin);
    g_free (filename_utf);
  } else
    title = g_strdup ("Turtle");
  gtk_window_set_title (GTK_WINDOW (win), title);
  g_free (title);
}
```

This is the callback for the "change-file" signal on the TfeTextView instance.

- It calls `tfe_text_view_get_file`.
- If it returns a GFile instance, the title is set to "Turtle (filename)".
- Otherwise, the title defaults to "Turtle".
- Note that `g_file_get_basename` does not necessarily return a UTF-8 string; therefore, the return value must be converted to UTF-8 using `g_filename_display_name`.

The rest of `turtleapplication.c` is very simple and similar to the code in earlier sections. The complete source code for `turtleapplication.c` can be found in the turtle directory.

3.4.3 What Does the Interpreter Do?

Suppose the turtle application runs the following program:

```
distance = 100
fd distance*2
```

The application processes the program as follows:

- Generally, a program consists of tokens. In the example above, the tokens are "distance", "=", "100", "fd", "*" and "2".
- The parser calls the `yylex` function to read each token from the source file. `yylex` returns a code representing the "token kind" and sets a global variable, `yylval`, to what is called a semantic value. The `yylval` variable is a union type. The types of `yylval.ID` and `yylval.NUM` are string and double, respectively. Since there are seven tokens in the program, `yylex` is called seven times:

	token kind	yylval.ID	yylval.NUM
1	ID	distance	
2	=		
3	NUM		100
4	FD		
5	ID	distance	
6	*		
7	NUM		2

- While `yylex` returns a token kind on every call, it does not always set `yylval.ID` or `yylval.NUM`. This is because keywords (`FD`) and symbols (`=` and `*`) do not have semantic values.
- The `turtle` application then constructs a tree data structure using its internal parser.
- `turtle` analyzes the tree (shown in the image above) and executes it. This part of the application is called the runtime routine or interpreter. The tree consists of rectangles and line segments between

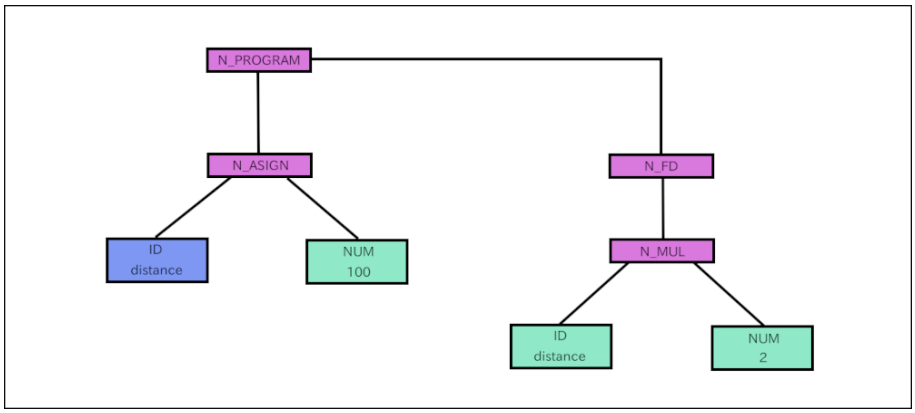


Figure 3.11: turtle parser tree

the rectangles. The rectangles are called nodes. For example, `N_PROGRAM`, `N_ASSIGN`, `N_FD`, and `N_MUL` are nodes.

1. The interpreter goes down from `N_PROGRAM` to `N_ASSIGN`.
2. The `N_ASSIGN` node has two children, `ID` and `NUM`. This node comes from `distance = 100`, which is `ID = NUM` syntactically. First, `turtle` checks if the first child is an `ID`. If it is, `turtle` looks for the variable in the variable table. If it does not exist, it registers the `ID` (`distance`) to the table. It then goes back up to the `N_ASSIGN` node.
3. `turtle` calculates the second child. In this case, it is the number `100`. It saves `100` to the variable table at the `distance` record.
4. `turtle` goes back to `N_PROGRAM`, then goes to the next node, `N_FD`. This node has only one child. The interpreter goes down to the child `N_MUL`.
5. The first child is an `ID` (`distance`). `turtle` searches the variable table for the variable `distance` and gets the value 100. The second child is the number 2. It multiplies 100 by 2 and gets 200. `turtle` then goes back to `N_FD`.
6. Now `turtle` knows the distance is 200. It moves the cursor forward by 200 pixels. The segment is drawn on the surface.
7. There is no following node. The runtime routine returns to the function `run_cb`.

- The function `run_cb` calls `gtk_widget_queue_draw` and puts the GtkDrawingArea widget into the queue.
- The system redraws the widget. At that time, the drawing function `draw_func` is called. The function copies the `surface` to the surface in the GtkDrawingArea.

An actual `turtle` program is more complicated than the example above. However, what `turtle` does is basically the same. Interpretation consists of three parts:

- Lexical analysis
- Syntax parsing and tree generation
- Interpretation and execution of the tree

3.4.4 Compilation Flow

The source files are:

- flex source file: `turtle.lex`
- bison source file: `turtle.y`
- C header file: `turtle_lex.h`, `tfetextview.h`
- C source file: `turtleapplication.c`, `tfetextview.c`
- Other files: `turtle.ui`, `turtle.gresource.xml`, and `meson.build`

The compilation process involves several steps, as illustrated in the diagram below:

1. `glib-compile-resources` compiles `turtle.ui` into `resources.c` and generates `resources.h`, according to the rules in `turtle.gresource.xml`.
2. `bison` compiles `turtle.y` into `turtle_parser.c` and generates `turtle_parser.h`.
3. `flex` compiles `turtle.lex` into `turtle_lex.c`.
4. The C compiler and linker process `turtleapplication.c`, `tfetextview.c`, `resources.c`, `turtle_parser.c`, and `turtle_lex.c`, alongside their respective headers (`tfetextview.h`, `resources.h`, `turtle_parser.h` and `turtle_lex.h`), to generate the `turtle` executable file.

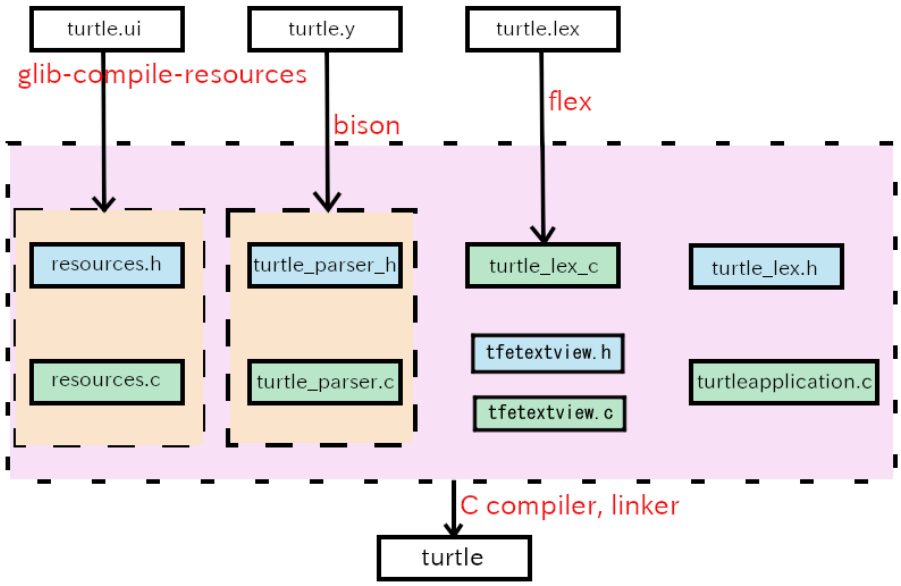


Figure 3.12: compile process

Meson controls this entire process. The instructions are defined in `meson.build`:

```
1  project('turtle', 'c')
2
3  compiler = meson.get_compiler('c')
4  mathdep = compiler.find_library('m', required : true)
5
6  gtkdep = dependency('gtk4')
7
8  gnome = import('gnome')
9  resources = gnome.compile_resources('resources','turtle.gresource.xml')
10
11 flex = find_program('flex')
12 bison = find_program('bison')
13 turtleparser = custom_target('turtleparser', input: 'turtle.y', output: ['turtle_parser.c', 'turtle_
14 turtlelexer = custom_target('turtlelexer', input: 'turtle.lex', output: 'turtle_lex.c', command: [fl
15
16 sourcefiles=files('turtleapplication.c', 'tfetextview.c')
17
18 executable('turtle', sourcefiles, resources, turtleparser, turtlelexer, turtleparser[1], dependencie
```

- 1: Defines the project name as “turtle” and the programming language as C.
- 3: Retrieves the C compiler, which is typically `gcc` on Linux.
- 4: Locates the math library. Because this program uses trigonometric functions, it requires the math

library. While included via `#include <math.h>` in the source code, it must also be explicitly linked during the build process.
- 6: Locates the `gtk4` library dependency.
- 8: Imports the GNOME module. See the Meson build system website – GNOME module for further information.
- 9: Compiles the UI file into a C source file according to the `turtle.gresource.xml` file.
- 11: Locates the `flex` program.
- 12: Locates the `bison` program.
- 13: Uses `bison` to compile `turtle.y` into `turtle_parser.c` and `turtle_parser.h`. The `custom_target` function creates a custom top-level target. See the Meson build system website – custom target for further information.
- 14: Uses `flex` to compile `turtle.lex` into `turtle_lex.c`.
- 16: Creates a file object named `sourcefiles` that contains the C source files.
- 18: Compiles the C source files, including those generated by `glib-compile-resources`, `bison`, and `flex`. The argument `turtleparser[1]` refers to `turtle_parser.h`, which is the second output defined in line 13.

3.4.5 Turtle.lex

3.4.5.1 What Does Flex Do?

Flex generates a lexical analyzer (also known as a scanner) from a flex source file, which is simply a text file. (We will explain its syntax later.) Flex saves the generated scanner as a C source file. This scanner reads an input text file — typically programming language source code — and extracts tokens like variable names, numbers, and symbols. Consider the following Turtle source file:

```
fc (1,0,0) # Foreground color is red, rgb = (1,0,0).
pd         # Pen down.
distance = 100
angle = 90
fd distance    # Go forward by distance (100) pixels.
tr angle     # Turn right by angle (90) degrees.
```

The content of this text file is broken down into individual units, such as `fc`, `(`, `1`, and so on.

The keywords, identifiers, and numbers — such as `fc`, `pd`, `distance`, `angle`, `tr`, `1`, `0`, `100`, and `90` — are called tokens. The characters `(` (left parenthesis), `,` (comma), `)` (right parenthesis), and `=` (equal sign) are classified as symbols. (Note that in lexical analysis, these symbols are also generally referred to as tokens.)

Flex reads `turtle.lex` and generates the C source code for the scanner. This `turtle.lex` file defines the tokens and symbols, along with the specific action to execute when each is encountered. As you will see, `turtle.lex` is a relatively small and straightforward file.

```
1  %top{
2  #include <string.h>
3  #include <stdlib.h>
4  #include <glib.h>
5  #include "turtle_parser.h"
6
7    static int nline = 1;
8    static int ncolumn = 1;
9    static void get_location (char *text);
10
11   /* Dinamically allocated memories are added to the single list. They will be freed in the finalize
12   extern GSList *list;
13 }
14
```

```
%option noyywrap

REAL_NUMBER (0|[1-9][0-9]*)(\.[0-9]+)?
IDENTIFIER [a-zA-Z][a-zA-Z0-9]*
%%
  /* rules */
#.*               ; /* comment. Be careful. Dot symbol (.) matches any character but new line. */
[ ]               ncolumn++; /* white space. [ and ] is a "character class". */
\t                ncolumn += 8; /* assume that a tab is 8 spaces. */
\n                nline++; ncolumn = 1;
  /* reserved keywords */
pu                get_location (yytext); return PU; /* pen up */
pd                get_location (yytext); return PD; /* pen down */
pw                get_location (yytext); return PW; /* pen width = line width */
fd                get_location (yytext); return FD; /* forward */
tr                get_location (yytext); return TR; /* turn right */
tl                get_location (yytext); return TL; /* turn left, since ver 0.5 */
bc                get_location (yytext); return BC; /* background color */
fc                get_location (yytext); return FC; /* foreground color */
dp                get_location (yytext); return DP; /* define procedure */
if                get_location (yytext); return IF; /* if statement */
rt                get_location (yytext); return RT; /* return statement */
rs                get_location (yytext); return RS; /* reset the status */
rp                get_location (yytext); return RP; /* repeat, since ver 0.5 */
  /* constant */
{REAL_NUMBER}     get_location (yytext); yylval.NUM = atof (yytext); return NUM;
  /* identifier */
{IDENTIFIER}      { get_location (yytext); yylval.ID = g_strdup(yytext);
                    list = g_slist_prepend (list, yylval.ID);
                    return ID;
                  }
"=="              get_location (yytext); return EQ; /* equality operator */
"="               get_location (yytext); return '='; /* assignment operator */
">"               get_location (yytext); return '>';
"<"               get_location (yytext); return '<';
"+"               get_location (yytext); return '+';
"-"               get_location (yytext); return '-'; /* subtraction or unary minus */
"*"               get_location (yytext); return '*';
"/"               get_location (yytext); return '/';
"("               get_location (yytext); return '(';
")"               get_location (yytext); return ')';
"{"               get_location (yytext); return '{';
"}"               get_location (yytext); return '}';
","               get_location (yytext); return ',';
.                 ncolumn++;             return YYUNDEF;
%%

static void
get_location (char *text) {
  yylloc.first_line = yylloc.last_line = nline;
  yylloc.first_column = ncolumn;
  yylloc.last_column = (ncolumn += strlen(text)) - 1;
}
```

```
68
69 static YY_BUFFER_STATE state;
70
71 void
72 init_flex (const char *text) {
73   state = yy_scan_string (text);
74   nline = ncolumn = 1;
75 }
76
77 void
78 finalize_flex (void) {
79   yy_delete_buffer (state);
80 }
```

The file consists of three sections separated by `%%` (lines 19 and 60). These are the definitions, rules, and user code sections.

3.4.5.2 Definitions Section

- 1-13: The lines between `%top{` and `}` contain C source code. This code will be copied directly to the top of the generated C source file.
- 2-3: The program uses the `strlen` (line 66) and `atof` (line 40) functions, which are defined in `string.h` and `stdlib.h`, respectively. These header files are included here.
- 4: The program uses GLib functions and structures, such as `g_strdup` and `GSList`. The main GLib header file, `glib.h{}`, is included here.
- 5: The `turtle_parser.h` header file is generated from `turtle.y` by Bison. It defines constants and variables like `PU` and `yylloc`. This header is included here so the scanner can use them.
- 7-9: The variables `nline` and `ncolumn` track the current input position. The `get_location` function is declared here (as a forward declaration) so it can be called before its actual definition on lines 62-67.
- 12: `GSList` is a structure for a singly-linked list. The `list` variable is defined in `turtle.y`, so it is declared here with the `extern` storage-class specifier. It serves as the head of the list, which is used to keep track of dynamically allocated memory.
- 15: The `%option noyywrap` directive must be specified when the scanner reads from a single input file. For more details, refer to the "Generated Scanner" section in the Flex documentation.
- 17-18: `REAL_NUMBER` and `IDENTIFIER` are named definitions. In Flex, a name must begin with a letter or an underscore, followed by zero or more letters, digits, underscores (`_`), or dashes (`-`). These names are followed by the regular expressions that define them. They are used in the rules section, where they automatically expand to their corresponding regular expressions.

3.4.5.3 Rules Section

This section is the core of the scanner. Lexical rules consist of patterns and their corresponding actions. Patterns are either regular expressions or definitions enclosed in braces. These definitions must be declared beforehand in the definitions section. The syntax for writing these regular expressions is detailed in the Flex documentation.

For example, consider the rule on line 40:

- `{REAL_NUMBER}` is the pattern
- `get_location (yytext); yylval.NUM = atof (yytext); return NUM;` is the action.

The `{REAL_NUMBER}` pattern is defined on line 17, so it expands to the regular expression `(0|[1-9][0-9]*)(\.[0-9]+)?`. This expression matches numeric strings like `0`, `12`, and `1.5`.

When the input stream matches this pattern, the matched text is stored in the `yytext` variable, and the corresponding action is executed. First, the `get_location` function updates the location variables

to track the current position in the text. Then, the matched string is converted into a double-precision floating-point number using `atof(yytext)` and assigned to `yylval.NUM`. Finally, the scanner returns `NUM`, a token kind defined in `turtle.y` that represents double-precision numbers.

The scanner generated by Flex contains the `yylex` function. When `yylex` is called and encounters the input string "123.4", it processes it as follows:

1. The string "123.4" matches the `{REAL_NUMBER}` pattern.
2. The `get_location` function is called, updating the location variable `ncolumn` and setting the tracking coordinates within the `yylloc` structure.
3. The `atof` function converts the string "123.4" into the double value `123.4`.
4. This numeric value is assigned to the variable, `yylval.NUM`.
5. Finally, `yylex` returns the `NUM` token kind to the caller.

As a result, the caller knows the input is a number (`NUM`) and its value is `123.4`.

- 20-59: The rules section.
- 21: Defines a comment as a # followed by any characters except a newline. Because no corresponding action is specified, the matched text is simply ignored.
- 22: Matching a white space just increases `ncolumn` by one.
- 23: A tab character is assumed to equal eight spaces.
- 24: Matching a newline character increases `nline` by one and resets `ncolumn` to 1.
- 26-38: When the scanner matches one of these keywords, it updates the location variables (`ncolumn` and `yylloc`) and returns the corresponding token kind.
- 40: Real number constants. The action converts the matched text in `yytext` to a double-precision floating-point number, stores it in `yylval.NUM`, and returns `NUM`.
- 42: `IDENTIFIER` is defined on line 18. An identifier is the name of a variable or procedure, beginning with a letter and followed by zero or more letters or digits. The location variables are updated, and the identifier string is duplicated using the `g_strdup` function and assigned to `yylval.ID`. This newly allocated memory is then prepended to `list` (a `GSList`) so it can be automatically freed when `list` is freed (`g_slist_free_full` is called). Finally, the `ID` token kind is returned.
- 46-58: Symbols. Each line Updates the location variable and returns the token kind. For single-character symbols (like `=`, `+`, `-`), the token kind is simply the character itself. The exception is the two-character equality operator `==`, which returns the `EQ` token kind.
- 59: If the input does not match any of the preceding patterns, it triggers an error. A special token kind, `YYUNDEF`, is returned.

3.4.5.4 User Code Section

This section is simply copied into the generated C source file.

- 62-67: The `get_location` function. The input location is tracked by `nline` and `ncolumn`. The parser refers to the variable `yylloc`, which is a C structure with four members: `first_line`, `first_column`, `last_line`, and `last_column`. These point to the start and end of the current input text.
- 69: `YY_BUFFER_STATE` is a pointer to the input buffer. Flex defines `YY_BUFFER_STATE` in the generated C file (`turtle_lex.c`). See "Section 11: Multiple Input Buffers" in the Flex documentation for more information.
- 71-75: The `init_flex` function is called by `run_cb`, which is the "clicked" signal handler for the `Run` button. It takes one string parameter, which the caller fills with the contents of the GtkTextBuffer. The `yy_scan_string` function sets this string as the input buffer for the scanner. The `nline` and `ncolumn` variables are initialized to 1.
- 77-80: The `finalize_flex` function is called after the runtime routine finishes. It deletes the input buffer.

3.4.6 Turtle.y

Because `turtle.y` is over 800 lines long, explaining the entire source code is impractical. Instead, I will focus on the key points and omit the less important parts.

3.4.6.1 What Does Bison Do?

Bison generates the C source code for a parser from a Bison source file (`turtle.y`). A parser analyzes the source code according to its defined grammar. Consider the following Turtle source file:

```
fc (1,0,0) # Foreground color is red, rgb = (1,0,0).
pd         # Pen down.
distance = 100
angle = 90
fd distance    # Go forward by distance (100) pixels.
tr angle     # Turn right by angle (90) degrees.
```

The parser calls `yylex` to get the next token. Each token consists of its type (the token kind) and its value (the semantic value). Therefore, each time the parser calls `yylex`, it receives the information shown in the following table:

	token kind	yylval.ID	yylval.NUM
1	FC		
2	(		
3	NUM		1.0
4	,		
5	NUM		0.0
6	,		
7	NUM		0.0
8	)		
9	PD		
10	ID	distance	
11	=		
12	NUM		100.0
13	ID	angle	
14	=		
15	NUM		90.0
16	FD		
17	ID	distance	
18	TR		
19	ID	angle	

The Bison source code specifies the grammar rules of the Turtle language. For example, `fc (1,0,0)` is a primary procedure. A procedure is similar to a void function in C because it does not return a value. While programmers can define their own custom procedures, built-in commands like `fc` are classified as primary procedures.

This grammar rule is defined in the Bison source code as follows:

```
primary_procedure: FC '(' expression ',' expression ',' expression ')';
expression: ID | NUM;
```

This defines the following rules:

- A `primary_procedure` consists of the `FC` token followed by three expressions enclosed in parentheses and separated by commas.

- An expression can be either an `ID` or a `NUM`.

The notation above is known as BNF (Backus-Naur Form). Strictly speaking, Bison's syntax is not exactly standard BNF, but the differences are minor.

If the expressions are numbers, the first line expands to:

```
FC '(' NUM ',' NUM ',' NUM ')';
```

The parser analyzes the Turtle source code. If the input matches the definition above, the parser recognizes it as a primary procedure.

The complete grammar of the Turtle language is detailed in the Turtle manual. The following is an excerpt from that document:

```
program:
  statement
| program statement
;

statement:
  primary_procedure
| procedure_definition
;

primary_procedure:
  PU
| PD
| PW expression
| FD expression
| TR expression
| TL expression
| BC '(' expression ',' expression ',' expression ')'
| FC '(' expression ',' expression ',' expression ')'
| ID '=' expression
| IF '(' expression ')' '{' primary_procedure_list '}'
| RT
| RS
| RP '(' expression ')' '{' primary_procedure_list '}'
| ID '(' ')'
| ID '(' argument_list ')'
;

procedure_definition:
  DP ID '('  ')' '{' primary_procedure_list '}'
| DP ID '(' parameter_list ')' '{' primary_procedure_list '}'
;

parameter_list:
  ID
| parameter_list ',' ID
;

argument_list:
  expression
| argument_list ',' expression
```

```
;

primary_procedure_list:
  primary_procedure
| primary_procedure_list primary_procedure
;

expression:
  expression EQ expression
| expression '>' expression
| expression '<' expression
| expression '+' expression
| expression '-' expression
| expression '*' expression
| expression '/' expression
| '-' expression %prec UMINUS
| '(' expression ')'
| ID
| NUM
;
```

The grammar rule defines a `program` first:

- A `program` is either a single `statement`, or a `program` followed by another `statement`.

Because this definition is recursive, it evaluates as follows:

- `statement` is a program.
- `statement statement` becomes `program statement`, which is a `program`.
- For `statement statement statement`, the first two `statements` reduce to a `program`. This leaves `program statement`, which is also a `program`.

Ultimately, this recursive logic means that a `program` is simply any continuous sequence of `statements`.

The symbols `program` and `statement` are not tokens because they do not appear directly in the input. In grammar rules, these are called non-terminal symbols (often abbreviated as `nterm`).

Conversely, tokens are called terminal symbols. In this context, the word "token" has a broad meaning: it includes both named tokens (like `ID` or `NUM`) and single-character symbols (like `=`) that appear in the input.

Let's analyze the previous Turtle program exactly as Bison does:

	token kind	yylval.ID	yylval.NUM	parse	S/R
1	FC			FC	S
2	(			FC(	S
3	NUM		1.0	FC(NUM	S
				FC(expression	R
4	,			FC(expression,	S
5	NUM		0.0	FC(expression,NUM	S
				FC(expression,expression	R
6	,			FC(expression,expression,	S
7	NUM		0.0	FC(expression,expression,NUM	S
				FC(expression,expression,expression	R
8	)			FC(expression,expression,expression)	S
				primary_procedure	R
				statement	R

	token kind	yylval.ID	yylval.NUM	parse	S/R
				program	R
9	PD			program PD	S
				program primary_procedure	R
				program statement	R
				program	R
10	ID	distance		program ID	S
11	=			program ID=	S
12	NUM		100.0	program ID=NUM	S
				program ID=expression	R
				program primary_procedure	R
				program statement	R
				program	R
13	ID	angle		program ID	S
14	=			program ID=	S
15	NUM		90.0	program ID=NUM	S
				program ID=expression	R
				program primary_procedure	R
				program statement	R
				program	R
16	FD			program FD	S
17	ID	distance		program FD ID	S
				program FD expression	R
				program primary_procedure	R
				program statement	R
				program	R
18	TR			program TR	S
19	ID	angle		program TR ID	S
				program TR expression	R
				program primary_procedure	R
				program statement	R
				program	R

The rightmost column indicates the parser's action: Shift (`S`) or Reduce (`R`).

- A shift operation appends the next input token to the parser's stack.
- A reduce operation substitutes a recognized pattern on the stack with its corresponding non-terminal symbol (`nterm`).

For example, in the fourth row, `NUM` is replaced by `expression`. This substitution process is a reduce operation.

The parser repeats shift and reduce operations until it reaches the end of the input. If the final result reduces to a `program`, the entire input is syntactically valid. Whenever a reduction occurs, the parser executes a corresponding action. These actions build a syntax tree, which is later analyzed and executed by a runtime routine.

Bison source files are called Bison grammar files. A grammar file consists of four sections: prologue, declarations, rules, and epilogue. The format is as follows:

```
%{
prologue
%}
declarations
```

```
%%
rules
%%
epilogue
```

3.4.6.2 Prologue

The prologue section contains C code that is copied directly into the generated parser file. The following is an excerpt from `turtle.y`:

```
%code top{
  #include <stdarg.h>
  #include <setjmp.h>
  #include <math.h>
  #include <glib.h>
  #include <cairo.h>
  #include "turtle_parser.h"

  /* The following line defines 'debug' so that debug information is printed out during the run time
  /* However it makes the program slow. */
  /* If you want to debug on, uncomment the line. */

  /* #define debug 1 */

  extern cairo_surface_t *surface;

  /* error reporting */
  static void yyerror (char const *s) { /* for syntax error */
    g_printerr ("%s from line %d, column %d to line %d, column %d\n",s, yylloc.first_line, yylloc.fi
  }
  /* Node type */
  enum {
    N_PU,
    N_PD,
    N_PW,
 ... ... ...
  };
}
```

The directive `%code top` copies its contents to the top of the parser implementation file. It usually includes `#include` directives, declarations of functions, and definitions of constants. The function `yyerror` reports a syntax error and is called by the parser. The node type identifies a node in the tree.

The `%code requires` directive copies its contents into both the generated parser file and the generated header file. This header file is then included by the Flex scanner and other C source files.

```
%code requires {
  int yylex (void);
  int yyparse (void);
  void run (void);

  /* semantic value type */
  typedef struct _node_t node_t;
  struct _node_t {
    int type;
    union {
```

```
      struct {
        node_t *child1, *child2, *child3;
      } child;
      char *name;
      double value;
    } content;
  };
}
```

- `yylex` is shared between the parser and the scanner source files.
- `yyparse` and `run` are called by the `run_cb` function in `turtleapplication.c`.
- `node_t` is the semantic value type used for non-terminal symbols (`nterms`).

The generated header file uses these definitions to create `YYSTYPE`, the global semantic value type that accommodates all token and non-terminal value types. The following is an excerpt from the generated header file:

```
/* Value type.  */
#if ! defined YYSTYPE && ! defined YYSTYPE_IS_DECLARED
union YYSTYPE
{
  char * ID;                            /* ID  */
  double NUM;                           /* NUM  */
  node_t * program;                     /* program  */
  node_t * statement;                   /* statement  */
  node_t * primary_procedure;           /* primary_procedure  */
  node_t * primary_procedure_list;      /* primary_procedure_list  */
  node_t * procedure_definition;        /* procedure_definition  */
  node_t * parameter_list;              /* parameter_list  */
  node_t * argument_list;               /* argument_list  */
  node_t * expression;                  /* expression  */
};
```

Other useful macros and declarations are placed in the `%code` directive.

```
%code {
/* The following macro is convenient to get the member of the node. */
  #define child1(n) (n)->content.child.child1
  #define child2(n) (n)->content.child.child2
  #define child3(n) (n)->content.child.child3
  #define name(n) (n)->content.name
  #define value(n) (n)->content.value

  /* start of nodes */
  static node_t *node_top = NULL;
  /* functions to generate trees */
  static node_t *tree1 (int type, node_t *child1, node_t *child2, node_t *child3);
  static node_t *tree2 (int type, double value);
  static node_t *tree3 (int type, char *name);
}
```

3.4.6.3 Bison declarations

The declarations section defines all terminal and non-terminal symbols used in the grammar. It is also used to specify other parser directives, such as `%locations`, `%define`, `%left`, and `%precedence`.

```
%locations
```

```
%define api.value.type union /* YYSTYPE, the type of semantic values, is union of following types */
 /* key words */
%token PU
%token PD
%token PW
%token FD
%token TR
%token TL /* ver 0.5 */
%token BC
%token FC
%token DP
%token IF
%token RT
%token RS
%token RP /* ver 0.5 */
%token EQ /* equality operator added on 2026/08/10 */
 /* constant */
%token <double> NUM
 /* identifier */
%token <char *> ID
 /* non terminal symbol */
%nterm <node_t *> program
%nterm <node_t *> statement
%nterm <node_t *> primary_procedure
%nterm <node_t *> primary_procedure_list
%nterm <node_t *> procedure_definition
%nterm <node_t *> parameter_list
%nterm <node_t *> argument_list
%nterm <node_t *> expression
 /* logical relation symbol */
%left '=' '<' '>'
 /* arithmetic symbol */
%left '+' '-'
%left '*' '/'
%precedence UMINUS /* unary minus */
```

The `%locations` directive inserts a location-tracking structure into the generated header file. This structure is defined as follows:

```
typedef struct YYLTYPE YYLTYPE;
struct YYLTYPE
{
  int first_line;
  int first_column;
  int last_line;
  int last_column;
};
```

This type is shared between the scanner and the parser files. The error-reporting function, `yyerror`, uses it to indicate exactly where a syntax error occurred.

The `%define api.value.type union` directive generates the semantic value type (`YYSTYPE`) by combining the types of all tokens and non-terminals, and inserts it into the generated header file. This generated code is the `YYSTYPE` excerpt shown in the previous subsection.

The `%token` and `%nterm` directives define terminal tokens and non-terminal symbols, respectively.

```
%token PU
... ...
%token <double> NUM
```

These directives define the tokens `PU` and `NUM`. The numeric values for these token kinds are defined as enumeration constants in the generated header file:

```
  enum yytokentype
  {
  ... ... ...
    PU = 258,                      /* PU  */
  ... ... ...
    NUM = 269,                     /* NUM  */
  ... ... ...
  };
  typedef enum yytokentype yytoken_kind_t;
```

Additionally, the `<double>` tag ensures that the semantic value for `NUM` is typed as a double in the generated header file:

```
union YYSTYPE
{
  char * ID;                                /* ID  */
  double NUM;                               /* NUM  */
  ... ...
}
```

All non-terminal symbols (`nterms`) share the same semantic value type, `node_t *`.

The `%left` and `%precedence` directives define the precedence and associativity of operators:

```
 /* logical relation symbol */
%left '=' '<' '>'
 /* arithmetic symbol */
%left '+' '-'
%left '*' '/'
%precedence UMINUS /* unary minus */
```

The `%left` directive defines the following symbols as left-associative operators. If the operator `+` is left-associative, then:

```
A + B + C = (A + B) + C
```

In other words, the leftmost operation is evaluated first.

Conversely, if an operator `*` is right-associative, it will be evaluated from right to left:

```
A * B * C = A * (B * C)
```

The definitions above determine the exact behavior of the parser. Although addition and multiplication are mathematically associative — meaning `(A + B) + C and A + (B + C)` yield the same result — the parser requires a strict evaluation order to build the syntax tree. Without explicitly specifying left (or right) associativity, the parser will encounter ambiguity and result in a shift/reduce conflict.

The order of the `%left` and `%precedence` directives defines the precedence of the operators. Operators declared later have a higher precedence than those declared earlier (from lowest to highest).

For example, based on the declarations above:

```
v=w+z*5+7 is the same as v=((w+(z*5))+7)
```

Note: The =operator in the example above is an assignment. In the Turtle language, an assignment is not evaluated as an `expression`; instead, it is parsed as a `primary_procedure`. Conversely, the `==` operator (not `=`) is used as a logical operator within expressions. This logical equality operator is typically used in conditional expressions, such as in an `if` statement.

3.4.6.4 Grammar Rules

The Grammar Rules section defines the syntactic grammar of the language. It is written in a format similar to BNF (Backus-Naur Form):

```
result: components { action };
```

- `result`: A non-terminal symbol (`nterm`).
- `components`: A sequence of tokens and/or non-terminal symbols.
- `action`: C code that is executed whenever the parser reduces the components to the result. This action block is optional and can be omitted.

The following is a simplified excerpt of the grammar rules from `turtle.y`:

```
program:
  statement { node_top = $$ = $1; }
;
statement:
  primary_procedure
;
primary_procedure:
  FD expression    { $$ = tree1 (N_FD, $2, NULL, NULL); }
;
expression:
  NUM   { $$ = tree2 (N_NUM, $1); }
;
```

- The first rule defines `program` as a `statement`. Whenever a `statement` is reduced to a `program`, the action `node_top = $$ = $1;` is executed.
- `node_top` is a static variable that points to the top (root) node of the tree.
- The symbol `$$` represents the semantic value of the result (the left-hand side of the rule). For example, in the first rule, `$$` is the semantic value of program, which is a pointer to a `node_t` structure.
- The symbol `$1` represents the semantic value of the first component on the right-hand side. In the first rule, `$1` is the semantic value of `statement`, which is also a `node_t` pointer.
- The second rule defines `statement` as a `primary_procedure`. Because no action is specified, the default action (`$$ = $1;`) is executed automatically.
- The third rule defines `primary_procedure` as the token `FD` followed by an `expression`. The action calls the `tree1` function to create a new tree node, and assigns the returned pointer to `$$`. This tree node contains a type identifier and a union (which can hold three child node pointers, a string, or a double).

```
node --+-- type
       +-- union contents
                   +---struct {node_t *child1, *child2, *child3;};
                   +---char *name
                   +---double value
```

- The `tree1` function assigns its four arguments to the node's type, `child1`, `child2`, and `child3` members.
- The final rule defines an `expression` as a `NUM` token.

- The action for this rule calls `tree2` to create a tree node. The parameters for `tree2` are the node type and the numeric semantic value (passed via `$1`).

Suppose the parser reads the following program:

```
fd 100
```

How does the parser process this?

1. The parser reads the `FD` token. It recognizes that this might be the start of a `primary_procedure`, but it needs to read the next token (the lookahead) to be sure.
2. The scanner (`yylex`) returns the `NUM` token and sets its semantic value (`yylval.NUM`) to the double `100.0`. The parser then reduces `NUM` to an `expression`. During this reduction, it creates a new tree node of type `N_NUM` holding the value `100.0`, and sets the `expression`'s semantic value to point to this new node.
3. The parser's stack now contains `FD` followed by `expression`. The parser reduces these two symbols to a `primary_procedure`. It creates a new node of type `N_FD` and sets its `child1` member to point to the `N_NUM` node. The semantic value of the `primary_procedure` is set to point to this new `N_FD` node.
4. The parser reduces `primary_procedure` to statement. The semantic value of statement simply copies the pointer from `primary_procedure`, so it also points to the `N_FD` node.
5. The parser reduces `statement` to `program`. The semantic value of `statement` is assigned to both the semantic value of `program` and the static variable `node_top`.
6. Finally, `node_top` points to the root `N_FD` node, and the `N_FD` node points to the child `N_NUM` node, successfully building the syntax tree.

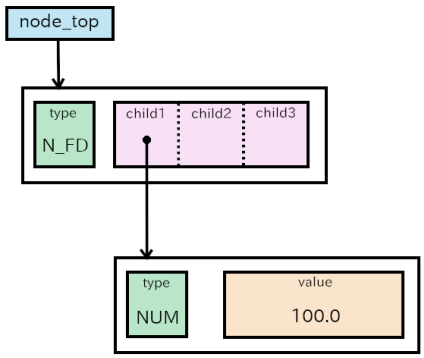


Figure 3.13: tree

The following grammar rules are excerpted from `turtle.y`. Because these rules are based on the principles explained above, they are presented here without line-by-line explanations. Please review the code carefully to understand how the remaining rules and actions are structured.

```
program:
  statement { node_top = $$ = $1; }
| program statement {
        node_top = $$ = tree1 (N_program, $1, $2, NULL);
#ifdef debug
if (node_top == NULL) g_printerr ("program: node_top is NULL.\n"); else g_printerr ("program: node_t
#endif
        }
;

statement:
  primary_procedure
```

```
| procedure_definition
;

primary_procedure:
  PU    { $$ = tree1 (N_PU, NULL, NULL, NULL); }
| PD    { $$ = tree1 (N_PD, NULL, NULL, NULL); }
| PW expression    { $$ = tree1 (N_PW, $2, NULL, NULL); }
| FD expression    { $$ = tree1 (N_FD, $2, NULL, NULL); }
| TR expression    { $$ = tree1 (N_TR, $2, NULL, NULL); }
| TL expression    { $$ = tree1 (N_TL, $2, NULL, NULL); } /* ver 0.5 */
| BC '(' expression ',' expression ',' expression ')' { $$ = tree1 (N_BC, $3, $5, $7); }
| FC '(' expression ',' expression ',' expression ')' { $$ = tree1 (N_FC, $3, $5, $7); }
 /* assignment */
| ID '=' expression   { $$ = tree1 (N_ASSIGN, tree3 (N_ID, $1), $3, NULL); }
 /* control flow */
| IF '(' expression ')' '{' primary_procedure_list '}' { $$ = tree1 (N_IF, $3, $6, NULL); }
| RT    { $$ = tree1 (N_RT, NULL, NULL, NULL); }
| RS    { $$ = tree1 (N_RS, NULL, NULL, NULL); }
| RP '(' expression ')' '{' primary_procedure_list '}'    { $$ = tree1 (N_RP, $3, $6, NULL); }
 /* user defined procedure call */
| ID '(' ')'  { $$ = tree1 (N_procedure_call, tree3 (N_ID, $1), NULL, NULL); }
| ID '(' argument_list ')'  { $$ = tree1 (N_procedure_call, tree3 (N_ID, $1), $3, NULL); }
;

procedure_definition:
  DP ID '('  ')' '{' primary_procedure_list '}'  {
         $$ = tree1 (N_procedure_definition, tree3 (N_ID, $2), NULL, $6);
        }
| DP ID '(' parameter_list ')' '{' primary_procedure_list '}'  {
         $$ = tree1 (N_procedure_definition, tree3 (N_ID, $2), $4, $7);
        }
;

parameter_list:
  ID { $$ = tree3 (N_ID, $1); }
| parameter_list ',' ID  { $$ = tree1 (N_parameter_list, $1, tree3 (N_ID, $3), NULL); }
;

argument_list:
  expression
| argument_list ',' expression { $$ = tree1 (N_argument_list, $1, $3, NULL); }
;

primary_procedure_list:
  primary_procedure
| primary_procedure_list primary_procedure {
         $$ = tree1 (N_primary_procedure_list, $1, $2, NULL);
        }
;

expression:
  expression EQ expression { $$ = tree1 (N_EQ, $1, $3, NULL); }
| expression '>' expression { $$ = tree1 (N_GT, $1, $3, NULL); }
```

```
| expression '<' expression { $$ = tree1 (N_LT, $1, $3, NULL); }
| expression '+' expression { $$ = tree1 (N_ADD, $1, $3, NULL); }
| expression '-' expression { $$ = tree1 (N_SUB, $1, $3, NULL); }
| expression '*' expression { $$ = tree1 (N_MUL, $1, $3, NULL); }
| expression '/' expression { $$ = tree1 (N_DIV, $1, $3, NULL); }
| '-' expression %prec UMINUS { $$ = tree1 (N_UMINUS, $2, NULL, NULL); }
| '(' expression ')' { $$ = $2; }
| ID    { $$ = tree3 (N_ID, $1); }
| NUM   { $$ = tree2 (N_NUM, $1); }
;
```

3.4.6.5 Epilogue

The epilogue is written in C and is copied directly into the generated parser implementation file. While you can generally include any arbitrary C code in this section, the Turtle interpreter uses it to contain the runtime routines and other auxiliary functions.

3.4.6.5.1 Tree Node Creation Functions The parser uses three functions to create abstract syntax tree nodes: `tree1`, `tree2`, and `tree3`.

- `tree1` creates a node and sets its type, along with pointers to up to three child nodes (`NULL` is permitted).
- `tree2` creates a node and sets its type, along with a numeric value (`double`).
- `tree3` creates a node and sets its type, along with a string pointer.

Each function dynamically allocates memory for the new node. These allocations are tracked in a list so they can be safely freed when the runtime routine finishes.

These functions are called directly within the semantic actions of the grammar rules section.

```
/* Dynamically allocated memories are added to the single list. They will be freed in the finalize f
GSList *list = NULL;

node_t *
tree1 (int type, node_t *child1, node_t *child2, node_t *child3) {
  node_t *new_node;

  list = g_slist_prepend (list, g_malloc (sizeof (node_t)));
  new_node = (node_t *) list->data;
  new_node->type = type;
  child1(new_node) = child1;
  child2(new_node) = child2;
  child3(new_node) = child3;
  return new_node;
}

node_t *
tree2 (int type, double value) {
  node_t *new_node;

  list = g_slist_prepend (list, g_malloc (sizeof (node_t)));
  new_node = (node_t *) list->data;
  new_node->type = type;
  value(new_node) = value;
  return new_node;
}
```

```
node_t *
tree3 (int type, char *name) {
  node_t *new_node;

  list = g_slist_prepend (list, g_malloc (sizeof (node_t)));
  new_node = (node_t *) list->data;
  new_node->type = type;
  name(new_node) = name;
  return new_node;
}
```

3.4.6.5.2 Symbol Table Variables and user-defined procedures are registered in the symbol table. Currently, this table is implemented as a simple C array. In a future version, it should be replaced with a more efficient data structure, such as a hash table.

- Variables are registered with their name and value.
- Procedures are registered with their name and a pointer to the procedure node.

To support this, each entry in the table contains the following fields:

- type: Identifies whether the entry is a variable or a procedure.
- name: The identifier string.
- value: Either a numeric value (for variables) or a node pointer (for procedures).

```
#define MAX_TABLE_SIZE 100
enum {
  PROC,
  VAR
};

struct {
  int type;
  char *name;
  union {
    node_t *node;
    double value;
  } object;
} table[MAX_TABLE_SIZE];
int tp;

void
init_table (void) {
  tp = 0;
}
```

The `init_table` function initializes the symbol table. This function must be called before registering any entries.

There are five functions used to access the table:

- `proc_install` installs a procedure.
- `var_install` installs a variable.
- `proc_lookup` searches for a procedure. If found, it returns a pointer to the procedure node; otherwise, it returns `NULL`.
- `var_lookup` searches for a variable. If found, it returns `TRUE` and assigns the value to the provided pointer argument; otherwise, it returns `FALSE`.

- `var_replace` updates the value of a variable. If the variable has not been registered yet, this function installs it.

```
int
tbl_lookup (int type, char *name) {
  int i;

  if (tp == 0)
    return -1;
  for (i=0; i<tp; ++i)
    if (type == table[i].type && strcmp(name, table[i].name) == 0)
      return i;
  return -1;
}

void
tbl_install (int type, char *name, node_t *node, double value) {
  if (tp >= MAX_TABLE_SIZE)
    runtime_error ("Symbol table overflow.\n");
  else if (tbl_lookup (type, name) >= 0)
    runtime_error ("Name %s is already registered.\n", name);
  else {
    table[tp].type = type;
    table[tp].name = name;
    if (type == PROC)
      table[tp++].object.node = node;
    else
      table[tp++].object.value = value;
  }
}

void
proc_install (char *name, node_t *node) {
  tbl_install (PROC, name, node, 0.0);
}

void
var_install (char *name, double value) {
  tbl_install (VAR, name, NULL, value);
}

void
var_replace (char *name, double value) {
  int i;
  if ((i = tbl_lookup (VAR, name)) >= 0)
    table[i].object.value = value;
  else
    var_install (name, value);
}

node_t *
proc_lookup (char *name) {
  int i;
  if ((i = tbl_lookup (PROC, name)) < 0)
```

```
    return NULL;
  else
    return table[i].object.node;
}

gboolean
var_lookup (char *name, double *value) {
  int i;
  if ((i = tbl_lookup (VAR, name)) < 0)
    return FALSE;
  else {
    *value = table[i].object.value;
    return TRUE;
  }
}
```

3.4.6.5.3 Stack for Parameters and Arguments

A stack is a last-in, first-out (LIFO) data structure. Turtle uses a stack to store parameters and arguments whenever a procedure is called. This LIFO structure is essential for supporting recursive calls.

Each element on the stack contains a name and a value.

```
#define MAX_STACK_SIZE 500
struct {
  char *name;
  double value;
} stack[MAX_STACK_SIZE];
int sp, sp_biggest;

void
init_stack (void) {
  sp = sp_biggest = 0;
}
```

The `sp` variable serves as the stack pointer. It is an index for the `stack` array that always points to the next available position to store data.

The `sp_biggest` variable tracks the highest value reached by `sp`. This allows us to determine the maximum number of elements used in the array at runtime. Its primary purpose is to help find an appropriate value for `MAX_STACK_SIZE`. This variable will become unnecessary in a future version if the stack is updated to use a better data structure and dynamic memory allocation.

The runtime routine pushes data onto the stack whenever it executes a procedure call node (type `N_procedure_call`).

For example:

```
dp drawline (angle, distance) { ... ... ... }
drawline (90, 100)
```

- The first line defines the procedure `drawline`. The runtime routine stores the name “drawline” and its corresponding procedure node in the symbol table.
- The second line calls the procedure. First, it looks up the procedure in the symbol table to retrieve its node. Then, it examines the node to retrieve the parameter names, angle and distance.
- Next, it pushes (“angle”, `90.0`) onto the stack.
- Then, it pushes (“distance”, `100.0`) onto the stack.

- Finally, it pushes (NULL, `2.0`) onto the stack. The number `2.0` represents the total parameter count. This count is used to properly clean up (pop) the arguments off the stack when the procedure returns.

The following diagram illustrates the structure of the stack during a series of nested procedure calls. First, `procedure 1` is called, which takes two parameters. Inside `procedure 1`, `procedure 2` is called with a single parameter. Finally, inside `procedure 2`, `procedure 3` is called with three parameters. Notice how each procedure's block ends with a `NULL` entry storing its total parameter count.

In this implementation, data is pushed onto the stack from lower memory addresses to higher memory addresses. Therefore, in the diagram, the addresses increase from top to bottom.

However, standard stack terminology can be slightly counterintuitive here. The "top of the stack" always refers to the most recently pushed data, while the "bottom of the stack" is the very first item pushed. As a result, the logical "top of the stack" is visually located at the bottom of the diagram, while the logical "bottom of the stack" is at the top.

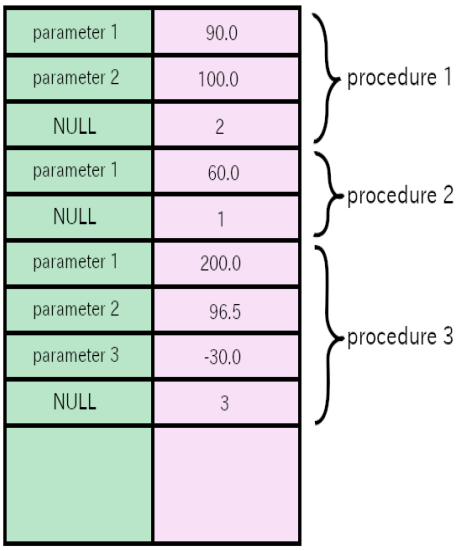


Figure 3.14: Stack

There are four functions used to access and manage the stack:

- `stack_push` pushes data onto the stack.
- `stack_lookup` searches the stack for a variable by its name. It restricts this search to the parameters of the most recently called procedure. If the variable is found, it returns `TRUE` and assigns the value to the provided `value` pointer argument; otherwise, it returns `FALSE`.
- `stack_replace` updates the value of a variable currently on the stack. It returns `TRUE` upon success, or `FALSE` if the variable is not found.
- `stack_return` discards the parameters from the most recent procedure call. The stack pointer is restored to its previous position, pointing back to the parameters of the calling procedure.
- `stack_search` is an internal helper for lookup and replace.

```
void
stack_push (char *name, double value) {
  if (sp >= MAX_STACK_SIZE)
    runtime_error ("Stack overflow.\n");
  else {
    stack[sp].name = name;
    stack[sp++].value = value;
```

```
    sp_biggest = sp > sp_biggest ? sp : sp_biggest;
  }
}

int
stack_search (char *name) {
  int depth, i;

  if (sp == 0)
    return -1;
  depth = (int) stack[sp-1].value;
  if (depth + 1 > sp) /* something strange */
    runtime_error ("Stack error.\n");
  for (i=0; i<depth; ++i)
    if (strcmp(name, stack[sp-(i+2)].name) == 0) {
      return sp-(i+2);
    }
  return -1;
}

gboolean
stack_lookup (char *name, double *value) {
  int i;

  if ((i = stack_search (name)) < 0)
    return FALSE;
  else {
    *value = stack[i].value;
    return TRUE;
  }
}

gboolean
stack_replace (char *name, double value) {
  int i;

  if ((i = stack_search (name)) < 0)
    return FALSE;
  else {
    stack[i].value = value;
    return TRUE;
  }
}

void
stack_return(void) {
  int depth;

  if (sp <= 0)
    return;
  depth = (int) stack[sp-1].value;
  if (depth + 1 > sp) /* something strange */
    runtime_error ("Stack error.\n");
```

```
  sp -= depth + 1;
}
```

3.4.6.5.4 Surface and Cairo The global variable `surface` is shared between `turtleapplication.c` and `turtle.y`. It is initialized in `turtleapplication.c`.

The runtime routine uses its own Cairo context, which is distinct from the context used by the GtkDrawingArea instance. The runtime routine uses this context to draw shapes directly onto `surface`.

After the runtime routine returns to `run_cb`, `run_cb` queues the GtkDrawingArea widget for a redraw. When the widget is redrawn, the `draw_func` function is called to copy the contents of `surface` onto the surface of the GtkDrawingArea object.

The `turtle.y` file contains two functions to manage the Cairo context:

- `init_cairo` initializes the Cairo context and the static variables that track the pen status (up or down), direction, initial location, line width, and color. `init_cairo` first retrieves the current dimensions of the surface to set the turtle's initial location (at the center of the surface) and configure the transformation matrix (see below).
- `destroy_cairo` simply destroys the Cairo context.

Turtle uses its own coordinate system. Its origin is located at the center of the surface, and the positive X and Y axes point right and up, respectively.

However, the drawing surface has a different coordinate system. Its origin is at the top-left corner, and the positive X and Y axes point right and down, respectively.

The plane that uses the Turtle coordinate system is called user space, which corresponds to Cairo's user space. The plane that uses the surface coordinate system is called device space.

Cairo utilizes an affine transformation to map user-space coordinates (x, y) to device-space coordinates (z, w). This relationship is expressed mathematically as:

$$\begin{bmatrix} z \\ w \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} p \\ q \end{bmatrix}$$

Figure 3.15: transformation

The `init_cairo` function retrieves the width and height of `surface` (as shown in the code snippet below) to configure this mapping:

- The center is (0, 0) in the user-space coordinates, which corresponds to (width / 2, height / 2) in the device-space coordinates.
- The positive direction of the X axis is the same in both spaces. Therefore, the user-space point (1, 0) is transformed into (1 + width / 2, height / 2).
- The positive direction of the Y axis is inverted between the two spaces. Therefore, the user-space point (0, 1) is transformed into (width / 2, -1 + height / 2).

By substituting these coordinate pairs for x, y, z, and w into the matrix equation above, we can determine the transformation parameters. Solving this system of linear equations yields:

`a = 1, b = 0, c = 0, d = -1, p = width/2, q = height/2`

Cairo provides the `cairo_matrix_t` structure to define this affine transformation. The `init_cairo` function initializes this structure and applies it to the Cairo context (as shown in the code below). Once the matrix is applied, this transformation is automatically performed whenever drawing functions like `cairo_stroke` are called.

```
/* status of the surface */
static gboolean pen = TRUE;
static double angle = 90.0; /* angle starts from x axis and measured counterclockwise */
                  /* Initially facing to the north */
static double cur_x = 0.0;
static double cur_y = 0.0;
static double line_width = 2.0;

struct color {
  double red;
  double green;
  double blue;
};
static struct color bc = {0.95, 0.95, 0.95}; /* white */
static struct color fc = {0.0, 0.0, 0.0}; /* black */

/* cairo */
static cairo_t *cr;
gboolean
init_cairo (void) {
  int width, height;
  cairo_matrix_t matrix;

  pen = TRUE;
  angle = 90.0;
  cur_x = 0.0;
  cur_y = 0.0;
  line_width = 2.0;
  bc.red = 0.95; bc.green = 0.95; bc.blue = 0.95;
  fc.red = 0.0; fc.green = 0.0; fc.blue = 0.0;

  if (surface) {
    width = cairo_image_surface_get_width (surface);
    height = cairo_image_surface_get_height (surface);
    matrix.xx = 1.0; matrix.xy = 0.0; matrix.x0 = (double) width / 2.0;
    matrix.yx = 0.0; matrix.yy = -1.0; matrix.y0 = (double) height / 2.0;

    cr = cairo_create (surface);
    cairo_transform (cr, &matrix);
    cairo_set_source_rgb (cr, bc.red, bc.green, bc.blue);
    cairo_paint (cr);
    cairo_set_source_rgb (cr, fc.red, fc.green, fc.blue);
    cairo_move_to (cr, cur_x, cur_y);
    return TRUE;
  } else
    return FALSE;
}

void
destroy_cairo () {
  cairo_destroy (cr);
}
```

3.4.6.5.5 The `eval` Functions The `eval` function recursively evaluates an expression and returns its resulting value. For example, if the node type is `N_ADD`, the function performs the following steps:

1. It calls `eval(child1(node))` to retrieve the first value (`value1`).
2. It calls `eval(child2(node))` to retrieve the second value (`value2`).
3. It returns the sum of those two values (`value1 + value2`).

This arithmetic logic is handled by the `calc` macro, which is defined on the sixth line of the following code snippet.

```
double
eval (node_t *node) {
  double value = 0.0;

  if (node == NULL)
    runtime_error ("No expression to evaluate.\n");

#define calc(op) eval (child1(node)) op eval (child2(node))

  switch (node->type) {
    case N_EQ:
      value = (double) calc (==);
      break;
    case N_GT:
      value = (double) calc (>);
      break;
    case N_LT:
      value = (double) calc (<);
      break;
    case N_ADD:
      value = calc (+);
      break;
    case N_SUB:
      value = calc (-);
      break;
    case N_MUL:
      value = calc (*);
      break;
    case N_DIV:
      if (eval (child2(node)) == 0.0)
        runtime_error ("Division by zero.\n");
      else
        value = calc (/);
      break;
    case N_UMINUS:
      value = -(eval (child1(node)));
      break;
    case N_ID:
      if (! (stack_lookup (name(node), &value)) && ! var_lookup (name(node), &value) )
        runtime_error ("Variable %s not defined.\n", name(node));
      break;
    case N_NUM:
      value = value(node);
      break;
    default:
```

```
      runtime_error ("Illegal expression.\n");
  }
  return value;
}
```

3.4.6.5.6 The execute Function

The `execute` function analyzes and executes primary procedures and procedure definitions. It does not return a value and operates recursively.

The logic for handling the `N_RT` and `N_procedure_call` nodes is complex, so these specific operations will be explained in detail after the following code block. The remaining operations are straightforward and easy to understand. Please review the code below to understand the general execution flow.

```
/* procedure - return status */
static int proc_level = 0;
static int ret_level = 0;

void
execute (node_t *node) {
  double d, x, y;
  char *name;
  int n, i;
  int counter; /* ver 0.5, for repeat procedure */

  if (node == NULL)
    runtime_error ("Node is NULL.\n");
  if (proc_level > ret_level)
    return;
  switch (node->type) {
    case N_program:
      execute (child1(node));
      execute (child2(node));
      break;
    case N_PU:
      pen = FALSE;
      break;
    case N_PD:
      pen = TRUE;
      break;
    case N_PW:
      line_width = eval (child1(node)); /* line width */
      break;
    case N_FD:
      d = eval (child1(node)); /* distance */
      x = d * cos (angle*M_PI/180);
      y = d * sin (angle*M_PI/180);
      /* initialize the current point = start point of the line */
      cairo_move_to (cr, cur_x, cur_y);
      cur_x += x;
      cur_y += y;
      cairo_set_line_width (cr, line_width);
      cairo_set_source_rgb (cr, fc.red, fc.green, fc.blue);
      if (pen)
        cairo_line_to (cr, cur_x, cur_y);
      else
```

```
        cairo_move_to (cr, cur_x, cur_y);
      cairo_stroke (cr);
      break;
    case N_TR:
      angle -= eval (child1(node));
      for (; angle < 0; angle += 360.0);
      for (; angle>360; angle -= 360.0);
      break;
    case N_TL: /* ver 0.5 */
      angle += eval (child1(node));
      for (; angle < 0; angle += 360.0);
      for (; angle>360; angle -= 360.0);
      break;
    case N_BC:
      bc.red = eval (child1(node));
      bc.green = eval (child2(node));
      bc.blue = eval (child3(node));

#define fixcolor(c)  c = c < 0 ? 0 : (c > 1 ? 1 : c)

      fixcolor (bc.red);
      fixcolor (bc.green);
      fixcolor (bc.blue);
      /* clear the shapes and set the background color */
      cairo_set_source_rgb (cr, bc.red, bc.green, bc.blue);
      cairo_paint (cr);
      break;
    case N_FC:
      fc.red = eval (child1(node));
      fc.green = eval (child2(node));
      fc.blue = eval (child3(node));
      fixcolor (fc.red);
      fixcolor (fc.green);
      fixcolor (fc.blue);
      break;
    case N_ASSIGN:
      name = name(child1(node));
      d = eval (child2(node));
      if (! stack_replace (name, d)) /* First, tries to replace the value in the stack (parameter).*/
        var_replace (name, d); /* If the above fails, tries to replace the value in the table. If th
      break;
    case N_IF:
      if (eval (child1(node)))
        execute (child2(node));
      break;
    case N_RT:
      ret_level--;
      break;
    case N_RS:
      pen = TRUE;
      angle = 90.0;
      cur_x = 0.0;
      cur_y = 0.0;
```

```
      line_width = 2.0;
      fc.red = 0.0; fc.green = 0.0; fc.blue = 0.0;
      /* To change background color, use bc. */
      break;
    case N_RP: /* ver 0.5 */
      counter = (int) eval (child1(node));
      if (counter < 0)
        runtime_error ("Repeat number %d is negative.\n", counter);
      if (counter > 100)
        runtime_error ("Repeat number %d is too big.\n", counter);
      for (i=0; i<counter; ++i)
        execute (child2(node));
      break;
    case N_procedure_call:
      name = name(child1(node));

      node_t *proc = proc_lookup (name);
      if (! proc)
        runtime_error ("Procedure %s not defined.\n", name);
      if (strcmp (name, name(child1(proc))) != 0)
        runtime_error ("Unexpected error. Procedure %s is called, but invoked procedure is %s.\n", na
/* make tuples (parameter (name), argument (value)) and push them to the stack */

      node_t *param_list;
      node_t *arg_list;
      param_list = child2(proc);
      arg_list = child2(node);
      if (param_list == NULL) {
        if (arg_list == NULL) {
          stack_push (NULL, 0.0); /* number of argument == 0 */
        } else
          runtime_error ("Procedure %s has different number of argument and parameter.\n", name);
      }else {

/* Don't change the stack until finish evaluating the arguments. */
#define TEMP_STACK_SIZE 20

        char *temp_param[TEMP_STACK_SIZE];
        double temp_arg[TEMP_STACK_SIZE];
        n = 0;
        for (; param_list->type == N_parameter_list; param_list = child1(param_list)) {
          if (arg_list->type != N_argument_list)
            runtime_error ("Procedure %s has different number of argument and parameter.\n", name);
          if (n >= TEMP_STACK_SIZE)
            runtime_error ("Too many parameters. the number must be %d or less.\n", TEMP_STACK_SIZE)
          temp_param[n] = name(child2(param_list));
          temp_arg[n] = eval (child2(arg_list));
          arg_list = child1(arg_list);
          ++n;
        }
        if (param_list->type == N_ID && arg_list -> type != N_argument_list) {
          temp_param[n] = name(param_list);
          temp_arg[n] = eval (arg_list);
```

```
          if (++n >= TEMP_STACK_SIZE)
            runtime_error ("Too many parameters. the number must be %d or less.\n", TEMP_STACK_SIZE)
          temp_param[n] = NULL;
          temp_arg[n] = (double) n;
          ++n;
        } else
          runtime_error ("Unexpected error.\n");
        for (i = 0; i < n; ++i)
          stack_push (temp_param[i], temp_arg[i]);
      }
      ret_level = ++proc_level;
      execute (child3(proc));
      ret_level = --proc_level;
      stack_return ();
      break;
    case N_procedure_definition:
      name = name(child1(node));
      proc_install (name, node);
      break;
    case N_primary_procedure_list:
      execute (child1(node));
      execute (child2(node));
      break;
    default:
      runtime_error ("Unknown statement.\n");
  }
}
```

The parser creates an `N_procedure_call` node when it encounters a call to a user-defined procedure. This procedure must have been defined in a previous statement. Suppose the parser reads the following example code:

```
dp drawline (angle, distance) {
  tr angle
  fd distance
}
drawline (90, 100)
drawline (90, 100)
drawline (90, 100)
drawline (90, 100)
```

This example draws a square.

When the parser reads lines 1 through 4, it generates the following node structure:

The runtime routine simply stores the procedure in the symbol table, associating its name with its corresponding node structure.

When the parser encounters the procedure call on line 5, it generates the following node structure:

When the runtime encounters an `N_procedure_call node`, it performs the following sequential steps:

1. Lookup: Searches the symbol table to find the procedure's name.
2. Retrieve: Retrieves pointers to the parameter list node and the procedure body node.
3. Bind Arguments: Creates a temporary stack and pairs each parameter name with its corresponding argument value. It pushes these tuples onto the stack, followed by a final terminating entry: (NULL, number_of_parameters). If no errors occur, it copies these bound values from the temporary stack

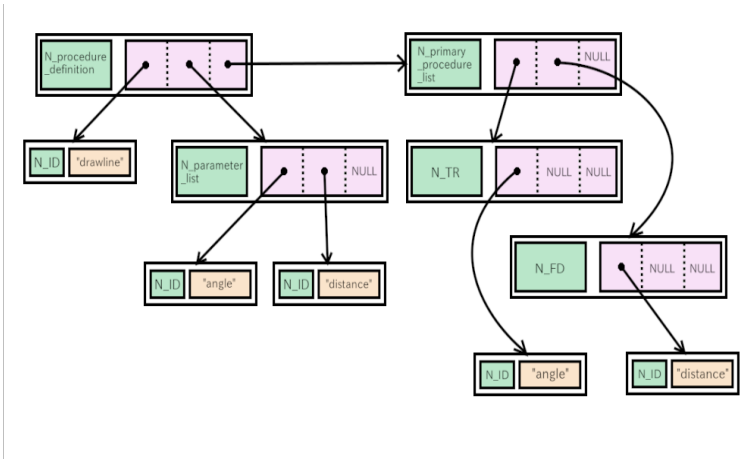


Figure 3.16: Nodes of drawline

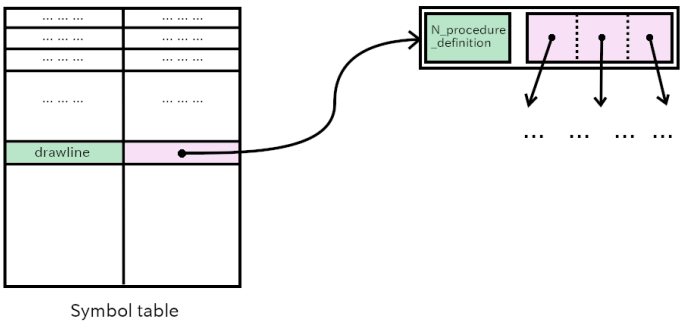


Figure 3.17: Symbol table

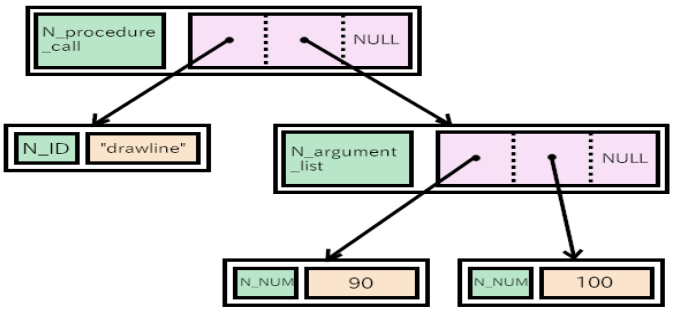


Figure 3.18: Nodes of procedure call

to the active parameter stack.
4. Update Execution Levels: Increments `proc_level` by 1 and sets `ret_level` to match `proc_level`. `proc_level` tracks the depth of the procedure call (it is 0 when running in the main routine). `ret_level` controls the execution flow. The runtime only executes commands when `ret_level` equals `proc_level`. If `ret_level` is smaller than `proc_level`, the runtime halts execution of the current block until the levels match again.
5. Execute: Executes the procedure's body node.
6. Cleanup and Return: Decrements `proc_level` by 1, updates `ret_level` to match the new `proc_level`, and calls `stack_return`.

When the runtime encounters an `N_RT` (return) node, it decrements `ret_level` by 1. Because `ret_level` is now smaller than the current `proc_level`, the runtime automatically ignores all subsequent commands in the procedure, effectively forcing an early return.

3.4.6.5.7 Runtime Entry and Error Functions The `run` function serves as the entry point for the runtime routine.

The `runtime_error` function is responsible for reporting errors that occur during execution. (By contrast, errors encountered during the parsing phase are classified as syntax errors and are handled by `yyerror`.) When `runtime_error` is called, it reports the issue, immediately halts command execution, and returns control to the `run` function to safely exit.

The runtime error handling utilizes the `setjmp` and `longjmp` functions, which are declared in the `<setjmp.h>` header.

The `setjmp(buf)` function saves the current execution context into `buf` and initially returns 0. When `longjmp(buf, 1)` is called later by the `runtime_error` function, it restores that saved context. This effectively rewinds the program, causing execution to resume immediately after the original `setjmp` call — but this time, `setjmp` returns 1.

In the run function below, this mechanism controls the execution flow:

- Normal Execution: When `setjmp` is initially called, it returns 0, which is assigned to `i`. The program safely proceeds to call `execute(node_top)`, and then continues to the cleanup phase.
- Error Recovery: If an error occurs, `runtime_error` prints the formatted message and calls `longjmp`. The program jumps back, and `setjmp` returns 1, which is assigned to `i`. Because `i` is now 1, the program bypasses the `execute(node_top)` call and jumps directly to the cleanup phase.
- Cleanup: Regardless of whether the execution succeeded or failed, the function safely concludes by calling `destroy_cairo()` and using `g_slist_free_full` to free all dynamically allocated memory.

```
static jmp_buf buf;

void
run (void) {
  int i;

  if (! init_cairo()) {
    g_print ("Cairo not initialized.\n");
    return;
  }
  init_table();
  init_stack();
  ret_level = proc_level = 1;
  i = setjmp (buf);
  if (i == 0)
    execute(node_top);
  /* else ... get here by calling longjmp */
```

```
  destroy_cairo ();
  g_slist_free_full (g_steal_pointer (&list), g_free);
}

/* format supports only %s, %f and %d */
static void
runtime_error (char *format, ...) {
  va_list args;
  char *f;
  char b[3];
  char *s;
  double v;
  int i;

  va_start (args, format);
  for (f = format; *f; f++) {
    if (*f != '%') {
      b[0] = *f;
      b[1] = '\0';
      g_print ("%s", b);
      continue;
    }
    switch (*++f) {
      case 's':
        s = va_arg(args, char *);
        g_print ("%s", s);
        break;
      case 'f':
        v = va_arg(args, double);
        g_print("%f", v);
        break;
      case 'd':
        i = va_arg(args, int);
        g_print("%d", i);
        break;
      default:
        b[0] = '%';
        b[1] = *f;
        b[2] = '\0';
        g_print ("%s", b);
        break;
    }
  }
  va_end (args);

  longjmp (buf, 1);
}
```

The `runtime_error` function utilizes a variable-length argument list.

```
static void
runtime_error (char *format, ...)
```

This mechanism is implemented using the standard `<stdarg.h>` header. A `va_list` variable named `args` is used to iterate through the provided arguments:

- `va_start` initializes the `args` list.
- `va_arg` retrieves the current argument and advances the reference pointer to the next one.
- `va_end` performs the necessary cleanup operations when the iteration is complete.

While `runtime_error` mimics the behavior of the standard `printf` function, it is intentionally simplified to support only the `%s`, `%f`, and `%d` format specifiers. The implementations of both `<setjmp.h>` and `<stdarg.h>` in this interpreter are modeled closely after the conventions described in the classic text, *The C Programming Language* by Brian W. Kernighan and Dennis M. Ritchie.

3.4.6.5.8 Further Reading and Resources This turtle program serves as a foundational exercise rather than a polished production language. Develping a robust, custom programming language requires extensive study. While analyzing modern open-source codebases (such as the C source code for Ruby) is arguably the most practical way for programmers to learn today, the following historical and technical resources remain highly valuable:

- The official Flex and Bison documentation
- *Software Tools* by Brian W. Kernighan and P. J. Plauger (1976)
- *The UNIX Programming Environment* by Brian W. Kernighan and Rob Pike (1984)

In addition, the following books are widely recommended, although I have not read them personally:

- *Crafting Interpreters* by Robert Nystrom. It is completely free to read on the author's website.
- *Compilers: Principles, Techniques, and Tools* (often called the "Dragon Book") by Aho, Lam, Sethi, and Ullman. This is the classic, deeply academic reference for parsing theory.

Chapter 4

Drag and Drop

4.1 Drag and Drop

4.1.1 What is Drag and Drop?

Drag and drop is also written as “Drag-and-Drop”, or “DND” in short. DND is like “copy and paste” or “cut and paste”. If a user drags a UI element, which is a widget, selected part or other elements, data is transferred from the source to the destination.

You probably have experience moving a file with DND.

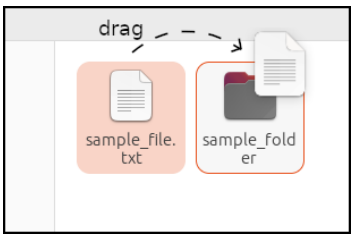


Figure 4.1: DND on the GUI file manager

When the DND starts, the file `sample_file.txt` is provided to the system. When it ends, the system provides it to the `sample_folder` in the file manager. Therefore, it is like “cut and paste”. The actual behavior may be different from the explanation here, but the concept is similar.

4.1.2 Example for DND

This tutorial provides a simple example in the `src/dnd` directory. It has three labels for the source and one label for the destination. The source labels have “red”, “green” or “blue” labels. If a user drags the label to the destination label, the font color will be changed.

4.1.3 UI File

The widgets are defined in the XML file `dnd.ui`.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <interface>
3   <object class="GtkApplicationWindow" id="win">
4     <property name="default-width">800</property>
5     <property name="default-height">600</property>
6     <property name="resizable">FALSE</property>
7     <property name="title">Drag and Drop</property>
```

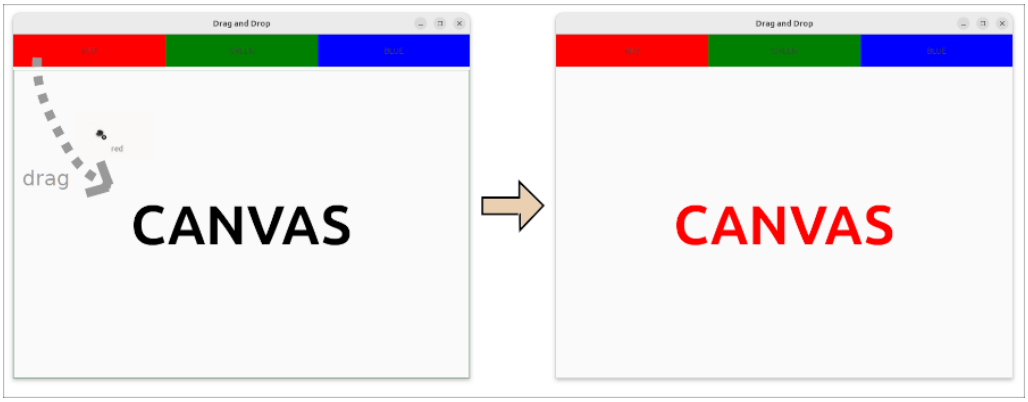


Figure 4.2: DND example

```
8       <child>
9         <object class="GtkBox">
10          <property name="hexpand">TRUE</property>
11          <property name="vexpand">TRUE</property>
12          <property name="orientation">GTK_ORIENTATION_VERTICAL</property>
13          <property name="spacing">5</property>
14          <child>
15            <object class="GtkBox">
16              <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
17              <property name="homogeneous">TRUE</property>
18              <child>
19                <object class="GtkLabel" id="red">
20                  <property name="label">RED</property>
21                  <property name="justify">GTK_JUSTIFY_CENTER</property>
22                  <property name="name">red</property>
23                </object>
24              </child>
25              <child>
26                <object class="GtkLabel" id="green">
27                  <property name="label">GREEN</property>
28                  <property name="justify">GTK_JUSTIFY_CENTER</property>
29                  <property name="name">green</property>
30                </object>
31              </child>
32              <child>
33                <object class="GtkLabel" id="blue">
34                  <property name="label">BLUE</property>
35                  <property name="justify">GTK_JUSTIFY_CENTER</property>
36                  <property name="name">blue</property>
37                </object>
38              </child>
39            </object>
40          </child>
41          <child>
42            <object class="GtkLabel" id="canvas">
43              <property name="label">CANVAS</property>
44              <property name="justify">GTK_JUSTIFY_CENTER</property>
45              <property name="name">canvas</property>
```

```
            <property name="hexpand">TRUE</property>
            <property name="vexpand">TRUE</property>
          </object>
        </child>
      </object>
    </child>
  </object>
</interface>
```

It is converted to a resource file by `glib-compile-resources`. The compiler uses an XML file `dnd.gresource.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>
<gresources>
  <gresource prefix="/com/github/ToshioCP/dnd">
    <file>dnd.ui</file>
  </gresource>
</gresources>
```

4.1.4 C File dnd.c

The C file `dnd.c` isn't a big file. The number of the lines is less than a hundred. A GtkApplication object is created in the function `main`.

```
int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
  g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
}
```

The application ID is defined as:

```
#define APPLICATION_ID "com.github.ToshioCP.dnd"
```

4.1.4.1 Startup Signal Handler

Most of the work is done in the "startup" signal handler.

Two objects GtkDragSource and GtkDropTarget are used for DND implementation.

- Drag source: A drag source (GtkDragSource instance) is an event controller. It initiates a DND operation when the user clicks and drags the widget. If data, in the form of GdkContentProvider, is set in advance, it gives the data to the system at the beginning of the drag.
- Drop target: A drop target (GtkDropTarget) is also an event controller. You can get the data in the GtkDropTarget::drop signal handler.

The example below uses these objects in a very simple way. You can use a number of features that the two objects have. See the following links for more information.

- Drag-and-Drop in GTK
- GtkDragSource
- GtkDropTarget

```
1  static void
2  app_startup (GApplication *application) {
3    GtkApplication *app = GTK_APPLICATION (application);
4    GtkBuilder *build;
5    GtkWindow *win;
6    GtkLabel *src_labels[3];
7    int i;
8    GtkLabel *canvas;
9    GtkDragSource *src;
10   GdkContentProvider *content;
11   GtkDropTarget *tgt;
12   GdkDisplay *display;
13   char *s;
14
15   build = gtk_builder_new_from_resource ("/com/github/ToshioCP/dnd/dnd.ui");
16   win = GTK_WINDOW (gtk_builder_get_object (build, "win"));
17   src_labels[0] = GTK_LABEL (gtk_builder_get_object (build, "red"));
18   src_labels[1] = GTK_LABEL (gtk_builder_get_object (build, "green"));
19   src_labels[2] = GTK_LABEL (gtk_builder_get_object (build, "blue"));
20   canvas = GTK_LABEL (gtk_builder_get_object (build, "canvas"));
21   gtk_window_set_application (win, app);
22   g_object_unref (build);
23
24   for (i=0; i<3; ++i) {
25     src = gtk_drag_source_new ();
26     content = gdk_content_provider_new_typed (G_TYPE_STRING, gtk_widget_get_name (GTK_WIDGET (src_lab
27     gtk_drag_source_set_content (src, content);
28     g_object_unref (content);
29     gtk_widget_add_controller (GTK_WIDGET (src_labels[i]), GTK_EVENT_CONTROLLER (src)); // The owner
30   }
31
32   tgt = gtk_drop_target_new (G_TYPE_STRING, GDK_ACTION_COPY);
33   g_signal_connect (tgt, "drop", G_CALLBACK (drop_cb), NULL);
34   gtk_widget_add_controller (GTK_WIDGET (canvas), GTK_EVENT_CONTROLLER (tgt)); // The ownership of t
35
36   provider = gtk_css_provider_new ();
37   s = g_strdup_printf (format, "black");
38   gtk_css_provider_load_from_string (provider, s);
39   g_free (s);
40   display = gdk_display_get_default ();
41   gtk_style_context_add_provider_for_display (display, GTK_STYLE_PROVIDER (provider),
42                                               GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);
43   g_object_unref (provider); // The provider is still alive because the display owns it.
44 }
```

- 15-22: Builds the widgets. The array `src_labels[]` points to the source labels red, green and blue in the UI file. The variable `canvas` points to the destination label.
- 24-30: Sets the DND source widgets. The for-loop iterates through the array `src_labels[]` each of which points to the source widget, red, green or blue label.
- 25: Creates a new GtkDragSource instance.
- 26: Creates a new GdkContentProvider instance with the string “red”, “green” or “blue”. They are the name of the widgets. These strings are the data to transfer through the DND operation.
- 27: Sets the content of the drag source to the GdkContentProvider instance above.
- 28: Content is useless so it is destroyed.

- 29: Add the event controller, which is actually the drag source, to the widget. If a DND operation starts on the widget, the corresponding drag source works and the data is given to the system.
- 32-34: Sets the DND drop target.
- 32: Creates a new GtkDropTarget instance. The first parameter is the GType of the data. The second parameter is a GdkDragAction enumeration constant. The arguments here are string type and the constant for copy.
- 33: Connects the "drop" signal and the handler `drop_cb`.
- 34: Add the event controller, which is actually the drop target, to the widget.
- 36-43: Sets CSS.
- 37: A variable `format` is static and defined at the top of the program. Static variables are shown below.

```
static GtkCssProvider *provider = NULL;
static const char *format = "label {padding: 20px;} label#red {background: red;} "
  "label#green {background: green;} label#blue {background: blue;} "
  "label#canvas {color: %s; font-weight: bold; font-size: 72pt;}";
```

4.1.4.2 Activate Signal Handler

```
1 static void
2 app_activate (GApplication *application) {
3   GtkApplication *app = GTK_APPLICATION (application);
4   GtkWindow *win;
5
6   win = gtk_application_get_active_window (app);
7   gtk_window_present (win);
8 }
```

This handler just shows the window.

4.1.4.3 Drop Signal Handler

```
1 static gboolean
2 drop_cb (GtkDropTarget *self, const GValue *value, gdouble x, gdouble y, gpointer user_data) {
3   char *s;
4
5   s = g_strdup_printf (format, g_value_get_string (value));
6   gtk_css_provider_load_from_string (provider, s);
7   g_free (s);
8   return TRUE;
9 }
```

The "drop" signal handler has five parameters.

- GtkDropTarget instance on which the signal has been emitted.
- GValue that holds the data from the source.
- The arguments `x` and `y` are the coordinate of the mouse when released.
- User data was set when the signal and the handler were connected.

The string from the GValue is "red", "green" or "blue". It replaces "%s" in the variable `format`. That means the font color of the label `canvas` will be updated to the color.

4.1.5 Meson.build

The file `meson.build` controls the building process.

```
1 project('dnd', 'c')
2
```

```
3 gtkdep = dependency('gtk4')
4
5 gnome = import('gnome')
6 resources = gnome.compile_resources('resources','dnd.gresource.xml')
7
8 executable(meson.project_name(), 'dnd.c', resources, dependencies: gtkdep, export_dynamic: true, ins
```

You can build it from the command line.

```
$ cd src/dnd
$ meson setup _build
$ ninja -C _build
$ _build/dnd
```

The source files are under the directory `src/dnd` of the repository. Download it and see the directory.

Chapter 5

List Widgets

5.1 GtkListView

GTK 4 has added new list objects GtkListView, GtkGridView and GtkColumnView. The new feature is described in Gtk API Reference – List Widget Overview.

GTK 4 has other means to implement lists. They are GtkListBox and GtkTreeView which are took over from GTK 3. There's an article in Gtk Development blog about list widgets by Matthias Clasen. He described why GtkListView are developed to replace GtkTreeView. GtkTreeView is deprecated since version 4.10.

GtkListView, GtkGridView, GtkColumnView and related objects are described in Section 29 to 33.

5.1.1 Outline

A list is a sequential data structure. For example, an ordered string sequence "one", "two", "three", and "four" is a list. Each element is called an item. A list is like an array, but in many cases it is implemented with pointers which point to the next items of the list. And it has a start point. So, each item can be referred to by the index of the item (first item, second item, ..., nth item, ...).

Gio provides GListModel interface. It is a zero-based list and its items are the same type of GObject descendants, or objects that implement the same interface. An object that implements GListModel is not a widget. So, the list is not displayed on the screen directly. There's another object GtkListView which is a widget to display the list. The items in the list need to be connected to the items in GtkListView. GtkListItemFactory instance maps items in the list to GtkListView.

5.1.2 GListModel and GtkStringList

If you want to make a list of strings with GListModel, for example, "one", "two", "three", "four", note that strings can't be items of the list. Because GListModel is a list of GObject objects and strings aren't GObject objects. The word "GObject" here means GObject class or its derived class. So, you need a wrapper which is a GObject and contains a string. GtkStringObject is the wrapper object and GtkStringList, which implements GListModel, is a list of GtkStringObject.

```
char *array[] = {"one", "two", "three", "four", NULL};
GtkStringList *stringlist = gtk_string_list_new ((const char * const *) array);
```

The function `gtk_string_list_new` creates a GtkStringList object. Its items are GtkStringObject objects which contain the strings "one", "two", "three" and "four". There are functions to add items to the list or remove items from the list.

- `gtk_string_list_append` appends an item to the list

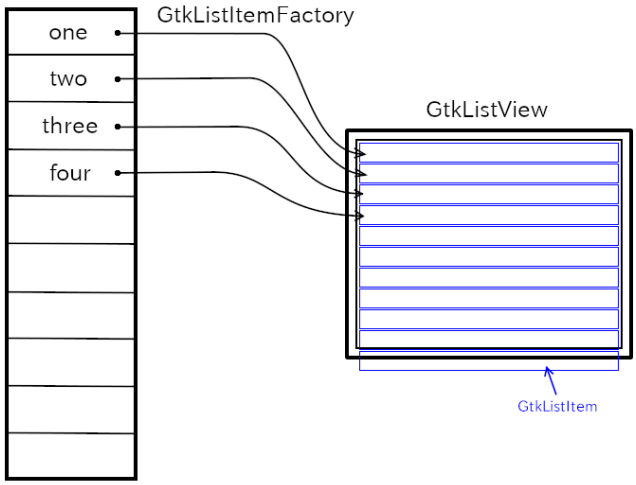


Figure 5.1: List

- `gtk_string_list_remove` removes an item from the list
- `gtk_string_list_get_string` gets a string in the list

See GTK 4 API Reference – GtkStringList for further information.

Other list objects will be explained later.

5.1.3 GtkSelectionModel

GtkSelectionModel is an interface to support for selection. Thanks to this model, a user can select items by clicking on them. It is implemented by GtkMultiSelection, GtkNoSelection and GtkSingleSelection objects. These three objects are usually enough to build an application. They are created with another GListModel. You can also create them alone and add a GListModel later.

- GtkMultiSelection supports multiple selection.
- GtkNoSelection supports no selection. This is a wrapper to GListModel when GtkSelectionModel is needed.
- GtkSingleSelection supports single selection.

5.1.4 GtkListView

GtkListView is a widget to show GListModel items. GtkListItem is used by GtkListView to represent items of a list model. But, GtkListItem is not a widget (not a descendant of GtkWidget), so a user needs to set a widget, for example GtkLabel, as a child of GtkListItem to display an item of the list model. “item” property of GtkListItem points to an object that belongs to the list model.

In case the number of items is very big, for example more than a thousand, GtkListItem is recycled and connected to another item which is newly displayed. This recycle makes the number of GtkListItem objects fairly small, less than 200. This is very effective to restrain the growth of memory consumption so that GListModel can contain lots of items, for example, more than a million items.

5.1.5 GtkListItemFactory

GtkListItemFactory creates or recycles GtkListItem and connects it with an item of the list model. There are two child classes of this factory, GtkSignalListItemFactory and GtkBuilderListItemFactory.

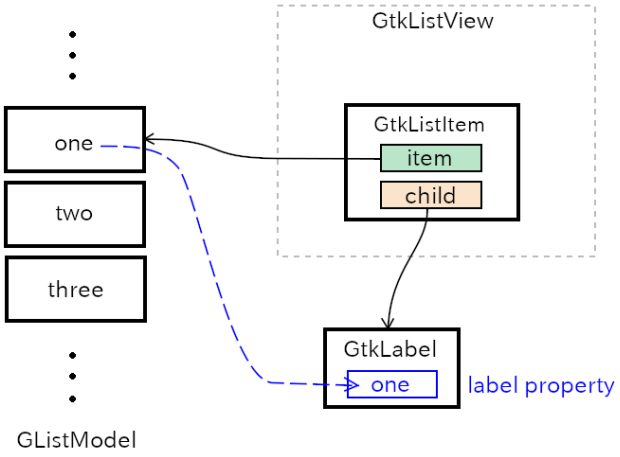


Figure 5.2: GtkListItem

5.1.5.1 GtkSignalListItemFactory

GtkSignalListItemFactory provides signals for users to configure a GtkListItem object. There are four signals.

1. "setup" is emitted to set up GtkListItem object. A user sets its child widget in the handler. For example, creates a GtkLabel widget and sets the child property of the GtkListItem to it. This setting is kept even when the GtkListItem instance is recycled (to bind to another item of GListModel).
2. "bind" is emitted to bind an item in the list model to the widget. For example, a user gets the item from "item" property of the GtkListItem instance. Then gets the string of the item and sets the label property of the GtkLabel instance with the string. This signal is emitted when the GtkListItem is newly created, recycled or some changes have happened to the item of the list.
3. "unbind" is emitted to unbind an item. A user undoes everything done in step 2 in the signal handler. If some object are created in step 2, they must be destroyed.
4. "teardown" is emitted to undo everything done in step 1. So, the widget created in step 1 must be destroyed. After this signal, the list item will be destroyed.

The following program `list1.c` shows a list of strings "one", "two", "three" and "four". GtkNoSelection is used, so user can't select any item.

```
#include <gtk/gtk.h>

static void
setup_cb (GtkSignalListItemFactory *self, GtkListItem *listitem, gpointer user_data) {
  GtkWidget *lb = gtk_label_new (NULL);
  gtk_list_item_set_child (listitem, lb);
  /* Because gtk_list_item_set_child sunk the floating reference of lb, releasing (unref) isn't nece
}

static void
bind_cb (GtkSignalListItemFactory *self, GtkListItem *listitem, gpointer user_data) {
  GtkWidget *lb = gtk_list_item_get_child (listitem);
  /* Strobj is owned by the instance. Caller mustn't change or destroy it. */
  GtkStringObject *strobj = gtk_list_item_get_item (listitem);
  /* The string returned by gtk_string_object_get_string is owned by the instance. */
  gtk_label_set_text (GTK_LABEL (lb), gtk_string_object_get_string (strobj));
```

```
17  }
18
19  static void
20  unbind_cb (GtkSignalListItemFactory *self, GtkListItem *listitem, gpointer user_data) {
21    /* There's nothing to do here. */
22  }
23
24  static void
25  teardown_cb (GtkSignalListItemFactory *self, GtkListItem *listitem, gpointer user_data) {
26    /* There's nothing to do here. */
27    /* GtkListItem instance will be destroyed soon. You don't need to set the child to NULL. */
28  }
29
30  static void
31  app_activate (GApplication *application) {
32    GtkApplication *app = GTK_APPLICATION (application);
33    GtkWidget *win = gtk_application_window_new (app);
34    gtk_window_set_default_size (GTK_WINDOW (win), 600, 400);
35    GtkWidget *scr = gtk_scrolled_window_new ();
36    gtk_window_set_child (GTK_WINDOW (win), scr);
37
38    char *array[] = {
39      "one", "two", "three", "four", NULL
40    };
41    /* sl is owned by ns */
42    /* ns and factory are owned by lv. */
43    /* Therefore, you don't need to care about their destruction. */
44    GtkStringList *sl =  gtk_string_list_new ((const char * const *) array);
45    GtkNoSelection *ns =  gtk_no_selection_new (G_LIST_MODEL (sl));
46
47    GtkListItemFactory *factory = gtk_signal_list_item_factory_new ();
48    g_signal_connect (factory, "setup", G_CALLBACK (setup_cb), NULL);
49    g_signal_connect (factory, "bind", G_CALLBACK (bind_cb), NULL);
50    /* The following two lines can be left out. The handlers do nothing. */
51    g_signal_connect (factory, "unbind", G_CALLBACK (unbind_cb), NULL);
52    g_signal_connect (factory, "teardown", G_CALLBACK (teardown_cb), NULL);
53
54    GtkWidget *lv = gtk_list_view_new (GTK_SELECTION_MODEL (ns), factory);
55    gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), lv);
56    gtk_window_present (GTK_WINDOW (win));
57  }
58
59  /* ----- main ----- */
60  #define APPLICATION_ID "com.github.ToshioCP.list1"
61
62  int
63  main (int argc, char **argv) {
64    GtkApplication *app;
65    int stat;
66
67    app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
68
69    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
```

```
70
71    stat = g_application_run (G_APPLICATION (app), argc, argv);
72    g_object_unref (app);
73    return stat;
74  }
```

The file `list1.c` is located under the directory /src/misc. Make a shell script below and save it to your bin directory, for example `$HOME/bin`.

```
gcc `pkg-config --cflags gtk4` $1.c `pkg-config --libs gtk4`
```

Change the current directory to the directory includes `list1.c` and type as follows.

```
$ bash comp list1
$ ./a.out
```

Then, the following window appears.



Figure 5.3: list1

The program is not so difficult. If you feel some difficulty, read this section again, especially GtkSignalListItemFactory subsubsection.

5.1.5.2 GtkBuilderListItemFactory

GtkBuilderListItemFactory is another GtkListItemFactory. Its behavior is defined in a UI file.

```
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkLabel">
        <binding name="label">
          <lookup name="string" type="GtkStringObject">
            <lookup name="item">GtkListItem</lookup>
          </lookup>
        </binding>
      </object>
    </property>
  </template>
</interface>
```

Template tag is used to define GtkListItem. And its child property is GtkLabel object. The factory sees this template and creates GtkLabel and sets the child property of GtkListItem. This is the same as what setup handler of GtkSignalListItemFactory did.

Then, bind the label property of the GtkLabel to the string property of a GtkStringObject. The string object refers to the item property of the GtkListItem. So, the lookup tag is like this:

`label <- string <- GtkStringObject <- item <- GtkListItem`

The last lookup tag has a content `GtkListItem`. Usually, C type like `GtkListItem` doesn't appear in the content of tags. This is a special case. There is an explanation in the GTK Development Blog by Matthias Clasen.

> Remember that the classname (GtkListItem) in a ui template is used as the "this" pointer referring to the object that is being instantiated.

Therefore, GtkListItem instance is used as the `this` object of the lookup tag when it is evaluated. `this` object will be explained in Section 34.

The C source code is as follows. Its name is `list2.c` and located under /src/misc directory.

```
#include <gtk/gtk.h>

static void
app_activate (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  gtk_window_present (gtk_application_get_active_window(app));
}

static void
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GtkWidget *win = gtk_application_window_new (app);
  gtk_window_set_default_size (GTK_WINDOW (win), 600, 400);
  GtkWidget *scr = gtk_scrolled_window_new ();
  gtk_window_set_child (GTK_WINDOW (win), scr);

  char *array[] = {
    "one", "two", "three", "four", NULL
  };
  GtkStringList *sl = gtk_string_list_new ((const char * const *) array);
  GtkSingleSelection *ss = gtk_single_selection_new (G_LIST_MODEL (sl));

  const char *ui_string =
"<interface>"
  "<template class=\"GtkListItem\">"
    "<property name=\"child\">"
      "<object class=\"GtkLabel\">"
        "<binding name=\"label\">"
          "<lookup name=\"string\" type=\"GtkStringObject\">"
            "<lookup name=\"item\">GtkListItem</lookup>"
          "</lookup>"
        "</binding>"
      "</object>"
    "</property>"
  "</template>"
"</interface>"
;
  GBytes *gbytes = g_bytes_new_static (ui_string, strlen (ui_string));
  GtkListItemFactory *factory = gtk_builder_list_item_factory_new_from_bytes (NULL, gbytes);

  GtkWidget *lv = gtk_list_view_new (GTK_SELECTION_MODEL (ss), factory);
  gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), lv);
```

```
}

/* ----- main ----- */
#define APPLICATION_ID "com.github.ToshioCP.list2"

int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);

  g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);

  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
}
```

No signal handler is needed for GtkBuilderListItemFactory. GtkSingleSelection is used, so user can select one item at a time.

Because this is a small program, the UI data is given as a string.

5.1.6 GtkDirectoryList

GtkDirectoryList is a list model containing GFileInfo objects which are information of files under a certain directory. It uses `g_file_enumerate_children_async()` to get the GFileInfo objects. The list model is created by `gtk_directory_list_new` function.

```
GtkDirectoryList *gtk_directory_list_new (const char *attributes, GFile *file);
```

`attributes` is a comma separated list of file attributes. File attributes are key-value pairs. A key consists of a namespace and a name. For example, "standard::name" key is the name of a file. "standard" means general file information. "name" means filename. The following table shows some example.

key	meaning
standard::type	file type. for example, regular file, directory, symbolic link, etc.
standard::name	filename
standard::size	file size in bytes
access::can-read	read privilege if the user is able to read the file
time::modified	the time the file was last modified in seconds since the UNIX epoch

The current directory is ".". The following program makes GtkDirectoryList `dl` and its contents are GFileInfo objects under the current directory.

```
GFile *file = g_file_new_for_path (".");
GtkDirectoryList *dl = gtk_directory_list_new ("standard::name", file);
g_object_unref (file);
```

It is not so difficult to make a file listing program by changing `list2.c` in the previous subsection. One problem is that GFileInfo doesn't have properties. Lookup tag look for a property, so it is useless for looking for a filename from a GFileInfo object. Instead, a closure tag is appropriate in this case. A closure tag specifies a function and the type of the return value of the function.

```
char *
get_file_name (GtkListItem *item, GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_strdup (g_file_info_get_name (info)) : NULL;
}
... ...
... ...

"<interface>"
  "<template class=\"GtkListItem\">"
    "<property name=\"child\">"
      "<object class=\"GtkLabel\">"
        "<binding name=\"label\">"
          "<closure type=\"gchararray\" function=\"get_file_name\">"
            "<lookup name=\"item\">GtkListItem</lookup>"
          "</closure>"
        "</binding>"
      "</object>"
    "</property>"
  "</template>"
"</interface>"
```

- The string “gchararray” is a type name registered in the GObject type system. While the type “gchar” is the same as the C “char” type, “gchararray” specifically represents a NULL-terminated string (char *). In GtkBuilder XML files, you must use these registered type names instead of native C types to let the GValue system handle the data correctly. For example, common type names include gboolean, gchar, gint, gfloat, gdouble, and gchararray. See GObject tutorial for further information.
- Closure tag has type attribute and function attribute. Function attribute specifies the function name and type attribute specifies the type of the return value of the function. The content of closure tag (it is between <closure…> and</closure>) consists of the parameters for the function. `<lookup name="item">GtkListItem</lookup>` gives the value of the item property of the GtkListItem. This will be the second argument of the function. The first parameter is always the GListItem instance, which is a ‘this’ object. The ‘this’ object is explained in Section 31.
- `get_file_name` function is the callback function for the closure tag. It first checks the `info` parameter. Because it can be NULL when GListItem `item` is unbounded. If it’s GFileInfo, it returns the copied filename. Because the return value (filename) of `g_file_info_get_name` is owned by GFileInfo object. So, the the string needs to be duplicated to give the ownership to the caller. Binding tag binds the “label” property of the GtkLabel to the closure tag.

The whole program (`list3.c`) is as follows. The program is located in /src/misc directory.

```
1  #include <gtk/gtk.h>
2
3  char *
4  get_file_name (GtkListItem *item, GFileInfo *info) {
5    return G_IS_FILE_INFO (info) ? g_strdup (g_file_info_get_name (info)) : NULL;
6  }
7
8  static void
9  app_activate (GApplication *application) {
10   GtkApplication *app = GTK_APPLICATION (application);
11   gtk_window_present (gtk_application_get_active_window(app));
12 }
13
14 static void
```

```
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GtkWidget *win = gtk_application_window_new (app);
  gtk_window_set_default_size (GTK_WINDOW (win), 600, 400);
  GtkWidget *scr = gtk_scrolled_window_new ();
  gtk_window_set_child (GTK_WINDOW (win), scr);

  GFile *file = g_file_new_for_path (".");
  GtkDirectoryList *dl = gtk_directory_list_new ("standard::name", file);
  g_object_unref (file);
  GtkNoSelection *ns = gtk_no_selection_new (G_LIST_MODEL (dl));

  const char *ui_string =
"<interface>"
  "<template class=\"GtkListItem\">"
    "<property name=\"child\">"
      "<object class=\"GtkLabel\">"
        "<binding name=\"label\">"
          "<closure type=\"gchararray\" function=\"get_file_name\">"
            "<lookup name=\"item\">GtkListItem</lookup>"
          "</closure>"
        "</binding>"
      "</object>"
    "</property>"
  "</template>"
"</interface>"
;
  GBytes *gbytes = g_bytes_new_static (ui_string, strlen (ui_string));
  GtkListItemFactory *factory = gtk_builder_list_item_factory_new_from_bytes (NULL, gbytes);

  GtkWidget *lv = gtk_list_view_new (GTK_SELECTION_MODEL (ns), factory);
  gtk_scrolled_window_set_child (GTK_SCROLLED_WINDOW (scr), lv);
}

/* ----- main ----- */
#define APPLICATION_ID "com.github.ToshioCP.list3"

int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);

  g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);

  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
}
```

The ui data (xml data above) is used to build the GListItem template at runtime. GtkBuilder refers to the symbol table to find the function `get_file_name`.

Generally, a symbol table is used by a linker to link objects to an executable file. It includes function names and their location. A linker usually doesn't put a symbol table into the created executable file. But if `--export-dynamic` option is given, the linker adds the symbol table to the executable file.

To accomplish it, an option `-Wl,--export-dynamic` is given to the C compiler.

- `-Wl` is a C compiler option that passes the following option to the linker.
- `--export-dynamic` is a linker option. The following is cited from the linker document. "When creating a dynamically linked executable, add all symbols to the dynamic symbol table. The dynamic symbol table is the set of symbols which are visible from dynamic objects at run time."

Compile and execute it.

```
$ gcc -Wl,--export-dynamic `pkg-config --cflags gtk4` list3.c `pkg-config --libs gtk4`
```

You can also make a shell script to compile `list3.c`

```
gcc -Wl,--export-dynamic `pkg-config --cflags gtk4` $1.c `pkg-config --libs gtk4`
```

Save this one liner to a file `comp`. Then, copy it to `$HOME/bin` and give it executable permission.

```
$ cp comp $HOME/bin/comp
$ chmod +x $HOME/bin/comp
```

You can compile `list3.c` and execute it, like this:

```
$ comp list3
$ ./a.out
```

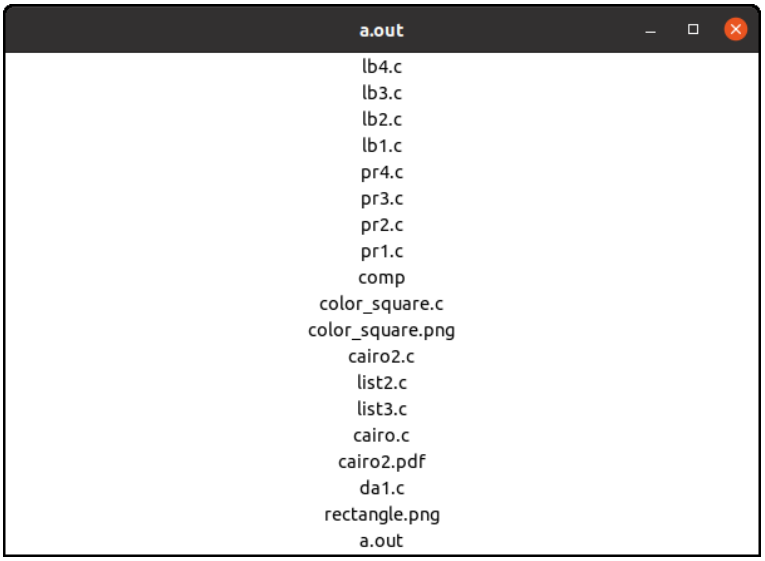


Figure 5.4: screenshot list3

5.2 GtkGridView and Activate Signal

GtkGridView is similar to GtkListView. It displays a GListModel as a grid, which is like a square tessellation.

This is often seen when you use a file browser like GNOME Files (Nautilus).

In this section, let's make a very simple file browser `list4`. It just shows the files in the current directory. And a user can choose either list or grid layout by clicking on buttons in the tool bar. Each item in the list or grid has an icon and a filename. In addition, `list4` provides the way to open the `tfe` text editor to show a text file. A user can do that by double clicking on an item or pressing enter key when an item is selected.

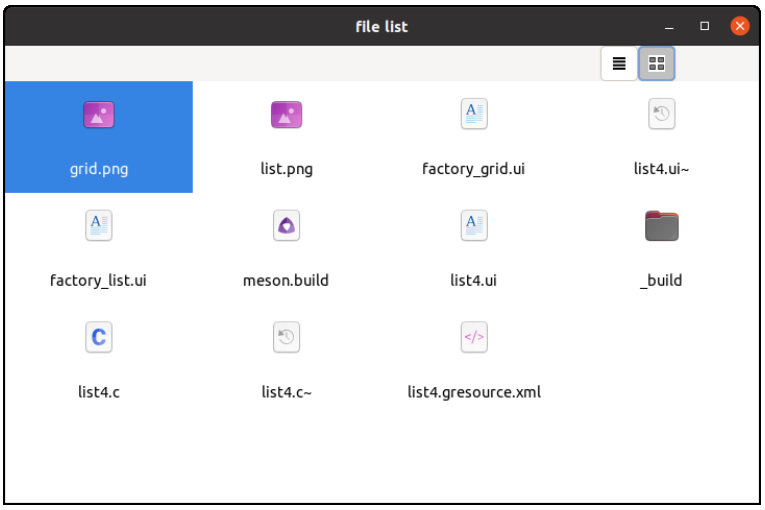


Figure 5.5: Grid

5.2.1 GtkDirectoryList

GtkDirectoryList implements GListModel and it contains information of files in a certain directory. The items of the list are GFileInfo objects.

In the `list4` source files, GtkDirectoryList is described in a UI file and built by GtkBuilder. The GtkDirectoryList instance is assigned to the "model" property of a GtkSingleSelection instance. And the GtkSingleSelection instance is assigned to the "model" property of a GListView or GGridView instance.

```
GtkListView (model property) => GtkSingleSelection (model property) => GtkDirectoryList
GtkGridView (model property) => GtkSingleSelection (model property) => GtkDirectoryList
```

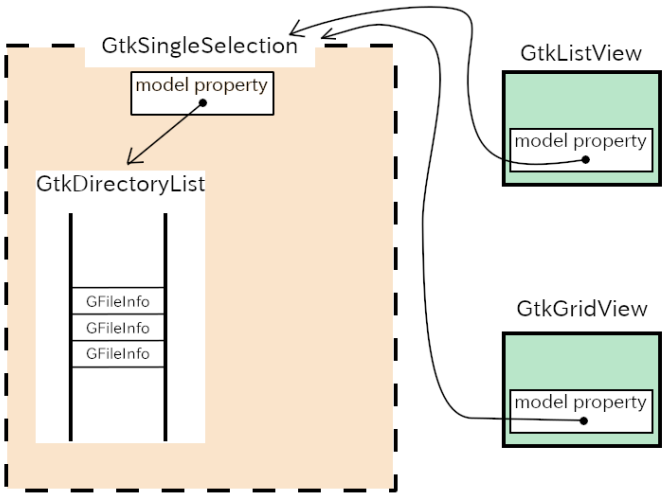


Figure 5.6: DirectoryList

The following is a part of the UI file `list4.ui`.

```
<object class="GtkListView" id="list">
  <property name="model">
    <object class="GtkSingleSelection" id="singleselection">
      <property name="model">
        <object class="GtkDirectoryList" id="directory_list">
```

```
          <property name="attributes">standard::name,standard::icon,standard::content-type</property>
        </object>
      </property>
    </object>
  </property>
</object>
<object class="GtkGridView" id="grid">
  <property name="model">singleselection</property>
</object>
```

GtkDirectoryList has an “attributes” property. They are attributes of GFileInfo such as “standard::name”, “standard::icon” and “standard::content-type”.

- standard::name is a filename.
- standard::icon is an icon of the file. It is a GIcon object.
- standard::content-type is a content-type. Content-type is the same as MIME type. For example, “text/plain” is a text file, “text/x-csrc” is a C source code and so on. (“text/x-csrc”is not registered to IANA media types. Such “x-” subtype is not a standard mime type.) Content type is also used by desktop systems.

GtkGridView uses the same GtkSingleSelection instance (`singleselection`). So, its model property is set to it.

5.2.2 Ui File of the Window

The window is built with the following UI file. (See the screenshot at the beginning of this section).

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="GtkApplicationWindow" id="win">
    <property name="title">file list</property>
    <property name="default-width">600</property>
    <property name="default-height">400</property>
    <child>
      <object class="GtkBox" id="boxv">
        <property name="orientation">GTK_ORIENTATION_VERTICAL</property>
        <child>
          <object class="GtkBox" id="boxh">
            <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
            <child>
              <object class="GtkLabel" id="dmy1">
                <property name="hexpand">TRUE</property>
              </object>
            </child>
            <child>
              <object class="GtkButton" id="btnlist">
                <property name="name">btnlist</property>
                <property name="action-name">win.view</property>
                <property name="action-target">&apos;list&apos;</property>
                <child>
                  <object class="GtkImage">
                    <property name="resource">/com/github/ToshioCP/list4/list.png</property>
                  </object>
                </child>
              </object>
            </child>
```

```
30            <child>
31              <object class="GtkButton" id="btngrid">
32                <property name="name">btngrid</property>
33                <property name="action-name">win.view</property>
34                <property name="action-target">&apos;grid&apos;</property>
35                <child>
36                  <object class="GtkImage">
37                    <property name="resource">/com/github/ToshioCP/list4/grid.png</property>
38                  </object>
39                </child>
40              </object>
41            </child>
42            <child>
43              <object class="GtkLabel" id="dmy2">
44                <property name="width-chars">10</property>
45              </object>
46            </child>
47          </object>
48        </child>
49        <child>
50          <object class="GtkScrolledWindow" id="scr">
51            <property name="hexpand">TRUE</property>
52            <property name="vexpand">TRUE</property>
53          </object>
54        </child>
55      </object>
56    </child>
57  </object>
58  <object class="GtkListView" id="list">
59    <property name="model">
60      <object class="GtkSingleSelection" id="singleselection">
61        <property name="model">
62          <object class="GtkDirectoryList" id="directory_list">
63            <property name="attributes">standard::name,standard::icon,standard::content-type</proper
64          </object>
65        </property>
66      </object>
67    </property>
68  </object>
69  <object class="GtkGridView" id="grid">
70    <property name="model">singleselection</property>
71  </object>
72</interface>
```

The file consists of two parts. The first part begins at the line 3 and ends at line 57. This part is the widgets from the top level window to the scrolled window. It also includes two buttons. The second part begins at line 58 and ends at line 71. This is the part of GtkListView and GtkGridView.

- 13-17, 42-46: Two labels are dummy labels. They just work as a space to put the two buttons at the appropriate position.
- 18-41: GtkButton `btnlist` and `btngrid`. These two buttons work as selection buttons to switch from list to grid and vice versa. These two buttons are connected to a stateful action `win.view`. This action has a parameter. Such action consists of prefix, action name and parameter. The prefix of the action is `win`, which means the action belongs to the top level window (GtkApplicationWindow).

The prefix defines the scope of the action. The action name is `view`. The parameters are `list` or `grid`, which show the state of the action. A parameter is also called a target, because it is a target to which the action changes its state. We often write the detailed action like "win.view::list" or "win.view::grid".

- 21-22: The properties "action-name" and "action-target" belong to GtkActionable interface. The GtkButton class implements the GtkActionable interface. The action name is "win.view" and the target is "list". Generally, a target is GVariant, which can be string, integer, float and so on. You need to use GVariant text format to write GVariant value in UI files. If the type of the GVariant value is string, then the value with GVariant text format is enclosed in single quotes or double quotes. Because UI files are xml format text, single quote cannot be written without escaping it. Its escape sequence is '. Therefore, the target 'list' is written as 'list'. Because the button is connected to the action, "clicked" signal handler isn't needed.
- 23-27: The child widget of the button is GtkImage. GtkImage has a "resource" property. It is a GResource and GtkImage reads an image data from the resource and sets the image. This resource is built from 24x24-sized png image data, which is an original icon.
- 50-53: GtkScrolledWindow. Its child widget will be set with GtkListView or GtkGridView.

The action `view` is created, connected to the "activate" signal handler and inserted to the window (action map) as follows.

```
act_view = g_simple_action_new_stateful ("view", g_variant_type_new("s"), g_variant_new_string ("lis
g_signal_connect (act_view, "activate", G_CALLBACK (view_activated), NULL);
g_action_map_add_action (G_ACTION_MAP (win), G_ACTION (act_view));
```

The signal handler `view_activated` will be explained later.

5.2.3 Factories

Each view (GtkListView and GtkGridView) has its own factory because its items have different structure of widgets. The factories are GtkBuilderListItemFactory objects. Their UI files are as follows.

factory_list.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkBox">
        <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
        <property name="spacing">20</property>
        <child>
          <object class="GtkImage">
            <binding name="gicon">
              <closure type="GIcon" function="get_icon">
                <lookup name="item">GtkListItem</lookup>
              </closure>
            </binding>
          </object>
        </child>
        <child>
          <object class="GtkLabel">
            <property name="hexpand">TRUE</property>
            <property name="xalign">0</property>
            <binding name="label">
              <closure type="gchararray" function="get_file_name">
                <lookup name="item">GtkListItem</lookup>
```

```
              </closure>
            </binding>
          </object>
        </child>
      </object>
    </property>
  </template>
</interface>
```

factory_grid.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkBox">
        <property name="orientation">GTK_ORIENTATION_VERTICAL</property>
        <property name="spacing">20</property>
        <child>
          <object class="GtkImage">
            <property name="icon-size">GTK_ICON_SIZE_LARGE</property>
            <binding name="gicon">
              <closure type="GIcon" function="get_icon">
                <lookup name="item">GtkListItem</lookup>
              </closure>
            </binding>
          </object>
        </child>
        <child>
          <object class="GtkLabel">
            <property name="hexpand">TRUE</property>
            <property name="xalign">0.5</property>
            <binding name="label">
              <closure type="gchararray" function="get_file_name">
                <lookup name="item">GtkListItem</lookup>
              </closure>
            </binding>
          </object>
        </child>
      </object>
    </property>
  </template>
</interface>
```

The two files above are almost same. The difference is:

- The orientation of the box
- The icon size
- The position of the text of the label

```
$ cd list4; diff factory_list.ui factory_grid.ui
6c6
<         <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
---
>         <property name="orientation">GTK_ORIENTATION_VERTICAL</property>
9a10
```

```
>             <property name="icon-size">GTK_ICON_SIZE_LARGE</property>
20c21
<             <property name="xalign">0</property>
---
>             <property name="xalign">0.5</property>
$
```

Two properties “gicon” (property of GtkImage) and “label” (property of GtkLabel) are in the UI files above. Because GFileInfo doesn't have properties correspond to icon or filename, the factory uses closure tag to bind “gicon” and “label” properties to GFileInfo information. A function `get_icon` gets GIcon from the GFileInfo object. And a function `get_file_name` gets a filename from the GFileInfo object.

```
GIcon *
get_icon (GtkListItem *item, GFileInfo *info) {
  GIcon *icon;

   /* g_file_info_get_icon can return NULL */
  icon = G_IS_FILE_INFO (info) ? g_file_info_get_icon (info) : NULL;
  return icon ? g_object_ref (icon) : NULL;
}

char *
get_file_name (GtkListItem *item, GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_strdup (g_file_info_get_name (info)) : NULL;
}
```

One important thing is the ownership of the return values. The return value is owned by the caller. So, `g_object_ref` or `g_strdup` is necessary.

5.2.4 An Activate Signal Handler of the Button Action

An activate signal handler `view_activate` switches the view. It does two things.

- Changes the child widget of GtkScrolledWindow.
- Changes the CSS of buttons to show the current state.

```
static void
view_activated(GSimpleAction *action, GVariant *parameter) {
  const char *view = g_variant_get_string (parameter, NULL);
  const char *other;
  char *css;

  if (strcmp (view, "list") == 0) {
    other = "grid";
    gtk_scrolled_window_set_child (scr, GTK_WIDGET (list));
  }else {
    other = "list";
    gtk_scrolled_window_set_child (scr, GTK_WIDGET (grid));
  }
  css = g_strdup_printf ("button#btn%s {background: silver;} button#btn%s {background: white;}", vie
  gtk_css_provider_load_from_string (provider, css);
  g_free (css);
  g_action_change_state (G_ACTION (action), parameter);
}
```

The second parameter of this handler is the target value associated with the clicked button. Its type is GVariant.

- If `btnlist` has been clicked, then `parameter` is a GVariant of the string "list".
- If `btngrid` has been clicked, then `parameter` is a GVariant of the string "grid".

The third parameter `user_data` points to NULL and it is ignored here.

- 3: `g_variant_get_string` gets the string from the GVariant variable.
- 7-13: Sets the child of `scr`. The function `gtk_scrolled_window_set_child` decreases the reference count of the old child by one. And it increases the reference count of the new child by one.
- 14-16: Sets the CSS for the buttons. The background of the clicked button will be silver color and the other button will be white.
- 17: Changes the state of the action.

5.2.5 Activate Signal on GtkListView and GtkGridView

Views (GtkListView and GtkGridView) have an "activate" signal. It is emitted when an item in the view is double clicked or the enter key is pressed. You can do anything you like by connecting the "activate" signal to the handler.

The example `list4` launches `tfe` text file editor if the item of the list is a text file.

```
static void
list_activate (GtkListView *list, int position, gpointer user_data) {
  GFileInfo *info = G_FILE_INFO (g_list_model_get_item (G_LIST_MODEL (gtk_list_view_get_model (list)
  launch_tfe_with_file (info);
}

static void
grid_activate (GtkGridView *grid, int position, gpointer user_data) {
  GFileInfo *info = G_FILE_INFO (g_list_model_get_item (G_LIST_MODEL (gtk_grid_view_get_model (grid)
  launch_tfe_with_file (info);
}

... ...
... ...

  g_signal_connect (GTK_LIST_VIEW (list), "activate", G_CALLBACK (list_activate), NULL);
  g_signal_connect (GTK_GRID_VIEW (grid), "activate", G_CALLBACK (grid_activate), NULL);
```

The second parameter of each handler is the position of the item (GFileInfo) of the GListModel. So you can get the item with `g_list_model_get_item` function.

5.2.5.1 Content Type and Application Launch

The function `launch_tfe_with_file` gets a file from the GFileInfo instance. If the file is a text file, it launches `tfe` with the file.

GFileInfo has information about file type. The file type is like "text/plain", "text/x-csrc" and so on. This is referred to as the content type. Content type can be got with `g_file_info_get_content_type` function.

```
1 static void
2 launch_tfe_with_file (GFileInfo *info) {
3   GError *err = NULL;
4   GFile *file;
5   GList *files = NULL;
6   const char *content_type;
7   const char *text_type = "text/";
8   GAppInfo *appinfo;
```

```
9   int i;
10
11  if (! info)
12    return;
13  content_type = g_file_info_get_content_type (info);
14 #ifdef debug
15  g_print ("%s\n", content_type);
16 #endif
17  if (! content_type)
18    return;
19  for (i=0;i<5;++i) { /* compare the first 5 characters */
20    if (content_type[i] != text_type[i])
21      return;
22  }
23  appinfo = g_app_info_create_from_commandline ("tfe", "tfe", G_APP_INFO_CREATE_NONE, &err);
24  if (err) {
25    g_printerr ("%s\n", err->message);
26    g_error_free (err);
27    return;
28  }
29  err = NULL;
30  file = g_file_new_for_path (g_file_info_get_name (info));
31  files = g_list_append (files, file);
32  if (! (g_app_info_launch (appinfo, files, NULL, &err))) {
33    g_printerr ("%s\n", err->message);
34    g_error_free (err);
35  }
36  g_list_free_full (files, g_object_unref);
37  g_object_unref (appinfo);
38 }
```

- 13: Gets the content type of the file from GFileInfo.
- 14-16: Prints the content type if "debug" is defined. This is only useful to know a content type of a file. If you don't want this, delete or uncomment the definition `#define debug 1` at line 6 in the source file.
- 17-22: If there is no content type, or if it doesn't begin with "text/", the function returns.
- 23: Creates GAppInfo object of `tfe` application. GAppInfo is an interface and the variable `appinfo` points to a GDesktopAppInfo instance. GAppInfo is a collection of information of applications.
- 32: Launches the application (`tfe`) with an argument `file`. `g_app_info_launch` has four parameters. The first parameter is GAppInfo object. The second parameter is a list of GFile objects. In this function, only one GFile instance is given to `tfe`, but you can give more arguments. The third parameter is GAppLaunchContext, but this program gives NULL instead. The last parameter is the pointer to the pointer to a GError.
- 36: `g_list_free_full` frees the memories used by the list and items.

If your distribution supports GTK 4, using `g_app_info_launch_default_for_uri` is convenient. The function automatically determines the default application from the file and launches it. For example, if the file is text, then it launches GNOME text editor with the file. Such feature comes from desktop.

5.2.6 Compilation and Execution

The source files are located in /src/list4 directory. To compile and execute list4, type as follows.

```
$ cd list4 # or cd src/list4. It depends your current directory.
$ meson setup _build
```

```
$ ninja -C _build
$ _build/list4
```

Then a file list appears as a list style. Click on a button on the tool bar so that you can change the style to grid or back to list. Double click “list4.c” item, then `tfe` text editor runs with the argument “list4.c”. The following is the screenshot.

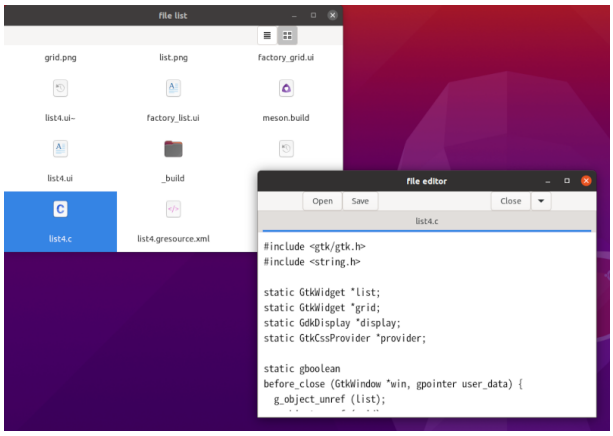


Figure 5.7: Screenshot

5.2.7 “gbytes” Property of GtkBuilderListItemFactory

GtkBuilderListItemFactory has “gbytes” property. The property contains a byte sequence of UI data. If you use this property, you can put the contents of `factory_list.ui` and `factory_grid.ui` into `list4.ui`. The following shows a part of the new UI file (`list5.ui`).

```
  <object class="GtkListView" id="list">
    <property name="model">
      <object class="GtkSingleSelection" id="singleselection">
        <property name="model">
          <object class="GtkDirectoryList" id="directory_list">
            <property name="attributes">standard::name,standard::icon,standard::content-type</proper
          </object>
        </property>
      </object>
    </property>
    <property name="factory">
      <object class="GtkBuilderListItemFactory">
        <property name="bytes"><![CDATA[
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkBox">
        <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
        <property name="spacing">20</property>
        <child>
          <object class="GtkImage">
            <binding name="gicon">
              <closure type="GIcon" function="get_icon">
                <lookup name="item">GtkListItem</lookup>
              </closure>
```

```
            </binding>
          </object>
        </child>
        <child>
          <object class="GtkLabel">
            <property name="hexpand">TRUE</property>
            <property name="xalign">0</property>
            <binding name="label">
              <closure type="gchararray" function="get_file_name">
                <lookup name="item">GtkListItem</lookup>
              </closure>
            </binding>
          </object>
        </child>
      </object>
    </property>
  </template>
</interface>
        ]]></property>
      </object>
    </property>
  </object>
```

CDATA section begins with "<![CDATA[" and ends with "]]>". The contents of CDATA section is recognized as a string. Any character, even if it is a key syntax marker such as '<' or '>', is recognized literally. Therefore, the text between "<![CDATA[" and "]]>" is inserted to "bytes" property as it is.

This approach reduces the number of UI files. But, the new UI file is a bit complicated especially for the beginners. If you feel some difficulty, it is better for you to separate the UI file.

A directory /src/list5 includes the UI file above.

5.3 GtkExpression

The GtkExpression is a fundamental type. It is not a descendant of GObject. GtkExpression provides a way to describe references to values. GtkExpression needs to be evaluated to obtain a value.

It is similar to arithmetic calculation.

`1 + 2 = 3`

`1+2` is an expression. It shows the way how to calculate. `3` is the value that comes from the expression. Evaluation is to calculate the expression and get the value.

GtkExpression is a way to get a value. Evaluation is like a calculation. A value is obtained by evaluating the expression.

5.3.1 Constant Expression

A constant expression (GtkConstantExpression) provides a constant value or an instance when it is evaluated.

```
GValue value = G_VALUE_INIT;
expression = gtk_constant_expression_new (G_TYPE_INT, 100);
gtk_expression_evaluate (expression, NULL, &value);
```

- GtkExpression uses GValue to hold a value. GValue is a structure and container to hold a type and value. It must be initialized with `G_VALUE_INIT`, first. Be careful that `value` is a structure, not a

pointer to a structure.

- Constant expression is created with `gtk_constant_expression_new` function. The parameter of the function is a type (GType) and a value (or instance). This expression holds a constant value. `G_TYPE_INT` is a type that is registered to the type system. It is integer type. Some types are shown in the following table.
- `gtk_expression_evaluate` evaluates the expression. It has three parameters, the expression to evaluate, `this` instance and a pointer to a GValue for being set with the value. `this` instance isn't necessary for constant expressions. Therefore, the second argument is NULL. `gtk_expression_evaluate` returns TRUE if it successfully evaluates the expression. Otherwise it returns FALSE.
- If it returns TRUE, the GValue `value` is set with the value of the expression. The type of the value is int.

GType	C type	type name	notes
G_TYPE_CHAR	char	gchar	
G_TYPE_BOOLEAN	int	gboolean	
G_TYPE_INT	int	gint	
G_TYPE_FLOAT	float	gfloat	
G_TYPE_DOUBLE	double	gdouble	
G_TYPE_POINTER	void *	gpointer	general pointer
G_TYPE_STRING	char *	gchararray	null-terminated Cstring
G_TYPE_OBJECT		GObject	
GTK_TYPE_WINDOW		GtkWindow	

A sample program `exp_constant_simple.c` is in `/src/expression` directory.

```
1  #include <gtk/gtk.h>
2
3  int
4  main (int argc, char **argv) {
5    GtkExpression *expression;
6    GValue value = G_VALUE_INIT;
7
8    /* Create an expression */
9    expression = gtk_constant_expression_new (G_TYPE_INT,100);
10   /* Evaluate the expression */
11   if (gtk_expression_evaluate (expression, NULL, &value))
12     g_print ("The value is %d.\n", g_value_get_int (&value));
13   else
14     g_print ("The constant expression wasn't evaluated correctly.\n");
15   gtk_expression_unref (expression);
16   g_value_unset (&value);
17
18   return 0;
19 }
```

- 9: A constant expression is created. It holds an int value 100. The variable `expression` points to the expression.
- 11-14: Evaluates the expression. If it successes, show the value to the stdout. Otherwise show an error message.
- 15-16: Releases the expression and unsets the GValue.

Constant expression is usually used to give a constant value or instance to another expression.

5.3.2 Property Expression

A property expression (GtkPropertyExpression) looks up a property in a GObject instance. For example, a property expression that refers "label" property in a GtkLabel object is created like this.

```
expression = gtk_property_expression_new (GTK_TYPE_LABEL, another_expression, "label");
```

The second parameter `another_expression` is one of:

- An expression that gives a GtkLabel instance when it is evaluated.
- NULL. When NULL is given, a GtkLabel instance will be given when it is evaluated. The instance is called `this` object.

For example,

```
label = gtk_label_new ("Hello");
another_expression = gtk_constant_expression_new (GTK_TYPE_LABEL, label);
expression = gtk_property_expression_new (GTK_TYPE_LABEL, another_expression, "label");
```

If `expression` is evaluated, its second parameter `another_expression` is evaluated first in the evaluation process. The value of `another_expression` is the `label` (GtkLabel instance). Then, `expression` looks up `label` property of the label and the evaluation results in "Hello".

In the example above, the second argument of `gtk_property_expression_new` is `another_expression`. But the second argument can be NULL. If it is NULL, `this` instance is used instead. `this` is given by `gtk_expression_evaluate` function.

There's a simple program `exp_property_simple.c` in `/src/expression` directory.

```
1  #include <gtk/gtk.h>
2
3  int
4  main (int argc, char **argv) {
5    GtkWidget *label;
6    GtkExpression *expression;
7    GValue value = G_VALUE_INIT;
8
9    gtk_init ();
10   label = gtk_label_new ("Hello world.");
11   /* Create an expression */
12   expression = gtk_property_expression_new (GTK_TYPE_LABEL, NULL, "label");
13   /* Evaluate the expression */
14   if (gtk_expression_evaluate (expression, label, &value))
15     g_print ("The value is %s.\n", g_value_get_string (&value));
16   else
17     g_print ("The property expression wasn't evaluated correctly.\n");
18   gtk_expression_unref (expression);
19   g_value_unset (&value);
20
21   return 0;
22 }
```

- 9-10: `gtk_init` initializes GTK GUI toolkit. It isn't usually necessary because the GtkApplication default startup handler does the initialization. A GtkLabel instance is created with the text "Hello world.".
- 12: A property expression is created. It looks up a "label" property of a GtkLabel instance. But at the creation, no instance is given because the second argument is NULL. The expression just knows how to take the property from a future-given GtkLabel instance.

- 14-17: The function `gtk_expression_evaluate` evaluates the expression with a 'this' instance `label`. The result is stored in the GValue `value`. The function `g_value_get_string` gets a string from the GValue. But the string is owned by the GValue so you must not free the string.
- 18-19: Releases the expression and unset the GValue. At the same time the string in the GValue is freed.

If the second argument of `gtk_property_expression_new` isn't NULL, it is another expression. The expression is owned by a newly created property expression. So, when the expressions are useless, you just release the last expression. Then it releases another expression it has.

5.3.3 Closure Expression

A closure expression calls a closure when it is evaluated. A closure is a generic representation of a callback (a pointer to a function). For information about closure, see GObject API Reference – The GObject messaging system. There are simple closure example files `closure.c` and `closure_each.c` in the `/src/expression` directory.

There are two types of closure expressions, GtkCClosureExpression and GtkClosureExpression. They corresponds to GCClosure and GClosure respectively. When you program in C language, GtkCClosureExpression and GCClosure are appropriate.

A closure expression is created with `gtk_cclosure_expression_new` function.

```
GtkExpression *
gtk_cclosure_expression_new (GType value_type,
                             GClosureMarshal marshal,
                             guint n_params,
                             GtkExpression **params,
                             GCallback callback_func,
                             gpointer user_data,
                             GClosureNotify user_destroy);
```

- `value_type` is the type of the value when it is evaluated.
- `marshal` is a marshaller. You can assign NULL. If it is NULL, then `g_cclosure_marshal_generic ()` is used as a marshaller. It is a generic marshaller function implemented via libffi.
- `n_params` is the number of parameters.
- `params` is an array of pointers to expressions. When the C closure expression is evaluated, each of these expressions is evaluated first, and their resulting values are passed to the callback function as arguments.
- `callback_func` is a callback function. It receives the `this` object followed by the evaluated params above. So, if `n_params` is 3, the number of arguments for the callback function is 4. (`this` and `params`. See below.)
- `user_data` is user data. You can add it for the closure. It is like `user_data` in `g_signal_connect`. If it is not necessary, assign NULL.
- `user_destroy` is a destroy notify for `user_data`. It is called to destroy `user_data` when it is no longer needed. If NULL is assigned to `user_data`, assign NULL to `user_destroy`, too.

Call back functions have the following format.

```
C-type
callback (this, param1, param2, ...)
```

For example,

```
int
callback (GObject *object, int x, const char *s)
```

The following is `exp_closure_simple.c` in `/src/expression`.

```c
#include <gtk/gtk.h>

static int
calc (GtkLabel *label) { /* this object */
  const char * s;
  int i, j;

  s = gtk_label_get_text (label); /* s is owned by the label. */
  sscanf (s, "%d+%d", &i, &j);
  return i+j;
}

int
main (int argc, char **argv) {
  GtkExpression *expression;
  GValue value = G_VALUE_INIT;
  GtkLabel *label;

  gtk_init ();
  label = GTK_LABEL (gtk_label_new ("123+456"));
  g_object_ref_sink (label);
  expression = gtk_cclosure_expression_new (G_TYPE_INT, NULL, 0, NULL,
                 G_CALLBACK (calc), NULL, NULL);
  if (gtk_expression_evaluate (expression, label, &value)) /* 'this' object is the label. */
    g_print ("%d\n", g_value_get_int (&value));
  else
    g_print ("The closure expression wasn't evaluated correctly.\n");
  gtk_expression_unref (expression);
  g_value_unset (&value);
  g_object_unref (label);

  return 0;
}
```

- 3-11: A call back function. The parameter is only one and it is a 'this' object. It is a GtkLabel and its label is assumed to be "(number)+(number)".
- 8-10: Retrieves two integers from the label and returns the sum of them. This function has no error report. If you want to return error report, change the return value type to be a pointer to a structure of gboolean and integer. One is for errors and the other is for the sum. The first argument of `gtk_cclosure_expression_new` is `G_TYPE_POINTER`. There is a sample program `exp_closure_with_error_report` in `/src/expression` directory.
- 19: The function 'gtk_init" initializes GTK. It is necessary for GtkLabel.
- 20: A GtkLabel instance is created with "123+456".
- 21: The instance has floating reference. It is changed to an ordinary reference count.
- 22-23: Creates a closure expression. The value type is `G_TYPE_INT` when it is evaluated. This expression has no parameters. The callback function `calc` recieves only `this` object (GtkLabel) as an argument.
- 24: Evaluates the expression with the label as a 'this' object.
- 25: If the evaluation succeeds, the sum of "123+456", which is 579, is shown.
- 27: If it fails, an error message appears.
- 28-30: Releases the expression and the label. Unsets the value.

A closure expression is more flexible than other type of expressions because you can specify your own callback function.

5.3.4 GtkExpressionWatch

GtkExpressionWatch is a structure, not an object. It represents a watched GtkExpression. Two functions create GtkExpressionWatch structure. They are `gtk_expression_bind` and `gtk_expression_watch`.

5.3.4.1 gtk_expression_bind Function

This function binds the target object's property to the expression. If the value of the expression changes, the property reflects the value immediately.

```
GtkExpressionWatch*
gtk_expression_bind (
  GtkExpression* self,
  GObject* target,
  const char* property,
  GObject* this_
)
```

This function takes the ownership of the expression. So, if you want to own the expression, call `gtk_expression_ref ()` to increase the reference count of the expression. And you should unref it when it becomes no longer needed. If you don't own the expression, you don't care about releasing the expression.

An example `exp_bind.c` and `exp_bind.ui` is in /src/expression directory.

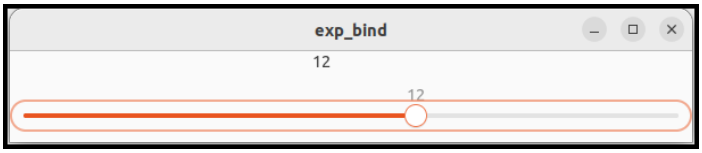


Figure 5.8: exp_bind

It includes a label and a scale. If you move the slider to the right, the scale value increases and the number on the label also increases. In the same way, if you move it to the left, the number on the label decreases. The label is bound to the scale value via an adjustment.

```
<?xml version='1.0' encoding='UTF-8'?>
<interface>
  <object class='GtkApplicationWindow' id='win'>
    <property name='default-width'>600</property>
    <child>
      <object class='GtkBox'>
        <property name='orientation'>GTK_ORIENTATION_VERTICAL</property>
        <child>
          <object class='GtkLabel' id='label'>
            <property name="label">10</property>
          </object>
        </child>
        <child>
          <object class='GtkScale'>
            <property name='adjustment'>
              <object class='GtkAdjustment' id='adjustment'>
                <property name='upper'>20.0</property>
                <property name='lower'>0.0</property>
                <property name='value'>10.0</property>
                <property name='step-increment'>1.0</property>
                <property name='page-increment'>5.0</property>
```

```
                <property name='page-size'>0.0</property>
              </object>
            </property>
            <property name='digits'>0</property>
            <property name='draw-value'>true</property>
            <property name='has-origin'>true</property>
            <property name='round-digits'>0</property>
          </object>
        </child>
      </object>
    </child>
  </object>
</interface>
```

The UI file describes the following parent-child relationship.

```
GtkApplicationWindow (win) -- GtkBox -+- GtkLabel (label)
                                      +- GtkScale
```

Four GtkScale properties are defined.

- adjustment. GtkAdjustment provides the followings.
  - upper and lower: the range of the scale.
  - value: current value of the scale. It reflects the value of the scale.
  - step increment and page increment: When a user presses an arrow key or page up/down key, the scale moves by the step increment or page increment respectively.
  - page-size: When an adjustment is used with a scale, page-size is zero.
- digits: The number of decimal places that are displayed in the value.
- draw-value: Whether the value is displayed.
- has-origin: Whether the scale has the origin. If it's true, an orange bar appears between the origin and the current point.
- round-digits: The number of digits to round the value to when it changes. For example, if it is zero, the slider moves to an integer point.

```c
#include <gtk/gtk.h>

GtkExpressionWatch *watch;

static int
f2i (GObject *object, double d) {
  return (int) d;
}

static int
close_request_cb (GtkWindow *win) {
  gtk_expression_watch_unwatch (watch);
  return false;
}

static void
app_activate (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  gtk_window_present (gtk_application_get_active_window(app));
}

static void
```

```
23 app_startup (GApplication *application) {
24   GtkApplication *app = GTK_APPLICATION (application);
25   GtkBuilder *build;
26   GtkWidget *win, *label;
27   GtkAdjustment *adjustment;
28   GtkExpression *expression, *params[1];
29
30   /* Builds a window with exp.ui resource */
31   build = gtk_builder_new_from_resource ("/com/github/ToshioCP/exp/exp_bind.ui");
32   win = GTK_WIDGET (gtk_builder_get_object (build, "win"));
33   label = GTK_WIDGET (gtk_builder_get_object (build, "label"));
34   // scale = GTK_WIDGET (gtk_builder_get_object (build, "scale"));
35   adjustment = GTK_ADJUSTMENT (gtk_builder_get_object (build, "adjustment"));
36   gtk_window_set_application (GTK_WINDOW (win), app);
37   g_signal_connect (win, "close-request", G_CALLBACK (close_request_cb), NULL);
38   g_object_unref (build);
39
40   /* GtkExpressionWatch */
41   params[0] = gtk_property_expression_new (GTK_TYPE_ADJUSTMENT, NULL, "value");
42   expression = gtk_cclosure_expression_new (G_TYPE_INT, NULL, 1, params, G_CALLBACK (f2i), NULL, NULL
43   watch = gtk_expression_bind (expression, label, "label", adjustment); /* watch takes the ownership
44 }
45
46 #define APPLICATION_ID "com.github.ToshioCP.exp_watch"
47
48 int
49 main (int argc, char **argv) {
50   GtkApplication *app;
51   int stat;
52
53   app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
54
55   g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
56   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
57
58   stat = g_application_run (G_APPLICATION (app), argc, argv);
59   g_object_unref (app);
60   return stat;
61 }
```

The point of the program is:

- 41-42: Two expressions are defined. One is a property expression and the other is a closure expression. The property expression looks up the "value" property of the adjustment instance. The closure expression just converts the double into an integer.
- 43: `gtk_expression_bind` binds the label property of the GtkLabel instance to the integer returned by the closure expression. It creates a GtkExpressionWatch structure. The binding remains active while the watch exists. When the window is destroyed, the scale and adjustment are also destroyed. The watch recognizes a change in the value of the expression and tries to change the property of the label. Obviously, it is not a correct behavior. The watch should be unwatched before the window is destroyed.
- 37: Connects the "close-request" signal on the window to a handler `close_request_cb`. This signal is emitted when the close button is clicked. The "close-request" handler is called just before the window closes. It is the right moment to make the GtkExpressionWatch unwatched.

- 10-14: “close-request” signal handler. The function `gtk_expression_watch_unwatch (watch)` makes the watch stop watching the expression. It also releases the expression.

If you want to bind a property to an expression, `gtk_expression_bind` is the best choice. You can do it with `gtk_expression_watch` function, but it is less suitable.

5.3.4.2 gtk_expression_watch Function

```
GtkExpressionWatch*
gtk_expression_watch (
  GtkExpression* self,
  GObject* this_,
  GtkExpressionNotify notify,
  gpointer user_data,
  GDestroyNotify user_destroy
)
```

The function doesn’t take the ownership of the expression. It differs from `gtk_expression_bind`. So, you need to release the expression when it is useless. It creates a GtkExpressionWatch structure. The third parameter `notify` is a callback to invoke when the expression changes. You can set `user_data` to give it to the callback. The last parameter is a function to destroy the `user_data` when the watch is unwatched. Put NULL if you don’t need them.

Notify callback has the following format.

```
void
notify (
  gpointer user_data
)
```

This function is used to do something when the value of the expression changes. But if you want to bind a property to the value, use `gtk_expression_bind` instead.

There’s a sample program `exp_watch.c` in /src/expression directory. It outputs the width of the window to the standard output.

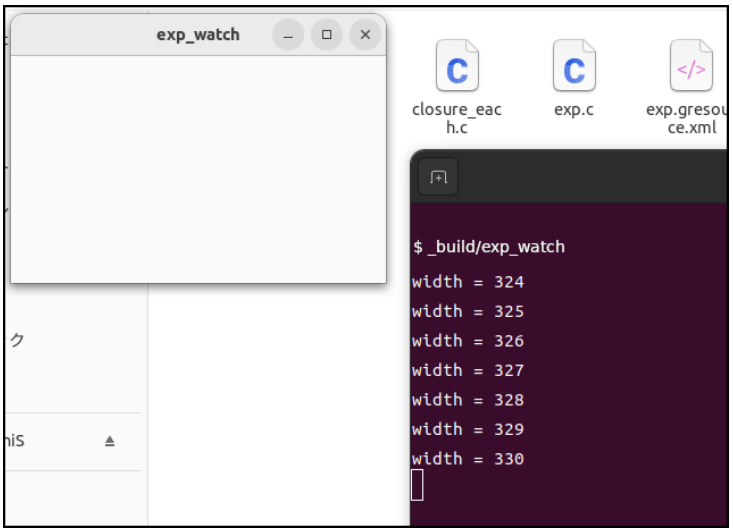


Figure 5.9: exp_watch

When you resize the window, the width is displayed in the terminal.

```
1 #include <gtk/gtk.h>
2
```

```
GtkExpression *expression;
GtkExpressionWatch *watch;

static void
notify (gpointer user_data) {
  GValue value = G_VALUE_INIT;

  if (gtk_expression_watch_evaluate (watch, &value))
    g_print ("width = %d\n", g_value_get_int (&value));
  else
    g_print ("evaluation failed.\n");
}

static int
close_request_cb (GtkWindow *win) {
  gtk_expression_watch_unwatch (watch);
  gtk_expression_unref (expression);
  return false;
}

static void
app_activate (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  gtk_window_present (gtk_application_get_active_window(app));
}

static void
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GtkWidget *win;

  win = GTK_WIDGET (gtk_application_window_new (app));
  g_signal_connect (win, "close-request", G_CALLBACK (close_request_cb), NULL);

  expression = gtk_property_expression_new (GTK_TYPE_WINDOW, NULL, "default-width");
  watch = gtk_expression_watch (expression, win, notify, NULL, NULL);
}

#define APPLICATION_ID "com.github.ToshioCP.exp_watch"

int
main (int argc, char **argv) {
  GtkApplication *app;
  int stat;

  app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);

  g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
  g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);

  stat = g_application_run (G_APPLICATION (app), argc, argv);
  g_object_unref (app);
  return stat;
```

```
56 }
```

- 37: A property expression looks up the “default-width” property of the window.
- 38: Create a watch structure for the expression. The callback `notify` is called every time the value of the expression changes. The ‘this’ object is `win`, so the expression returns the default width of the window.
- 6-14: The callback function `notify`. It uses `gtk_expression_watch_evaluate` to get the value of the expression. The ‘this’ object is given in advance (when the watch is created). It outputs the window width to the standard output.
- 16-21: A handler for the “close-request”signal on the window. It stops the watch. In addition, it releases the reference to the expression. Because `gtk_expression_watch` doesn’t take the ownership of the expression, you own it. So, the release is necessary.

5.3.5 Gtkexpression in Ui Files

GtkBuilder supports GtkExpressions. There are four tags.

- constant tag to create constant expression. Type attribute specifies the type name of the value. If no type is specified, the type is assumed to be an object. The content is the value of the expression.
- lookup tag to create property expression. Type attribute specifies the type of the object. Name attribute specifies the property name. The content is an expression or object which has the property to look up. If there’s no content, ‘this’ object is used.
- closure tag to create closure expression. Type attribute specifies the type of the returned value. Function attribute specifies the callback function. The contents of the tag are arguments that are expressions.
- binding tag to bind a property to an expression. It is put in the content of an object tag. Name attribute specifies the property name of the object. The content is an expression.

```
<constant type="gchararray">Hello world</constant>
<lookup name="label" type="GtkLabel">label</lookup>
<closure type="gint" function="callback_function"></closure>
<bind name="label">
  <lookup name="default-width">win</lookup>
</bind>
```

These tags are usually used for GtkBuilderListItemFactory.

```
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkLabel">
        <binding name="label">
          <lookup name="string" type="GtkStringObject">
            <lookup name="item">GtkListItem</lookup>
          </lookup>
        </binding>
      </object>
    </property>
  </template>
</interface>
```

GtkBuilderListItemFactory uses GtkBuilder to extends the GtkListItem with the XML data.

In the xml file above, “GtkListItem” is an instance of the GtkListItem template. It is the ‘this’ object given to the expressions. Please refer to the GTK Development Blog for further information about “this” objects.

GtkBuilder calls `gtk_expression_bind` function when it finds a binding tag. The function sets the 'this' object like this:

1. If the binding tag has object attribute, the object will be the 'this' object.
2. If the current object of the GtkBuilder exists, it will be the 'this' object. That's why a GtkListItem instance is the 'this' object of the XML data for a GtkBuilderListItemFactory.
3. Otherwise, the target object of the binding tag will be the 'this' object.

GTK 4 document doesn't describe information about "this" object when expressions are defined in a UI file. The information above is found from the GTK 4 source files and it is possible to include mistakes. If you have accurate information, please let me know.

A sample program `exp.c` and a UI file `exp.ui` is in /src/expression directory. The UI file includes lookup, closure and bind tags. No constant tag is included. However, constant tags are not used so often.



Figure 5.10: exp.c

If you resize the window, the size is shown at the title of the window. If you type characters in the entry, the same characters appear on the label.

The UI file is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="GtkApplicationWindow" id="win">
    <binding name="title">
      <closure type="gchararray" function="set_title">
        <lookup name="default-width" type="GtkWindow"></lookup>
        <lookup name="default-height" type="GtkWindow"></lookup>
      </closure>
    </binding>
    <property name="default-width">600</property>
    <property name="default-height">400</property>
    <child>
      <object class="GtkBox">
        <property name="orientation">GTK_ORIENTATION_VERTICAL</property>
        <child>
          <object class="GtkLabel">
            <binding name="label">
```

```
18                <lookup name="text">
19                  buffer
20                </lookup>
21              </binding>
22            </object>
23          </child>
24          <child>
25            <object class="GtkEntry">
26              <property name="buffer">
27                <object class="GtkEntryBuffer" id="buffer"></object>
28              </property>
29            </object>
30          </child>
31        </object>
32      </child>
33    </object>
34  </interface>
```

- 4-9: The title property of the main window is bound to a closure expression. Its callback function `set_title` is defined in the C source file. It returns a string because the type attribute of the tag is "gchararray". Two parameters are given to the function. They are width and height of the window. Lookup tags don't have contents, so 'this' object is used to look up the properties. The 'this' object is `win`, which is the target of the binding (`win` includes the binding tag).
- 17-21: The "label" property of the GtkLabel instance is bound to the "text" property of `buffer`, which is the buffer of GtkEntry defined in line 25. If a user types characters in the entry, the same characters appear on the label.

The C source file is as follows.

```
1  #include <gtk/gtk.h>
2
3  char *
4  set_title (GtkWidget *win, int width, int height) {
5    return g_strdup_printf ("%d x %d", width, height);
6  }
7
8  static void
9  app_activate (GApplication *application) {
10   GtkApplication *app = GTK_APPLICATION (application);
11   gtk_window_present (gtk_application_get_active_window(app));
12 }
13
14 static void
15 app_startup (GApplication *application) {
16   GtkApplication *app = GTK_APPLICATION (application);
17   GtkBuilder *build;
18   GtkWidget *win;
19
20   build = gtk_builder_new_from_resource ("/com/github/ToshioCP/exp/exp.ui");
21   win = GTK_WIDGET (gtk_builder_get_object (build, "win"));
22   gtk_window_set_application (GTK_WINDOW (win), app);
23   g_object_unref (build);
24 }
25
26 #define APPLICATION_ID "com.github.ToshioCP.exp"
```

```
27
28 int
29 main (int argc, char **argv) {
30   GtkApplication *app;
31   int stat;
32
33   app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
34
35   g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
36   g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
37
38   stat = g_application_run (G_APPLICATION (app), argc, argv);
39   g_object_unref (app);
40   return stat;
41 }
```

- 4-6: The callback function. It returns a string (w)x(h), where the w and h are the width and height of the window. String duplication is necessary.

The C source file is very simple because almost everything is done in the UI file.

5.3.6 Conversion between GValues

If you bind different type properties, type conversion is automatically done. Suppose the `label` property (string) is bound to the `default-width` property (int).

```
<object class="GtkLabel">
  <binding name="label">
    <lookup name="default-width">
      win
    </lookup>
  </binding>
</object>
```

The expression created by the lookup tag returns a int type GValue. On the other hand "label" property holds a string type GValue. When a GValue is copied to another GValue, the type is automatically converted if possible. If the current width is 100, an int `100` is converted to a string `"100"`.

If you use `g_object_get` and `g_object_set` to copy properties, the value is automatically converted.

5.3.7 Meson.build

The source files are in `/src/expression` directory. You can build all the files at once.

```
$ cd src/expression
$ meson setup _build
$ ninja -C _build
```

For example, if you want to run "exp", which is the executable file from "exp.c", type `_build/exp`. You can run other programs as well.

The file `meson.build` is as follows.

```
1 project('exp', 'c')
2
3 gtkdep = dependency('gtk4')
4
5 gnome=import('gnome')
6 resources = gnome.compile_resources('resources','exp.gresource.xml')
```

```
7
8  sourcefiles=files('exp.c')
9
10 executable('exp', sourcefiles, resources, dependencies: gtkdep, export_dynamic: true, install: false)
11 executable('exp_constant', 'exp_constant.c', dependencies: gtkdep, export_dynamic: true, install: fa
12 executable('exp_constant_simple', 'exp_constant_simple.c', dependencies: gtkdep, export_dynamic: tru
13 executable('exp_property_simple', 'exp_property_simple.c', dependencies: gtkdep, export_dynamic: tru
14 executable('closure', 'closure.c', dependencies: gtkdep, export_dynamic: true, install: false)
15 executable('closure_each', 'closure_each.c', dependencies: gtkdep, export_dynamic: true, install: fa
16 executable('exp_closure_simple', 'exp_closure_simple.c', dependencies: gtkdep, export_dynamic: true,
17 executable('exp_closure_with_error_report', 'exp_closure_with_error_report.c', dependencies: gtkdep,
18 executable('exp_bind', 'exp_bind.c', resources, dependencies: gtkdep, export_dynamic: true, install:
19 executable('exp_watch', 'exp_watch.c', dependencies: gtkdep, export_dynamic: true, install: false)
20 executable('exp_test', 'exp_test.c', resources, dependencies: gtkdep, export_dynamic: true, install:
```

5.4 GtkColumnView

5.4.1 GtkColumnView

GtkColumnView is like GtkListView, but it has multiple columns. Each column is GtkColumnViewColumn.

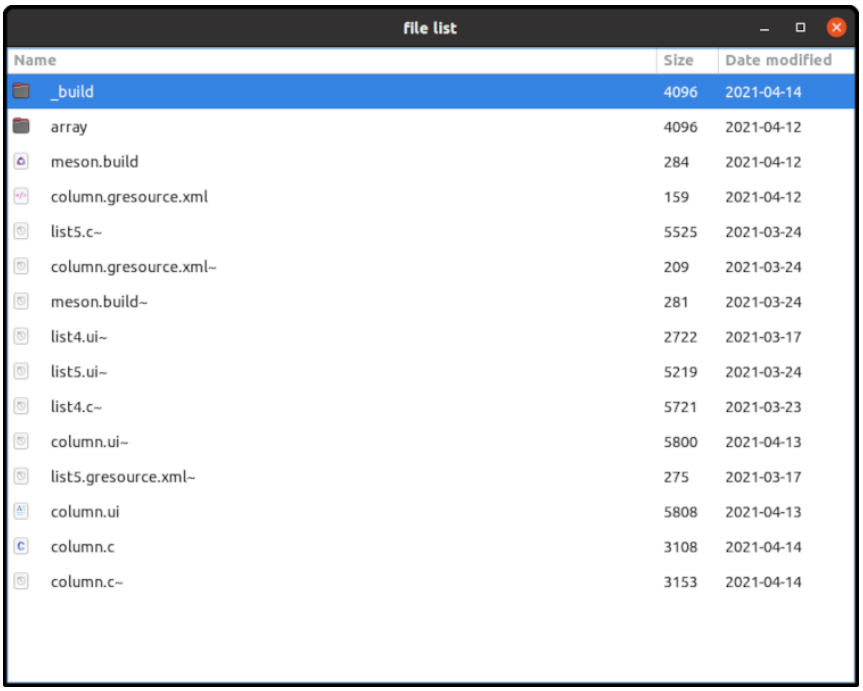


Figure 5.11: Column View

- GtkColumnView has “model” property. The property points to a GtkSelectionModel object.
- Each GtkColumnViewColumn has the “factory” property. The property points to a GtkListItemFactory (GtkSignalListItemFactory or GtkBuilderListItemFactory).
- The factory connects GtkListItem and items of GtkSelectionModel. And the factory builds the descendant widgets of GtkColumnView to display the items on the screen. This process is the same as the one in GtkListView.

The following diagram shows how it works.

The example in this section is a window that displays information of files in the current directory. The information is the name, size and last modified datetime of files. So, there are three columns.

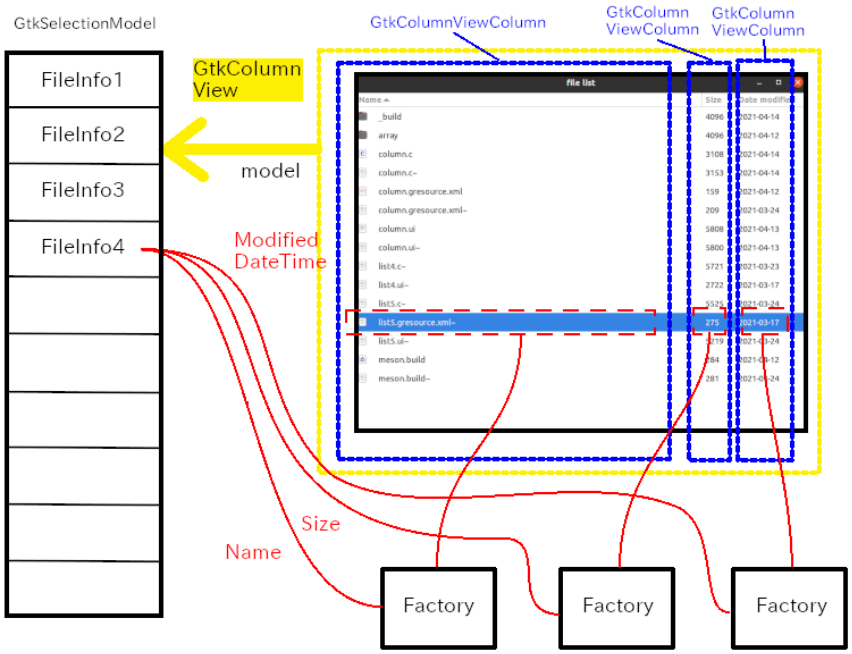


Figure 5.12: ColumnView

In addition, the example uses GtkSortListModel and GtkSorter to sort the information.

5.4.2 column.ui

UI file specifies widgets and list item templates.

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="GtkApplicationWindow" id="win">
    <property name="title">file list</property>
    <property name="default-width">800</property>
    <property name="default-height">600</property>
    <child>
      <object class="GtkScrolledWindow">
        <property name="hexpand">TRUE</property>
        <property name="vexpand">TRUE</property>
        <child>
          <object class="GtkColumnView" id="columnview">
            <property name="model">
              <object class="GtkNoSelection">
                <property name="model">
                  <object class="GtkSortListModel">
                    <property name="model">
                      <object class="GtkDirectoryList" id="directorylist">
                        <property name="attributes">standard::name,standard::icon,standard::size,tim
                      </object>
                    </property>
                    <binding name="sorter">
                      <lookup name="sorter">columnview</lookup>
                    </binding>
                  </object>
```

```
                </property>
              </object>
            </property>
            <child>
              <object class="GtkColumnViewColumn">
                <property name="title">Name</property>
                <property name="expand">TRUE</property>
                <property name="factory">
                  <object class="GtkBuilderListItemFactory">
                    <property name="bytes"><![CDATA[
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkBox">
        <property name="orientation">GTK_ORIENTATION_HORIZONTAL</property>
        <property name="spacing">20</property>
        <child>
          <object class="GtkImage">
            <binding name="gicon">
              <closure type="GIcon" function="get_icon_factory">
                <lookup name="item">GtkListItem</lookup>
              </closure>
            </binding>
          </object>
        </child>
        <child>
          <object class="GtkLabel">
            <property name="hexpand">TRUE</property>
            <property name="xalign">0</property>
            <binding name="label">
              <closure type="gchararray" function="get_file_name_factory">
                <lookup name="item">GtkListItem</lookup>
              </closure>
            </binding>
          </object>
        </child>
      </object>
    </property>
  </template>
</interface>
                    ]]></property>
                  </object>
                </property>
                <property name="sorter">
                  <object class="GtkStringSorter">
                    <property name="expression">
                      <closure type="gchararray" function="get_file_name">
                      </closure>
                    </property>
                  </object>
                </property>
              </object>
```

```
            </child>
            <child>
              <object class="GtkColumnViewColumn">
                <property name="title">Size</property>
                <property name="factory">
                  <object class="GtkBuilderListItemFactory">
                    <property name="bytes"><![CDATA[
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkLabel">
        <property name="hexpand">TRUE</property>
        <property name="xalign">0</property>
        <binding name="label">
          <closure type="gint64" function="get_file_size_factory">
            <lookup name="item">GtkListItem</lookup>
          </closure>
        </binding>
      </object>
    </property>
  </template>
</interface>
                    ]]></property>
                  </object>
                </property>
                <property name="sorter">
                  <object class="GtkNumericSorter">
                    <property name="expression">
                      <closure type="gint64" function="get_file_size">
                      </closure>
                    </property>
                    <property name="sort-order">GTK_SORT_ASCENDING</property>
                  </object>
                </property>
              </object>
            </child>
            <child>
              <object class="GtkColumnViewColumn">
                <property name="title">Date modified</property>
                <property name="factory">
                  <object class="GtkBuilderListItemFactory">
                    <property name="bytes"><![CDATA[
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <template class="GtkListItem">
    <property name="child">
      <object class="GtkLabel">
        <property name="hexpand">TRUE</property>
        <property name="xalign">0</property>
        <binding name="label">
          <closure type="gchararray" function="get_file_time_modified_factory">
            <lookup name="item">GtkListItem</lookup>
```

```
132           </closure>
133         </binding>
134       </object>
135     </property>
136   </template>
137 </interface>
138                     ]]></property>
139                   </object>
140                 </property>
141                 <property name="sorter">
142                   <object class="GtkNumericSorter">
143                     <property name="expression">
144                       <closure type="gint64" function="get_file_unixtime_modified">
145                       </closure>
146                     </property>
147                     <property name="sort-order">GTK_SORT_ASCENDING</property>
148                   </object>
149                 </property>
150               </object>
151             </child>
152           </object>
153         </child>
154       </object>
155     </child>
156   </object>
157 </interface>
```

- 3-12: GtkApplicationWindow has a child widget GtkScrolledWindow. GtkScrolledWindow has a child widget GtkColumnView.
- 12-18: GtkColumnView has “model” property. It points to GtkSelectionModel interface. GtkNoSelection class is used as GtkSelectionModel. Similarly, it also has a “model” property. It points to GtkSortListModel. This list model supports sorting the list. It will be explained in the later subsection. And it also has “model” property. It points to GtkDirectoryList. Therefore, the chain is: GtkColumnView => GtkNoSelection => GtkSortListModel => GtkDirectoryList.
- 18-20: GtkDirectoryList. It is a list of GFileInfo, which holds information of files under a directory. It has “attributes” property. It specifies which attributes are kept in each GFileInfo.
  - “standard::name” is the name of the file.
  - “standard::icon” is the GIcon object of the file
  - “standard::size” is the file size.
  - “time::modified” is the date and time the file was last modified.
- 29-79: The first GtkColumnViewColumn object. There are four properties, “title”, “expand”, factory” and “sorter”.
- 31: Sets the “title” property to “Name”. This is the title on the header of the column.
- 32: Sets the “expand” property to TRUE to allow the column to expand as much as possible. (See the image above).
- 33- 69: Sets the “factory” property to GtkBuilderListItemFactory. The factory has “bytes” property which holds a UI string to define a template to extend GtkListItem class. The CDATA section (line 36-66) is the UI string to put into the “bytes” property. The contents are the same as the UI file `factory_list.ui` in Section 30.
- 70-77: Sets the “sorter” property to GtkStringSorter object. This object provides a sorter that compares strings. It has “expression” property. A closure tag with a string type function `get_file_name` is used here. The function will be explained later.
- 80-115: The second GtkColumnViewColumn object. Its sorter property is set to GtkNumericSorter.
- 116-151: The third GtkColumnViewColumn object. Its sorter property is set to GtkNumericSorter.

5.4.3 GtkSortListModel and GtkSorter

GtkSortListModel is a list model that sorts its elements according to a GtkSorter instance assigned to the "sorter" property. The property is bound to "sorter" property of GtkColumnView in line 22 to 24.

```
<object class="GtkSortListModel" id="sortlist">
... ... ...
  <binding name="sorter">
    <lookup name="sorter">columnview</lookup>
  </binding>
```

Therefore, `columnview` determines how to sort the list model. The "sorter" property of GtkColumnView is a read-only GtkSorter and it always refers to the current column's "sorter" property. When a user clicks a column header, GtkColumnView updates its own "sorter" property to point to that column's sorter.

The binding above makes an indirect connection between the "sorter" property of GtkSortListModel and the "sorter" property of each column.

GtkSorter compares two items (GObject or its descendant). GtkSorter has several child objects.

- GtkStringSorter compares strings taken from the items.
- GtkNumericSorter compares numbers taken from the items.
- GtkCustomSorter uses a callback to compare.
- GtkMultiSorter combines multiple sorters.

The example uses GtkStringSorter and GtkNumericSorter.

GtkStringSorter uses GtkExpression to get the strings from the items (objects). The GtkExpression is stored in the "expression" property of the GtkStringSorter. When GtkStringSorter compares two items, it evaluates the expression by calling `gtk_expression_evaluate` function. It assigns each item to the second argument ('this' object) of the function.

In the UI file above, the GtkExpression is in the line 71 to 76.

```
<object class="GtkStringSorter">
  <property name="expression">
    <closure type="gchararray" function="get_file_name">
    </closure>
  </property>
</object>
```

The GtkExpression calls `get_file_name` function when it is evaluated.

```
1 char *
2 get_file_name (GFileInfo *info) {
3   return G_IS_FILE_INFO (info) ? g_strdup (g_file_info_get_name (info)) : NULL;
4 }
```

The function is given the item (GFileInfo) of the GtkSortListModel as an argument (`this` object). But you need to be careful that it can be NULL while the list item is being recycled. So, `G_IS_FILE_INFO (info)` is always necessary in callback functions. The function retrieves a filename from `info`. The string is owned by `info` so it is necessary to duplicate. And it returns the copied string.

GtkNumericSorter compares numbers. It is used in the line 106 to 112 and line 142 to 148. The lines from 106 to 112 is:

```
<object class="GtkNumericSorter">
  <property name="expression">
    <closure type="gint64" function="get_file_size">
    </closure>
  </property>
```

```
  <property name="sort-order">GTK_SORT_ASCENDING</property>
</object>
```

The closure tag specifies a callback function `get_file_size`.

```
goffset
get_file_size (GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_file_info_get_size (info): -1;
}
```

It just returns the size of `info`. The type of the size is `goffset`. The type `goffset` is the same as `gint64`.

The lines from 142 to 148 is:

```
<object class="GtkNumericSorter" id="sorter_datetime_modified">
  <property name="expression">
    <closure type="gint64" function="get_file_unixtime_modified">
    </closure>
  </property>
  <property name="sort-order">GTK_SORT_ASCENDING</property>
</object>
```

The closure tag specifies a callback function `get_file_unixtime_modified`.

```
gint64
get_file_unixtime_modified (GFileInfo *info) {
  GDateTime *dt;

  dt = G_IS_FILE_INFO (info) ? g_file_info_get_modification_date_time (info) : NULL;
  return dt ? g_date_time_to_unix (dt) : -1;
}
```

It gets the modification date and time (GDateTime type) of `info`. Then it gets a Unix time from `dt`. Unix time, sometimes called Unix epoch, is the number of seconds that have elapsed since 00:00:00 UTC on 1 January 1970. It returns the Unix time as a gint64.

5.4.4 column.c

`column.c` is as follows. The code is simple and short thanks to `column.ui`.

```
#include <gtk/gtk.h>

/* functions (closures) for GtkBuilderListItemFactory */
GIcon *
get_icon_factory (GtkListItem *item, GFileInfo *info) {
  GIcon *icon;

   /* g_file_info_get_icon can return NULL */
  icon = G_IS_FILE_INFO (info) ? g_file_info_get_icon (info) : NULL;
  return icon ? g_object_ref (icon) : NULL;
}

char *
get_file_name_factory (GtkListItem *item, GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_strdup (g_file_info_get_name (info)) : NULL;
}

/* goffset is defined as gint64 */
```

```
/* It is used for file offsets. */
goffset
get_file_size_factory (GtkListItem *item, GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_file_info_get_size (info) : -1;
}

char *
get_file_time_modified_factory (GtkListItem *item, GFileInfo *info) {
  GDateTime *dt;

   /* g_file_info_get_modification_date_time can return NULL */
  dt = G_IS_FILE_INFO (info) ? g_file_info_get_modification_date_time (info) : NULL;
  return dt ? g_date_time_format (dt, "%F") : NULL;
}

/* Functions (closures) for GtkSorter */
char *
get_file_name (GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_strdup (g_file_info_get_name (info)) : NULL;
}

goffset
get_file_size (GFileInfo *info) {
  return G_IS_FILE_INFO (info) ? g_file_info_get_size (info): -1;
}

gint64
get_file_unixtime_modified (GFileInfo *info) {
  GDateTime *dt;

  dt = G_IS_FILE_INFO (info) ? g_file_info_get_modification_date_time (info) : NULL;
  return dt ? g_date_time_to_unix (dt) : -1;
}

static void
app_activate (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  gtk_window_present (gtk_application_get_active_window(app));
}

static void
app_startup (GApplication *application) {
  GtkApplication *app = GTK_APPLICATION (application);
  GFile *file;
  GtkBuilder *build = gtk_builder_new_from_resource ("/com/github/ToshioCP/column/column.ui");
  GtkWidget *win = GTK_WIDGET (gtk_builder_get_object (build, "win"));
  GtkDirectoryList *directorylist = GTK_DIRECTORY_LIST (gtk_builder_get_object (build, "directorylis
  g_object_unref (build);

  gtk_window_set_application (GTK_WINDOW (win), app);

  file = g_file_new_for_path (".");
  gtk_directory_list_set_file (directorylist, file);
```

```
72    g_object_unref (file);
73  }
74
75  #define APPLICATION_ID "com.github.ToshioCP.columnview"
76
77  int
78  main (int argc, char **argv) {
79    GtkApplication *app;
80    int stat;
81
82    app = gtk_application_new (APPLICATION_ID, G_APPLICATION_DEFAULT_FLAGS);
83
84    g_signal_connect (app, "startup", G_CALLBACK (app_startup), NULL);
85    g_signal_connect (app, "activate", G_CALLBACK (app_activate), NULL);
86
87    stat = g_application_run (G_APPLICATION (app), argc, argv);
88    g_object_unref (app);
89    return stat;
90  }
```

5.4.5 Compilation and Execution.

All the source files are in src/column directory. Change your current directory to the directory and type the following.

```
$ cd src/column
$ meson setup _build
$ ninja -C _build
$ _build/column
```

Then, a window appears.

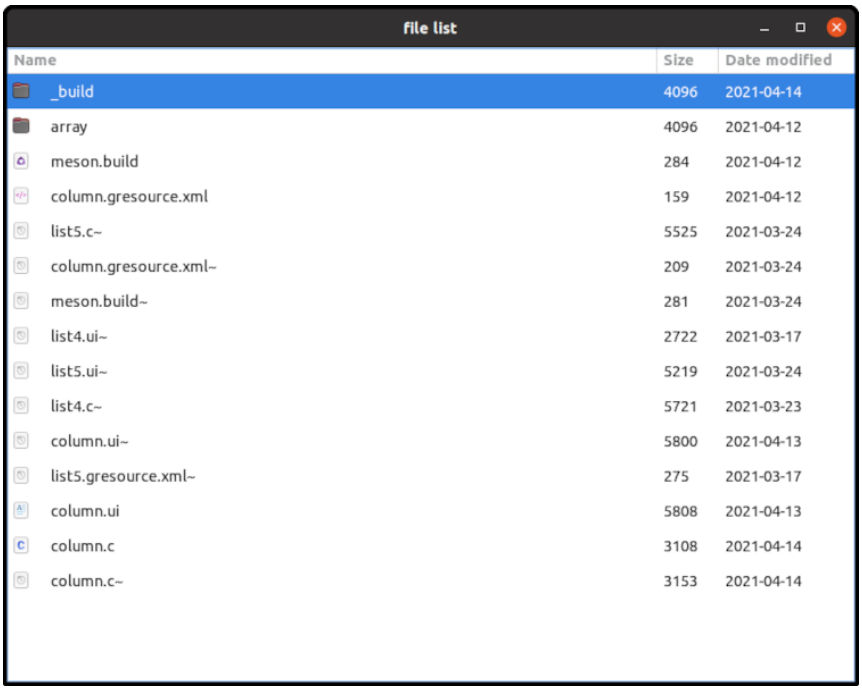


Figure 5.13: Column View

If you click the header of a column, then the entire list is sorted by the column. If you click the header of

another column, then the list is sorted by the newly selected column.

GtkColumnView is very useful and it can manage very big GListModel. It is possible to use it for file list, application list, database frontend and so on.

5.5 GtkSignalListItemFactory

5.5.1 GtkSignalListItemFactory vs GtkBuilderListItemFactory

GtkBuilderlistItemFactory is convenient when GtkListView just shows the contents of a list model. Its binding direction is always from an item of the list to a child of GtkListItem.

However, it is insufficient for dynamic connections. For example, suppose you want to edit the contents of a list. You can set a GtkText widget as the child of a GtkListItem to allow users to edit text. You need to update the list item when the text in the GtkText is edited. The data flow is in the opposite direction compared to GtkBuilderListItemFactory — it goes from the GtkText widget back to the list item. You can achieve this using GtkSignalListItemFactory, which provides more flexibility.

This section shows just some parts of the source file listeditor.c. If you want to see the whole codes, see src/listeditor directory of the Gtk4 tutorial repository.

5.5.2 List Editor

The sample program is a list editor and the data in the list are strings. It's similar to a line editor. It reads a text file line by line. Each line is an item of the list. The list is displayed with a GtkListView. The list view contains strings which are the contents of the corresponding items in the list model.

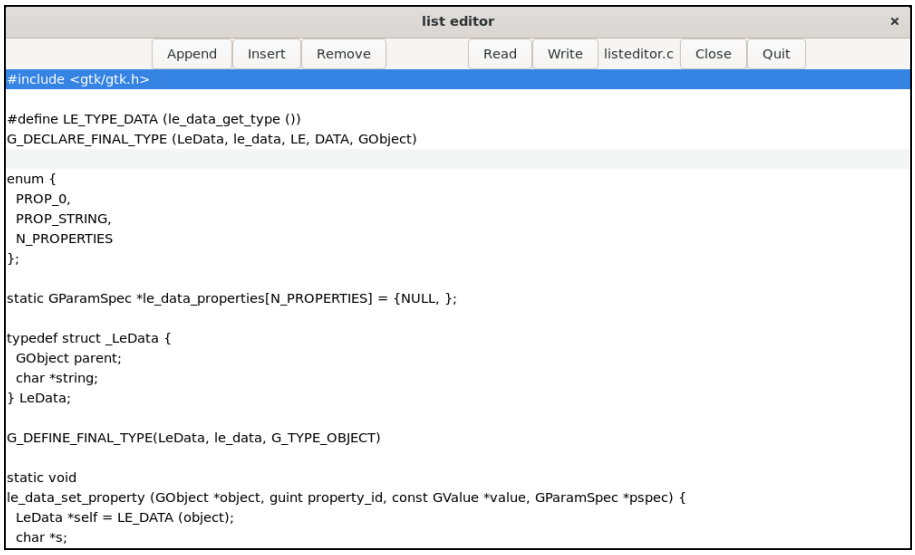


Figure 5.14: List editor

You can compile the source file and execute it as follows.

- Download the program from the repository.
- Change your current directory to `src/listeditor`.
- Type the following on your commandline.

```
$ meson setup _build
$ ninja -C _build
$ _build/listeditor
```

The window has buttons:

- **Append button:** appends a data after the current item, or at the end of the model if no current item exists.
- **Insert button:** inserts a row before the current item, or at the top of the model if no current item exists.
- **Remove button:** removes the current item.
- **Read button:** reads a file.
- **Write button:** Writes the contents to the file shown on the toolbar. If no filename is set, a file dialog opens to let you choose one.
- **Close button:** remove all the items from the model.
- **Quit button:** quits the application.

You can click on the content area and select a row. The corresponding item to the row will be the current item. The current position (zero-based) is shown at the left of the tool bar.

5.5.3 Connect a GtkText Instance and an Item in the Model

The ListView sets its factory property to GtkSignalListItemFactory. Its two signals "setup" and "bind" are connected to the handlers `setup_cb` and `bind_cb` respectively. The remaining two signals "unbind" and "teardown" aren't used in this program. The following shows the signal handlers.

```
1  static void
2  setup_cb (GtkListItemFactory *factory, GtkListItem *listitem) {
3    GtkWidget *text = gtk_text_new ();
4
5    gtk_list_item_set_child (listitem, GTK_WIDGET (text));
6    gtk_editable_set_alignment (GTK_EDITABLE (text), 0.0);
7    gtk_widget_set_can_target (GTK_WIDGET (text), FALSE);
8
9    g_signal_connect (text, "changed", G_CALLBACK (text_changed_cb), listitem);
10   g_signal_connect (listitem, "notify::selected", G_CALLBACK (item_selected_cb), text);
11 }
12
13 static void
14 bind_cb (GtkListItemFactory *factory, GtkListItem *listitem) {
15   GtkWidget *text = gtk_list_item_get_child (listitem);
16   LeData *data = LE_DATA (gtk_list_item_get_item(listitem));
17
18   if (data) {
19     g_signal_handlers_block_by_func (text, (gpointer) text_changed_cb, listitem);
20     gtk_editable_set_text (GTK_EDITABLE (text), le_data_look_string (data));
21     gtk_editable_set_position (GTK_EDITABLE (text), -1);
22     g_signal_handlers_unblock_by_func (text, (gpointer) text_changed_cb, listitem);
23   }
24 }
```

- 5-7: `setup_cb` creates the child widget of the list item. This widget will be a row of the GtkListView instance.
- 9-10: `setup_cb` connects the "changed" signal on the GtkText instance to the handler `text_changed_cb`. This handler copies the text in the GtkText to the item when the GtkText is changed. `setup_cb` also connects the "notify::selected" signal on the GtkListItem to the handler `item_selected_cb`. The handler make the GtkText grab the keyboard focus to prepare keyboard input.
- These instance and signals created by `setup_cb` will be automatically destroyed when the list item disposes itself. So, teardown signal handler is not necessary.

- 83-86: `bind_cb` just copy the string in the item in the model to the GtkText. The macro function `g_signal_block_by_func` avoids unnecessary copy from GtkText to item at the update of the GtkText. No instances are created in this process, so "unbind" signal handler is not necessary.
- 5-7: `setup_cb` creates the child widget for the list item, which serves as a row in the GtkListView.
- 9-10: `setup_cb` connects the GtkText's "changed" signal to the `text_changed_cb` handler. This handler updates the item in the model with the new text whenever the GtkText is modified. Additionally, it connects the "notify::selected" signal of the GtkListItem to `item_selected_cb`, which makes the GtkText grab keyboard focus to prepare for immediate input. The "notify" signal is inherited from GObject class.
- The instances and signal connections created in `setup_cb` are automatically cleaned up when the list item is disposed. Therefore, a "teardown" signal handler is unnecessary.
- 19-22: `bind_cb` simply copies the string from the model's item to the GtkText. The `g_signal_handlers_block_by_func` macro prevents an unnecessary write-back from the GtkText to the item during this update. Since no new resources are allocated in this step, an "unbind" signal handler is not needed.

The two signal handlers `text_changed_cb` and `item_selected_cb` are shown below.

```
1  static void
2  text_changed_cb (GtkEditable *self, gpointer user_data) {
3    GtkListItem *listitem = GTK_LIST_ITEM (user_data);
4    LeData *data = LE_DATA (gtk_list_item_get_item(listitem));
5    const char *s;
6
7    if (data) {
8      s = gtk_editable_get_text (self);
9      le_data_set_string (data, s);
10   }
11 }
12
13 static void
14 item_selected_cb (GtkListItem *listitem, GParamSpec *pspec, gpointer user_data) {
15   GtkWidget *text = GTK_WIDGET (user_data);
16
17   if (gtk_list_item_get_selected (listitem)) {
18     gtk_widget_grab_focus (text);
19   }
20 }
```

5.5.4 GtkSingleSelection

In the `liststore.ui` file, the GtkListView's model property is set to a GtkSingleSelection, which in turn wraps the underlying GListModel.

Clicking a row in the list view selects and highlights it, automatically focusing the GtkText widget for immediate editing. Since GtkSingleSelection handles all the selection logic internally, no additional code is required for this behavior.

5.5.5 Another Possible Implementation

Alternatively, you can use a GBinding object to bind the GtkText's "text" property to the LeData's "string" property. To do this, you first need to implement the "string" property in the LeData class.

The code for this version is available in the src/listeditor_binding/ directory. The GBinding instance is created in the "bind" handler and unbound in the "unbind" handler. We use the `g_object_set_data` function to store the binding object within the list item.

This is just one example, and other implementations are possible. For instance, you could subclass GtkText and store the binding instance directly inside the subclass.

However, these alternatives require more code. Ultimately, using the "changed" signal remains the simplest and most straightforward approach.

Appendix A

Appendix

A.1 Author Guide

A.1.1 Quick Start Guide

This section describes the steps to view the GTK 4 Tutorial on your local machine.

1. You will need a Linux operating system (or a Linux environment like WSL on Windows), Ruby (ver 4.0.5), Rake, Pandoc, and a LaTeX system (TexLive2026 is recommended).
2. Download this repository and extract its contents. See the next section for instructions on how to download the repository.
3. Change your current directory to the root directory of the source files.
4. Run `bundle install` to install the required Gems. `bundle` is a command from Bundler, a Ruby library management tool included with Ruby.
5. To view the tutorial in a web browser on your local PC:

- Run `bundle exec rake html` to generate HTML files for local use.
- Run `bundle exec rake serve` and access `http://localhost:8000` in your browser to view the HTML files.

6. To create a PDF version of the tutorial:

- Run `bundle exec rake pdf` to generate the PDF file.
- The PDF file will be generated in the `pdf` directory.

A.1.2 Recommended Environment and Software Setup

To generate the tutorial documents in your local repository, the following environment and software are required. I have also included the versions used in my own environment for reference.

- Linux Operating System: The programs in this repository were tested on Ubuntu 26.04 LTS.
- Ruby, Rake, Pandoc, and LaTeX: These are used to generate the documentation.
  - Ruby: Version 4.0.5 is used. Ruby version 3 or later may work, but you may need to modify the Gemfile, .ruby-version, and mise.toml.
  - Pandoc: Version 3.7.0.2 was used, but the version provided by your Linux distribution's package manager is likely sufficient. Pandoc converts Markdown (Pandoc's Markdown) into HTML or PDF.
  - LaTeX: When Pandoc converts Markdown to PDF, it generates LaTeX as an intermediate step. A LaTeX system then converts that into a PDF. For this tutorial, TeXLive 2026 with LuaLaTeX (LuaHBTex 1.24.0 engine) was used.
  - If you want to use other versions of the programs mentioned above, the issue "PDF Build Failures on Ubuntu 24.04 with Ruby 4.0.5, XeLaTeX and pandoc #65" might be useful for you.

There are two ways to download the repository to your local PC: - Using Git: Run `git clone https://github.com/ToshioCP/Gtk4-tutorial.git` in your terminal. - Downloading as a Zip: Click the green `Code` button on the repository's main page and select "Download ZIP." Extract the ZIP file after downloading.

A.1.3 How to Write .src.md Files

The source files for this tutorial are located in the `src` directory and have the `.src.md` extension. From here on, these will be referred to as ".src.md files."

A.1.3.1 The Document Generation Process

Three types of documents are generated and stored in the `gfm`, `docs`, and `pdf` directories, corresponding to GFM (GitHub Flavored Markdown), HTML, and PDF formats.

These three output formats—`gfm`, `html`, and `pdf` — are referred to as targets. Note that for the `html` target, the output directory is named `docs` instead of `html`. This is to align with the directory name used for hosting files on GitHub Pages.

Conversion to a target can happen in two ways: direct generation (no intermediate steps) or indirect generation (using intermediate steps).

Direct Generation

- gfm target: .src.md => GFM

Indirect Generation

- html target: .src.md => Pandoc's Markdown => HTML
- pdf target: .src.md => Pandoc's Markdown => LaTeX => PDF

As seen in this process, the .src.md format is converted into either GFM or Pandoc's Markdown. This conversion is handled by Ruby scripts.

GFM and Pandoc's Markdown are both extensions of the original Markdown. Since Markdown has several such "dialects," they share common features but also have distinct differences. While GFM and Pandoc's Markdown are identical in their basic syntax, they differ in areas such as how info strings and attributes are handled in fenced code blocks.

A.1.3.2 The Two Components of .src.md

The content of a .src.md file is divided into two parts:

1. @@@ commands
2. Markdown: specifically, GFM or Pandoc's Markdown

Let's look at a concrete example:

```
This line belongs to "2. Markdown."

@@@shell
echo Hello World.
@@@

This line also belongs to "2. Markdown."
```

The three lines starting from `@@@shell` to `@@@` belong to "1. @@@ commands."

A.1.3.3 @@@ Commands

@@@ commands are defined as blocks with the following structures:

- @@@if command: Start with `@@@if` and end with `@@@end`.
- @@@include command: Start with `@@@include` and end with `@@@`.
- @@@shell command: Start with `@@@shell` and end with `@@@`.
- @@@table command: Start with `@@@table` and end with `@@@`.

The table command (`@@@table`) isn't used in the conversion and will be explained in a later section.

A.1.3.4 The @@@if Command

This command begins with `@@@if`, optionally followed by `@@@elif` or `@@@else`, and ends with `@@@end`. These work similarly to the C preprocessor directives `#if`, `#elif`, `#else`, and `#endif`.

`@@@if` and `@@@elif` are followed by a condition. Conditions can be:

- One of the generation targets: gfm, html, or pdf.
- Two targets joined by the `|` (OR) operator. The condition is true if the generation target matches either one.

Example:

````
@@@if gfm
```ruby
@@@else
```{.ruby .numberLines}
@@@end
def hello
  print("Hello World.\n")
end
```
````

This example resolves the difference in how fenced code blocks handle metadata between GFM and Pandoc's Markdown:

- In GFM, you can append an info string (usually the language name) after the backticks.
- In Pandoc's Markdown, you can append attributes enclosed in curly braces after the backticks. This allows for language specification as well as additional features like line numbering.

A.1.3.5 @@@include

The `@@@include` command imports external files.

```
@@@include
sample.c
@@@
```

In this example, the C source file `sample.c` specified between `@@@include` and the `@@@` marker is imported. The source file path must be relative from the `src` directory. If the content of `sample.c` is as shown below, the @@@include command is replaced by that C code. To ensure it fits within the Markdown text, it is wrapped in a fenced code block.

```
#include <stdio.h>

void
hello() {
    printf("Hello World.\n");
}

int
main() {
```
```

```
    hello();
    return 0;
}
```

You can extract a specific function by adding the function name after the filename.

```
@@@include
sample.c hello
@@@
```

This will only include the `hello` function from `sample.c`. The resulting replacement will look like this (wrapped in a fenced code block, which is omitted here for clarity):

```
void
hello() {
    printf("Hello World.\n");
}
```

While you can import source files from other languages, function-level extraction is only supported for C source files. Additionally, the C function extraction feature assumes the following formatting:

```
Return_Type
Function_Name(Arguments) {
  Function body
}
```

Specifically, the return type must be on the line immediately preceding the function name, and the function must end with a closing brace `}` at the beginning of a line. If these conditions are not met, the function cannot be extracted correctly.

Inserted text is converted into a fenced code block, surrounded by lines starting with three backticks.

If the output target is `gfm`, an info string (language name) is added to the starting fence. Using the previous C code example:

```
@@@include
sample.c hello
@@@
```

Since the file extension `.c` indicates C, the info string will be `c`. The resulting output becomes:

````
```c
void
hello() {
    printf("Hello World.\n");
}
```
````

Info strings like c, ruby, or xml are determined by the file extension:

- `.c` => c
- `.rb` => ruby
- `.xml` => xml

A list of supported languages is defined in /data/ext.yml. Note that this path starts with a slash to represent the repository root, not the operating system root.

If the output target is `html` or `pdf`, attributes of Pandoc's Markdown are added after the starting fence.

```
@@@include
sample.c hello
@@@
```

This @@@ command is converted as follows:

````
```{.c .numberLines}
void
hello() {
    printf("Hello World.\n");
}
```
````

These Pandoc's Markdown attributes provide both syntax highlighting for the C code and line numbers. Line numbering is only available for `html` and `pdf` targets that use Pandoc's Markdown. Unfortunately, GFM does not have a Markdown extension for line numbers, so only the code itself is output for that target.

The @@@include command must be written in exactly three lines (start marker, pathname with optional function names, end marker). Consequently, you cannot include multiple files in a single command block.

A.1.3.6 @@@shell

This command consists of the following three lines:

```
@@@shell
Shell command to execute
@@@
```

Make sure your current directory is the src directory of the local repository. All paths in the shell commands must be within src; any attempt to access files outside this directory will cause an error.

The three lines of the @@@shell command are replaced by the command line, its standard output, and the subsequent prompt.

For example:

```
@@@shell
wc author_guide.src.md
@@@
```

This is converted into:

````
```
$ wc author_guide.src.md
  509  2850 18698 author_guide.src.md
$
```
````

No info strings or attributes are added to this fenced code block.

A.1.3.7 How to Write Links

Links use URLs to direct the browser to a destination. There are three ways to write a URL:

- Absolute URL: A URL starting with http or https. (e.g., `https://example.com/a/b/c.html`)
- Root-relative URL: A path starting with a slash, representing the root of the site. In the example above, the root-relative version would be `/a/b/c.html`.
- Relative URL: A path relative to the current URL. If the current page is `https://example.com/a/index.html`, the relative URL to `https://example.com/a/b/c.html` would be `b/c.html`. Note that a relative URL does not start with a slash.

A.1.3.7.1 External links Use absolute URLs for links to external sites. For example, the following sentence includes an external link to the GFM specification page:

```
The GFM documentation can be found in the [GitHub Flavored Markdown Spec](https://github.github.com/g
```

A.1.3.7.2 Internal links For internal links, use root-relative or relative URLs to ensure portability. When writing root-relative URLs in `.src.md` files, treat the `src` directory as the site root. The Ruby script automatically adjusts these paths to refer to the appropriate locations during the conversion.

We recommend using root-relative links for image files, though relative links also work.

Examples:

Root-relative URL:

- `[Section 1](/sec1.src.md)`
- `[Image file](/images/sample.png)`
- `[C file pr1.c](/misc/pr1.c)`
- `[Directory misc](/misc/)`

Relative URL:

- `[Section 1](sec1.src.md)`
- `[Image file](images/sample.png)`
- `[C file pr1.c](misc/pr1.c)`
- `[Directory misc](misc/)`

The requirements for the pdf target differ from the gfm or html targets:

- Section links: Must use fragment identifiers (e.g., `#link-to-files`). They link to ATX header IDs created by Pandoc. For further information, see Pandoc User's Manual - Extension: auto_identifiers.
- Other files: Both root-relative and relative URLs are supported. The links usually refer to C source files or directories. The Ruby script converts these links to point to the corresponding blob or tree pages in the GitHub Gtk4-tutorial repository.

Example:

- `[Section 1](#prerequisite-and-license)`
- `[Image file](/images/image.png)`
- `[C source file](misc/pr1.c)`
- `[Directory](/misc/)`

Path Validation:

Ensure that all internal links remain within the `src` directory. The Ruby script will detect and warn you if a path (e.g., ../../../etc/passwd) attempts to escape the directory.

Note: This check prevents broken links and helps avoid "path traversal" issues, ensuring all referenced resources are correctly included in the final output.

For the pdf target, you can specify image sizes in units like cm.

```
![Image a](/images/a.png){width=6cm height=5cm}
```

This attribute is a Pandoc's Markdown extension and is not available in GFM. The units (cm, % etc.) depend on the target (HTML or PDF). These attributes are preserved only when the target is html or pdf. If the target is gfm, the Ruby script removes the attribute.

A.1.3.8 Other Notes on Writing .src.md

The conversion from .src.md to GFM or Pandoc's Markdown by the Ruby script follows this order:

1. Pass 1: Replacement of @@@if commands
2. Pass 2: Replacement of @@@include and @@@shell commands
3. Pass 3: Separation into code and non-code sections; code is left as-is, while links in non-code sections are converted.

Failure to account for this order can lead to unexpected results.

```
```
@@@if gfm
Some statement for the target gfm
@@@elif html
Some statement for the target html
@@@else
Some statement for the target pdf
@@@end
```
```

Even if you intend to display the @@@if command itself, it will not be preserved as-is. This is because @@@if replacement (Pass 1) occurs before code block protection (Pass 3). If the target is gfm, the resulting output will be:

```
```
Some statement for the target gfm
```
```

To achieve the intended output, use indented code blocks instead of fenced code blocks. @@@ commands must start at the beginning of a line. If they are indented, they are not recognized as commands, allowing them to pass through Pass 1 unchanged and be treated as an (indented) code block in Pass 3.

As explained later, section files (`sec<num>.src.md`) must be placed directly under the root of the source directory. However, other .src.md files can be organized in subdirectories within the source directory. This directory structure is mirrored in the target directory when the target is gfm or html. If the target is pdf, these .src.md files are included in the appendices. Note that only directories containing .src.md files are copied. C programs are not copied, and image files are copied to the `docs` directory only when generating HTML.

Finally, since .src.md uses GFM or Pandoc's Markdown, please refer to their respective manuals for proper syntax. If the dialects differ, use @@@if to write the appropriate version for each. Links to the manuals are below:

- GitHub Flavored Markdown Spec
- Pandoc User's Guide

A.1.3.9 Table Formatting

Formatting tables is not strictly required, but it can improve the readability of .src.md files.

For example, suppose you have the following table in a text file named `sample.src.md`:

```
Price List

@@@table
|item|price|
|:---:|:---:|
|mouse|$10|
|PC|$500|
@@@
```

The @@@table command formats it as follows, aligning items vertically for better aesthetics:

```
Price List

|item |price|
|:---:|:---:|
```

```
|mouse| $10 |
| PC  |$500 |
```

This command only affects the visual alignment within the Markdown file and does not change the final HTML or PDF output. Note that this command only supports the specific table syntax shown above.

The @@@table command is not supported during Rakefile-based conversion. You must use the standalone script `bin/table_formatter.rb`.

```
$ bundle exec ruby bin/table_formatter.rb sample.src.md
```

Whether to format tables is a matter of personal preference. If you don't find it necessary, you don't need to use the @@@table command.

A.1.4 Directory Structure

This section covers the primary directory structure. This structure can be modified, but if you do so, you must also update `/path_manager.rb`, which manages these directories.

- `/src`: Contains all source files (.src.md files, C source code, etc.).
- `/src/images`: Contains image files like PNG and JPG.
- `/gfm`: Contains generated GFM files.
- `/docs`: Contains generated HTML files and copies of image files.
- `/pdf`: Contains generated PDF files.
- `/lib`: Contains Ruby library files.
- `/test`: Contains test files. Run all tests by executing `bundle exec rake -f test/Rakefile` from the terminal.
- `/data`: Contains `chapters.yml` (defining chapters for PDF generation), other definition files, and templates.
- `/bin`: Contains standalone Ruby scripts.

A.1.4.1 Source Directory Structure

A.1.4.1.1 index.src.md `index.src.md` is placed directly in the source directory. In GFM generation, it is converted to `README.md` at the repository root. In HTML generation, it becomes `/docs/index.html`, serving as the website's homepage. In PDF generation, it forms the initial introductory section.

A.1.4.1.2 Section Files Section files are named with the prefix "sec" followed by the section number and the `.src.md` extension (e.g., `sec1.src.md`, `sec5.src.md`, `sec12.src.md`). These correspond to Section 1, Section 5, and Section 12, respectively.

Section numbers must be consecutive natural numbers starting from 1. However, if you want to insert a new section in the middle, renaming many files manually is tedious. For example, if you want to insert a new section between `sec8.src.md` and `sec9.src.md`, first create a temporary section using a decimal, like `sec8.5.src.md`. Then, use the `/bin/renumber.rb` script to automatically renumber the sections:

- `sec8.src.md` remains `sec8.src.md`
- `sec8.5.src.md` becomes `sec9.src.md`
- `sec9.src.md` becomes `sec10.src.md`

All subsequent sections will also be renumbered accordingly.

A.1.5 Rake

This repository's conversion system uses Rake, a Ruby build tool. Execute Rake by passing a task as an argument:

```
$ bundle exec rake <task name>
```

Available tasks:

- `gfm`: Generates GFM files. (Default task; executed if the task name is omitted).
- `html`: Generates HTML files.
- `pdf`: Generates PDF files.
- `all`: Generates GFM, HTML, and PDF files.
- `serve`: Starts a local server with `docs` as the site root (Port 8000).

A.1.5.1 Modes

The environment variable `G4T_ENV` determines the mode used when generating HTML files.

- development: Development mode. Generates files for local serving.
- production: Production mode. Generates files for GitHub Pages.

The primary difference is the base URL, which affects root-relative link URLs. GitHub Pages uses the repository name as the base URL, while the local environment has no base URL. The Ruby script determines the mode by reading `G4T_ENV`.

```
$ G4T_ENV=development bundle exec rake html # Development mode
$ G4T_ENV=production bundle exec rake html # Production mode
```

You must generate the HTML files in production mode before pushing to GitHub. If the environment variable assignment is omitted, the files will be generated in development mode.

A.2 TfeTextView API reference

A.2.1 Description

TfeTextView is a child object of GtkTextView. If its contents comes from a file, it holds the pointer to the GFile. Otherwise, the pointer is NULL.

A.2.2 Hierarchy

```
GObject -- GInitiallyUnowned -- GtkWidget -- GtkTextView -- TfeTextView
```

A.2.3 Ancestors

- GtkTextView
- GtkWidget
- GInitiallyUnowned
- GObject

A.2.4 Constructors

- tfe_text_view_new ()
- tfe_text_view_new_with_file ()

A.2.5 Instance methods

- tfe_text_view_get_file ()
- tfe_text_view_set_file ()
- tfe_text_view_get_modified
- tfe_text_view_read ()
- tfe_text_view_write ()

A.2.6 Signals

A.2.7 API for constructors, instance methods and signals

constructors

A.2.7.1 tfe_text_view_new()

```
GtkWidget *
tfe_text_view_new (void);
```

Creates a new TfeTextView instance and returns the pointer to it as GtkWidget.

Return value

- a new TfeTextView. The caller of the function takes ownership of the data, and is responsible for freeing it.

A.2.7.2 tfe_text_view_new_with_file()

```
GtkWidget *
tfe_text_view_new_with_file (GFile *file, GError **err);
```

If `file` is NULL, just returns a new TfeTextView instance as GtkWidget. If `file` is not NULL, creates a new TfeTextView, reads the contents of the `file` and set it into the GtkTextBuffer associated with the newly created TfeTextView. Then returns the pointer to the TfeTextView as GtkWidget. If an error happens during the read process, it returns `NULL` and set `err`.

Parameters

- file: type: `*GFile`. The caller owns the argument.
- err: type: `**GError`. It can be NULL and if it is NULL the function ignores this argument. If it is not NULL, `*err` must be NULL. If there are no errors, the pointer remains unchanged (it remains NULL). In case of error, the function will set `*err` to a newly allocated GError that contains an error domain, a code, and a message. The caller will take ownership of the GError, and be responsible for freeing it.

Return value

- a new TfeTextView. The caller of the function takes ownership of the data, and is responsible for freeing it. If an error, for example a read file error, happens, NULL will be returned.

Instance methods

A.2.7.3 tfe_text_view_get_file()

```
GFile *
tfe_text_view_get_file (TfeTextView *tv);
```

Returns the copy of the GFile associated with the TfeTextView. If no file is set, i.e. GFile is NULL, NULL will be returned.

Parameters

- tv: type: `*TfeTextView` The caller owns the argument.

Return value

- the pointer to the GFile or NULL. The returned data is owned by The instance.

A.2.7.4 tfe_text_view_set_file()

```
void
tfe_text_view_set_file (TfeTextView *tv, GFile *file);
```

Set the file associated with the TfeTextView instance.

Parameters

- tv: type: `*TfeTextView` The caller owns the argument.
- file: type: `*GFile` The argument can be NULL. The function releases the GFile instance in the object if it exists. Then, it sets file in the object to the argument. If the argument is NULL, the function just releases the old file instance.

A.2.7.5 tfe_text_view_get_modified()

```
gboolean
tfe_text_view_get_modified (TfeTextView *tv);
```

Indicates whether the buffer has been modified since the modification flag was last set to FALSE.

Parameters

- tv: type: `*TfeTextView` The caller owns the argument.

Return value

- TRUE if the buffer has been modified, FALSE otherwise.

A.2.7.6 tfe_text_view_read()

```
gboolean
tfe_text_view_read (TfeTextView *tv, GError **err);
```

Read the content of the file referenced by the GFile in the object. If the GFile is NULL or an error happens during the read process, it returns FALSE and `*err` is set to a GError.

parameters

- tv: type: `*TfeTextView` The caller owns the argument.
- err: type: `**GError` It can be NULL and if it is NULL the function ignores this argument. If it is not NULL, `*err` must be NULL. If there are no errors, the pointer remains unchanged (it remains NULL). In case of error, the function will set `*err` to a newly allocated GError that contains an error domain, a code, and a message. The caller will take ownership of the GError, and be responsible for freeing it.

return value

- Gboolean. If it successfully reads the file, it returns TRUE, otherwise FALSE

A.2.7.7 tfe_text_view_write()

```
gboolean
tfe_text_view_write (TfeTextView *tv, GError **err);
```

Write the content of the object buffer to a file referenced by the GFile in the object. If the GFile is NULL or an error happens during the write process, it returns FALSE and `*err` is set to a GError.

Parameters

- tv: type: `*TfeTextView` The caller owns the argument.
- err: type: `**GError` It can be NULL and if it is NULL the function ignores this argument. If it is not NULL, `*err` must be NULL. If there are no errors, the pointer remains unchanged (it remains NULL). In case of error, the function will set `*err` to a newly allocated GError that contains an error

domain, a code, and a message. The caller will take ownership of the GError, and be responsible for freeing it.

return value

- Gboolean. If it successfully writes the file, it returns TRUE, otherwise FALSE

A.2.7.8 Signals

No signal except the ones from the ancestors

A.3 Turtle Manual

Turtle is a simple interpreter for turtle graphics.

A.3.1 Requirements and Installation

Turtle is written in C language. You need:

- Linux. Turtle is tested on ubuntu 26.04 LTS
- gcc, meson and ninja
- gtk4. Turtle is tested on GTK 4.22.4

It is easy to compile Turtle's source files and install it.

For a user-area installation, put an option `--prefix=$HOME/.local` to your meson command line. Then, it will be installed under `$HOME/.local/bin`. The instruction is:

```
$ meson setup --prefix=$HOME/.local _build
$ ninja -C _build
$ ninja -C _build install
```

For a system-wide installation (under `/usr/local/bin`), just run it without any options. You'll need root privileges for the install step, so don't forget to add 'sudo' to the last command.

```
$ meson setup _build
$ ninja -C _build
$ sudo ninja -C _build install
```

Type the following command, and the following window appears.

```
$ turtle
```

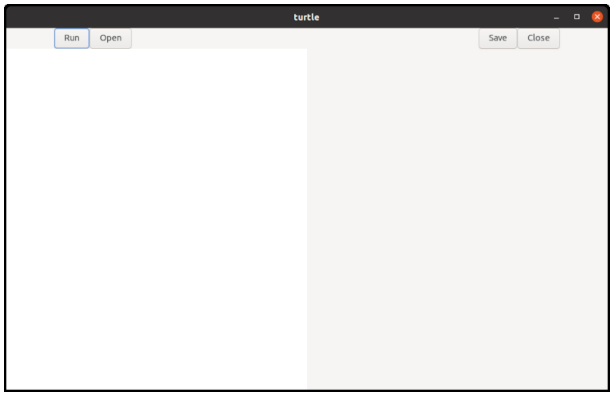


Figure A.1: Screenshot just after it's executed

The left half is a text editor and the right half is a surface. Surface is like a canvas to draw shapes.

Write Turtle language in the text editor and click the `run` button. Then, the program will be executed and it draws shapes on the surface.

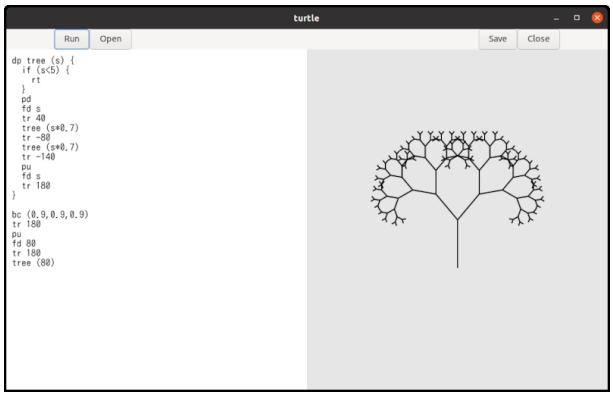


Figure A.2: Tree

If you uncomment the following line in `turtle.y`, the codes for debugging will also be compiled. Turtle shows the status to the standard output, but the speed is quite slow. It is not recommended except you are developing the program.

```
/* # define debug 1 */
```

A.3.2 Example

Imagine a turtle. The turtle has a pen and initially it is at the center of the screen, facing to the north (to the north means up on the screen). You can let the turtle down the pen or up the pen. You can order the turtle to move forward.

```
pd
fd 100
```

- pd: Pen Down. The turtle put the pen down so that the turtle will draw a line if it moves.
- fd 100: ForwarD 100. The turtle goes forward 100 pixels.

If you click the `run` button, a line segment appears on the screen. One of the endpoints of the line segment is at the center of the surface and the other is 100 pixels above the center. The point at the center is the start position of the turtle and the other endpoint is its final position.

If the turtle picks the pen up, no line segment appears.

```
pu
fd 100
```

The command `pu` means “Pen Up”.

The turtle can change its direction.

```
pd
fd 100
tr 90
fd 100
```

The command `tr` is “Turn Right”. The argument is an angle in degrees. Therefore, `tr 90` means “Turn right by 90 degrees”. If you click the `run` button, two line segments appear. One is vertical and the other is horizontal.

You can use `tl` (Turn Left) to change the turtle’s direction as well.

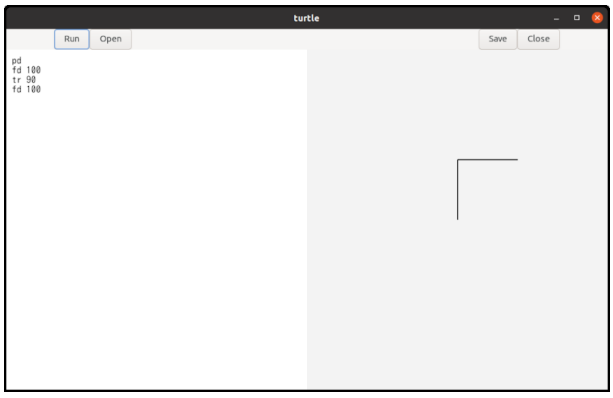


Figure A.3: Two line segments on the surface

A.3.3 Background and Foreground Colors

Colors are specified using RGB and are represented by an (r, g, b) value. Each component is a real number between 0 and 1.

- Red is (1.0, 0.0, 0.0). You can also write (1, 0, 0).
- Green is (0.0, 1.0, 0.0)
- Blue is (0.0, 0.0, 1.0)
- Black is (0.0, 0.0, 0.0)
- White is (1.0, 1.0, 1.0)

You can create a variety of colors by changing the values of the three components.

There are two commands to change colors.

- bc: Background Color. `bc (1,0,0)` changes the background color to red. This command clears the surface and changes its background color. Any shapes on the surface disappear.
- fc: Foreground Color. `fc (0,1,0)` changes the foreground color to green. This command changes the pen color. Existing shapes on the surface are not affected. After this command, the turtle draws lines in the new color.



Figure A.4: Change the foreground color

A.3.4 Other Commands

- pw: Pen Width. This is the same as pen size or line width. For example, if the current line width is 3, `pw 5` makes newly drawn lines thicker, while `pw 1` makes them thinner.
- rs: ReSet. The turtle moves back to the initial position and direction. In addition, The command initialize the pen, line width (pen size), and foreground color. The pen is down, the line width is 2 and the foreground color is black.

An instruction such as `fd 100`or `pd` is called a statement. Statements are executed in order, from the beginning to the end of the program.

A.3.5 Comments and Spaces

Text following a `#` (hash mark) to the next `\n` (new line) is treated as a comment. It includes the beginning hash mark but doesn't include the new line. If a comment appears on the last line of a file, the trailing new line can be omitted. The program ignores all comments.

```
# draw a triangle<NEW LINE>
fd 100 # forward 100 pixels<NEW LINE>
tr 120 # turn right by 120 degrees<NEW LINE>
fd 100<NEW LINE>
tr 120<NEW LINE>
fd 100 # Now a triangle appears.<EOF>
```

Here, `<NEW LINE>` and `<EOF>` represent a newline character and the end of te file respectively. The comments on lines 1, 2, 3 and 6 are all syntactically correct.

Whitespace characters (spaces, tabs and newlines) are ignored and serve only as delimiters between commands and values. When calculating column numbers, each tab is treated as eight spaces.

A.3.6 Variables and Expressions

Variable names begin with a letter, followed by any comnibnation of letters and digits. Keywords like `fd` or `tr` can't be used as variable names. `Distance` and `angle5` are variables, but `1step` is not, because it starts with a number instead of a letter.

Variable names are case-sensitive.

All variables store real numbers and function identically to the `double` type in C language. Integers are automatically cast to real numbers, meaning 1 and 1.0 represent the same value.

When writing numeric numbers, they must begin with a digit rather than a sign (`+` or `-`).

- `100`, `20.34` and `0.01` are valid numbers
- `+100` is not a valid number and will cause a syntax error. You should simply use `100` instead.
- `-100` is technically not a single number. However, turtle recognizes the `-` as a unary minus operaor and applies it to the number `100`, so the calculated result is still `-100`.
- `100 + -20` is a valid expression and will be recognized as `100 + (- 20)` since unary minus has higher precedence than addition. However, using parentheses — written as `100 + (-20)` — is recommended to make your code easier to read.

```
distance = 100
fd distance
```

In the first line, the value `100` is assigned to the variable `distance`. An assignment is a type of statement. While most statements must begin with a command like `fd`, assignments are the only exception to this rule.

The example above draws a line segment 100 pixels long.

You can use variables in expressions. There are 8 types of operations available.

- Addition: `x + y`
- Subtraction: `x - y`
- Multiplication: `x * y`
- Division: `x / y`
- Unary minus: `- x`
- Logical equality: `x == y`

- Greater than: `x > y`
- Less than: `x < y`

The last three operators are primarily used as conditions in `if` statements.

A variable is registered in the symbol table the first time it is assigned a value. Evaluating a variable before it has been registered is not allowed and will cause an error.

A.3.7 The if Statement

The Turtle language has a very simple `if` statement.

```
if (x > 50) {
  fd x
}
```

There is no else clause.

A.3.8 The loop Statement

The Turtle language has a very simple loop construct: `rp` (RePeat) statement.

```
rp (4) {
  fd 100
  tr 90
}
```

The code repeats the statements enclosed the curly braces four times.

A.3.9 Procedures

Procedures are similar to functions in the C language. The main difference is that procedures do not return a value.

```
dp triangle (side) {
  fd side
  tr 120
  fd side
  tr 120
  fd side
  tr 120
}

triangle (100)
```

The `dp` (Define Procedure) keyword is followed by a procedure name, a list of parameters, and the procedure's body. Procedure names must begin with a letter, followed by letters or digits. Parameters are provided as a comma-separated list of variables inside parentheses.

For example:

```
dp abc (a) { ... ... }
dp abc (a, b) { ... ... }
dp abc (a, b, c) { ... ... }
```

The body is a sequence of statements. These statements are not executed when the procedure is defined. Instead, they are executed later when the procedure is called.

Procedures are called by their names followed by arguments.

```
dp proc (a, b, c) { ... ... }

proc (100, 0, -20*3)
```

The number of arguments must match the number of parameters. An argument can be any valid expression. When calling a procedure, you must include parentheses after the procedure name,even if there are no arguments.

Procedure names and variable names do not conflict.

```
dp a () {fd a}
a=100
a ()
```

This is a valid program.

- 1: Defines a procedure name `a`. A variable name `a` is used in its body.
- 2: Assigns the value `100` to the variable `a`.
- 3: Calls the Procedure `a`.

However, using the same name for a procedure and a variable causes confusion. You should avoid doing this.

A.3.10 Recursive Calls

Procedures can be called recursively.

```
dp repeat (n) {
  n = n - 1
  if (n < 0) {
    rt
  }
  fd 100
  tr 90
  repeat (n)
}

repeat (4)
```

In this example, `repeat` is called from within its own body. A procedure calling itself is known as a recursive call.

New parameters are created and set each time the procedure is called. Therefore, the parameter `n` is 4 during the first call, but it is 3 during the second call. Each time the procedure is called, `n` decreases by one. Finally, when `n` becomes less than zero, the procedure returns.

Notice that the condition uses `< 0` rather than `== 0`. Because all numbers in the Turtle language are floating-point values (`double`), they are subject to tiny precision errors (rounding errors) during calculations. Checking for exact equality with `==` might fail due to these microscopic errors, which would cause an infinite loop. Using `<` ensures the recursion terminates safely.

The program above draws a square. This demonstrates that we can program loops using recursive calls.

A.3.11 Fractal Curves

A fractal is a special pattern that looks exactly the same even as you zoom in. To draw one, you start with a basic shape and apply a simple rule repeatedly. With every step, you add a smaller version of the shape, scaling it down at a constant rate (like exactly half the size of the previous one).

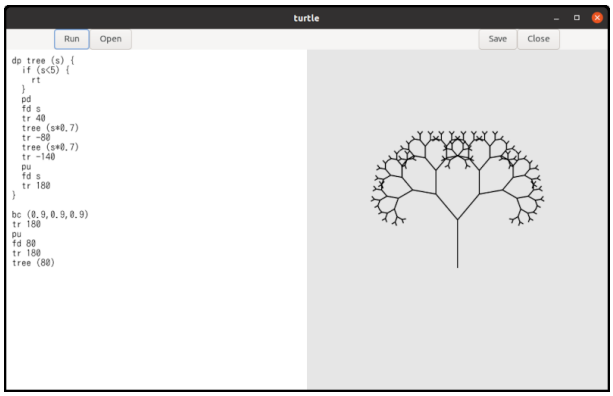


Figure A.5: Tree

This shape is called a fractal tree, or simply 'tree'. The base pattern is a single line segment, which represents the first stage. In the second stage, two shorter line segments are added to the endpoint of the original segment. Each new segment is 70% of the original length and is rotated by +30 or -30 degrees. The third stage applies this same rule, adding two even shorter segments to the ends of the second-stage segments. This process is then repeated several times to generate the complete tree.

This repetition is implemented using recursive calls. Two additional examples are shown below: the Koch curve and the Square Koch curve.

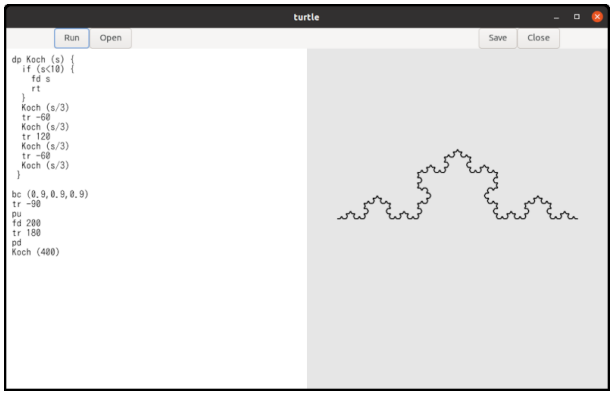


Figure A.6: Koch curve

The `src/turtle/example` folder contains some example Turtle code including the shapes above.

A.3.12 Tokens and Punctuation

Below is a list of the available tokens.

Keywords:

- `pu`: Pen up
- `pd`: Pen down
- `pw`: Pen width (line width)
- `fd`: Forward
- `tr`: Turn right
- `tl`: Turn left
- `bc`: Background color
- `fc`: Foreground color
- `if`: If statement
- `rt`: Return

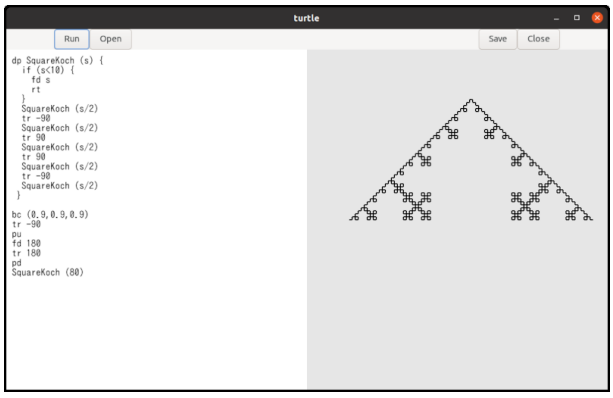


Figure A.7: Square Koch curve

- `rs`: Reset
- `rp`: Repeat
- `dp`: Define procedure

Identifiers and Numbers:

- Identifier: Used for the names of variables, parameters and procedures. It is denoted by the regular expression `[a-zA-Z][a-zA-Z0-9]*`.
- Number: Defined by the expression `(0|[1-9][0-9]*)(\.[0-9]+)?`. Numbers do not include a `+` or `-` sign to avoind syntactic ambiguity. However, a negative value such as `-10` is evaluated as a unary minus operator followed by a number.

Assignment:

- `=`: Assignment

Expression Symbols:

- `==`: Equality
- `>`: Greater than
- `<`: Less than
- `+`: Addition
- `-`: Subtraction or Unary Minus
- `*`: Multiplication
- `/`: Division
- `(`: Left parenthesis (Opening parenthesis)
- `)`: Right parenthesis (Closing parenthesis)

Delimiters:

- `(`: Left parenthesis (used to open parameter and argument lists, as well as conditions or values for statements such as `if`, `rp`, `bc`, and `fc`)
- `)`: Right parenthesis (used to close parameter and argument lists, as well as conditions or values for statements such as `if`, `rp`, `bc`, and `fc`)
- `{`: Left curly brace (used to open a block of statements, such as a procedure body)
- `}`: Right curly brace (used to close a block of statements)
- `,`: Comma (used to separate parameters or arguments)

Comments and Whitespace:

- Comment: A comment begins with a `#` symbol and continues until the next newline character. If a comment is placed at the very end of a file, the trailing newline may be omitted.
- Space: The standard space character.
- Horizontal tab: Recognized as eight spaces.

- Newline: Marks the end of a line.

Comments and whitespace are used to explicitly separate tokens. They do not have any syntactic meaning and are ignored by the parser.

A.3.13 Grammar

```
program:
  statement
| program statement
;

statement:
  primary_procedure
| procedure_definition
;

primary_procedure:
  pu
| pd
| pw expression
| fd expression
| tr expression
| tl expression
| bc '(' expression ',' expression ',' expression ')'
| fc '(' expression ',' expression ',' expression ')'
| ID '=' expression    /* ID is an identifier which is a name of variable */
| if '(' expression ')' '{' primary_procedure_list '}'
| rt
| rs
| rp '(' expression ')' '{' primary_procedure_list '}'
| ID '(' ')'    /* ID is an identifier which is a name of procedure */
| ID '(' argument_list ')'    /* the same as above */
;

procedure_definition:
  dp ID '('  ')' '{' primary_procedure_list '}'
| dp ID '(' parameter_list ')' '{' primary_procedure_list '}'
;

parameter_list:
  ID
| parameter_list ',' ID
;

argument_list:
  expression
| argument_list ',' expression
;

primary_procedure_list:
  primary_procedure
| primary_procedure_list primary_procedure
;
```

```
expression:
  expression '==' expression
| expression '>' expression
| expression '<' expression
| expression '+' expression
| expression '-' expression
| expression '*' expression
| expression '/' expression
| '-' expression %prec UMINUS
| '(' expression ')'
| ID
| NUM    /* NUM is a number */
;
```